

// ZPWR //

zpwr

v48.9.5

THE ENCYCLOPEDIA

a hacker's field guide to the terminal · 460+ verbs



MENKE
TECHNOLOGIES

Copyright and Attribution

zpwR — the Unix terminal environment

Copyright © 2026 MenkeTechnologies. All rights reserved.

No part of this book may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the publisher, except for brief quotations in reviews.

Published by MenkeTechnologies.

The trademarks and product names referenced in this book are the property of their respective owners. This book is an independent work: it is not affiliated with, authorized by, sponsored by, or otherwise endorsed by the Zsh project, the tmux project, the Vim or Neovim projects, or any other project named in these pages.

The software this book describes

zpwR is released under the MIT License, Copyright © 2017–2026 MenkeTechnologies. The source is at github.com/MenkeTechnologies/zpwR.

The software this book's software builds on

zpwR is an environment rather than a single program: it configures, extends, and dispatches to a large number of third-party tools, and it vendors some of them into its script directory so an install is self-contained. Every such tool remains the work of its own authors under its own license, recorded where it is vendored. Where this book names one of them it is describing how zpwR drives it, not claiming authorship of it.



MT PUBLISHING



MT OPEN SOURCE

Contents

Copyright and Attribution	1
Chapter 1 — The Red Pill	6
Chapter 2 — Navigation Protocol	10
Chapter 3 — The Git Dimension	15
Chapter 4 — The Search Engine	24
Chapter 5 — Editor Mastery	28
Chapter 6 — Performance Cult	35
Chapter 7 — Health & Diagnostics	38
Chapter 8 — Environment Mastery	47
Chapter 9 — The Janitor's Closet	52
Chapter 10 — Build System	58
Chapter 11 — Monitoring & Automation	66
Chapter 12 — Utility Arsenal	72
Chapter 13 — emacs Dimension	81
Chapter 14 — tmux Warfare	86
Chapter 15 — Network & Containers	91
Chapter 16 — The Logging System	96
Chapter 17 — Introspection	100
Chapter 18 — Installation & Reset	107
Chapter 19 — Batch Operations	110
Chapter 20 — Forgit Deep Dive	115
Chapter 21 — zpwr Environment Variables	120
Chapter 22 — Creative Tools	127
Chapter 23 — Exercism & External	133
Chapter 24 — Tips, Tricks & Easter Eggs	135

Chapter 25 — The Complete Reference	140
Chapter 26 — powerlevel10k & Instant Prompt	142
Chapter 27 — zinit & Plugin System	147
Chapter 28 — zsh Internals	156
Chapter 29 — Arb Architecture	163
Chapter 30 — tmux Deep Dive	168
Chapter 31 — vim Deep Dive	173
Chapter 32 — temprs — Temporary File Stack Manager	176
Chapter 33 — Isofrs — Enhanced Isof	178
Chapter 34 — eza — The Modern ls	180
Chapter 35 — bat — cat With Wings	182
Chapter 36 — fd-find — Find Replacement	185
Chapter 37 — ripgrep — grep At Light Speed	187
Chapter 38 — neovim	190
Chapter 39 — perl In zpw	192
Chapter 40 — Hidden Subsystems	195
Chapter 41 — Quality Assurance	199
Chapter 42 — Colorization Stack	200
Chapter 43 — The End	203
Chapter 44 — The Verb Dispatcher, Line By Line	205
Chapter 45 — Autoload As Architecture	209
Chapter 46 — The Completion Stack: _megacomplete And Friends	213
Chapter 47 — The Semicolon Trigger: fzf Completion Protocol	217
Chapter 48 — The Searchable Environment Index	220
Chapter 49 — Environment Resolution And The Override Protocol	224
Chapter 50 — The Cross-OS Layer	229
Chapter 51 — Logging As An Instrument	232
Chapter 52 — One Resolver For Everything	235
Chapter 53 — Manufacturing Completions	238
Chapter 54 — The Test Harness	241

Chapter 55 — What Happens When You Press Enter	244
Chapter 56 — The Keymap As A Namespace	248
Chapter 57 — Updating A Machine You Did Not Configure Today	252
Chapter 58 — roff In, Completions Out	255
Chapter 59 — tmux As A Composed Configuration	259
Chapter 60 — The Installer’s Ledger	262
Chapter 61 — The Prompt Is A Pair Of Arrays	267
Chapter 62 — Colorizing Programs You Did Not Write	270
Chapter 63 — Fan-Out And The Polling Daemon	273
Chapter 64 — Drawing Boxes That Do Not Break	277
Chapter 65 — Two Interpreters In One Script Directory	281
Chapter 66 — The Preview Command Is A Generated Program	286
Chapter 67 — Running A Script Outside The Shell That Configures It	291
Chapter 68 — The Vendored Usage Parser	296
Chapter 69 — Fifteen Ways To Watch Something	301
Chapter 70 — The Index That Outlived Its Files	307
Chapter 71 — Three Manifests, One Installer	310
Chapter 72 — Four Ways To Load A Vim Plugin	314
Chapter 73 — The Man Pages Nothing Installs	318
Chapter 74 — Sixty Copies Of One Box	321
Chapter 75 — The Instrument Panel	324
Chapter 76 — Capturing A Session And Replaying It	328
Chapter 77 — One Hundred Sixty-Four zstyles	331
Chapter 78 — The Palette Is A Permutation	335
Chapter 79 — Two Thousand Lines Of Cinema	339
Chapter 80 — The Picker Binary Is <code>arb --fzf</code>	343
Chapter 81 — The Turbo Ladder, As Actually Written	344
Chapter 82 — Installing A Unit File From The Repo That Ships It	346
Chapter 83 — The Git Configuration That Ships With The Shell	350
Chapter 84 — The Second Index	354

Chapter 85 — One Corpus, Three Renderers	358
Chapter 86 — Eighty-Three Batches	362
Chapter 87 — Two Thousand Lines Of Vim Script	364
Chapter 88 — Forty Functions In A Package List	368
Appendix A — Keybinding Reference	371
Appendix B — Complete Keybinding Dump	384
Colophon	402
Reproducing the Book's Claims	403
Also from MenkeTechnologies	404

Chapter 1 – The Red Pill

Welcome to the Machine

You found it. The rabbit hole. ZPWR is not a dotfiles repo. It is not a “framework.” It is a fully integrated shell operating system that happens to run inside your terminal.

What Is Zpwr?

A 460-verb command dispatcher built on top of zsh, tmux, vim/nvim, arb -fzf, and git. Every verb is a surgical tool. Every keybinding is a weapon. Every alias is a shortcut through the matrix.

The Philosophy

1. Verbs, not scripts. Everything is `zpwr <verb>`.
2. Discoverability. Tab completion on everything.
3. Symmetry. `vim*` and `emacs*` verbs mirror each other.
4. Speed. Compiled `zwc` files. Benchmarked startup.
5. No magic. Read the source. It is all zsh.

The Directory Tree

```
$ZPWR ..... root of the kingdom
$ZPWR_ENV ..... env vars and shell init
$ZPWR_SCRIPTS ..... executable scripts
$ZPWR_AUTOLOAD ..... autoloaded functions
$ZPWR_LOCAL ..... local config (gitignored)
$ZPWR_INSTALL ..... installer scripts
$ZPWR_TMUX ..... tmux configuration
$ZPWR_TEST ..... test suite
```

Your First Command

```
zpwr about    # behold the banner
zpwr help     # list every verb in color
```

Pro tip: `zpwr` is aliased to many short forms. Type `zpwr <tab>` and let the completions wash over you.

WARNING: Side effects may include mass alias adoption, involuntary tab-completing in other people’s terminals, and an irrational hatred of vanilla bash.

Your First 5 Minutes

You just installed `zpwr`. The terminal blinks. What now? Here is your survival kit for the first 300 seconds.

Step 1: See The Banner

```
zpwr about          # the ASCII art welcome screen
zpwr banner          # same thing, different alias
zpwr bannercounts    # banner + environment statistics
```

Step 2: Count Your Arsenal

```
zpwr environmentcounts # how many aliases, functions, etc.
```

This prints a breakdown of your loaded shell environment: aliases, functions, builtins, parameters, completions. If the numbers are big, congratulations. You are armed.

Step 3: Discover The Verbs

```
zpwr help           # colored list of ALL verbs
zpwr verbs          # arb --fzf fuzzy search through verbs
```

The verbs verb is your panic button (alias: subcommandsfzf). Forgot what a command is called? Fuzzy search it. This pipes every verb and its description into arb -fzf. Type a few characters. Watch the list collapse. Hit enter. Execute.

Step 4: Verify Your Environment

```
zpwr doctor          # run environment health check
zpwr envvars          # list all zpwr env vars
zpwr colortest        # verify your terminal colors
zpwr colortest256     # the full 256-color palette
```

Step 5: Tab Complete Everything

Type zpwr and press <tab>. You will see 460 verbs. Type zpwr git and press <tab>. Filtered to git verbs. Type zpwr vim and press <tab>. Filtered to vim verbs. The completion system knows every verb. Trust it.

If you only remember one command from this page: zpwr verbs It is the cheat code to everything else.

The Anatomy of a Verb

Every zpwr command follows one sacred pattern: zpwr <verb> [args...]

How Dispatch Works

When you type zpwr filesearch, here is what happens: 1. The zpwr function receives “filesearch” as \$1 2. It looks up ZPWR_VERBS[filesearch] in the verb table 3. The table entry maps to a function or alias 4. That function is invoked with any remaining args.

The Verb Table

The associative array ZPWR_VERBS is the brain. Each entry has the format:

```
ZPWR_VERBS[verb]='function_or_alias=description'
```


Many Names, One Function

Verbs often have aliases for muscle memory:

```
zpwr forward = zpwr f    = zpwr fwd
zpwr reverse = zpwr r    = zpwr rvs = zpwr cdup
zpwr github  = zpwr gh   = zpwr mygithub
zpwr repo    = zpwr zp   = zpwr zpwr
```

Tab Completion In Action

zpwr git<tab> # shows: gitcommit, gitconfig, gitsearch... zpwr vim<tab> # shows: vimall, vimrecent, vimctags...
zpwr home<tab> # shows: home, homeenv, homelocal... zpwr forgit<tab> # shows: forgitadd, forgitlog, forgitdiff...

Inspect The Table Yourself

```
zpwr subcommands      # dump all verbs to stdout
zpwr verbs            # fuzzy search verbs with arb --fzf
zpwr verbscount        # how many verbs exist
```

The Edit Suffix Pattern

Many verbs come in pairs: filesearch -> open file after selection filesearchedit -> type into command line for editing The “edit” variant puts the result on your command line instead of executing immediately. Power and precision.

Configuring Your Rig

Your terminal is a cockpit. These are the dials.

The Four Config Sessions

Each of these opens a curated vim session with the relevant config files already loaded in buffers:

```
zpwr brc    # shell aliases file vim session
```

Opens `$ZPWR_LOCAL/tokens.sh` and related files this is where YOUR personal aliases live.

```
zpwr zrc    # zshrc vim session
```

Opens the zshrc and related zsh configs the source of all shell initialization.

```
zpwr vrc    # vimrc/init.vim vim session
```

Opens vim/nvim config files your editor’s soul laid bare.

```
zpwr trc    # tmux.conf vim session
```

Opens tmux configuration pane and window management settings.

The Tokens File

```
zpwr tokens      # zmax the .tokens.sh file
zpwr vimtokens   # same file, explicit vim
zpwr emacs tokens # same thing, emacs edition
```

Located at `$ZPWR_LOCAL/.tokens.sh` This file is gitignored. Put secrets and custom aliases here. Never commit this file. It is YOUR local override.

Editor Commands

```
zpwr edit        # launch $EDITOR
zpwr editor      # same thing
zpwr vimconfig   # edit all zpwr configs in vim
```

The Full Config Stack

`$ZPWR_LOCAL/.tokens.sh` ... your aliases and secrets `$ZPWR_ENV/.zpwr_env.sh` .. env vars and paths

```
~/.zshrc ..... shell entry point
~/.vimrc / init.vim ..... editor config
~/.tmux.conf ..... tmux config
```

Think of it as: brc for shell, zrc for zsh, vrc for vim, trc for tmux. Four letters, four kingdoms.

Chapter 2 – Navigation Protocol

Directory Warfare

In the filesystem, speed is survival. Typing full paths is for people who have given up on life.

Forward And Reverse

```
zpwr forward <dir> # cd into a child directory
zpwr f <dir>       # same thing, shorter
zpwr fwd <dir>     # same thing, medium length
```

```
zpwr reverse <dir> # cd UP the tree to a parent matching <dir>
zpwr r <dir>       # same thing, one letter
zpwr rvs <dir>     # same thing, three letters
zpwr cdup <dir>    # same thing, cd-up style
```

Example: you are in `~/code/myproject/src/lib/utls`

```
zpwr r myproject # jumps UP to ~/code/myproject
```

No more `../..../..` chains. Navigate like a human.

Basic Cd

```
zpwr cd <path> # cd to directory arg
```

Seems redundant but integrates with zpwr's dispatch.

Z: Freccy Jumping

```
zpwr z <query> # jump to freccy-ranked directory
```

Z tracks where you go. The more you visit a directory, the higher it ranks. Type a fragment, land at the top match.

```
zpwr zcd # arb --fzf list of z-ranked dirs, pick one
```

When you cannot remember the exact fragment, zcd shows you the full ranked list in arb `-fzf`.

Practical Combos

zpwr z proj # land in your most-visited “proj” dir
zpwr r src # climb back up to the “src” parent
zpwr f tests # dive into the “tests” child.

The pattern is: z for teleportation, f for descent, r for ascent. Three directions. Total coverage.

PROTIP: If you are still typing `cd ../../..`, this is your intervention.

Home Base Operations

Every zpwr directory has a one-word teleporter. No paths. No thinking. Just say where.

The Home Verb Family

```
zpwr home          # cd $ZPWR (the root)
zpwr homelocal     # cd $ZPWR_LOCAL
zpwr homescripts   # cd $ZPWR_SCRIPTS
zpwr homeenv       # cd $ZPWR_ENV
zpwr homeinstall   # cd $ZPWR_INSTALL
zpwr homeautoload  # cd $ZPWR_AUTOLOAD
zpwr homeautoloadcommon # cd $ZPWR_AUTOLOAD_COMMON
zpwr homecomps     # cd $ZPWR_COMPS
zpwr hometest      # cd $ZPWR_TEST
zpwr hometmux      # cd $ZPWR_TMUX
zpwr homeexercism  # cd $HOME/Exercism
```

What Lives Where

```
home ..... the zpwr repo root, start of everything
homelocal ..... gitignored personal configs and tokens
homescripts ..... executable shell scripts
homeenv ..... environment variables and init files
homeinstall ..... installer scripts for dependencies
homeautoload ..... autoloaded functions (the engine room)
```

Homeautoloadcommon platform-independent autoloads.

```
homecomps ..... zsh completion definitions
hometest ..... test suite files
hometmux ..... tmux configuration files
```

Example Workflow

You want to edit an autoload function:

```
zpwr homeautoloadcommon
```

Now you are in the autoload directory # ls, grep, edit to your heart's content.

You want to check your local tokens:

```
zpwr homelocal
```

Lands you right next to .tokens.sh.

Every home* verb is a bookmark you never need to set. The bookmarks are built into the system.

Named Directories

Zsh has a secret weapon most people never learn: named directories. They turn env vars into tilde paths.

How It Works

When you do `hash -d ZPWR=$ZPWR`, zsh registers a named directory. Now `~ZPWR` expands to `$ZPWR` everywhere: in `cd`, in tab completion, in prompt, in arguments.

Zpwr's Named Directories

```
~ZPWR ..... $ZPWR root
~ZPWR_SCRIPTS ..... $ZPWR_SCRIPTS
~ZPWR_AUTOLOAD ..... $ZPWR_AUTOLOAD
~ZPWR_ENV ..... $ZPWR_ENV
~ZPWR_LOCAL ..... $ZPWR_LOCAL
~ZPWR_INSTALL ..... $ZPWR_INSTALL
~ZPWR_TMUX ..... $ZPWR_TMUX
~ZPWR_TEST ..... $ZPWR_TEST
~ZPWR_COMPS ..... $ZPWR_COMPS
```

What This Means In Practice

Your prompt might show: `~ZPWR_AUTOLOAD/common` instead of the full path. You can type:

```
cd ~ZPWR_SCRIPTS and it just works
vim ~ZPWR_ENV/.zpwir_env.sh tilde paths everywhere
```

Tab Completion With Tilde

Type `cd ~ZPWR<tab>` and watch zsh expand the options: `~ZPWR` `~ZPWR_SCRIPTS` `~ZPWR_ENV` `~ZPWR_LOCAL` ... This works in any command, not just `cd`.

The Mechanism

Zsh option `AUTO_NAME_DIRS` turns any exported scalar parameter into a named directory automatically. ZPWR exports its directory variables, so they become named directories for free.

Think of named directories as human-readable symlinks that live in your shell's brain instead of the filesystem. No files created. No cleanup needed. Pure memory.

Quick Jumps

The `home*` verbs land you in zpwr directories. These verbs land you in the ecosystem around zpwr.

zpwr Internals

```
zpwr scripts      # cd to scripts directory
```

Shortcut alias: `zs`.

```
zpwr autoload     # cd to autoload directory
```

Shortcut alias: `zal`.

```
zpwr completions  # cd to completion directory
zpwr comps        # same thing
```

Shortcut alias: `zco`.

Plugin Ecosystem

```
zpwr plugins      # cd to $ZSH_CUSTOM/plugins
```

Shortcut alias: `zpl` This is where oh-my-zsh custom plugins live. Your third-party zsh plugins are neighbors here.

```
zpwr forked      # cd to $ZPWR_FORKED_DIR
zpwr fp          # same thing
```

Forked repos that `zpwr` depends on live here. Useful when you need to hack on a dependency.

Special Directories

```
zpwr hidden      # cd to $ZPWR_HIDDEN_DIR
```

The hidden directory. Dot-prefixed internal state.

```
zpwr repo        # cd to $ZPWR_REPO_NAME
zpwr zp          # same thing
zpwr zpwr        # same thing (yes, zpwr zpwr)
```

The Navigation Cheat Sheet

Use case Verb ——— —

```
Edit a script ..... zpwr scripts
Fix a completion ..... zpwr comps
```

Add an autoload function .. `zpwr autoload`.

```
Debug a plugin ..... zpwr plugins
Patch a fork ..... zpwr forked
```

Each of these is one verb, zero path typing, and instant teleportation to where the work is.

Navigation Verbs

// 39 ways to get where you are going //

Navigate the filesystem, ZPWR directories, and frequency ranked locations with precision.

Directory Movement

```
zpwr cd          # cd to directory arg
zpwr f           # cd to directory arg
zpwr forward     # cd to directory arg
zpwr fwd         # cd to directory arg
zpwr cdup        # cd up tree to directory arg
zpwr r           # cd up tree to directory arg
zpwr reverse     # cd up tree to directory arg
zpwr rvs         # cd up tree to directory arg
zpwr catcd       # cat and cd to first dir
zpwr clone       # clone and cd to arg
```

zpwr Home Directories

```
zpwr home          # go to $ZPWR
zpwr homeautoload  # go to $ZPWR_AUTOLOAD
zpwr homeautoloadcommon # go to $ZPWR_AUTOLOAD_COMMON
zpwr homecomps     # go to $ZPWR_COMPS
zpwr homeenv       # go to $ZPWR_ENV
zpwr homeexercism  # cd to $HOME/Exercism
zpwr homeinstall   # go to $ZPWR_INSTALL
zpwr homelocal     # go to $ZPWR_LOCAL
zpwr homescripts   # go to $ZPWR_SCRIPTS
zpwr hometest      # go to $ZPWR_TEST
zpwr hometests     # go to $ZPWR_TEST
zpwr hometmux      # go to $ZPWR_TMUX
```

zpwr Subdirectories

```
zpwr autoload      # cd to autoload directory
zpwr comps         # cd to completion directory
zpwr completions   # cd to completion directory
zpwr forked        # cd to $ZPWR_FORKED_DIR
zpwr fp            # cd to $ZPWR_FORKED_DIR
zpwr hidden        # go to $ZPWR_HIDDEN_DIR
zpwr plugins       # cd to $ZSH_CUSTOM/plugins
zpwr repo          # cd to $ZPWR_REPO_NAME
zpwr scripts       # cd to scripts directory
zpwr web           # cd to web dir
zpwr zp            # cd to $ZPWR_REPO_NAME
zpwr zpwr          # cd to $ZPWR_REPO_NAME
zpwr zpz           # cd to repo, checkout, rebase, push
```

Frecency & Arb

```
zpwr z             # cd to z frecency ranked dir
zpwr zcd           # list then cd to z ranked dir
zpwr dirsearch     # cd to a sub dir via arb --fzf
```

Chapter 3 – The Git Dimension

Git Basics

Git is already a language. ZPWR makes it a dialect you can actually speak without consulting the man page.

The Commit Pipeline

```
zpwr gitcommit "your message here"
```

This is not just git commit. It does: git add -> git commit -> git push All in one verb. The full gacp (git add commit push). Because who commits without pushing? Cowards.

Config And Ignores

```
zpwr gitconfig          # edit git config file
zpwr gitglobalconfig    # same thing, explicit name
zpwr gitignore          # edit .git/info/exclude (local)
zpwr gitignores         # edit global git ignores file
zpwr gitglobalignores   # same as gitignores
```

Stash And Diff (VIA Forgit)

ZPWR delegates interactive stash and diff to forgit:

```
zpwr forgitdiff         # interactive git diff viewer
zpwr forgitstash        # interactive stash browser
```

See page 13 for the full forgit arsenal.

Remote Operations

```
zpwr gitremotes         # list all git remotes
zpwr gitdir             # check if pwd is a git dir
```

Tags

```
zpwr gitedittag         # print tag to $BUFFER
zpwr gitupdatetag       # print tag and latest msg to $BUFFER
```

Real-world Example

You just finished a feature. Ship it:

```
zpwr gitcommit "add quantum flux capacitor"
```


Stages everything, commits with message, pushes. # One line. Done. Next feature.

Need to check your git config:

```
zpwr gitconfig
```

Opens it in \$EDITOR. Fix, save, done.

Git verbs follow the pattern: git + action noun. If you can name it, zpwr probably has a verb for it.

GitHub Integration

GitHub is not a website. It is an API with a website. ZPWR treats it accordingly.

Create And Destroy

```
zpwr githubcreate    # create a remote GitHub repo
zpwr hc              # same thing, two letters
```

Creates a repo on GitHub from your current directory. Sets up the remote. Ready to push.

```
zpwr githubdelete    # delete a remote GitHub repo
zpwr hd              # same thing, two letters
```

This is permanent. It asks for confirmation. But once confirmed, the repo is gone. Poof.

Browse Your Github

```
zpwr github          # open your GitHub profile in browser
zpwr gh              # same thing
zpwr mygithub         # same thing
```

Opens your profile page. Quick ego check.

```
zpwr githubzpwr      # open zpwr repo on GitHub
zpwr ghz             # same thing
```

Jump straight to the zpwr source. File issues. Star it. You know you want to.

Contribution Stats

```
zpwr githubcontribcount # count GitHub contribs in last year
```

Tells you how green your contribution graph is. The number that keeps you pushing commits at 2 AM.

Reveal: The Browser Shortcut

```
zpwr reveal          # show current repo in browser
```

You are in a git repo. You want to see it on GitHub. One word. Browser opens. You are there.

Workflow Example

Start a new project from scratch:

```
mkdir new-project && cd new-project
git init
zpwr hc           # repo created on GitHub
zpwr gitcommit "initial commit"
zpwr reveal       # admire it in the browser
```

From zero to pushed repo in four commands. That is the GitHub integration promise.

Git Archaeology

The past is not dead. It is not even past. It is stored in `.git/objects` and `zpwr` can dig it up.

Searching The Log

```
zpwr gitsearch      # search for regex in git log
```

Uses `arb -fzf` to grep through your entire commit history. That one commit where you “fixed the thing”? Found.

```
zpwr gitcommits     # search git commits with arb --fzf
```

Interactive commit browser. Preview diffs inline. Like `git log` but you can actually find things.

Counting Commits

```
zpwr gitcommitcount # total number of commits
```

Just a number. But what a number. The sum total of your decisions in this repo.

Contribution Analysis

```
zpwr gitcontribcount # contributions by author
```

Who committed the most? The leaderboard of blame.

```
zpwr gitcontribcountdirs # contributions across dirs
```

Same analysis but across multiple repositories. Useful for org-wide contribution audits.

```
zpwr gitcontribcountlines # lines contributed by author
```

Lines of code per author. The metric that lies to you. But it is a fun lie.

Size Analysis

```
zpwr gittotallines  # total line count of git files
```

How big is this repo, really? Now you know.

```
zpwr gitlargest     # largest files in git history
```

Shows the biggest blobs stored by git, descending. That 50MB binary someone committed in 2019? There it is. Now you know what to purge (see page 12).

Dig Session Example

Who wrote the most code in this repo?

```
zpwr gitcontribcount
```

What is eating all the disk space?

```
zpwr gitlargest
```

Find that commit about the database migration:

```
zpwr gitsearch
```

Type "migration" in arb -fzf. There it is.

Git Surgery

Sometimes you need to rewrite history. Not the geopolitical kind. The git kind. These are destructive operations. Use with intent.

Clearing The Cache

```
zpwr gitclearcache    # clear old git refs and objects
```

Runs garbage collection on git's internal state. Prunes unreachable objects, packs loose refs. Your .git directory gets a deep clean.

Removing Commits

```
zpwr gitclearcommit  # remove matching commits from history
```

This is filter-branch territory. Rewrites history. Force push required after. Use when someone committed passwords, API keys, or that embarrassing commented-out code block.

Removing Files From All History

```
zpwr gitclearfile    # rm file from all git refs and objects
```

Nuclear option for files. That 200MB video someone committed in 2020? This removes it from every commit that ever existed. Your repo size drops. Your clone time drops. Your blood pressure drops.

Email Surgery

```
zpwr gitemail        # change email with git filter-branch
```

Changes both author and committer email across history.

```
zpwr gitaemail       # change author email only
```

When you committed with your personal email at work.

```
zpwr gitcemail       # change committer email only
```

When the committer field has the wrong identity.

When To Use These

Scenario: you committed with wrong@email.com.

```
zpwr gitemail
```

Follow the prompts to fix every commit.

Scenario: .env file in git history with secrets.

```
zpwr gitclearfile
```

Purges it from every ref and object.

REMEMBER: These rewrite history. Coordinate with your team before force pushing. Or do not coordinate and watch the chaos unfold.

Forgit: Git on Steroids

Forgit wraps common git operations with interactive previews. Every action gets a live preview pane. Every choice is informed. No more blind git commands.

Staging And Unstaging

```
zpwr forgitadd          # interactive git add
```

Shows unstaged files with inline diffs in the preview. Select files with tab. Enter to stage. Visual staging.

```
zpwr forgitreset        # interactive git reset HEAD
```

The reverse of forgitadd. Unstage files visually.

Viewing Changes

```
zpwr forgitdiff          # interactive git diff viewer
```

Browse changed files. Preview shows colorized diffs. Side-by-side comparison in your terminal.

```
zpwr forgitlog           # interactive git log
```

Commit history with preview. See the diff for each commit without leaving the log view. Beautiful.

Stash Operations

```
zpwr forgitstash         # interactive stash browser
```

Lists stashes. Preview shows what each stash contains. Pick one, apply it. No more "was it stash@{3}?"

Cleanup

```
zpwr forgitclean         # interactive git clean
```

Shows untracked files. Select which to delete. Preview before you destroy. Civilized cleanup.

```
zpwr forgitrestore       # interactive git restore
```

Discard changes to selected files. With preview.

Gitignore Management

```
zpwr forgitignore      # browse gitignore templates
zpwr forgitignoreget   # download a gitignore template
zpwr forgitignorelist  # list available templates
zpwr forgitignoreupdate # update gitignore templates
zpwr forgitignoreclean # clean gitignore cache
```

Extras

```
zpwr forgitinfo      # show git repo information
zpwr forgitwarn      # show forgit warnings
```

Forgit turns git into a visual experience. Every forgit* verb is a regular git command with eyes.

Git Battalion Operations

One repo is easy. Twenty repos is a war. These verbs let you run commands across all of them.

The General: Foralldir

```
zpwr gitforalldir <cmd> # run cmd in ALL git dirs
```

Walks every git directory it can find and runs your command inside each one. The nuclear option.

Cached Repos

```
zpwr gitrepoexec <cmd> # run cmd in cached git dirs
```

Uses the cached list of known repos. Faster than foralldir because it does not rescan the filesystem.

Clean Vs Dirty Targeting

```
zpwr gitreposcleanexec <cmd> # refresh + run in clean repos
zpwr gitreposcleancacheexec <cmd> # run in cached clean repos
zpwr gitreposdirtyexec <cmd> # refresh + run in dirty repos
zpwr gitreposdirtycacheexec <cmd> # run in cached dirty repos
```

Clean repos have no uncommitted changes. Dirty repos have pending work. Target your commands accordingly.

Mass Update

```
zpwr gitupdatefordir # run git updates in all git dirs
```

Pulls latest, updates submodules, the full sync. Run this on a Monday morning. Coffee. Wait.

Branch Wipe Operations

```
zpwr gitfordirmain # wipe to main branch in git dirs
zpwr gitfordirdevelop # wipe to develop branch in git dirs
```

```

zpwir gitzfdir <branch> [z]      # wipe to any branch + z jump
zpwir gitzfdirmain [z]          # z jump + wipe to main
zpwir gitzfdirdevelop [z]       # z jump + wipe to develop

```

The Repos File

```

zpwir gitreposfile      # edit the git repos file

```

This file lists all known repositories. Edit it to add or remove repos from batch operations.

Example: Status Check All Repos

```

zpwir gitreposexec git status -s

```

Runs git status -s in every cached repo. Instantly see which repos have uncommitted work.

Battalion ops turn you from a developer into a fleet commander.

Git Add-Commit-Push

// 318 aliases for the git power user //

The zsh-git-acp plugin provides 318 aliases that collapse multi-step git workflows into single commands.

The Gacp Pattern

```

gacp "message"      # git add -A, commit, push
gac "message"       # git add -A, commit (no push)
gcp                 # git commit, push

```

Branch Shortcuts

```

gcm                 # git checkout main
gcb <branch>        # git checkout -b <branch>
gbd <branch>        # git branch -d <branch>

```

zpwir Integration

```

zpwir gitcommit "msg"      # commit and push with message
zpwir gitcommitcount       # count commits in current repo

```

The full alias list is enormous — 318 combinations of git operations compressed into 2-4 character shortcuts. Most follow the pattern: g + operation letter(s).

Plugin: MenkeTechnologies/zsh-git-acp.

Git Operations Reference

// 33 verbs for git mastery //

Core git verbs for commits, config, history rewriting, contributor stats, and multi-repo operations.

Commit & Push

```
zpw! gitcommit          # commit and push with arg message
zpw! gitcommitcount     # count commits in current repo
zpw! gitcommits         # search git commits with arb --fzf
zpw! clone              # clone and cd to arg
```

Config & Ignores

```
zpw! gitconfig          # edit git config file
zpw! gitglobalconfig    # edit git config file
zpw! gitignore          # $EDITOR ~/.git/info/exclude
zpw! gitignores         # edit git ignores file
zpw! gitglobalignores   # edit git ignores file
```

Email & History Rewriting

```
zpw! gitemail          # change email with git filter-branch
zpw! gitcemail         # change committer email
zpw! gitaemail         # change author email
zpw! gitclearcommit    # remove matching commits from history
zpw! gitclearcache     # clear old git refs and objects
zpw! gitclearfile      # rm file from all git refs/objects
```

Stats & Info

```
zpw! gitcontribcount   # count git contributions by author
zpw! gitcontribcountdirs # contributions by author for dirs
zpw! gitcontribcountlines # count lines contributed by author
zpw! gittotallines     # total line count of git files
zpw! gitlargest        # show largest files stored by git
zpw! gitremotes        # list all git remotes
zpw! gitdir            # check if pwd is git dir
zpw! gitwho            # aggregate contributor stats
zpw! gitsearch         # search for regex in git log
```

Multi-repo & Tags

```
zpw! gitforallldir     # run cmd in all git dirs
zpw! gitfordirdevelop  # git wipe to develop in git dirs
zpw! gitfordirmain     # git wipe to main in git dirs
zpw! gitupdatefordir   # run git updates in all git dirs
zpw! gitzfordir        # git wipe on branch in git dirs
zpw! gitzfordirdevelop # z and git wipe to develop
zpw! gitzfordirmain    # z and git wipe to main
zpw! gitupdatetag      # print tag and latest msg
zpw! gitedittag        # print tag to $BUFFER
```

GitHub & Travis CI

// 16 verbs for the hosted world //

Verbs for GitHub repo management, contribution counting, and Travis CI browser integration.

Github Navigation

```
zpwr gh          # open github profile
zpwr github      # open github profile
zpwr mygithub    # open github profile
zpwr ghz         # open zpwr github repo
zpwr githubzpwr  # open zpwr github repo
zpwr zpwrgithub  # open zpwr github
```

Github Repo Management

```
zpwr githubcreate  # create remote github repo
zpwr githubdelete  # delete remote github repo
zpwr hc            # create remote github repo
zpwr hd            # delete remote github repo
```

Contribution Stats

```
zpwr ghcontribcount  # count github contribs in last year
zpwr githubcontribcount  # count github contribs in last year
```

Travis Ci

```
zpwr travis        # open current travis project
zpwr travisbranch   # open current travis branches
zpwr travisbuild    # open current travis builds
zpwr travispr       # open current travis PRs
```


Chapter 4 – The Search Engine

Finding Files

The filesystem is vast. You are small. But you have `arb -fzf` and a verb for every search strategy.

Filesearch: Local Tree

```
zplr filesearch          # find and open a file in a sub dir
```

Recursively lists files under `$PWD` into `arb -fzf`. Type fragments of the filename. Fuzzy match. Enter. Opens the file in your pager.

```
zplr filesearchedit      # same but puts path on command line
```

Instead of opening immediately, the selected path lands on your `$BUFFER` for further editing. Append commands, pipe it, whatever you want.

Dirsearch: Directories Only

```
zplr dirsearch           # cd to a sub directory via arb --fzf
```

Same idea but only directories. Pick one, land in it. Faster than `cd-tab-tab-tab-tab`.

Findsearch: Full Disk

```
zplr findsearch          # find across the entire drive
```

Uses `find(1)` to search the whole filesystem. Results feed into `arb -fzf`. Pick one, open it. Slow on first run. Worth the wait.

```
zplr findsearchedit      # same but result goes to $BUFFER
```

Locatesearch: Index-based

```
zplr locatesearch        # search the locate database
```

Uses `locate(1)` which searches a pre-built index. Blazing fast. But only as fresh as your last updatedb.

```
zplr locatesearchedit    # same but result goes to $BUFFER
```

The Search Hierarchy

Speed Scope Verb — — — Fast ... current tree .. filesearch

```
Fast ... dirs only ..... dirsearch
```

Slow ... entire disk ... findsearch Instant indexed disk .. locatesearch

Start with filesearch for local work. Escalate to locatesearch when you need the whole disk. Fall back to findsearch when the index is stale.

Searching Content

Finding files is one thing. Finding the line inside the file? That is the real game.

Wordsearch: Content In Local Tree

```
zpwr wordsearch          # find file by content in sub dir
```

Greps through files in the current tree. Results show matching lines with filenames. Fzf preview shows the file with match highlighted. Select one, jump straight to it.

```
zpwr wordsearchedit      # same but result goes to $BUFFER
```

Puts the selected file path on your command line. Useful when you want to pipe or chain commands.

Grep: Ag Into Arb

```
zpwr grep                # grep through pwd with ag into arb --fzf
```

This is the heavy artillery. Uses the silver searcher (ag) piped into arb –fzf for interactive filtering. Respects .gitignore. Skips binaries. Fast.

Filesearch Vs Wordsearch

Know the filename? Use filesearch Know the content? Use wordsearch Know nothing? Use grep and improvise.

The Edit Suffix Pattern

Every search verb has an edit variant: filesearch -> filesearchedit wordsearch -> wordsearchedit findsearch -> findsearchedit locatesearch -> locatesearchedit.

Without edit: opens the result immediately. With edit: puts the path on your command line. This lets you compose: vim \$(zpwr filesearchedit)

Real-world Examples

Find where “ZPWR_VERBS” is defined:

```
zpwr wordsearch
```

Type ZPWR_VERBS in arb –fzf, browse the matches.

Quick grep for TODO comments:

```
zpwr grep
```

Type TODO, see every file that has one.

The search verbs are your eyes inside the codebase. Use them before you reach for a GUI.

The ARB Dimension

ARB is the nervous system of zpwr. It is wired into search, selection, navigation, git, vim, environment browsing, and verb dispatch. If zpwr has a UI, arb -fzf IS that UI.

Verb Discovery

```
zpwr verbs      # fuzzy search all zpwr verbs
```

Every verb, every description, in one arb -fzf window. This is the god-mode cheat sheet. Learn it first.

Environment Browsing

```
zpwr envaccept   # accept from arb --fzf env
```

Browse all shell environment entries (aliases, functions, parameters, builtins) in arb -fzf. Select one, accept it onto the command line.

```
zpwr envedit     # edit from arb --fzf env
```

Same browsing but puts the result on \$BUFFER for editing.

```
zpwr envcachesearch # search cached aliases, functions, etc.
```

Searches the pre-built cache of your entire environment. Faster than envedit because it uses the cache file.

Arb Patterns Across zpwr

ARB appears in dozens of verbs. The pattern is always: 1. A data source generates lines (files, commits, dirs) 2. Lines pipe into arb -fzf with a preview command 3. You fuzzy-filter, select, and take an action.

Verbs with arb -fzf built in: filesearch file list -> arb -fzf -> open wordsearch grep results -> arb -fzf -> open dirsearch dir list -> arb -fzf -> cd gitcommits git log -> arb -fzf -> show diff forgitadd unstaged files -> arb -fzf -> stage forgitlog commit log -> arb -fzf -> preview zcd z database -> arb -fzf -> cd subcommandsfzf all verbs -> arb -fzf -> execute.

The Arb Keybindings

```
zpwr vimsearch   # search vim keybindings in arb --fzf
zpwr allsearch   # search ALL keybindings in arb --fzf
```

When you want to find a keybinding but do not remember where it is defined. Search them all.

ARB is not optional in zpwr. It is the interface. Master arb -fzf and you master zpwr.

The Learning System

// your shell remembers what you learn //

The zsh-learn plugin turns your shell into a persistent knowledge base backed by MySQL. Save commands, search them, quiz yourself, and build a personal learning collection.

Core Verbs

```
zpwri learn          # save a learning entry
zpwri searchl        # search the collection
zpwri quiz           # quiz yourself
zpwri quizall        # quiz all entries
zpwri redo           # redo last search as SQL update
```

Search Variants

```
zpwri se             # basic search
zpwri see            # category search
zpwri seee           # timestamp search
zpwri ser            # random search
zpwri sef            # search into arb --fzf
zpwri searchall      # entire collection into arb --fzf
zpwri searchfull     # entire collection into arb --fzf
zpwri searchle       # category search (alias for see)
zpwri searchlee      # timestamp search (alias for seee)
zpwri learnsearch    # search the collection
zpwri rsql           # redo into vim
zpwri redosql        # redo into vim
```

Management

```
zpwri learndel       # delete entry
zpwri learndelete    # delete entry (alias)
zpwri delid          # delete by ID
zpwri editl          # edit entry by ID
zpwri getlearn        # get all entries
zpwri learnget       # get all entries
zpwri createlearningcollection # create MySQL table
zpwri droplearningcollection  # drop MySQL table
```

How It Works

Entries are stored in a MySQL database (configurable via `$ZPWR_SCHEMA_NAME` and `$ZPWR_TABLE_NAME`). Each entry has a category, timestamp, and content. The quiz system pulls random entries and tests your recall.

Plugin: MenkeTechnologies/zsh-learn.

Chapter 5 – Editor Mastery

Vim Integration Overview

Vim is not just an editor in zpwr. It is a first-class citizen with its own verb family. Over 20 verbs, each opening vim in a specific context.

The Philosophy

Every vim verb answers one question: “What do I want to edit right now?” The answer is always one verb away.

The Vim* Verb Family

```
vimall ..... open ALL zpwr files (for :argdo)
vimalledit ..... pick and edit 1+ zpwr configs
vimautoload ..... open all autoloads (for :argdo)
vimcd ..... vim edit, then cd to first dir
vimconfig ..... edit all zpwr config files
vimctags ..... vim zpwr ctags
vimemacsconfig ..... vim edit emacs zpwr configs
vimfilesearch ..... vim a file found via arb --fzf
vimfilesearchedit ... vim edit a file found via arb --fzf
vimfindsearch ..... vim a file from find(1) via arb --fzf
vimfindsearchedit ... vim edit find result
vimgtags ..... vim zpwr GNU global tags
vimgtagsedit ..... vim edit zpwr GNU global tags
vimlocatesearch ..... vim a file from locate via arb --fzf
vimlocatesearchedit . vim edit locate result
vimplugincount ..... count vim plugins
vimpluginlist ..... list all vim plugins
vimrecent ..... edit most recent files
vimrecentcd ..... cd + edit most recent files
vimrecentsudo ..... sudo edit most recent files
vimrecentsudocd ..... cd + sudo edit recent files
vimscripts ..... open all zpwr scripts (for :argdo)
vimscriptedit ..... pick and edit 1+ scripts
vimsearch ..... search vim keybindings via arb --fzf
vimtests ..... edit all zpwr tests
vimtokens ..... vim the .tokens.sh file
vimwordsearch ..... vim a file found by content
vimwordsearchedit ... vim edit file found by content
```

The :argdo Pattern

Verbs like vimall and vimscripts load many files into vim’s arglist. Then you can:

```
:argdo %s/old/new/g | update
```

Mass find-and-replace across the entire zpwr codebase.

If it can be edited, there is a vim verb for it.

Vim File Operations

Opening vim is easy. Opening vim with the right files already loaded? That is where zpwr earns its keep.

The Big Opens

```
zpwr vimall          # load ALL zpwr files into arglist
```

Every .zsh, .sh, .py, .pl file in the zpwr tree. Hundreds of files. Ready for :argdo mass operations. This is the tactical nuke of text editing.

```
zpwr vimalledit      # arb --fzf pick 1+ configs to edit
```

Like vimall but you choose which files to open. Surgical precision instead of carpet bombing.

Recent Files

```
zpwr vimrecent       # edit most recently opened files
```

Pulls from vim's file history. Opens the files you were just working on. Resume where you left off.

```
zpwr vimrecentcd     # cd to dir + open recent files
```

Same as vimrecent but also changes to the directory. Your shell and editor are now in the same place.

```
zpwr vimrecentsudo   # sudo edit recent files
```

When you need root to edit those files.

```
zpwr vimrecentsudocd # cd + sudo edit recent files
```

The full combo: navigate, elevate, edit.

zpwr Subsystems

```
zpwr vimscripts      # load all zpwr scripts into arglist
zpwr vimscriptedit    # pick and edit 1+ scripts
zpwr vimautoload      # load all autoloader into arglist
zpwr vimtests         # edit all zpwr test files
```

vim Cd

```
zpwr vimcd           # edit file, cd to its directory
```

Opens a file in vim, and when you exit, your shell is cd'd to the directory containing that file. Edit something, then be where it lives. Elegant.

Workflow Example

Resume yesterday's work:

```
zpwr vimrecentcd
```

Opens recent files, lands you in the right dir.

Mass rename a function across all scripts:

```
zpwr vimscripts  
:argdo %s/oldFunc/newFunc/g | update
```

Done. Every script updated.

Vim Search and Tags

Finding files is step one. Finding definitions inside those files is step two. Tags are step three: jump to any symbol by name.

vim File Search

```
zpwr vimfilesearch      # arb --fzf find file -> open in vim
```

Lists files in the current tree via arb –fzf. Select one, it opens in vim. No path typing.

```
zpwr vimfilesearchedit  # arb --fzf find file -> path to $BUFFER
```

Same search but puts the path on your command line.

vim Word Search

```
zpwr vimwordsearch      # arb --fzf grep content -> open in vim
```

Searches file contents, shows matches in arb –fzf. Select a match, vim opens at that location.

```
zpwr vimwordsearchedit  # same but path to $BUFFER
```

Full Disk Search Into vim

```
zpwr vimfindsearch      # find(1) -> arb --fzf -> vim  
zpwr vimfindsearchedit  # find(1) -> arb --fzf -> $BUFFER  
zpwr vimlocatesearch    # locate -> arb --fzf -> vim  
zpwr vimlocatesearchedit # locate -> arb --fzf -> $BUFFER
```

The search escalation ladder from page 15 but with vim as the final destination instead of a pager.

Ctags: Jump To Definition

```
zpwr vimctags           # open zpwr ctags in vim
```

Ctags indexes function and variable definitions. With tags loaded, Ctrl-] on any symbol jumps to its definition. Ctrl-T jumps back. Code navigation at the speed of thought.

Gnu Global Tags

```
zpwr vimtags          # open zpwr GNU global tags in vim
zpwr vimtagsedit      # edit zpwr GNU global tags
```

GNU Global is ctags on steroids. It indexes not just definitions but references too. Find every caller of a function, not just its definition.

The Search Matrix

What you know Verb to use ———- ———- Filename fragment vimfilesearch String in a file vimwordsearch
File somewhere vimlocatesearch Symbol name vimctags then Ctrl-] All references vimtags.

Vim Config and Sessions

Configuration is not a one-time event. It is a lifestyle. ZPWR gives you session commands that open the right config files as a group.

The Session Verbs

Each of these opens a curated set of config files in vim buffers, ready for synchronized editing:

```
zpwr vrc              # vim/nvim config session
```

Opens your vimrc or init.vim along with related files. All your editor settings in one session.

```
zpwr brc              # shell aliases session
```

Opens the tokens file and shell alias configs. Where you tune your personal shell experience.

```
zpwr zrc              # zshrc session
```

Opens the zshrc and zsh init chain. The root of your shell's boot sequence.

```
zpwr trc              # tmux.conf session
```

Opens tmux configuration files. Pane layouts, keybindings, status bar.

Config Editing

```
zpwr vimconfig        # edit all zpwr configs in vim
```

Opens every zpwr configuration file. The master session. Use :bn and :bp to navigate between buffers.

```
zpwr vimtokens        # vim the .tokens.sh file
zpwr tokens            # same file, opens in zmax
```

Quick jump to your personal aliases and secrets file. This is the file that is gitignored. Your sanctuary.

Plugin Inventory

```
zpwr vimplugincount   # how many vim plugins installed
zpwr vimpluginlist     # list all vim plugins
```

Know thy plugins. Count them. Name them. If the count scares you, it should.

Typical Config Workflow

Add a new shell alias:

```
zpwr brc
```

Edit .tokens.sh, add your alias, :wq # Source it or open a new terminal.

Tweak a vim keybinding:

```
zpwr vic
```

Navigate to the mapping, change it, :wq.

Three letters to open any config. No file paths. No guessing. Just intent.

Emacs Mirror Universe

For every vim verb, there is an emacs verb. Identical semantics. Different editor. Perfect symmetry. ZPWR does not take sides in the editor war. It arms both factions equally.

The Symmetry Table

vim verb emacs verb ——— ————

```
vimall ..... emacsall
vimalledit ..... emacsalledit
vimautoload ..... emacsautoload
vimcd ..... emacscd
vimconfig ..... emacsconfig
vimctags ..... emacsctags
vimfilesearch ..... emacsfilesearch
vimfilesearchedit ... emacsfilesearchedit
vimfindsearch ..... emacsfindsearch
vimfindsearchedit ... emacsfindsearchedit
vimgtags ..... emacsgtags
vimgtagsedit ..... emacsgtagsedit
vimlocatesearch ..... emacslocatesearch
vimlocatesearchedit . emacslocatesearchedit
vimrecent ..... emacsrecent
vimrecentcd ..... emacsrecentcd
vimrecentsudo ..... emacsrecentsudo
vimrecentsudocd ..... emacsrecentsudocd
vimscripts ..... emacsscripts
vimscriptedit ..... emacsscriptedit
vimtests ..... emacs tests
vimtokens ..... emacs tokens
vimwordsearch ..... emacswordsearch
vimwordsearchedit ... emacswordsearchedit
vimplugincount ..... emacsplugincount
vimpluginlist ..... emacspluginlist
```

Emacs-specific

```
zpwr emacsallserver      # emacs all zpwr files (server mode)
zpwr emacsconfig         # emacs edit emacs-specific configs
zpwr emacspluginlist     # emacs the zpwr directory
```

The lesson: zpwr is editor-agnostic by design. Replace “vim” with “emacs” in any verb. It works. Both sides get the same firepower.

Vim Integration

// 35 verbs for the vim faithful //

The vim verb family mirrors emacs with full parity. Open, edit, search, navigate — all wired to vim/nvim.

Open & Edit

```
zpwr vimall          # vim all zpwr files for :argdo
zpwr vimalldedit     # vim edit 1 or more configs
zpwr vimautoload     # vim all autoloads :argdo
zpwr vimscripts     # vim all zpwr scripts for :argdo
zpwr vimscriptedit   # vim edit 1 or more scripts
```

Config & Tokens

```
zpwr vimconfig       # edit all zpwr configs
zpwr vimemacsconfig  # vim edit emacs zpwr configs
zpwr vimtokens       # vim the .tokens.sh file
zpwr vimtests        # edit all zpwr tests
zpwr vrc             # vimrc vim session
zpwr brc             # shell aliases file vim session
zpwr trc             # tmux.conf vim session
zpwr zrc             # zshrc vim session
zpwr tokens          # zmax the .tokens.sh file
```

Navigation

```
zpwr vimcd          # vim edit and cd to first dir
zpwr vimrecent       # vim edit most recent editor files
zpwr vimrecentcd     # cd and vim edit most recent files
zpwr vimrecentsudo   # sudo vim edit most recent files
zpwr vimrecentsudocd # cd and sudo vim most recent files
```

Search

```
zpwr vimsearch       # search vim keybindings
zpwr vimfilesearch   # vim a file in a sub dir
zpwr vimfilesearchedit # edit vim a file in a sub dir
zpwr vimfindsearch   # vim accept find drive for file
zpwr vimfindsearchedit # vim edit find drive for file
zpwr vimlocatesearch # vim accept locate drive for file
zpwr vimlocatesearchedit # vim edit locate drive for file
zpwr vimwordsearch   # vim a file in a sub dir by word
zpwr vimwordsearchedit # edit vim a file by word
```

Tags, Plugins & Viminio

```
zpwr vimctags          # vim zpwr ctags tags
zpwr vimgtags          # vim zpwr GNU global tags
zpwr vimgtagsedit      # vim edit zpwr GNU global tags
zpwr vimplugincount    # total number of vim plugins
zpwr vimpluginlist     # total list of vim plugins
zpwr catviminfo        # cat custom vim info file
zpwr catrecentfviminfo # cat recentf then custom vim info
zpwr catnviminforecentf # cat custom vim info then recentf
```

Chapter 6 – Performance Cult

Benchmarking Startup

A shell that takes 2 seconds to start is a shell that has already lost. ZPWR takes startup time seriously enough to build a benchmarking verb.

The Bench Verb

```
zpwr bench
```

Runs multiple shell startup iterations and reports: - Mean startup time - Percentiles (p50, p95, p99) - Baseline diff (zpwr overhead vs bare zsh)

What The Numbers Mean

```
< 100ms .... imperceptible. You are golden.
```

100-200ms .. noticeable if you open many terminals. 200-500ms .. getting sluggish. Time to investigate.

```
> 500ms .... something is wrong. Check your plugins.
```

How It Works

The bench verb spawns N fresh zsh instances, each running through the full init chain. It measures wall-clock time from fork to prompt. Then it does the same for bare zsh (no zpwr). The diff tells you exactly what zpwr costs.

What Slows You Down

Common culprits in order of likelihood: 1. Too many vim/emacs plugins loading at shell init 2. nvm/pyenv/rbenv hooking into every prompt 3. Uncompiled completion files (run zpwr compile) 4. Network calls in your init chain (stop that) 5. Massive HISTFILE being loaded synchronously.

Optimization Checklist

```
zpwr compile      # recompile all cache and completions
zpwr compilefpath # recompile all fpath files
zpwr compilefiles # recompile all files
```

Compiled .zwc files load faster than source. This is the single biggest performance lever.

Benchmark Regularly

Run zpwr bench after any config change. If the numbers go up, you broke something. If they go down, you are a performance wizard.

Your shell starts thousands of times a year. Every millisecond is multiplied by every terminal open. Measure. Optimize. Measure again.

Flame Charts

Bench tells you HOW LONG startup takes. Flame tells you WHERE the time goes.

The Flame Verb

```
zpwr flame
```

Generates a startup flame chart from zprof data. Shows you exactly which functions and files are consuming your startup time, in descending order.

Reading The Output

The flame chart shows a ranked list of functions: Each entry shows: - Function/file name - Time consumed (milliseconds) - Percentage of total startup time.

The top of the list is where optimization effort pays off the most. If one function takes 40% of startup, that is your target.

How Zprof Works

Zsh has a built-in profiler: zprof. When enabled, it instruments every function call during shell init and records the wall-clock time. The flame verb wraps this into a readable report.

Common Hotspots

What you will typically see at the top:

```
compinit ..... completion system init
compaudit ..... completion security check
compdump ..... writing completion cache
_zpwr_* ..... zpwr autoload functions
nvm/pyenv/rbenv .... version manager hooks
```

Optimization Tips

1. If compinit is slow, run `zpwr compile` to cache it
2. If a version manager dominates, lazy-load it
3. If an autoload is slow, check for disk I/O in it
4. If compaudit is slow, fix the directory permissions
5. After changes, run `zpwr bench` to verify improvement

The Optimization Loop

```
zpwr flame      # identify the bottleneck
```

... fix it ...

```
zpwr bench      # measure the improvement
zpwr flame      # find the next bottleneck
```

Repeat until you hit diminishing returns.

Flame charts turn gut feelings into data. Stop guessing. Start profiling.

Live Dashboard

Bench is a snapshot. Flame is a profile. Top is a live feed of your shell's vital signs.

The Top Verb

```
zpwr top
```

Launches a live shell resource dashboard. Think htop, but for your zsh environment.

What Each Section Shows

Shell Environment - Number of aliases currently defined - Number of functions loaded - Number of parameters (variables) in scope - Builtin and keyword counts.

Completion System - Completion functions registered - Cache file sizes and freshness.

Resource Usage - Memory footprint of the shell process - History file size and entry count - Autoload function count.

Interpreting The Data

High alias count? Normal for zpwr. 1000+ is fine. High function count? Also normal. Autoloaded functions are lazy-loaded, they cost nothing until called. Large HISTFILE? Consider zpwr backuphistory and trim. Stale cache? Run zpwr compile to refresh.

Companion Verbs

```
zpwr environmentcounts # one-shot environment stats
zpwr autoloadcount     # total number of autoloads
zpwr autoloadlist      # list all autoloads
zpwr vimplugincount     # count vim plugins
zpwr emacsplugincount  # count emacs plugins
```

Use these for quick spot checks without the full dashboard.

The Performance Quartet

zpwr bench -> how fast is total startup? zpwr flame -> ASCII flame chart of function hotspots zpwr startup -> per-function zprof breakdown zpwr top -> what is loaded right now?

Four verbs. Four perspectives. Complete observability. Your shell is no longer a black box. It is an open book.

Chapter 7 – Health & Diagnostics

The Doctor Is In (1/2)

The Doctor Is In

// every system needs a physician. yours lives at zpwr doctor. //

Your shell is a living organism. It accumulates cruft, grows tumors in PATH, and occasionally develops broken symlinks. The doctor runs a full-body scan and tells you what hurts.

Invoke The Doctor

```
zpwr doctor          # full diagnostic scan
zpwr doctor -h       # help with detailed check list
```

What It Checks

1. STALE .zwc FILES

Compiled zsh files older than their source. Fix: zpwr recompile.

2. ZCOMPDUMP FRESHNESS

Completion dump older than 1 week. Fix: zpwr regenzzsh.

3. DUPLICATE PATH ENTRIES

Every duplicate is a wasted hash lookup. Fix: zpwr deduppaths.

4. DUPLICATE/MISSING FPATH ENTRIES

Same disease, different organ.

5. BROKEN SYMLINKS

Dangling symlinks in `$ZPWR`.

Continued on next page...

The Doctor Is In (2/2)

The Doctor (continued)

6. NAMED DIRECTORIES

Invalid named dirs and p10k pollution from auto_name_dirs leaking gitstatus vars.

7. DEPENDENCIES

Checks: git, eza, arb -fzf, tmux, perl, python3, zinit.

8. HISTORY & BACKUPS

Is HISTFILE present? How many backups exist?

9. CONFIG SYMLINKS

Checks .zshrc, .vimrc, .tmux.conf are symlinks.

Reading The Results

Green checkmark = healthy yellow warning = degraded but functional red X = broken, fix now.

The Signal Bar

= HEALTHY #####— = WARNINGS ##—— = CRITICAL.

Companion Verbs

```
zpwr stale           # stale .zwc, orphan caches
zpwr pathaudit       # deep PATH/FPATH audit
zpwr lint            # shellcheck + zsh -n
zpwr startup         # zprof breakdown
zpwr ports           # listening ports
```

Pro Move

Add zpwr doctor to weekly cron. Catch rot early.

Dependency Cartography

// mapping the call graph. who calls whom. who dies alone. //

zpwr has hundreds of functions. They call each other in a tangled web of dependencies. The deps verb maps that web. Think of it as Google Maps for your function call graph.

Basic Usage

```
zpwr deps             # summary overview
zpwr deps zpwrClearList # what does zpwrClearList call?
zpwr deps -h          # full help
```

The Flags

-r, --reverse Who calls this function? (reverse lookup) -o, --orphans Functions called by nobody internally -s, --summary Overview stats (default when no args) -d, --dot Output dot format for graphviz rendering

Forward Lookup: What Does X Call?

```
zpwr deps zpwrDoctor
```

Shows every function zpwrDoctor calls, plus depth-2 callees. Output: zpwrDoctor -> zpwrPrettyPrint -> [zpwrLogDebug, ...]

Reverse Lookup: Who Calls X?

```
zpwr deps -r zpwrPrettyPrint
```

Answers: “Who depends on zpwrPrettyPrint?” If the answer is 30 functions, that function is load-bearing. Touch it carefully.

Orphan Detection

```
zpwr deps --orphans
```

Lists functions that no other zpwr function calls. These are either entry points (user-invoked) or dead code. Orphans are not necessarily bad – many are top-level verbs or zinit hooks. But if you wrote zpwrFoo and nobody calls it, maybe zpwrFoo was a fever dream.

Graphviz Export

```
zpwr deps --dot > graph.dot  
dot -Tsvg graph.dot -o graph.svg
```

Produces a visual dependency graph. Nodes are functions, edges are call relationships. The dot output uses cyberpunk colors: dark background, cyan nodes, magenta edges. Open graph.svg in a browser and behold the nervous system.

The Summary View

With no args, deps shows two ranked lists: TOP 10 MOST DEPENDENCIES (functions that call the most others) TOP 10 MOST DEPENDED ON (functions called by the most others) Plus stats: total functions, orphan count, total call edges. The bar charts use Unicode block characters. Very aesthetic.

How It Works

Scans autoload dirs: common, darwin, linux, comp_utils. Reads each function body and string-matches against all other function names. Also scans `$ZPWR_SCRIPTS/lib.sh`. It is $O(n^2)$ and proud of it. Your function count is not big enough to matter.

Command Autopsy

// when you need to know why a command is slow, fat, or dead //

The trace verb is a command profiler. It wraps any command in instrumentation and reports wall time, CPU time, memory, context switches, page faults, and more.

Basic Usage

```
zpwr trace ls -la      # full trace of ls -la  
zpwr trace -q make     # quick: timing + resources only  
zpwr trace -h          # full help
```

The Flags

-q, -quick Timing + resource stats only (runs once) -s, -strace Syscall counts only (Linux; no ltrace) -l, -ltrace C library calls only (Linux; no strace) -a, -all Both strace + ltrace (default behavior)

What You Get: The Resource Card

Exit code green = 0, red = nonzero wall time real clock time with rating: INSTANT (<100ms), FAST (<1s), NOMINAL (<5s), SLOW (<30s), HEAVY (30s+) user/sys time CPU time in user/kernel space peak RSS max resident memory (auto-scaled B/KB/MB/GB) ctx switches voluntary + involuntary context switches page reclaims soft faults (memory mapped in from cache) page faults hard faults (actual disk I/O for memory) block I/O filesystem reads and writes signals rcv number of signals delivered IPC messages sent/received inter-process messages shell delta memory change of the parent shell child procs before/after child process count.

Platform Differences

MacOS: `/usr/bin/time -l` for resource stats. SIP blocks dtruss/dtrace on most binaries. You get timing + memory + kernel stats. Linux: `strace -c` for syscall counts, `ltrace -c` for C library calls. Command re-runs for each tool.

Practical Examples

```
zpwr trace git status           # is git status slow in this repo?
zpwr trace -q rm -rf node_modules # quick mode for destructive ops
zpwr trace -s npm install        # syscalls only (Linux)
zpwr trace zsh -c 'source ~/.zshrc' # profile shell startup
```

The Signal Bar

Bottom of output: a 10-block performance signal. ##### score 10 = under 100ms. Lightning. #####— score 6 = under 2s. Acceptable. ##——— score 2 = under 30s. Go get coffee.

WARNING: Default mode re-runs the command for strace/ltrace. Use -q for destructive commands (rm, truncate, etc).

Usage Intelligence

// your shell keeps a diary. these verbs read it. //

You have 460 verbs and thousands of aliases. Which ones do you actually use? funcrank and aliasrank mine your shell history to answer that question.

Funcrank: Function & Verb Profiler

```
zpwr funcrank           # top 20 functions and verbs
zpwr funcrank 40        # top 40
zpwr funcrank --reset    # nuke cache, rebuild from scratch
zpwr funcrank --reset 50 # rebuild then show top 50
```

Funcrank scans your HISTFILE for occurrences of known function names and zpwr verbs. It produces two ranked lists:

1. Top functions by history occurrence count
2. Top zpwr verbs by how often you type “zpwr <verb>”

Each entry gets a Unicode bar chart. The biggest user wins.

Aliasrank: Reverse Expansion Profiler

```
zpwr aliasrank          # top 20 by expanded usage
zpwr aliasrank 50       # top 50
zpwr aliasrank --reset  # force cache rebuild
```

Here is the trick: zpwr uses zsh-expand to expand aliases before they hit history. So history contains the expanded form, not the alias name. aliasrank does a reverse lookup: it matches history lines against alias values to figure out which alias you would have typed.

The Cache System

`$ZPWR_LOCAL/zpwr-aliasrank-cache.zsh` command counts `$ZPWR_LOCAL/zpwr-verbrank-cache.zsh` verb counts `$ZPWR_LOCAL/zpwr-rank-stamp` history line watermark

The cache auto-rebuilds when history grows by 1000+ lines. The watermark file stores the last-seen history line count. No need to rebuild every time – incremental is the way. Use `--reset` to force a full rebuild if cache seems stale.

Interpreting The Results

If your top verb is `help`, you are still learning. Good. If your top verb is `cleanall`, you break things a lot. If your top function is `cd`, you need more named dirs. If an alias never appears, maybe it is too long to remember.

Practical Intelligence

Run funcrank monthly. Notice patterns: - Verbs you never use might need better names or keybindings - Functions with high counts are candidates for optimization - Low-count zpwr verbs might benefit from shorter aliases.

Your history is a behavioral log. These verbs turn it into actionable intelligence. Know thyself, operative.

Startup Profile

// bench tells you HOW FAST. startup tells you WHERE the time goes. //

zpwr bench measures total wall time over multiple iterations. zpwr startup gives a single-shot, per-function breakdown using zsh's built-in zprof profiler.

The Verb

```
zpwr startup          # top 30 functions (default)
zpwr startup 50       # top 50 functions
zpwr startup 100      # full breakdown
```

How It Works

1. Launches a fresh zsh with ZPWR_PROFILING=true
2. zprof runs at the end of the init sequence
3. Output is captured and colorized by self-time: $\geq 10\%$ RED — hot path, optimize here $\geq 3\%$ YELLOW — warm, worth investigating $\geq 1\%$ CYAN — normal $< 1\%$ DIM — negligible

The Performance Quartet

zpwr bench -> how fast is total startup? zpwr flame -> ASCII flame chart of function hotspots zpwr startup -> per-function zprof breakdown zpwr top -> what is loaded right now?

Four verbs. Four perspectives. Your shell is transparent.

Shell Lint

// if it parses, it ships. but does it REALLY parse? //

zpwr lint runs two levels of analysis on every zpwr file: syntax checking with zsh -n, and static analysis with shellcheck.

The Verb

```
zpwr lint          # run all checks
zpwr lint -z       # zsh -n syntax check only
zpwr lint -s       # shellcheck only
```

What It Checks

zsh -n (syntax check) Every file in autoload/common, autoload/darwin, autoload/linux, env/*.sh, env/*.zsh, install/.zshrc. This catches unmatched brackets, bad redirections, and broken quoting. Fast — no execution, just parsing.

Shellcheck (static analysis) Every .sh and .zsh file in scripts/ and install/. Catches quoting bugs, unused variables, subshell pitfalls. Zsh-incompatible warnings are suppressed (SC1071 etc).

When To Run It

After editing any zpwr file. Before committing. Before pushing. Catch it at the gate, not in production.

Companion Verbs

```
zpwr doctor        # broader health check
zpwr stale         # find stale compiled files
zpwr recompile     # rebuild .zwc files after fixes
```

Listening Ports

// who is listening? and on which door? //

zpwr ports wraps lsof to show every TCP port in LISTEN state with the process name, PID, user, and bind address.

The Verb

```
zpwr ports          # show all listening ports
zpwr ports 8080      # filter to port 8080
zpwr ports -k 3000   # kill whatever is on port 3000
```

Color Coding

Privileged ports (< 1024) — RED These require root. sshd on 22, httpd on 80. Common ports (< 10000) — YELLOW Dev servers, databases. 3000, 5432, 8080. High ports (>= 10000) — GREEN Ephemeral and user-space. Generally safe.

The Kill Flag

zpwr ports -k 3000 finds the PID on port 3000 and sends SIGTERM. Shows process name before killing. Use when a dev server refuses to die.

Companion Verbs

```
zpwr lsof           # interactive arb --fzf over all lsof output
zpwr ps             # grep through ps output
zpwr kill           # arb --fzf kill from ps
```

Stale File Finder

// compiled cruft is invisible cruft. make it visible. //

zpwr stale scans the entire zpwr tree for artifacts that have gone stale, orphaned, or broken.

The Verb

```
zpwr stale          # scan and report
zpwr stale -f        # delete all stale files
```

What It Finds

STALE .zwc Source file is newer than its compiled .zwc. Your shell is running yesterday's bytecode.

ORPHAN .zwc The source file was deleted but the .zwc remains. Dead weight taking up space and confusing zsh.

BROKEN SYMLINKS Symlinks pointing to files that no longer exist. Source of cryptic 'file not found' errors.

EMPTY CACHE FILES Zero-byte files in cache directories. Usually left behind by interrupted operations.

The Fix Flag

zpwr stale -f deletes everything it finds in one pass. Follow up with zpwr recompile to rebuild .zwc files.

Workflow

```
zpwr stale          # see what is stale
zpwr stale -f       # nuke the cruft
zpwr recompile      # rebuild fresh .zwc files
zpwr doctor         # verify everything is clean
```

Path Audit

// your PATH is your attack surface. audit it. //

zpwr pathaudit inspects PATH, FPATH, and MANPATH for duplicates, missing directories, and world-writable entries.

The Verb

```
zpwr pathaudit      # audit all three arrays
zpwr pathaudit -v    # verbose: show every entry
```

What It Checks

DUPLICATES Same directory appearing more than once. Wastes hash lookups and obscures precedence. Fix: zpwr deduppaths.

MISSING DIRECTORIES PATH entry pointing to a dir that does not exist. Slows command lookup — zsh checks every entry.

WORLD-WRITABLE DIRECTORIES Any user can write to this directory. If it is in your PATH, an attacker can plant a malicious binary that shadows a real command. Security risk.

All Three Arrays

PATH — command lookup for executables FPATH — function autoload lookup for zsh MANPATH — man page lookup for documentation.

Companion Verbs

```
zpwr doctor         # broader health check (also checks paths)
zpwr deduppaths      # remove duplicate entries
zpwr environmentcounts # count all env arrays
```

Doctor checks paths as part of a broader scan. pathaudit goes deeper with world-writable detection.

Expansion Stats Dashboard

Your aliases are working for you — here's the proof.

Zpwrexpandstats

Every time zsh-expand fires — alias, global alias, suffix alias, self-ref escape, native glob/hist, or spelling correction — it logs the event. zpwrExpandStats reads that log and shows you a ranked dashboard.

```
zpwr expandstats          # show full dashboard
zpwr expandstats -h       # help with all options
zpwr expandstats -t 10    # top 10 aliases
zpwr expandstats -w 80    # wider box (80 cols)
zpwr expandstats -r       # reset all stats
zpwr expandstats -f FILE  # custom stats file
```

What It Tracks

```
By trigger ..... space expansions vs history expansions
By type ..... alias, global, suffix, escape, native
Corrections ..... spelling fixes applied
```

Total expansions ... how many times the engine fired Keystrokes saved ... chars you didn't have to type

```
Top aliases ..... ranked bar chart with expansions shown
```

How It Works

The zsh-expand plugin hooks into the zle accept-line widget. Before each command runs, it expands aliases, globals, and suffixes inline. Each expansion is logged to `$ZPWR_LOCAL/zpwr-expand-stats.dat` with the trigger type, expansion type, and original alias name.

Config

```
ZPWR_EXPAND_STATS_FILE    # override stats file path
ZPWR_EXPAND_STATS_TOP     # default top N (default: 20)
```

Every alias that fires is a keystroke investment paying dividends. This dashboard shows your ROI.

Chapter 8 – Environment Mastery

Know Your Shell

// you cannot master what you cannot measure //

Three verbs to audit your environment. Each one reveals a different cross-section of the machine you live inside.

Envvars: The zpwr Variables

```
zpwr envvars          # list all ZPWR_* env vars
zpwr ev              # same thing, shorter
```

Shows every environment variable starting with ZPWR_. These are the knobs and dials of the framework: paths, flags, plugin manager config, colorscheme, etc. If something is misconfigured, this is where you look.

Envvarsall: The Full Picture

```
zpwr envvarsall       # every env var, all of them
zpwr eva              # same thing, shorter
```

Dumps every exported variable. This includes ZPWR_*, PATH, HOME, EDITOR, LANG, and everything else. Pipe it to `arb -fzf` when you are hunting for a specific var:

```
zpwr eva | arb --fzf
```

Environmentcounts: The Numbers Game

```
zpwr environmentcounts # full environment statistics
zpwr envcounts         # alias
zpwr e                 # shortest form
```

This is the census. It counts everything loaded in your current shell session:

Aliases regular + global + suffix aliases functions autoloaded + defined functions builtins zsh builtin commands parameters shell variables (scalar, array, assoc) completions registered completion functions modules loaded zsh modules named dirs hash -d entries PATH entries directories in PATH FPATH entries directories in FPATH.

The numbers tell you how armed you are. A fresh zpwr install might show thousands of aliases, hundreds of functions, and many registered completions. If the numbers are low, something failed to load.

Pro Tip: Pair Them

```
zpwr doctor          # is anything broken?
zpwr e               # how much loaded?
zpwr ev              # are my zpwr vars right?
```


Run all three after installing or updating zpw. They are your post-deployment smoke test.

The Option Matrix

// zsh has 200+ options. most people know 5. time to fix that. //

The zsh option system is a maze of boolean switches that control everything from globbing to history to error handling. These verbs help you navigate the labyrinth.

Options: The Boolean Switches

```
zpw options # list all set options
```

Shows every zsh option that is currently ON. The important ones for zpw: extendedglob (#, ~, ^ in globs) – zpw needs this autopushd (cd pushes to dirstack) histignorealldups (no duplicate history) sharehistory (history shared across sessions) completeinword (tab complete mid-word) promptsubst (parameter expansion in prompts)

If extendedglob is off, half of zpw's glob tricks die. If sharehistory is off, your sessions are islands.

Modules: The Loaded Extensions

```
zpw modules # list loaded zsh modules
```

Zsh modules are C libraries that add features: zsh/datetime EPOCHREALTIME, strftime zsh/stat zstat builtin for file metadata zsh/zutil zparseopts for flag parsing zsh/complint menuselect completion listing zsh/parameter \$functions, \$aliases, \$parameters hashes zsh/terminfo terminal capability strings.

If zsh/parameter is not loaded, zpw env verbs break. If zsh/datetime is missing, timing-based verbs fail. Load a missing module: zmodload zsh/modulename.

Zstyle: The Fine-tuning Knobs

```
zpw zstyle # list all zstyle settings
```

Zstyle is zsh's configuration registry. It controls: :completion:* how tab completion behaves :vcs_info:* git/svn info in prompts :zpw:* zpw-specific settings.

Example styles zpw sets: zstyle ':completion:*' menu select zstyle ':completion:*' matcher-list 'm:{a-z}={A-Z}' The first enables arrow-key menu selection in tab complete. The second makes completion case-insensitive.

Detective Work

Completions acting weird? Check:

```
zpw options # is completeinword on?
zpw modules # is zsh/complint loaded?
zpw zstyle # what completion styles are set?
```

The answer is usually hiding in one of these three.

Cache Search

// the environment is a haystack. these are your magnets. //

zpwr maintains an environment cache for fast lookup. These verbs let you search, inspect, and edit that cache without grepping through raw variable dumps.

Environmentcachesearch: Arb Env Navigation

```
zpwr environmentcachesearch      # arb --fzf through cached env
```

Opens an arb `--fzf` interface over the environment cache. Type a few characters to filter. The cache includes: - All exported environment variables - `ZPWR_*` configuration variables - `PATH` and `FPATH` entries - Named directory mappings.

This is faster than `env | grep` because it uses the pre-built cache file rather than querying the live env.

Envaccept: Accept A Cache Entry

```
zpwr envaccept                  # accept/apply cached env entry
```

After finding a variable in the cache, this verb applies it to your current session. Useful when the cache has a value you want but the live env does not.

Envedit: Edit The Environment

```
zpwr envedit                    # open env cache in $EDITOR
```

Opens the environment cache file in your editor. Make changes, save, and the cache is updated. This is the surgical approach: when you know exactly which variable needs tweaking.

The Cache Lifecycle

1. `zpwr regenenvcache` builds the cache from live env
2. `environmentcachesearch` fuzzy-searches the cache
3. `envaccept` applies an entry
4. `envedit` manual edits

The cache lives at `$ZPWR_LOCAL`. It is regenerated during shell startup and when you run `zpwr regenenvcache`.

Workflow Example

“What is `ZPWR_EDITOR` set to?”

```
zpwr environmentcachesearch      # type 'EDITOR' in arb --fzf
```

Found it: `ZPWR_EDITOR=nvim`.

“I want to change a bunch of `ZPWR_vars`.”

```
zpwr envedit                      # opens in $EDITOR
```

Edit the vars, save, done.

“Something is off, rebuild the cache.”

```
zpwr regenenvcache                # fresh cache from live env
```

Pro Tip

If you are debugging a missing env var, check two places: `zpwrc environmentcachesearch` (cached) and `zpwrc eva` (live). If cache has it but live does not, run `zpwrc regenenvcache`.

Snapshots & Time Travel

// save your universe. restore it when everything goes wrong. //

The snapshot verb captures your entire terminal environment to a portable directory. The restore verb brings it back. This is git for your shell state.

Taking A Snapshot

```
zpwrc snapshot                # timestamp-named snapshot
zpwrc snapshot myproject      # named snapshot
zpwrc snapshot before-refactor # meaningful name
```

What Gets Captured

Hash dirs named directory mappings (hash -d) aliases regular + global + suffix aliases env vars all exported variables (secrets excluded) `zpwrc` vars `ZPWR_VARS` associative array functions user function count tracking history full shell history dump tmux sessions, windows, panes, resurrect state vim sessions saved vim session files dir stack current directory and pushd stack git status branch, commit, dirty state of cwd.

Storage: `$ZPWR_LOCAL/snapshots/<name>/` Each snapshot is a directory of plain zsh files. Human-readable. Sourceable. Portable.

Restoring A Snapshot

```
zpwrc restore                # list available snapshots
zpwrc restore myproject      # restore everything
zpwrc restore myproject aliases env # restore only aliases+env
zpwrc restore myproject dirs  # just cd to saved pwd
```

Selective Restore Components

All everything (default when no component given) hash named directories only aliases regular, global, suffix aliases env environment variables tmux stage tmux-resurrect file (press prefix+Ctrl-r) vim vim session files to `~/.vim/sessions` history merge snapshot history into current session dirs cd to the saved working directory.

When To Use Snapshots

Before a big update: `zpwrc snapshot before-update` Before experiments: `zpwrc snapshot stable-state` Switching projects: `zpwrc snapshot projectA` New machine setup: copy snapshot dir, restore on new host.

NOTE: Sensitive vars (`PASSWORD`, `SECRET`, `TOKEN`, `AWS_SECRET*`, `GITHUB_TOKEN`) are automatically excluded from env snapshots. Your secrets stay secret.

History merge is additive – it does not overwrite your current history. It uses `fc -R` to load entries.

Environment & Search

// 22 verbs to inspect and search the shell //

List env vars, search keybindings, inspect zsh internals, and navigate the cached environment.

Environment Variables

```
zpw! ev                # list all zpw! env vars
zpw! envvars           # list all zpw! env vars
zpw! environmentvars   # list all zpw! env vars
zpw! environmentvariables # list all zpw! env vars
zpw! envvarsall        # list all env vars
zpw! environmentvarsall # list all env vars
zpw! eva               # list all env vars
```

Environment Counts & Cache

```
zpw! e                # count all zpw! env
zpw! envcounts        # count all zpw! env
zpw! environmentcounts # count all zpw! env
zpw! envcachesearch   # arb --fzf cached aliases, params, builtins
zpw! environmentcachesearch# arb --fzf search cached env
zpw! envaccept        # accept from arb --fzf env
zpw! envedit          # edit from arb --fzf env
```

Keybinding Search

```
zpw! allsearch        # search all keybindings
zpw! vimsearch        # search vim keybindings
zpw! zshsearch        # search zsh keybindings
```

zsh Internals

```
zpw! reload           # reparsing env
zpw! modules          # show zsh modules via arb --fzf
zpw! options          # show zsh options via arb --fzf
zpw! zstyle           # fuzzy search zstyle
zpw! zcompdump        # edit zcompdump
```

Chapter 9 – The Janitor’s Closet

Cleanup Operations

// entropy is the enemy. the janitor is your friend. //

Shell environments accumulate garbage like a hoarder’s attic. Cache files grow stale. Temp dirs fill up. Logs balloon. These verbs are the Marie Kondo of your `$ZPWR`.

Cleanall: The Thermonuclear Broom

```
zpwr cleanall                # clean everything at once
```

Calls all the individual clean verbs in sequence. This is the “I do not know what is wrong but I want a fresh start” button. Safe but thorough.

Individual Clean Verbs

CLEANCACHE.

```
zpwr cleancache              # clear general caches
```

Wipes cached data that zpwr generates during operation. Safe to run any time. Everything rebuilds on demand.

CLEANCOMPCACHE.

```
zpwr cleancompcache          # clear completion cache
```

Removes the zsh completion cache directory. Run this when tab completions are showing stale results or when a new command does not appear in completions. Follow up with `zpwr regenzzsh` to rebuild `zcompdump`.

CLEANDIRS.

```
zpwr cleandirs                # clear cached dir listings
```

Removes directory cache files used by navigation commands. If directory completions point to dirs that no longer exist, this verb fixes that.

CLEANTEMP.

```
zpwr cleantemp                # clear temp files
```

Removes temporary files zpwr created during operation. These are transient by nature – safe to nuke.

CLEANDIRSandTEMP.

```
zpwr cleandirsandtemp         # dirs + temp combined
```

Convenience combo of `cleandirs` and `cleantemp`.

CLEANLOG.

```
zpwr cleanlog                 # truncate log files
```

The zpwr logfile grows forever if you never clean it. This verb truncates or removes old log entries. Check log size: `ls -lh $ZPWR_LOGFILE`

The Clean Workflow

Light cleanup (weekly):

```
zpwr cleantemp && zpwr cleanlog
```

Medium cleanup (monthly):

```
zpwr cleancache && zpwr cleancompcache && zpwr regenzsh
```

Full cleanup (when things feel sluggish):

```
zpwr cleanall
```

None of these delete your configuration or history. They only remove regenerable cache and temp data.

Git Cache Hygiene

// zpwr tracks your git repos. the cache needs a bath sometimes. //

zpwr maintains caches of git repository locations so that repo-aware verbs (navigation, status, batch operations) do not have to scan the filesystem every time. Three verbs manage these caches.

Cleangitcache: The General Repo Cache

```
zpwr cleangitcache # clear all git repo caches
```

Wipes the master list of known git repositories. This cache is built by scanning for .git directories and is used by verbs like zpwr fordir and repo navigation commands.

When to use it: - You moved or deleted git repos - Navigation is suggesting repos that no longer exist - You cloned new repos and they are not showing up.

After clearing, run zpwr regengitrepocache to rebuild.

Cleangitcleancache: Clean Repos Only

```
zpwr cleangitcleancache # clear clean repo cache
```

zpwr separately tracks which repos have a clean working tree (no uncommitted changes). This cache speeds up operations that only care about clean repos.

Clear this when: - The “clean repos” list seems wrong - You committed in a repo but it still shows as dirty.

Cleangitdirtycache: Dirty Repos Only

```
zpwr cleangitdirtycache # clear dirty repo cache
```

The mirror of the clean cache. Tracks repos with uncommitted changes. Clear this when: - A repo shows as dirty but you already committed - The dirty count in the banner seems wrong.

Rebuild with: zpwr regengitdirtyrepocache.

The Git Cache Architecture

Regengitrepocache scans for all git repos regengitdirtyrepocache classifies clean vs dirty cleangitcache wipes the master list cleangitcleancache wipes the clean-only list cleangitdirtycache wipes the dirty-only list.

The caches live in `$ZPWR_LOCAL` as plain text files. One repo path per line. Grep-friendly. Human-readable.

The Full Refresh

```
zpwrm cleangitcache
zpwrm regengitrepopcache
zpwrm regengitdirtyrepopcache
```

Three commands. Fresh git cache. Takes a few seconds depending on how many repos you have scattered around.

Path Deduplication

// every duplicate entry is a tiny crime against performance //

PATH and FPATH grow duplicates like weeds. Every subshell, every sourced script, every tool that helpfully “adds itself to your PATH” can insert duplicates. The `deduppaths` verb removes them.

The Verb

```
zpwrm deduppaths # deduplicate PATH and FPATH
```

Removes duplicate entries from both PATH and FPATH while preserving the order of first occurrence. If `/usr/bin` appears 3 times, only the first one survives.

Why Duplicates Happen

1. Shell inheritance

Parent shell sets PATH. Child shell re-sources `.zshrc`. PATH grows. Nested tmux sessions make it worse.

2. Multiple init scripts

Homebrew, nvm, pyenv, rbenv, cargo – each one adds to PATH in their init hook. Source them twice, PATH grows.

3. /etc/zprofile on macOS

MacOS ships with `/etc/zprofile` that runs `path_helper`. `path_helper` reads `/etc/paths` and `/etc/paths.d/` and prepends entries. This runs on every login shell, BEFORE your `.zshrc`. Duplicates guaranteed.

4. Manual PATH manipulation

Export `PATH="/my/dir:$PATH"` in `.zshrc`. Every time `.zshrc` is sourced, another copy appears.

How To Prevent Duplicates

Zsh has a built-in mechanism: the `typeset -U` flag.

```
typeset -U path fpath
```

This marks the arrays as “unique” – zsh automatically removes duplicates when entries are added. `zpwrm` sets this early in startup. But if something adds to PATH before `zpwrm` loads, duplicates can sneak in.

Diagnosing The Problem

```
zpwr pathaudit          # full audit: dupes, missing, writable
zpwr doctor             # checks for duplicate PATH/FPATH
print -l $path | sort | uniq -d # show only duplicated entries
echo $#path             # count PATH entries
```

A healthy PATH has 10-30 entries. If you see 50+, something is adding entries in a loop.

The Fix

```
zpwr deduppaths          # instant dedup
```

Run it. Then check: `echo $#path` If the count dropped, you had duplicates. If it drops a lot, investigate the source.

For a permanent fix, find the script that adds duplicates and guard it with a check:

```
[[ -z ${path[(r)/my/dir]} ]] && path=(/my/dir $path)
```

The Nuclear Options

// when gentle cleanup is not enough. when you need to nuke from orbit. //

Most clean verbs are surgical. These two are orbital strikes. They clean, regenerate, rebuild, and update in one shot. Use them when you want a fresh start without reinstalling.

Cleanrefreshupdate: The Big Red Button

```
zpwr cleanrefreshupdate  # clean + regen + update
```

This is the full-cycle maintenance verb. It:

1. Cleans all caches (equivalent to `cleanall`)
2. Regenerates compiled files, completions, env cache
3. Runs the update cycle for `zpwr` and plugins

Think of it as “factory reset without losing data.” Your configuration, history, and local files survive. Everything else gets rebuilt from source.

When to use it: - Shell startup feels slow and you have tried individual fixes - After a major zsh or macOS upgrade - When individual clean verbs did not solve the problem - When you want to feel powerful.

TIME COST: Several minutes. This is not instant. It downloads plugin updates and recompiles everything.

Cleanupdatedeps: The Dependency Purge

```
zpwr cleanupdatedeps     # clean deps + update them
```

Focused on external dependencies rather than caches. This verb:

1. Cleans stale dependency state
2. Updates all managed dependencies

When to use it: - Plugin versions are stuck at old commits - A dependency update broke something - You want the latest of everything.

Decision Matrix

Symptom Verb.

Stale completions `zpwr cleancompcache` Slow startup `zpwr recompile` Missing new repos in nav `zpwr cleangit-cache` General weirdness `zpwr cleanall` Everything is broken `zpwr cleanrefreshupdate` Plugins are ancient `zpwr cleanupdatedeps`.

The Escalation Path

Always start small and escalate:

```
zpwr cleantemp           # level 1: temp files
zpwr cleancache          # level 2: all caches
zpwr cleanall            # level 3: everything
zpwr cleanrefreshupdate  # level 4: clean + rebuild + update
```

Do not start with level 4. That is like calling an airstrike because you saw a spider.

Cleanup Arsenal

// 14 verbs to sweep the floor //

Clean directories, caches, temp files, logs, and git state. From surgical precision to nuclear wipe.

Directory Cleaning

```
zpwr cleandirs           # clear all $ZPWR_DIRS_CLEAN
zpwr cleandirsandtemp    # clear dirs and temp
zpwr cleanall            # clear dirs, temp and cache
zpwr cleantemp           # clean all zpwr temp files
```

Cache Cleaning

```
zpwr cleancache          # clean all zpwr cache files
zpwr cleancompcache      # clean all zpwr compsys cache files
zpwr cleangitcache       # clean all git repo cache files
zpwr cleangitdirtycache  # clean git repo dirty cache files
zpwr cleangitcleancache  # clean git repo clean cache files
```

Log & Update Cleaning

```
zpwr cleanlog            # clear $ZPWR_LOGFILE
zpwr cleanrefreshupdate  # clean, refresh zwc, update deps
zpwr cleanupdatedeps     # clean and update deps
```


Chapter 10 – Build System

Compilation 101

// .zwc files: compiled zsh. faster loading. dark magic. //

Zsh can compile scripts to an internal bytecode format stored in .zwc (Zsh Word Code) files. Loading compiled files is faster than parsing source text every time.

What Is A .ZWC File?

A .zwc file is the compiled form of a .zsh script or an autoload function directory. When zsh finds foo.zwc next to foo.zsh, it loads the compiled version – skipping the parsing step entirely.

For individual files: `zcompile foo.zsh -> foo.zsh.zwc` For directories: `zcompile dir.zwc dir/* -> dir.zwc`.

The speed difference is measurable on large files and during startup when many files are sourced.

Recompile: Build All .ZWC Files

```
zpwr recompile      # compile all zpwr files
zpwr compile        # same thing
```

Scans all zpwr source files and compiles them to .zwc. This includes: - Autoload function files in `$ZPWR_AUTOLOAD` - Shell scripts in `$ZPWR_ENV` - Any .zsh file that has changed since last compile.

Related Compile Verbs

```
zpwr recompilefiles # compile specific file set
zpwr compilefiles   # alias for above
zpwr recompilefpath # compile FPATH directories
zpwr compilefpath    # alias for above
```

Decompile: Remove .ZWC Files

```
zpwr decompile      # remove all .zwc files
zpwr uncompile       # alias for above
```

Deletes all .zwc files in zpwr. After this, zsh falls back to parsing source files directly. Useful when: - Debugging a function that behaves differently than source - You suspect a stale .zwc file is loading old code - zpwr doctor reports stale .zwc files.

Refreshzwc: Smart Rebuild

```
zpwr refreshzwc      # refresh only stale .zwc files
```

Only recompiles .zwc files that are older than their source. Faster than a full recompile when only a few files changed.

The Compilation Workflow

After editing zpwr source files:

```
zpwr recompile          # rebuild .zwc files
```

If something is weird after an update:

```
zpwr decompile && zpwr recompile # clean rebuild
```

Quick check for stale files:

```
zpwr doctor            # reports stale .zwc files
```

The .zwc files are safe to delete. They are purely a performance cache. Source files are always the authority.

Compsys Cache

// tab completion is a religion. the zcompdump is its scripture. //

The zsh completion system (compsys) is the most complex part of zsh. It maintains a dump file (zcompdump) that caches all registered completions. When it breaks, tab completion goes haywire. Here is how to fix it.

What Is Zcompdump?

A file (usually `~/.zcompdump` or in `$ZPWR_LOCAL`) that contains all compdef registrations. When zsh starts, it sources this file instead of re-scanning all completion functions. This saves 100-500ms of startup time.

Location: `$ZSH_COMPDUMP` The file starts with `#files:TIMESTAMP` and contains compdef calls for every registered completion.

Regenzsh: The Completion Reset

```
zpwr regenzzsh         # regenerate zcompdump
```

Deletes the old zcompdump and forces compinit to rebuild it. This is the #1 fix for completion problems:

Symptom: New command installed but tab complete ignores it Fix: `zpwr regenzzsh`.

Symptom: Tab shows wrong completions for a command Fix: `zpwr cleancompcache && zpwr regenzzsh`.

Symptom: "command not found: compdef" errors on startup Fix: `zpwr regenzzsh`.

Symptom: Startup is slow (500ms+ for completion init) Fix: `zpwr regenzzsh` (forces fresh, optimized dump)

The Completion Cache Directory

Separate from zcompdump, zsh also caches individual completion results (like package names, hostnames, etc.) in a cache directory.

```
zpwr cleancompcache    # wipe the cache directory
```

This forces completions to re-query their sources. Useful when package lists or hostnames changed.

Why Completions Break

1. Stale zcompdump – new compdef not registered
2. Corrupted zcompdump – syntax error in dump file
3. Missing FPATH entry – completion function not found
4. Wrong function loaded – .zwc has old version
5. Plugin conflict – two plugins register for same cmd

The Nuclear Fix

```
zpwr cleancompcache
zpwr decompile
zpwr regenzzsh
zpwr recompile
```

Four commands: nuke cache, remove .zwc, regen dump, recompile. This fixes 99% of completion problems. The remaining 1% is probably a bug in the completion function itself.

Tag Generation

// ctrl-j in vim. M-. in emacs. the tags file makes it work. //

Tags files are indexes of symbol definitions: functions, variables, classes, methods. Your editor uses them to jump to definitions with a single keystroke. zpwr generates them.

Ctags: The Classic

```
zpwr regenc tags # regenerate ctags file
```

Runs exuberant-ctags or universal-ctags over the zpwr codebase. Produces a tags file that vim and emacs read.

In vim: Ctrl-J jumps to the definition under cursor. In vim: Ctrl-T jumps back to where you were. In vim: :tag zpwrDoctor jumps to zpwrDoctor.

Ctags understands shell functions, C, Python, Ruby, and dozens of other languages out of the box.

Gtags: The Gnu Global System

```
zpwr regeng tags # regenerate GNU Global tags
```

GNU Global is a more powerful tagging system. It supports: - Forward references (where is this defined?) - Backward references (where is this called from?) - Symbol completion.

Produces GTAGS, GRTAGS, GPATH files in the project root. Query from command line: global -x zpwrDoctor
Query references: global -rx zpwrDoctor.

Gtags With Pygments Parser

```
zpwr regeng tagspygments # gtags with Pygments backend
```

Uses the Python Pygments parser instead of the built-in one. Pygments understands more languages and file types. Requires: pip install pygments.

Gtags By File Type

```
zpwr regengtagstype          # gtags filtered by type
```

Generate tags for a specific file type only. Faster than a full regen when you only care about one language.

How Tags Work With Your Editor

vim reads 'tags' file. :set tags=.[/tags](#);;tags Ctrl-] to jump, Ctrl-T to return :tselect for multiple matches.

emacs reads TAGS file (etags format) or uses gtags M-. to find definition, M-* to return ggtags-mode or helm-gtags for GNU Global.

nvim same as vim, plus LSP which often replaces tags but tags still work as a fast fallback.

When To Regenerate

- After adding or renaming functions - After a zpwr update that adds new verbs - When "tag not found" errors appear for known functions - Use zpwr watch '*.zsh' 'zpwr regenctags' for auto-regen

Cache Regeneration

// the opposite of cleaning. building the caches back up. //

Cleaning removes caches. Regeneration rebuilds them. These verbs construct the indexes and precomputed data that make zpwr fast.

Regen: The Standard Rebuild

```
zpwr regen                  # regenerate common caches
```

Rebuilds the most frequently used caches: environment cache, config links, keybindings. This is the verb you reach for after editing config files.

Regenall: The Full Rebuild

```
zpwr regenall              # regenerate everything
```

Every cache. Every index. Every precomputed file. This is the thorough version of regen. Takes longer but leaves nothing stale.

Individual Regen Verbs

Regenenvcache.

```
zpwr regenenvcache         # rebuild environment cache
```

Snapshots all env vars to a cache file for fast lookup. Used by environmentcachesearch and other env verbs.

Regenkeybindings.

```
zpwr regenkeybindings          # rebuild keybinding index
```

Re-registers all zpwr keybindings. Run this after editing keybinding configuration or after a plugin clobbered your bindings.

Regenconfiglinks.

```
zpwr regenconfiglinks          # rebuild config symlinks
```

Re-creates the symlinks from `~/.zshrc`, `~/.vimrc`, etc. into the zpwr directory tree. Run this after a fresh clone or when zpwr doctor reports broken config links.

Regenhistory.

```
zpwr regenhistory              # clean up history file
```

Processes the HISTFILE to remove corrupted entries. Zsh history files can accumulate broken lines from concurrent writes or improper shutdowns.

Regenpowerline.

```
zpwr regenpowerline           # rebuild powerline config
```

Regenerates powerline/p10k configuration. Run when the prompt looks wrong after a terminal font change.

Regengitrepopocache.

```
zpwr regengitrepopocache      # scan and cache git repos
```

Walks the filesystem finding git repos and builds a master list. Used by repo navigation and batch verbs.

The Regen Cheat Sheet

After editing config: `zpwr regen` After zpwr update: `zpwr regenall` After env var changes: `zpwr regenenvcache`
After repo changes: `zpwr regengitrepopocache` After completion issues: `zpwr regenbash`.

Updates & Dependencies

// staying current without breaking everything. a balancing act. //

zpwr is a living system that depends on external tools, plugins, and its own git repository. These verbs keep everything synchronized.

Update: The Core Update

```
zpwr update                    # update zpwr itself
```

Pulls the latest zpwr from the git remote. This is a git pull on `$ZPWR`. Your local config in `$ZPWR_LOCAL` is not touched.

Updateall: The Everything Update

```
zpwr updateall                # update zpwr + all plugins
```

Updates zpwr, zinit plugins, and other managed components. This is the weekly maintenance verb. Run it when you want the latest of everything.

Updatedeps: Dependency Updates

```
zpwr updatedeps # update external dependencies
```

Updates the external tools zpwr depends on: zinit plugins, zsh-autosuggestions, zsh-syntax-highlighting, etc. Does not touch zpwr's own code.

Updatedepsclean: Clean + Update Deps

```
zpwr updatedepsclean # fresh dependency install
```

Cleans stale dependency state first, then updates. Use when updatedeps alone does not fix plugin issues.

Updatezinit: zinit Only

```
zpwr updatezinit # update zinit plugin manager
```

Updates just the zinit plugin manager binary, not the plugins it manages. Use when zinit itself has a bug or you need a newer zinit feature.

Updatepull: Git Pull All Repos

```
zpwr updatepull # pull all tracked git repos
```

Iterates over all cached git repos and runs git pull. Useful when you have many repos that track remotes and you want them all current.

The Update Hierarchy

Update just zpwr (git pull) updatedeps just plugins and tools updatezinit just the plugin manager updateall zpwr + plugins + tools updatepull all tracked git repos updatedepsclean clean slate + update deps.

Recommended Update Schedule

Weekly: zpwr updateall After update: zpwr recompile If broken: zpwr updatedepsclean.

Always recompile after updating. New source files need new .zwc files. The old compiled versions will load stale code until you recompile.

Regen & Compile

// 24 verbs to rebuild the world //

Regenerate caches, recompile zsh word code, and rebuild tags. The engine room of ZPWR maintenance.

Regen Caches

```
zpwr regen          # regen caches except git drive search
zpwr regenall        # regen all caches
zpwr regenpdf        # regenerate encyclopedia PDF
zpwr regenenvcache   # regen search env to zpwrEnv files
zpwr regengitrepocache # regen list of all git repos
zpwr regengitdirtyrepocache # regen list of dirty git repos
zpwr regenhistory    # regen $HISTFILE
zpwr regenkeybindings # regen all keybindings cache
zpwr regenconfiglinks # regen sym links to $HOME
zpwr regenpowerline  # regen powerline sym link
zpwr regenzsh        # regen compsys cache
```

Regen Tags

```
zpwr regengtags      # regen gtags files to $HOME
zpwr regenctags      # regen ctags files to $HOME
zpwr regengtagspygments # regen GNU gtags with pygments
zpwr regengtagstype  # regen GNU gtags with specified parser
```

Compile & Decompile

```
zpwr compile         # recompile all cache comps
zpwr compilefpath    # recompile all fpath cache comps
zpwr compilefiles    # recompile all files cache comps
zpwr recompile       # recompile all cache comps
zpwr recompilefpath  # recompile all fpath cache comps
zpwr recompilefiles  # recompile all files cache comps
zpwr refreshzwc      # delete then regen compiled zsh word code
zpwr decompile       # delete all cache comps
zpwr uncompile       # delete all cache comps
```

Update System

// 6 verbs to stay current //

Keep ZPWR, its dependencies, and zinit plugins up to date. From surgical single-dep pulls to full system updates.

Config Updates

```
zpwr update          # update zpwr custom configs
zpwr updatepull      # pull latest config from repo
```

Dependency Updates

```
zpwr updatedeps      # update all dependencies
zpwr updatedepsclean # update all dependencies then clean
zpwr updatezinit     # update zinit dependencies
```

Full Update

```
zplr updateall          # update zplr configs and all deps
```

Typical workflow: `zplr updateall` pulls the latest configs, updates `brew/pip/npm/gem/zinit`, and refreshes caches. Run `zplr updatedepsclean` afterward to reclaim disk space from stale packages.

Chapter 11 – Monitoring & Automation

File Watching

// your files change. your command runs. automatically. //

The watch verb monitors files for changes and auto-executes a command when something is modified. It is the poor man's CI pipeline, running locally in your terminal.

Usage

```
zplr watch '<GLOB>' '<COMMAND>'
```

Two arguments: what to watch, and what to run. Quote both arguments to prevent premature expansion.

Practical Examples

```
zplr watch '*.zsh' 'zplr recompile'
```

Recompile zplr whenever a .zsh file changes. Edit a function, save, and .zwc files auto-rebuild.

```
zplr watch 'src/**/*.ts' 'npm run build'
```

Build your TypeScript project on every source change.

```
zplr watch 'Makefile' 'make'
```

Run make whenever the Makefile changes.

```
zplr watch '.' 'echo changed'
```

Watch the entire current directory.

```
zplr watch '*.py' 'python -m pytest'
```

Run tests on every Python file change. TDD mode.

Platform Backend

MacOS: Uses fswatch (brew install fswatch) Linux: Uses inotifywait (apt install inotify-tools)

The verb auto-detects which tool is available. If neither is found, it tells you what to install.

The Output Format

Each trigger shows a box with: - Trigger count (how many times it has fired) - Timestamp (when the change was detected) - Filename (which file changed) - Exit status (green ok or red exit code) - Elapsed time (how long the command took)

How To Stop

Press Ctrl-C. The watcher process terminates cleanly.

Glob Handling

The glob is split into directory and file filter: 'src/**/*.*' -> watch src/ recursively, filter *.* '.' -> watch current dir, no filter '*.zsh' -> watch ./, filter *.zsh.

The glob-to-regex conversion handles * and ? patterns. For fswatch, it uses -e '.*' -i to exclude-then-include. For inotifywait, it uses zsh's glob matching on filenames.

Command Replay

// your history is a script. replay it on demand. //

The replay verb plays back the last N commands from your shell history. Three modes: look, execute, or tmux pane. It is a VCR for your terminal.

Basic Usage

```
zpwir replay                # preview last 10 commands
zpwir replay 20              # preview last 20
zpwir replay 5 exec          # execute last 5 with timing
zpwir replay 10 tmux         # replay in a tmux split pane
```

The Three Modes

PREVIEW MODE (default, safe)

```
zpwir replay
zpwir replay 20 preview
```

Lists the commands with line numbers. Nothing executes. Use this to review what you did before replaying it. The output shows helpful hints for switching to exec/tmux.

EXEC MODE (runs in current shell)

```
zpwir replay 5 exec
```

Executes each command sequentially in your current shell. Each command shows: - Command number [1/5] - The command being run - Exit status (green ok or red exit code) - Elapsed time in milliseconds.

WARNING: This runs real commands. If your last 5 commands included rm -rf something, it will do it again. Always preview first.

TMUX MODE (runs in a new pane)

```
zpwir replay 10 tmux
```

Opens a new tmux split pane and replays commands there. Your current pane is untouched. You can watch the replay happen side-by-side with your current work. The pane waits for Enter when done, then cleans up. Requires an active tmux session.

Use Cases

Reproduce a bug: `zpwrr replay 3 exec Demo` your workflow: `zpwrr replay 20 tmux Audit` what you did: `zpwrr replay 50` Re-run a build chain: `zpwrr replay 4 exec`.

How It Works

Uses `fc -rl -N` to extract the last N history entries. Reverses them to chronological order. In exec mode, each command is eval'd with timing via `EPOCHREALTIME`. In tmux mode, a temp script is built and executed in a split pane with a 0.5s delay between commands for readability. The script self-destructs after completion.

Log Surveillance

// watching logs in glorious technicolor. the zcolorizer way. //

`zpwrr` logs everything: startup, errors, plugin loads, debug messages. The `taillog` verb tails these logs with optional `zcolorizer` colorization for maximum readability.

Basic Usage

```
zpwrr taillog          # follow log with colors
zpwrr taillog -n 50    # last 50 lines, then follow
zpwrr taillog -f        # colorize without following
zpwrr taillog -r        # raw output with line numbers
```

The Flags

`-n`, `-lines N` Show last N lines before following `-f`, `-no-follow` Just colorize, do not tail `-f -r`, `-raw` No `zcolorizer`, plain output with line numbers

Custom Logfile

```
zpwrr taillog /var/log/system.log
zpwrr taillog /var/log/syslog
```

The last argument can be any logfile path. Default: `$ZPWR_LOGFILE` (the `zpwrr` logfile).

Zcolorizer: The Secret Weapon

`Zcolorizer` is a log colorizer that understands dozens of log formats: syslog, Apache, Postfix, PHP, kernel messages, and more. It auto-detects the format and colorizes:

Timestamps in one color warnings in yellow errors in red info in cyan.

```
hostnames      in magenta
```

Install: `brew install zcolorizer` If `zcolorizer` is not found, `taillog` falls back to plain output with a helpful install hint.

The zpwrr Logging Verbs

`Taillog` consumes logs. These verbs produce them:

```
zpwr logdebug <msg>          # debug level
zpwr loginfo <msg>           # info level
zpwr logerror <msg>          # error level
zpwr logtrace <msg>          # trace level (most verbose)
```

The *console variants also print to stderr:

```
zpwr logdebugconsole <msg>
zpwr loginfoconsole <msg>
zpwr logerrorconsole <msg>
zpwr logtraceconsole <msg>
```

Log Maintenance

Logs grow forever. Clean them periodically:

```
zpwr cleanlog                # truncate the logfile
```

Or check the size first: `ls -lh $ZPWR_LOGFILE`

A 100MB logfile means your debug logging is too chatty. A 1KB logfile means nothing is being logged. Both are worth investigating.

Backup & Restore

// your history is priceless. these verbs are the insurance policy. //

Your shell history is years of accumulated knowledge: every command you ran, every fix you found at 2am, every pipeline you crafted. Losing it is losing institutional memory. These verbs prevent that.

Backup: The General Backup

```
zpwr backup                  # backup zpwr configuration
```

Creates a timestamped backup of zpwr configuration files. This captures your local settings, tokens, and any customizations in `$ZPWR_LOCAL`.

Backup destination: `$ZPWR_LOCAL/rcBackups/` Each backup is timestamped so you can have many.

Backuphistory: The History Vault

```
zpwr backuphistory          # backup HISTFILE
```

Copies your HISTFILE to the backup directory with a timestamp. This is the most important backup in zpwr.

Your HISTFILE can be corrupted by: - Power loss during write - Concurrent sessions fighting over the file - Disk full causing truncated writes - Bad .zsh_history format from import/merge.

A weekly zpwr backuphistory gives you a safety net.

Restorehistory: The Time Machine

```
zpwr restorehistory         # restore from backup
```

Restores HISTFILE from the most recent backup. Use when your history is corrupted or accidentally truncated.

The restore process:

1. Finds the latest backup in `$ZPWR_LOCAL/rcBackups`
2. Backs up the current (possibly broken) `HISTFILE`
3. Copies the backup over `HISTFILE`
4. Reloads history with `fc -R`

Snapshots Vs Backups

Snapshot captures full env state (aliases, vars, tmux) backup copies config files to safety backuphistory specifically targets `HISTFILE`.

Use snapshots for environment state. Use backups for file safety. They complement each other.

The Safety Protocol

Before a big change:

```
zpwr snapshot before-change
zpwr backuphistory
zpwr backup
```

Now you have three layers of protection: env snapshot, history backup, and config backup. If the change goes sideways, you can recover everything.

Cron It

Add to your crontab (`crontab -e`):

```
0 3 * * 0 zsh -c 'source ~/.zshrc; zpwr backuphistory'
```

Every Sunday at 3am. Automatic insurance.

Live Dashboard & Sparklines

Your shell's vital signs, in real time.

zpwr Top

A live-updating dashboard of your shell's resource usage. Runs in alternate screen buffer. Press `q` to quit.

```
zpwr top      # refresh every 2s
zpwr top 1    # fast refresh
zpwr top 5    # slow refresh
zpwr dashboard # same verb
zpwr live     # same verb
```

What It Shows

```
Memory ..... RSS bar graph, virtual memory
Shell objects ..... functions, aliases, completions, params
Delta tracking ..... changes between refreshes in yellow
Loaded modules ..... all zsh modules currently loaded
Paths ..... fpath, path, named dirs, dir stack
Context ..... git branch, tmux info, startup time
```

Top 5 functions .. largest functions with size bars.

```
Top 5 paths ..... longest PATH entries
```

Sparklines

Startup time history: last 40 shell starts Color-coded: green=fast yellow=mid red=slow Shows min/max/avg stats below the graph.

Commit velocity: commits per day over 30 days Visualizes your development pace as a mini bar chart.

Startup times are logged automatically on each shell init to [\\$ZPWR_LOCAL/startup_history.log](#).

Chapter 12 – Utility Arsenal

File & Process Operations

// the everyday tools. sharpened and color-coded. //

zpwr wraps common file and process operations with enhanced output, color, and arb –fzf integration. These are the verbs you reach for twenty times a day.

File Listing

```
zpwr clearlist      # clear screen + list files
zpwr clearls        # alias for clearlist
zpwr info           # alias for clearlist
zpwr list           # list files (no clear)
zpwr ls             # alias for list
```

Clearlist is the most-used verb after help. It clears the terminal and shows a fresh directory listing. Uses eza if available, falls back to ls.

File Viewing

```
zpwr cat <file>      # colorized cat
zpwr c <file>         # shortest form
zpwr catcd <dir>      # cat files in a directory
```

Cat/c uses bat or pygmentize for syntax highlighting when available. Plain cat as fallback. catcd lists the contents of each file in a directory.

Process Management

```
zpwr ps             # enhanced process listing
zpwr kill           # arb-powered process kill
zpwr killedit       # kill then edit related file
```

zpwr kill is the crown jewel here. It pipes process listings through arb –fzf, letting you fuzzy-search for the process you want to terminate. Select it, confirm, dead. No more guessing PIDs.

File Descriptors

```
zpwr lsof           # list open files (enhanced)
zpwr lsosfedit      # lsof then edit result
```

zpwr lsosf wraps lsof with arb –fzf filtering. Useful when tracking down which process holds a file or which files a process has open.

File Opening

```
zpwr open                # open file/dir with system
```

Opens the target with the system handler: Finder on macOS, xdg-open on Linux. Works with files, directories, URLs.

The Combo Moves

Clear terminal, list files, check what is running:

```
zpwr clearlist && zpwr ps
```

Find and kill a zombie process:

```
zpwr kill      # arb --fzf search -> select -> terminate
```

Find what is holding a port:

```
zpwr lsof      # arb --fzf through open files and ports
```

Batch Execution

// one command, many targets. the industrial strength loop. //

When you need to run the same operation across multiple files, directories, or git repos, these verbs replace hand-rolled for loops with tested, colorized execution.

For: The Basic Iterator

```
zpwr for <command>      # run command in current dir context
```

Executes a command with zpwr's enhanced output formatting. Shows timing, exit status, and colorized output.

For10: The Stress Test

```
zpwr for10 <command>    # run command 10 times
```

Executes a command 10 times. Useful for: - Benchmarking (is the timing consistent?) - Flaky test detection (does it fail intermittently?) - Load testing (can it handle rapid repeated execution?)

Fordir: Iterate Over Git Repos

```
zpwr fordir <command>    # run command in each git repo
```

Iterates over all cached git repos, cd's into each one, and runs the command. Extremely powerful for bulk operations:

```
zpwr fordir 'git status -s'  # dirty files in all repos
zpwr fordir 'git pull'       # pull all repos
zpwr fordir 'git stash list'  # find forgotten stashes
```

Each repo's output is labeled with its path so you know which repo produced which output.

Fordirupdate: Pull All Repos

```
zpwr fordirupdate          # git pull in every repo
```

Specialized version of fordir that runs git pull in every cached repo. Same as zpwr fordir 'git pull' but optimized for the common case.

Execglob: Glob-based Execution

```
zpwr execglob <glob> <command>    # run command on glob matches
```

Finds files matching a glob and runs a command on each:

```
zpwr execglob '*.log' 'wc -l'      # line count each log file
zpwr execglob '*.bak' 'rm'        # delete all .bak files
```

Execglobparallel: Parallel Glob Execution

```
zpwr execglobparallel <glob> <cmd> # parallel execution
```

Same as execglob but runs commands in parallel. Useful when processing many files that are I/O-bound:

```
zpwr execglobparallel '*.png' 'optipng' # compress images
```

The Power Combos

Find all repos with uncommitted work:

```
zpwr fordir 'git diff --stat HEAD'
```

Count lines in all Python files:

```
zpwr execglob '**/*.py' 'wc -l'
```

Stress test a flaky command:

```
zpwr for10 'curl -s localhost:3000/health'
```

Text Operations

// find and replace. rename. count. transform. //

Text manipulation is the bread and butter of shell work. These verbs handle the common cases: renaming files, replacing strings, counting lines, and transforming text.

Rename: File Renaming

```
zpwr rename                # rename files with patterns
```

Renames files using pattern matching. Safer than raw mv because it shows what will happen before doing it.

Replacer: Find And Replace In Files

```
zpwr replacer                # search and replace in files
```

Performs find-and-replace across multiple files. The codebase refactoring tool. When you renamed a function and need to update every call site.

Changename: Rename With History

```
zpwr changename              # rename with tracking
```

Renames files or directories with additional tracking. More sophisticated than basic rename for complex renaming operations.

Changenamezpwr: Zpwr-specific Rename

```
zpwr changenamezpwr          # rename within zpwr codebase
```

Specialized rename for zpwr function and file names. Handles the autoload naming convention (file name must match function name) and updates references.

Linecount: Count Lines

```
zpwr linecount                # count lines in files
```

Enhanced line counting with formatting and totals. Shows per-file counts and a grand total. More informative than raw `wc -l`.

Pre & Post: Text Injection

```
zpwr pre                      # prepend text to input
zpwr post                     # append text to input
```

Pipe filters that add text before or after each line. Useful in pipelines for formatting:

```
ls | zpwr pre ' - '           # bullet-point file list
cat urls.txt | zpwr post '/api/v1' # append to each URL
```

Urlsafe: Url Encoding

```
zpwr urlsafe                  # URL-encode a string
```

Converts special characters to percent-encoded form. Spaces become `%20`, ampersands become `%26`, etc. Essential when constructing URLs from user input.

The Text Pipeline

zpwr text verbs compose naturally in pipelines:

```
cat data.txt | zpwr pre 'ROW: ' | zpwr post ' ;'
```

Result: ROW: line1 ; ROW: line2 ;

Rename all .txt to .md:

```
zpwr rename # interactive rename
```

Replace 'oldFunc' with 'newFunc' across the codebase:

```
zpwr replacer # interactive replace
```

Display & Banners

// because aesthetics matter. even in a terminal. //

zpwr ships with a fleet of display verbs that produce ASCII art, colorized output, and terminal animations. Some are informational. Some are just cool to look at.

The Banner Family

```
zpwr about # the welcome banner
zpwr banner # same as about
zpwr bannercounts # banner + environment stats
```

The core banner shows the zpwr ASCII logo with version, author, and system info. bannercounts adds the environment census (alias count, function count, etc.).

```
zpwr bannerlolcat # banner piped through lolcat
```

Rainbow gradient banner. Requires: `gem install lolcat` For when monochrome is not enough and you need to assert terminal dominance with a rainbow.

```
zpwr bannerpony # banner with ponysay
```

Displays the banner inside an ASCII pony speech bubble. Requires: `pip install ponysay` Peak terminal culture.

```
zpwr bannernopony # banner without the pony
```

For the pony-averse. All the info, none of the horse.

Pretty Printing

```
zpwr prettyprint # formatted text output
zpwr altprettyprint # alternate format
zpwr printmap # print associative array
```

Prettyprint is used internally by many zpwr verbs to format output with consistent color and spacing. printmap formats associative arrays as aligned key-value tables. Useful for debugging hash tables.

Color Testing

```
zpwr colortest # test basic terminal colors
zpwr colortest256 # full 256-color palette
```

Run these after changing terminals, fonts, or color schemes. colortest shows the 16 ANSI colors. colortest256 shows the full 256-color grid. If colors look wrong, your TERM or terminal emulator config needs adjustment.

Animation

```
zpwr animate # terminal animation
```

Plays an ASCII animation in the terminal. Because sometimes you need to entertain yourself while waiting for a build.

The Display Arsenal At A Glance

Impress a colleague: `zpwr bannerlolcat` Debug your terminal: `zpwr colortest256` Count your weapons: `zpwr bannercounts` Maximum absurdity: `zpwr bannerpony` Format a hash table: `zpwr printmap` Kill time aesthetically: `zpwr animate`.

File & Text Utilities

// 31 verbs for file wrangling //

Cat, list, grep, search, rename, and transform files. The everyday toolkit for text manipulation.

View & List

```
zpwr cat          # zpwr cat args
zpwr c            # zpwr cat args
zpwr ls           # list the files with no args
zpwr list         # list the files with no args
zpwr clearlist    # clear and list the files
zpwr clearls      # clear and list the files
zpwr info         # get info on command type with args
```

Search

```
zpwr grep         # grep through pwd with ag into arb --fzf
zpwr search       # recursive grep search with context
zpwr filesearch   # cat a file in a sub dir via arb --fzf
zpwr filesearchedit # edit a file in a sub dir via arb --fzf
zpwr wordsearch   # cat a file by word via arb --fzf
zpwr wordsearchedit # edit a file by word via arb --fzf
zpwr findsearch   # cat find drive for file via arb --fzf
zpwr findsearchedit # edit find drive for file via arb --fzf
zpwr locatesearch # accept locate drive for file
zpwr locatesearchedit # edit locate drive for file
zpwr linecount    # get line count of search term
zpwr cfasd        # cat the fasd frequency ranked file
```

Transform & Manage

```
zpwr replacer     # find and replace text in files
zpwr rename       # rename files
zpwr man          # arb --fzf through man pages
zpwr open         # open with system
zpwr opencommand  # get the command to open with system
```

Convert & Upload

```
zplr scripttopdf      # convert script to PDF
zplr creat/gif        # create GIF from file with ffmpeg
zplr upload           # upload with curl
zplr urlsafe          # base64 encode
zplr execglob         # exec on globbed files
zplr execglobparallel # parallel exec on globbed files
```

Miscellaneous Arsenal (1/2)

// the verbs that defy categorization //

Terminal & Color

```
zplr animate          # run terminal animation
zplr alacrity          # edit alacrity config
zplr color2           # turn on stderr filter
zplr return2          # turn off stderr filter
zplr colorsdiff        # colorized side diff
zplr colortest         # test colors
zplr colortest256      # test 256 colors
zplr figletfonts       # show all figlet fonts
zplr pygmentcolors     # show all pygment colors
```

Backup & History

```
zplr backup           # backup files
zplr backuphistory    # backup $HISTFILE
zplr hist             # exec history command via arb --fzf
zplr histedit         # edit history command via arb --fzf
zplr histfile         # edit history file
```

File & Dir Creation

```
zplr mkdir            # make a dir tree
zplr makefile         # make a dir tree with file at end
zplr scriptnew        # create new script without editor
zplr changename       # mv dir into namespace and sed sub
zplr changenamezplr   # mv into zplr namespace and sed sub
```

Docker & Go

```
zplr dfimage          # cat Dockerfile from IMAGE_ID
zplr dockerwipe       # wipe all docker images
zplr goclean          # clean Go package artifacts
```

Exercism

```
zpwr exercism          # go to Exercism home page
zpwr exercismedit      # edit file in $HOME/Exercism
zpwr exercismdownload  # download all for track
```

Miscellaneous Arsenal (2/2)

// looping, testing, system, and services //

Looping & Background

```
zpwr background      # run arg in background
zpwr for              # run first arg times for command
zpwr for10            # run 10 times for command
zpwr fordir           # run first arg in following dirs
zpwr fordirupdate     # run git updaters in following dirs
zpwr whiletrue        # run command forever
zpwr whilesleep       # run command and sleep N seconds
```

Testing

```
zpwr test             # run all zpwr tests
zpwr tests            # run all zpwr tests
zpwr testall          # run all env tests
zpwr testsall         # run all env tests
```

System & Misc

```
zpwr curl             # custom curl with cleaned output
zpwr deduppaths        # deduplicate PATH fpath manpath
zpwr edit              # exec $EDITOR
zpwr editor            # exec $EDITOR
zpwr install           # run configure, make, make install
zpwr loadjenv          # lazy load jenv
zpwr prettyprint       # pretty print with color
zpwr altprettyprint    # pretty with alternating color
zpwr reset             # reset repo to latest main
zpwr reveal            # show remote repo in browser
zpwr revealrecurse     # recursively reveal hidden dirs
zpwr rmconfiglinks     # remove sym links to $HOME
zpwr startauto         # start autocomplete
zpwr stopauto          # stop autocomplete
zpwr timer             # timer one or more commands
zpwr uninstall         # uninstall all zpwr configs
zpwr post              # postfix all output
zpwr pre               # prefix all output
zpwr zpwrCloneToForked # clone repo to $ZPWR_FORKED_DIR
```


Systemd Services

```
zpwir serviceup          # start and enable systemd service
zpwir servicedown        # stop and disable systemd service
zpwir restart             # restart zpwir systemd service
```

Chapter 13 – emacs Dimension

Emacs Integration Overview

Entering The emacs Dimension

Where M-x is a way of life and pinky fingers go to die.

You thought there were only vim verbs? Think again, operative. ZPWR mirrors every vim operation with an emacs equivalent. Same power, different religion.

The 28 emacs Verbs:

FILE OPS emacsall, emacsalledit, emacsautoload RECENT emacsrecent, emacsrecentcd, emacsrecentsudo SEARCH emacsfilesearch, emacsfilesearchedit emacsfindsearch, emacsfindsearchedit emacslocatesearch, emacslocatesearchedit emacswordsearch, emacswordsearchedit TAGS emacsctags, emacshtags, emacshtagsedit CONFIG emacsconfig, emacsemacsconfig, emacsokens SERVER emacsallserver SCRIPTS emacsscripts, emacsscriptedit META emacsplugincount, emacspluginlist, emacszipwr.

THE MIRROR PRINCIPLE:

Every vim verb has an emacs twin. The architecture is symmetrical by design. You can swap editors without losing any ZPWR functionality.

vim verb emacs twin zipwr vimall → zipwr emacsall zipwr vimrecent → zipwr emacsrecent zipwr vimscripts → zipwr emacsscripts.

Philosophy:

ZPWR does not care about your editor war allegiance. It arms both sides equally. The real enemy is notepad.

// the emacs dimension runs parallel to vim-space

// choose your weapon, the framework adapts

// \$ZPWR_EDITOR controls which universe you inhabit

Emacs File Operations

emacs File Ops: Opening Things Since 1976

emacs: an operating system lacking only a decent editor.

zipwr Emacsall

Opens all ZPWR framework files in emacs via arb -fzf. Every autoload, script, config – the whole kingdom.

```
zipwr emacsall          # browse all files
zipwr emacsalledit      # same but editable
```

zpwr Emacsrecent

Opens recently edited files. Emacs remembers where you have been, like a browser history for code.

```
zpwr emacsrecent      # arb --fzf recent files
zpwr emacsrecentcd    # cd to recent dir
zpwr emacsrecentsudo  # recent with sudo
zpwr emacsrecentsudocd # sudo + cd combo
```

zpwr Emacsscripts

Browse and open ZPWR scripts directory. Where the shell scripts live and breathe.

```
zpwr emacsscripts    # browse scripts
zpwr emacsscriptedit  # edit a script
```

zpwr Emacsautoload

Open autoloaded zsh functions in emacs. The beating heart of ZPWR's function library.

```
zpwr emacsautoload    # browse autoloads
```

// protip: emacsrecent is your daily driver

// it learns your habits and surfaces what matters

// sudo variants for when /etc needs surgery

Emacs Search & Tags

Search & Tags: Find Anything, Anywhere

grep is for amateurs, we have arb-powered search beams.

File Search (by Filename):

```
zpwr emacsfilesearch  # arb --fzf filename search
zpwr emacsfilesearchedit # search + edit
```

Searches file NAMES across the ZPWR tree. Fast, fuzzy, and feeds results into emacs.

Word Search (by Content):

```
zpwr emacswordsearch  # search file content
zpwr emacswordsearchedit # content search + edit
```

Searches INSIDE files for patterns. Like ag/rg but with arb -fzf preview and direct-to-emacs piping.

Find & Locate Search:

```
zpwr emacsfindsearch  # find-based search
zpwr emacsfindsearchedit # find + edit
zpwr emacslocatesearch # locate db search
zpwr emacslocatesearchedit # locate + edit
```

Tags – Code Navigation:

```
zpwr emacsctags          # ctags generation
zpwr emacshtags          # GNU Global tags
zpwr emacshtagsedit      # gtags + jump to def
```

Tags let emacs jump to function definitions instantly. ctags for basic tagging, gtags for cross-reference.

// the search hierarchy:

// wordsearch = grep inside files (content)

// filesearch = find by filename (path)

// locatesearch = system locate db (everywhere)

// findsearch = find command (directory tree)

// tags = jump to definitions (symbols)

Emacs Config & Server

Config & Server: The emacs Mothership

Because starting emacs fresh every time is a war crime.

zpwr Emacsconfig

Edit ZPWR's emacs configuration files. The bridge between the shell framework and emacs-land.

```
zpwr emacsconfig          # edit zpwr emacs cfg
```

zpwr Emacsemacsconfig

Edit your personal emacs init files (init.el, etc). Deeper than emacsconfig – this is YOUR emacs setup.

```
zpwr emacsemacsconfig     # edit ~/.emacs.d
```

zpwr Emacsallserver

Opens files using emacsclient, connecting to a running emacs daemon. Instant file opens, no startup penalty.

```
zpwr emacsallserver       # open via daemon
```

HOW THE SERVER WORKFLOW WORKS: 1. Start emacs daemon: emacs –daemon 2. Use emacsallserver to open files via emacsclient 3. Files open instantly in the running emacs instance 4. No more 3-second startup tax per file.

zpwr Emacstokens

Edit your tokens/secrets file in emacs. API keys, env vars, the classified stuff.

```
zpwr emacstokens          # edit $ZPWR_LOCAL/.tokens.sh
```

zpwr Emacszpwr

Open the entire ZPWR directory tree in emacs.

```
zpwr emacszpwr          # the whole framework
```

// emacsclient + daemon = the cheat code

// startup once, open files forever

// this is how emacs users actually survive

Emacs Integration

// 29 verbs for the church of emacs //

Every file operation in ZPWR has an emacs variant. Open, edit, search, and navigate — all wired to emacsclient.

Open & Edit

```
zpwr emacsall           # emacs all zpwr files for :argdo
zpwr emacsallserver     # emacs all zpwr files via server
zpwr emacsalledit       # emacs edit 1 or more configs
zpwr emacsautoload      # emacs all autoloads :argdo
zpwr emacsscripts       # emacs all zpwr scripts for :argdo
zpwr emacsscriptedit    # emacs edit 1 or more scripts
zpwr emacszpwr          # emacs zpwr dir
```

Config & Tokens

```
zpwr emacsconfig        # emacs all zpwr configs
zpwr emacsconfig        # emacs edit emacs zpwr configs
zpwr emacsconfig        # emacs edit emacs zpwr configs
zpwr emacsconfig        # emacs edit emacs zpwr configs
zpwr emacsconfig        # emacs edit emacs zpwr configs
zpwr emacsconfig        # emacs edit emacs zpwr configs
```

Navigation

```
zpwr emacsconfig        # emacs edit emacs zpwr configs
zpwr emacsconfig        # emacs edit emacs zpwr configs
zpwr emacsconfig        # emacs edit emacs zpwr configs
zpwr emacsconfig        # emacs edit emacs zpwr configs
zpwr emacsconfig        # emacs edit emacs zpwr configs
zpwr emacsconfig        # emacs edit emacs zpwr configs
```

Search

```
zpwr emacsconfig        # emacs edit emacs zpwr configs
zpwr emacsconfig        # emacs edit emacs zpwr configs
zpwr emacsconfig        # emacs edit emacs zpwr configs
zpwr emacsconfig        # emacs edit emacs zpwr configs
zpwr emacsconfig        # emacs edit emacs zpwr configs
zpwr emacsconfig        # emacs edit emacs zpwr configs
```

Tags & Plugins

```
zpwr emacsctags      # emacs zpwr ctags tags
zpwr emacsgtags      # emacs zpwr GNU global tags
zpwr emacsgtagsedit  # emacs edit zpwr GNU global tags
zpwr emacsplugincount # total number of emacs plugins
zpwr emacspluginlist  # total list of emacs plugins
```

Chapter 14 – tmux Warfare

Tmux Session Management

tmux Warfare: Terminal Multiplexer Combat

One terminal to rule them all, split them, and in panes bind them.

zpwr Attach

Attach to an existing tmux session. If no session exists, one will be summoned.

```
zpwr attach          # rejoin the matrix
```

zpwr Detach

Detach from current tmux session gracefully. Your processes keep running in the background.

```
zpwr detach          # leave but don't kill
```

zpwr Killmux

Kill the tmux server. Nuclear option. Everything dies. WARNING: all sessions, windows, and panes – gone.

```
zpwr killmux          # scorched earth
```

zpwr Start

Start a new tmux session with ZPWR's custom layout. Preconfigured panes, windows, and keybindings.

```
zpwr start            # bootstrap tmux
```

zpwr Starttabs

Start tmux with multiple pre-configured tabs. Each tab gets its own purpose – dev, logs, git, etc.

```
zpwr starttabs        # multi-tab setup
```

// the tmux lifecycle:

// start -> attach -> detach -> attach -> killmux

// sessions persist through SSH disconnects

// your terminal state is immortal

Key Duplication – Multi-Pane Workflow

Key Duplication: Type Once, Execute Everywhere

The closest thing to being in two places at once.

zpwr Startsend

Send keystrokes to ALL panes in the current window. Whatever you type goes to every pane simultaneously.

```
zpwr startsend          # enable key broadcast
```

USE CASE: You have 4 panes SSHed into 4 servers. You need to run the same command on all of them. startsend + type once = executed on all 4.

zpwr Startsendfull

Like startsend, but broadcasts across ALL windows and ALL panes in the entire tmux session.

```
zpwr startsendfull      # global broadcast
```

DANGER: every pane in every window receives input. Make sure you know what you are doing.

zpwr Stopsend

Turn off key broadcasting. Back to single-pane input.

```
zpwr stopsend          # disable broadcast
```

The Multi-pane Workflow:

1. \$ zpwr start # launch tmux
2. Split into 4 panes (Ctrl-b %)
3. SSH into different servers in each
4. \$ zpwr startsend # enable broadcast
5. Type: sudo apt update && sudo apt upgrade -y
6. All 4 servers update simultaneously
7. \$ zpwr stopsend # back to normal

// sysadmins call this poor mans ansible

// but honestly it is faster for quick ops

Clipboard & Buffer Operations

Clipboard Ops: Copy-paste On Steroids

Because Ctrl-C Ctrl-V is so last millennium.

zpwr Pastebuffer

Paste from the tmux paste buffer into the terminal. Tmux has its own clipboard, separate from the system.

```
zpwr pastebuffer        # paste tmux buffer
```


zpwr Pastestring

Paste a specific string into the active pane. Programmatic pasting – useful in scripts.

```
zpwr pastestring 'echo hello' # paste literal text
```

zpwr Pastecommand

Paste and execute a command in the active pane. Like pastestring but hits enter for you.

```
zpwr pastecommand 'ls -la' # paste + execute
```

zpwr Google

Open a Google search in your default browser. Because sometimes you need to leave the terminal.

```
zpwr google 'zsh completion' # search the web
```

zpwr Openurl

Open any URL in the default browser.

```
zpwr openurl https://github.com # open URL
```

// the clipboard pipeline:

// terminal output → tmux buffer → pastebuffer

// system clipboard → pastestring → pane

// brain → google → browser → back to terminal

Autocomplete Integration

Autocomplete: The Tab Key's Final Form

Zsh-autocomplete + tmux = sentient tab completion.

zpwr Startauto

Enable the ZPWR autocomplete integration. Completions appear as you type, no Tab needed.

```
zpwr startauto # enable live complete
```

zpwr Stopauto

Disable autocomplete. Sometimes you want silence.

```
zpwr stopauto # disable live complete
```

How It Works With Tmux:

The autocomplete system hooks into zsh's completion engine and renders suggestions in real-time.

FEATURES: * Live suggestions as you type * History-based predictions * Path completion with preview * Command argument completion * Works across all tmux panes independently.

tmux + Autocomplete Tips:

* Each tmux pane runs its own zsh instance * Autocomplete state is per-pane * startauto/stopauto affects current pane only * If completions feel slow, check `$ZPWR_COMPS_CACHE`

```
zpwr stopauto      # when it gets noisy
zpwr startauto     # when you miss it
```

// autocomplete is like a copilot for your shell

// it watches, it learns, it suggests

// toggle it off when you need to think in silence

Process Monitoring

Process Monitoring: Watching The Watchers

Every PID tells a story, most of them tragic.

zpwr Pstreeonitor

Displays a live process tree. Watch your processes branch, fork, and occasionally die unexpectedly.

```
zpwr pstreeonitor  # live process tree
```

Uses pstree under the hood with continuous refresh. Like htop's tree view but from the ZPWR command line.

zpwr Tty

Show info about the current terminal/tty.

```
zpwr tty          # current tty info
```

tmux + Monitoring Workflow:

The power move: dedicate a tmux pane to monitoring.

LAYOUT EXAMPLE: +-----+-----+ ||| your work | pstreeonitor ||| +-----+
+-----+ | zpwr taillog | +-----+

Process Hunting:

ZPWR integrates `arb -fzf` with `ps` and `lsuf` for killing.

`Ctrl-Y Ctrl-K # arb -fzf kill by ps Ctrl-Y Ctrl-L # arb -fzf kill by lsuf`

// pstreeonitor in a side pane = instant situational awareness

// you will see zombie processes before they see you

Clipboard, Send & Tmux

// 13 verbs for pane orchestration //

Clipboard access, key duplication across panes, URL handling, and tmux session management.

Clipboard

```
zpwri copycommand      # get the command to copy with system
zpwri pastecommand     # get the command to paste with system
zpwri pastebuffer      # paste clipboard to $BUFFER
zpwri pastestring      # paste clipboard to stdout
```

Key Duplication

```
zpwri startsend        # dup some keys to pane
zpwri startsendfull    # dup all keys to pane
zpwri stopsend         # stop dup keys to pane
```

Url & Browser

```
zpwri google           # google clipboard contents in tmux
zpwri openurl          # open URL from clipboard in tmux
```

Session Management

```
zpwri attach           # attach to tmux session
zpwri detach           # detach from all tmux sessions
zpwri killmux          # kill tmux server
zpwri killremote       # kill tmux server and ssh
```

Chapter 15 – Network & Containers

Network Recon

Network Recon: Scanning The Wire

Packets don't lie, but DNS sometimes does.

zpwr Digs

Enhanced DNS lookup. Wraps dig with cleaner output. When you need to know where a domain really points.

```
zpwr digs github.com      # DNS lookup
zpwr digs example.com     # check any domain
```

Shows A records, CNAME chains, TTLs – the works. Faster than remembering dig +short +noall +answer.

zpwr Pi / zpwr Ping

Ping a host. Simple, reliable, essential. 'pi' is the short alias because typing matters.

```
zpwr pi google.com        # quick ping
zpwr ping 8.8.8.8         # verbose ping
```

zpwr Pir

Ping with extended options. More detailed network diagnostics for when simple ping is not enough.

```
zpwr pir 192.168.1.1      # extended ping
```

Recon Workflow:

1. \$ zpwr digs suspect-domain.com # where does it resolve?
2. \$ zpwr pi <ip-from-dig> # is it alive?
3. \$ zpwr pir <ip> # deeper probe

// three verbs, complete network recon

// digs for DNS, pi for heartbeat, pir for details

// when in doubt, ping first – dead hosts tell no tales

Tor Integration

Tor Integration: Anonymous Mode Engaged

The onion router: because privacy is not a crime.

zpwr Torip

Check your current Tor exit node IP address. Confirms you're routing through the onion network.

```
zpwr torip # show tor exit IP
```

If Tor is running, this shows the exit node's IP. If not, it'll tell you Tor isn't connected.

zpwr Toriprenew

Request a new Tor circuit. Get a fresh exit node and a new apparent IP address.

```
zpwr toriprenew # new identity
```

BEFORE YOU START:

Tor must be installed and running as a service.

MacOS: brew install tor && brew services start tor Linux: sudo apt install tor && sudo systemctl start tor.

The Anonymous Workflow:

1. Start tor service
2. \$ zpwr torip # confirm you're connected
3. Do your thing
4. \$ zpwr toriprenew # rotate identity
5. \$ zpwr torip # verify new IP

DISCLAIMER: Tor provides anonymity, not invincibility. DNS leaks, browser fingerprinting, and OPSEC failures can still expose you. Use responsibly.

// "I am become anonymous, destroyer of logs"

// – J. Robert Oppenheimer, probably not

Docker Management

Docker Management: Container Warfare

It works on my machine → we ship the machine.

zpwr Dockerwipe

The nuclear option for Docker. Removes EVERYTHING: containers, images, volumes, networks – all of it.

```
zpwr dockerwipe # purge all docker
```

WARNING: This is not gentle. It will: - Stop all running containers - Remove all containers - Remove all images - Remove all volumes - Prune everything.

WHEN TO USE: * Disk space emergency (docker images love eating disk) * Fresh start after a botched experiment * When 'docker system df' shows 50GB of shame.

zpwr Dfimage

Reverse-engineer a Docker image to show its Dockerfile. See how an image was built without the source.

```
zpwr dfimage nginx:latest # reverse Dockerfile
zpwr dfimage myapp:v2     # inspect any image
```

Useful for auditing third-party images. Trust but verify what's in that container.

Docker Triage:

1. \$ docker system df # how bad is it?
2. \$ zpwr dfimage suspect:latest # what's inside?
3. \$ zpwr dockertwipe # burn it all down
4. \$ docker system df # ahh, fresh disk

// dockertwipe: for when docker system prune isn't angry enough

// dfimage: for when you don't trust the README

Clone & GitHub Workflow

Clone & Github: The Full Git Workflow

git clone is just the beginning of the relationship.

zpwr Clone

Clone a git repository with ZPWR's enhanced setup. Handles auth, directory placement, and post-clone hooks.

```
zpwr clone user/repo      # clone from GitHub
zpwr clone https://...    # clone any URL
```

zpwr Reveal

Open the current git repo's GitHub page in browser. Instant context switch from terminal to web.

```
zpwr reveal              # open repo in browser
```

zpwr Revealrecurse

Open GitHub pages for the repo AND its submodules. When you need the full picture of a multi-repo project.

```
zpwr revealrecurse      # open all repos
```

The Github Workflow:

1. \$ zpwr clone user/repo # get the code
2. \$ cd repo && hack hack hack
3. \$ zpwr reveal # check CI/issues
4. \$ git push # ship it

Related Tools:

```
zpwr killremote          # kill remote sessions
```

Killremote terminates remote SSH/tmux sessions that have gone stale. Cleanup for the disciplined operator.

// reveal is the bridge between terminal and browser

// faster than typing github.com/user/repo

// your fingers will thank you

Docker & Container Aliases

// 148 aliases for the containerized world //

The zsh-docker-aliases plugin provides 148 aliases for Docker and Docker Compose operations.

Docker Shortcuts

```
dps          # docker ps
dpsa         # docker ps -a
di           # docker images
drim         # docker rm
drmi         # docker rmi
dex          # docker exec -it
dl           # docker logs -f
```

Docker Compose

```
dcu          # docker compose up -d
dcd          # docker compose down
dcl          # docker compose logs -f
dcr          # docker compose restart
```

zpwrc Integration

```
zpwrc dockerwipe      # nuke all images/containers
zpwrc dfimage <ID>    # extract Dockerfile from image
```

Plugin: MenkeTechnologies/zsh-docker-aliases.

Process & Network

// 15 verbs for procs, ports, and packets //

Kill processes via arb -fzf, monitor process trees, toggle Tor, ping LANs, and run network diagnostics.

Process Management

```
zpwrc ps          # ps -ef | grep arg
zpwrc kill         # kill from ps output via arb --fzf
zpwrc killedit     # edit kill from ps output via arb --fzf
zpwrc lsof         # kill from lsof output via arb --fzf
zpwrc lsofedit     # edit kill from lsof output via arb --fzf
zpwrc pstreemonitor # monitor process tree with watch
```

Network Diagnostics

```
zpwrc digs        # run series of networking cmds on arg
zpwrc pi           # ping all LAN devices
zpwrc ping         # ping all LAN devices
zpwrc pir          # run command on all devices
```

Tor & Ip

```
zpwir torip          # get IP address via Tor and whois
zpwir toriprenew     # renew Tor identity
zpwir to             # toggle external ip
zpwir toggle         # toggle external ip
```

Polling

```
zpwir poll           # poll git remote and run command
```


Chapter 16 – The Logging System

Log Levels

Log Levels: Signal Vs Noise

if a tree falls in the forest and nobody tails the log...

ZPWR has a full structured logging system with four levels. Each level has a file variant and a console variant.

Level 1: Trace

```
zpwr logtrace 'entering func' # most verbose
```

The firehose. Every function entry, variable assignment, and internal state change. Use for deep debugging only.

Level 2: Debug

```
zpwr logdebug 'cache miss' # developer detail
```

Detailed info for development. Cache behavior, branch decisions, intermediate values.

Level 3: Info

```
zpwr loginfo 'update done' # normal operations
```

Standard operational messages. “Started”, “Completed”, “Loaded 47 plugins”. The daily heartbeat.

Level 4: Error

```
zpwr logerror 'file not found' # something broke
```

Things that should not have happened but did. Missing files, failed commands, broken assumptions.

Severity Pyramid:

Console vs File Logging

Console Vs File: Two Destinations

To stderr or to disk, that is the question.

Every log verb comes in two flavors:

File Variants (write To \$ZPWR_LOGFILE):

```
zpwr logtrace 'msg'      # --> logfile
zpwr logdebug 'msg'      # --> logfile
zpwr loginfo 'msg'       # --> logfile
zpwr logerror 'msg'      # --> logfile
```

These write silently to the log file. No terminal output. Perfect for background operations and scripts.

Console Variants (write To Stderr And Logfile):

```
zpwr logtraceconsole 'msg' # --> stderr + file
zpwr logdebugconsole 'msg' # --> stderr + file
zpwr loginfoconsole 'msg'  # --> stderr + file
zpwr logerrorconsole 'msg' # --> stderr + file
```

These print to your terminal AND write to the log. For interactive debugging when you need to see it live.

The Logfile:

```
$ZPWR_LOGFILE = $ZPWR_LOCAL/zpwr.log
```

```
cat $ZPWR_LOGFILE | tail -20 # recent entries
wc -l $ZPWR_LOGFILE          # how big is it?
```

Choosing Your Variant:

SCENARIO USE Background script running loginfo Debugging interactively logdebugconsole Tracking a rare bug logtraceconsole Production error handling logerror.

// file = silent, persistent, searchable

// console = loud, immediate, ephemeral

// use both strategically

Reading Logs

Reading Logs: The Art Of Log Diving

Somewhere in that wall of text is the answer you seek.

zpwr Taillog

Live tail of the ZPWR log file with zcolorizer colorization. Watch events stream by in glorious technicolor.

```
zpwr taillog # live colorized tail
```

Zcolorizer automatically highlights timestamps, levels, paths, and errors. Makes raw logs actually readable.

What To Look For:

ERROR lines = something broke, investigate immediately WARN lines = potential issue, worth noting INFO lines = normal operations, sanity check DEBUG lines = details for troubleshooting.

```
TRACE lines = deep internals, very verbose
```

Filtering Techniques:

```
grep ERROR $ZPWR_LOGFILE      # errors only
grep -c ERROR $ZPWR_LOGFILE   # count errors
tail -100 $ZPWR_LOGFILE       # last 100 lines
```

tmux Log Monitoring Setup:

Dedicate a tmux pane to zpwr taillog:

1. Split pane: Ctrl-b % 2. In new pane: zpwr taillog 3. Work in other pane, watch logs stream 4. Errors glow red thanks to zcolorizer

Log Maintenance:

Logs grow forever if you let them. ZPWR handles rotation, but you can manually truncate:

```
: > $ZPWR_LOGFILE          # truncate to zero
```

// taillog + zcolorizer = the cyberpunk log experience

// errors in red, info in green, timestamps in cyan

Debug Mode

Debug Mode: When Things Go Sideways

Enabling verbose output is how you stop guessing.

Zpwr_debug

Set this env var to enable debug-level logging. More detail in the log, more signal for your hunt.

```
export ZPWR_DEBUG=true      # enable debug
zpwr reload                 # reload with debug
zpwr taillog                 # watch debug output
```

Zpwr_trace

Maximum verbosity. Every function entry and exit, every variable, every decision point.

```
export ZPWR_TRACE=true      # enable trace
zpwr reload                 # reload with trace
```

WARNING: Trace mode generates MASSIVE log output. Your logfile will grow fast. Use sparingly.

Debugging Workflow:

1. Reproduce the problem once to confirm it 2. \$ export ZPWR_DEBUG=true 3. \$ zpwr reload 4. Reproduce the problem again 5. \$ zpwr taillog # find the smoking gun 6. Still lost? Escalate: 7. \$ export ZPWR_TRACE=true 8. \$ zpwr reload && reproduce

Disabling Debug:

```
unset ZPWR_DEBUG ZPWR_TRACE # back to normal
zpwr reload                  # reload clean
```

// debug mode: "trust but verify" in action

// trace mode: "verify everything, trust nothing"

// always disable after debugging or your log eats disk

Logging System

// 12 verbs for structured observability //

ZPWR has a four-level logging system: trace, debug, info, error. Each level writes to file or console.

Log To File

```
zpwr logtrace          # log trace to $ZPWR_LOGFILE
zpwr logdebug          # log debug to $ZPWR_LOGFILE
zpwr loginfo           # log info to $ZPWR_LOGFILE
zpwr logerror          # log error to $ZPWR_LOGFILE
```

Log To Console

```
zpwr logtraceconsole   # log trace to console
zpwr logdebugconsole   # log debug to console
zpwr loginfoconsole    # log info to console
zpwr logerrorconsole   # log error to console
```

Log Management

```
zpwr taillog           # colored tail of $ZPWR_LOGFILE
zpwr logs              # colored tail of $ZPWR_LOGFILE
zpwr cleanlog          # clear $ZPWR_LOGFILE
zpwr logincount        # count of logins by user
```

Log levels from most verbose to least: TRACE > DEBUG > INFO > ERROR.

Chapter 17 – Introspection

Counting Everything

Counting Everything: The Census Bureau

If you can't measure it, you can't brag about it.

ZPWR tracks the size of every component. These verbs tell you exactly how big your installation is.

zpwr Autoloadcount

Count of autoloaded zsh functions in the framework.

```
zpwr autoloadcount          # how many functions?
```

zpwr Scriptcount

Count of shell scripts in `$ZPWR/scripts`.

```
zpwr scriptcount           # how many scripts?
```

zpwr Zshplugincount

Count of installed zsh plugins (zinit, oh-my-zsh, etc).

```
zpwr zshplugincount        # zsh plugins loaded
```

zpwr Vimplugincount

Count of installed vim/neovim plugins.

```
zpwr vimplugincount        # vim plugins loaded
```

zpwr Emacsplugincount

Count of installed emacs packages.

```
zpwr emacsplugincount      # emacs pkgs loaded
```

The Full Census:

```
zpwr autoloadcount && zpwr scriptcount && \
```

zpwr zshplugincount && zpwr vimplugincount && \ zpwr emacsplugincount.

Run all five to get the full picture. ZPWR typically ships with 300+ autoloads, 50+ scripts, and dozens of plugins across all three editors.

```
// these counts tell you the scope of ZPWR
// hundreds of functions working in concert
// like a small city running on your laptop
```

Listing Everything

Listing Everything: The Inventory Manifest

Counting is nice, but names are power.

Counts tell you how many. Lists tell you WHAT. Every count verb has a list counterpart.

zpwr Autoloadlist

List every autoloaded function by name.

```
zpwr autoloadlist          # all function names
zpwr autoloadlist | grep arb --fzf # find arb --fzf funcs
```

zpwr Scriptlist

List every script in the scripts directory.

```
zpwr scriptlist           # all scripts
```

zpwr Zshpluginlist

List installed zsh plugins with their sources.

```
zpwr zshpluginlist        # all zsh plugins
```

zpwr Vimpluginlist

List installed vim/neovim plugins.

```
zpwr vimpluginlist        # all vim plugins
```

zpwr Emacspluginlist

List installed emacs packages.

```
zpwr emacspluginlist      # all emacs packages
```

Power Combos:

```
zpwr autoloadlist | wc -l      # quick count
zpwr autoloadlist | arb --fzf  # fuzzy browse
zpwr scriptlist | sort         # alphabetical
```

// lists are grep-able, arb-able, pipe-able
// the raw material for your own exploration
// pipe into arb -fzf for interactive discovery

History Archaeology

History Archaeology: Digging Up The Past

Your terminal remembers everything, even the mistakes.

zpwr Hist

Browse command history with arb -fzf. Search, preview, and re-execute past commands interactively.

```
zpwr hist # arb --fzf history browser
```

zpwr Histedit

Edit the history file directly. Surgically remove embarrassing commands or fix typos in saved history.

```
zpwr histedit # edit history file
```

Use with care. History surgery is delicate work.

zpwr Histfile

Show the path to the current history file. Useful when you're not sure which histfile is active.

```
zpwr histfile # show histfile path
```

zpwr Catvminfo

Display vim's recent file history from viminfo. What files has vim touched recently?

```
zpwr catvminfo # vim file history
```

zpwr Catrecentfviminfo / Catnviminfo recentf

Cross-reference vim/nvim recent files with the recentf list. Where vim and shell history intersect.

```
zpwr catrecentfviminfo # vim+recentf cross  
zpwr catnviminfo recentf # nvim+recentf cross
```

// your history is a goldmine of workflow patterns
// hist + arb -fzf = instant command recall
// histedit = revisionist history (use responsibly)

Verb Discovery

Verb Discovery: Finding What's Possible

You can't use what you don't know exists.

zpwr Subcommands

List all zpwr subcommands (verbs). The master index of everything zpwr can do.

```
zpwr subcommands          # all subcommands
```

zpwr Subcommandscount

How many subcommands exist? The number keeps growing.

```
zpwr subcommandscount     # total count
```

zpwr Verbs (alias: Subcommandsfzf)

Browse subcommands interactively with arb -fzf. The best way to explore when you're not sure what to try.

```
zpwr verbs                # arb --fzf subcommands
```

zpwr Verbslist / Verbsfzf / Verbscount

Alternative verb-centric views of available commands.

```
zpwr verbslist            # list all verbs
zpwr verbs                # arb --fzf browse verbs
zpwr verbscount           # count verbs
```

zpwr Wizard / Manual / Tutorial / Docs

You're reading it right now. The interactive encyclopedia with vim-style navigation. Very meta. All four names work.

```
zpwr wizard               # start the wizard
zpwr wizard --toc          # table of contents
zpwr wizard --search docker # search pages
zpwr wizard 42             # jump to page 42
```

// verbsfzf is the fastest way to discover (alias: subcommandsfzf)

// type a few letters, arb -fzf narrows it down

// when in doubt, arb -fzf it out

Shell State Inspection

Shell State: Know Thy zsh

Introspection: when the shell looks in the mirror.

zpwr Exists

Check if a ZPWR component exists (functions, files, etc).

```
zpwr exists zpwrReload      # does this exist?
```

zpwr Existscommand

Check if a command exists in PATH. Cleaner than 'which' or 'command -v' in scripts.

```
zpwr existscommand git      # is git installed?
zpwr existscommand rg       # is ripgrep here?
```

zpwr Reload

Reload the entire ZPWR framework without restarting. Re-resources configs, rebuilds caches, refreshes state.

```
zpwr reload                  # hot reload everything
```

zsh Inspection Tools:

These are built into zsh, but ZPWR makes them accessible:

```
setopt                       # list active options
zmodload                     # list loaded modules
zstyle -L                    # list all zstyles
```

zpwr Copycommand

Copy a command's path or definition to clipboard.

```
zpwr copycommand ls         # copy ls path
```

zpwr Opencommand

Open a command's source file in your editor. Works for functions, scripts, and binaries.

```
zpwr opencommand zpwrReload # view the source
```

// existscommand before you call, reload after you change

// opencommand when you need to understand the internals

Command Resolver

// what IS that command, really? //

zpwr resolve traces a command through its full resolution chain. Unlike 'which' or 'type', it follows the entire path from alias to function to builtin to binary.

The Verb

```
zpwr resolve ll          # where does ll go?
zpwr resolve ls cd git   # trace multiple commands
zpwr resolve -a grep     # show all type -a matches
zpwr which vim           # alias for resolve
```

What It Shows

Aliases expansion, then resolves first word.

```
Functions      source file path, line count, verb mapping
```

Builtins identified as builtin or reserved word Binaries full path, symlink chain, file type Global aliases detected and shown Suffix aliases detected and shown.

Why Not Just 'which'?

'Which' shows one answer. 'type -a' shows all candidates. Neither follows the chain. If 'll' is an alias for 'eza' and 'eza' is a symlink to '../Cellar/eza/0.18/bin/eza', resolve shows the entire path from ll to the Mach-O binary.

Companion Verbs

```
zpwr exists          # check if identifier is valid
zpwr deps            # function dependency graph
zpwr funcrank        # rank functions by usage
```

Shell Introspection

// 32 verbs to look inward //

Inspect ZPWR itself: banners, counts, plugin lists, subcommand discovery, and identity checks.

Existence Checks

```
zpwr exists          # check if identifier is valid
zpwr existscommand   # check if command is valid
```

Banners

```
zpwr about           # show zpwr banner
zpwr banner           # show zpwr banner
zpwr bannercounts     # show banner and env counts
zpwr bannerlolcat     # banner with ponysay and lolcat
zpwr bannernopony     # show banner without ponysay
zpwr bannerpony       # show banner with ponysay
```

Counts & Lists

```
zpwr autoloadcount      # total number of autoloads
zpwr autoloadlist       # total list of autoloads
zpwr scriptcount        # total number of scripts
zpwr scriptlist         # total list of scripts
zpwr zshplugincount     # total number of zsh plugins
zpwr zshpluginlist      # total list of zsh plugins
zpwr vimplugincount     # total number of vim plugins
zpwr vimpluginlist      # total list of vim plugins
zpwr emacsplugincount   # total number of emacs plugins
zpwr emacspluginlist    # total list of emacs plugins
```

Subcommands & Verbs

```
zpwr subcommands        # run subcommands for zpwr <tab>
zpwr subcommandscount   # number of subcommands
zpwr subcommandsedited  # edit subcommands for zpwr <tab>
zpwr subcommandsfzf     # arb --fzf subcommands for zpwr <tab>
zpwr subcommandsls      # list subcommands for zpwr <tab>
zpwr verbs              # run subcommands for zpwr <tab>
zpwr verbscount         # number of verbs
zpwr verbsedit          # edit subcommands for zpwr <tab>
zpwr verbsfzf           # arb --fzf subcommands for zpwr <tab>
zpwr verbslist          # list subcommands for zpwr <tab>
zpwr help               # list subcommands with color
```

Misc Introspection

```
zpwr tty                # print current tty
zpwr relpath            # get relative path to $PWD
zpwr printmap           # pretty print maps
```

Chapter 18 – Installation & Reset

Config Sessions

Config Sessions: Edit Everything From Here

One verb to open any config, no path memorization.

ZPWR provides shortcut verbs for every config file you'll ever need to edit. No more 'vim ~/.zshrc'.

zpwr Brc – Bash Rc

Edit your .bashrc. For when bash compatibility matters.

```
zpwr brc          # edit .bashrc
```

zpwr Zrc – zsh Rc

Edit your .zshrc. The most-touched config in ZPWR.

```
zpwr zrc          # edit .zshrc
```

zpwr Vrc – vim Rc

Edit your .vimrc or init.vim/init.lua.

```
zpwr vrc          # edit vim config
```

zpwr Trc – tmux Rc

Edit your .tmux.conf. Keybindings, statusbar, the works.

```
zpwr trc          # edit tmux config
```

zpwr Tokens – Secrets File

Edit \$ZPWR_LOCAL/.tokens.sh. API keys, secrets, env vars.

```
zpwr tokens      # edit tokens/secrets
zpwr vimtokens    # same, force vim
zpwr emacs tokens # same, force emacs
```

zpwr Zcompdump – Completion Cache

Inspect or regenerate the zsh completion dump file. When completions act weird, this is where you look.

```
zpwr zcompdump    # inspect comp cache
```

```
// brc, zrc, vrc, trc – four letters, four configs
// tokens for secrets, zcompdump for completion fixes
```

Install & Uninstall

Install & Uninstall: Setup And Teardown

Easy to install, clean to remove – no malware energy.

zpwr Install

Run the full ZPWR installation process. Installs dependencies, links configs, sets up plugins.

```
zpwr install                # full installation
```

WHAT IT DOES: 1. Installs system dependencies (brew/apt) 2. Links config files to their proper locations 3. Installs zsh, vim, and emacs plugins 4. Generates caches and completion dumps 5. Sets up tmux configuration.

zpwr Uninstall

Remove ZPWR configuration. Restores your shell to its pre-ZPWR state. Clean breakup, no hard feelings.

```
zpwr uninstall              # remove zpwr config
```

Config Link Management:

ZPWR uses symlinks to place configs where tools expect them.

How it works: `$ZPWR/configs/.zshrc` → `~/.zshrc` (symlink) `$ZPWR/configs/.vimrc` → `~/.vimrc` (symlink)
`$ZPWR/configs/.tmux` → `~/.tmux.conf` (symlink)

The source of truth lives in `$ZPWR`, the symlinks just point there. Version controlled, portable, clean.

Partial Operations:

Sometimes you don't want the full install/uninstall. You just need to fix the symlinks:

Regenconfiglinks = recreate all config symlinks rmconfiglinks = remove symlinks only.

// install is idempotent – run it again when things drift

// uninstall is clean – your system goes back to stock

Reset & Recovery

Reset & Recovery: When Things Go Wrong

Every system needs a panic button – here are yours.

zpwr Reset

Full framework reset. Clears caches, regenerates configs, rebuilds completion dumps, relinks everything.

```
zpwr reset # the big red button
```

WHEN TO USE: * Completions are broken or stale * Config changes aren't taking effect * Something feels "off" and you can't pin it down * After a major ZPWR update.

zpwr Reload

Hot reload without resetting. Re-sources all configs and refreshes the environment. Faster than reset.

```
zpwr reload # hot reload
```

Use reload when you've edited a config and want to apply changes without nuking caches.

Escalation Ladder:

Level 1: zpwr reload # re-source configs Level 2: zpwr reset # rebuild everything Level 3: zpwr install # full reinstall Level 4: rm -rf `$ZPWR` && clone # scorched earth.

Start at Level 1. Only escalate if the problem persists. Most issues are fixed by reload. Reset handles the rest.

Common Recovery Scenarios:

"Completions are wrong" → zpwr reset "alias isn't working" → zpwr reload "new plugin not loading" → zpwr reload "everything is broken" → zpwr reset "still broken after reset" → zpwr install.

// reload is ctrl-r for your shell framework

// reset is the factory reset you hope to never need

// but when you do need it, you'll be glad it exists

Chapter 19 – Batch Operations

Loop Warriors

Loop Warriors – The Repetition Toolkit

Because doing things once is for civilians.

The batch operations verbs are the assault rifles of the ZPWR arsenal. Point them at a problem, pull the trigger, and watch every target fall.

The For Loop – Your Basic Infantry

```
zpwr for <cmd> <args...>
```

Executes a command for each argument passed to it. Stdin becomes your ammo belt — pipe lines in, each one fires.

Example: run a command against multiple targets.

```
echo 'file1.txt\nfile2.txt' | zpwr for wc -l
```

For10 – The Burst-fire Variant

```
zpwr for10 <cmd>
```

Runs your command 10 times. No arguments, no mercy. Perfect for stress testing, benchmarking, or just asserting dominance over your CPU.

Example: benchmark a command 10 times.

```
zpwr for10 zpwr bench
```

Whiletrue – The Infinite Loop

```
zpwr whiletrue <cmd>
```

Runs your command forever. Ctrl-C is your only exit. For monitoring, watching, waiting. The sentinel loop.

Example: watch disk usage forever.

```
zpwr whiletrue df -h /
```

WARNING: there is no break condition. You ARE the break.

Whilesleep – The Patient Sniper

```
zpwr whilesleep <seconds> <cmd>
```

Like whiletrue, but sleeps between rounds. Polling with grace. Surveillance with manners.

Example: check a URL every 30 seconds.

```
zpwr whilesleep 30 curl -s -o /dev/null -w '%{http_code}' https://example.com
```

TACTICAL SUMMARY: for iterate over stdin lines for 10 run 10 times, no questions asked whiletrue infinite loop, Ctrl-C to escape whilesleep infinite loop with naptime between rounds.

// in the loop, there is no time, only iterations

// the loop does not end. the operator ends the loop.

Directory Sweeps

Directory Sweeps – Carpet Bombing Your Filesystem

One directory at a time, or all of them at once.

Single-file loops are amateur hour. Real operators sweep directories. They glob patterns. They parallelize. These verbs are your air support.

Fordir – The Directory Crawler

```
zpwr fordir <cmd>
```

Iterates into each subdirectory of the current dir and executes the command inside it. cd in, run, cd out. Like a SWAT team clearing rooms one by one.

Example: check git status in every project dir.

```
cd ~/repos && zpwr fordir git status -s
```

Fordirupdate – The Mass Updater

```
zpwr fordirupdate
```

Enters every subdirectory and runs git pull. Your entire project portfolio, synced in one command. Monday morning? Run this first. Coffee second.

Example: update all repos under ~/repos

```
cd ~/repos && zpwr fordirupdate
```

Execglob – Glob And Execute

```
zpwr execglob <glob> <cmd>
```

Finds files matching a glob pattern and runs your command against each one. Pattern matching meets execution. The regex sniper rifle of file operations.

Example: lint every Python file in the tree.

```
zpwr execglob '*.py' pylint
```


Execglobparallel – Concurrent Devastation

```
zplr execglobparallel <glob> <cmd>
```

Same as execglob, but fires all jobs in parallel. Every CPU core gets drafted into service. Your fan will spin up. This is normal. This is power.

Example: format every .rs file in parallel.

```
zplr execglobparallel '*.rs' rustfmt
```

CHOOSING YOUR WEAPON: fordir run cmd in each subdirectory fordirupdate git pull in each subdirectory
execglob glob match + execute sequentially execglobparallel glob match + execute in parallel.

// sequential is predictable. parallel is fast.

// choose based on whether output order matters.

// when in doubt, go parallel. life is short.

Polyglot Execution

Polyglot Execution – Beyond The Shell

zsh is the conductor, but the orchestra speaks many tongues.

Sometimes zsh loops aren't enough. Sometimes you need Python. Sometimes you need background jobs. Sometimes you need to poll a service until it wakes up or dies.

Execpy – The Python Bridge

```
zplr execpy <python_expression>
```

Fires a Python expression from the comfort of your shell. No need to open a REPL. No need for a .py file. Quick math, string ops, one-liners — python as a verb.

Example: quick calculation.

```
zplr execpy '2**128'
```

=> 340282366920938463463374607431768211456.

Background – The Shadow Operative

```
zplr background <cmd>
```

Sends a command into the background. Detached. Silent. It runs in the shadows while you keep working. Output goes to the void unless you redirect it.

Example: start a long build in the background.

```
zplr background make -j8
```

Timer – The Stopwatch

```
zplr timer <cmd>
```

Wraps any command in a high-resolution timer. How long did that take? Now you know. No guessing. Benchmarking made trivial.

Example: time your shell startup.

```
zpwr timer zsh -ic exit
```

Poll – The Watchdog

```
zpwr poll <cmd>
```

Runs a command repeatedly until it succeeds (exit 0). Waiting for a server to come up? A build to finish? Poll is your patient, relentless sentinel.

Example: wait for a port to open.

```
zpwr poll nc -z localhost 8080
```

THE BATCH OPS PHILOSOPHY: Loops for, for10, whiletrue, whilesleep Sweeps fordir, fordirupdate, execglob* Polyglot execpy, background, timer, poll.

// the terminal is not a single-threaded mind

// it is a factory floor. these verbs are the machines.

// feed them work. they do not tire. they do not complain.

Git Repo Cache System

// index every repo on your machine. operate on all of them. //

The zsh-git-repo-cache plugin scans your entire filesystem for git repos, caches the list, and lets you search, filter, and broadcast commands across all of them.

Cache Generation

```
zpwr regengitrepopcache      # scan filesystem with fd/find
zpwr regengitdirtyrepopcache # scan only dirty repos
```

Search & Browse

```
zpwr gitrepos      # all repos into arb --fzf
zpwr gitreposclean # clean repos into arb --fzf
zpwr gitreposdirty # dirty repos into arb --fzf
zpwr gitreposlist  # list all repos (no arb --fzf)
zpwr gitreposcleanlist # list clean repos
zpwr gitreposdirtylist # list dirty repos
```

Cached Variants

```
zpwr gitreposcleancache # clean repos from cache
zpwr gitreposdirtycache # dirty repos from cache
zpwr gitreposcleancachelist # list clean from cache
zpwr gitreposdirtycachelist # list dirty from cache
```

Batch Operations

```
zpwr gitreposexec <cmd>      # run cmd in all cached repos
zpwr gitreposdirtyexec <cmd> # run cmd in dirty repos
zpwr gitreposcleanexec <cmd> # run cmd in clean repos
zpwr gitwho                  # contributor stats across all repos
```

Cache Files

`$ZPWR_ALL_GIT_DIRS` — all repos `$ZPWR_ALL_GIT_DIRS_CLEAN` — clean repos only `$ZPWR_ALL_GIT_DIRS_DIRTY` — dirty repos only

Plugin: MenkeTechnologies/zsh-git-repo-cache.

Chapter 20 – Forgit Deep Dive

Forgit Add & Diff

Forgit Add & Diff – Interactive Staging

git add, but make it beautiful.

Forgit is the fusion of arb –fzf and git. It wraps the most common git operations in fuzzy, interactive, preview-enabled interfaces. You will never go back to raw git add.

Forgitadd – Interactive Staging

```
zpw❯ forgitadd
```

Opens an arb –fzf interface showing all unstaged changes. Each file gets a diff preview in the right pane. Tab to select multiple files. Enter to stage them.

The workflow: 1. Run `zpw forgitadd` 2. Browse files with arrow keys 3. See the diff preview for each file 4. Tab to multi-select 5. Enter to stage selected files.

No more git add -p scrolling through 500-line diffs. No more accidentally staging that debug print.

Forgitdiff – Visual Diff Browser

```
zpw❯ forgitdiff
```

Opens an arb –fzf browser of all changed files with a live diff preview. Read diffs without committing to anything. Pure reconnaissance.

Example workflow:

```
zpw❯ forgitdiff          # browse all diffs visually
zpw❯ forgitadd           # stage what you want
zpw❯ gitcommit 'msg'     # commit with message
```

WHY FORGIT > RAW GIT: Raw git add: type filenames from memory like a caveman forgitadd: browse, preview, multi-select, confirm Raw git diff: wall of text scrolling past forgitdiff: file-by-file preview with fuzzy search.

// staging should not be a memory test

// it should be a visual, interactive experience

// forgit turns git into a GUI without leaving the terminal

Forgit Log & Stash

Forgit Log & Stash – Visual History

Time travel with a preview window.

Git log is a river of hashes. Git stash is a pile of forgotten context. Forgit turns both into something you can actually navigate without losing your mind.

Forgitlog – The Commit Browser

```
zpwr forgitlog
```

Opens your git history in arb –fzf with commit previews. Each commit shows its full diff in the preview pane. Search commits by message, author, or hash.

What you see: Left pane: commit graph with short messages Right pane: full diff of highlighted commit Search: fuzzy match on any commit field.

“Who wrote this garbage?” — you, using forgitlog, discovering it was also you, three months ago.

Forgitstash – The Stash Manager

```
zpwr forgitstash
```

Browse your stash entries with preview diffs. See what is in each stash before applying it. No more git stash pop roulette.

The old way: git stash list # cryptic one-liners git stash show -p stash@{2} # which one was it again? git stash pop # pray.

The forgit way:

```
zpwr forgitstash          # browse, preview, apply
```

LOG + STASH TOGETHER: These two verbs give you eyes into the past. forgitlog shows committed history. forgitstash shows uncommitted history. Between them, nothing is lost.

// the commit graph is not a wall of text

// it is a timeline. forgit lets you walk it.

// stashes are not forgotten. they are filed.

Forgit Ignore & Clean

Forgit Ignore & Clean – .GITIGNORE Management

Because node_modules should never see the inside of a commit.

The .gitignore is your repo’s bouncer list. These verbs manage it with the precision of a nightclub velvet rope.

Forgitignore – The Ignore Browser

```
zpwr forgitignore
```

Interactive gitignore template browser powered by gitignore.io. Search for languages, frameworks, IDEs. Never hand-write a .gitignore again.

The Ignore Family:

Forgitignorelist list available gitignore templates forgitignoreget fetch a specific template by name forgitignore-update update your local template cache forgitignoreclean clean the template cache.

Example: generate a Python + macOS gitignore.

```
zpwr forgitignoreget python,macos > .gitignore
```

Example: browse all available templates.

```
zpwr forgitignorelist
```

Forgitclean – The Untracked File Purge

```
zpwr forgitclean
```

Shows untracked files in an arb –fzf interface with previews. Select which ones to delete. Surgical, not scorched earth. Unlike git clean -fd, you see before you destroy.

The old way: git clean -fd # delete everything. hope for the best.

The forgit way:

```
zpwr forgitclean # browse, select, confirm, then delete.
```

IGNORE WORKFLOW: 1. zpwr forgitignorelist browse templates 2. zpwr forgitignoreget fetch what you need 3. zpwr forgitclean purge untracked cruft 4. zpwr forgitadd stage the clean repo.

// a clean repo is a happy repo

// an ignored file is a file that cannot hurt you

// let the templates do the thinking

Forgit Reset & Restore

Forgit Reset & Restore – Undo With Eyes Open

The nuclear options, but with a safety preview.

Git reset and restore are the most terrifying commands in version control. Forgit wraps them in bubble wrap and gives you a preview before you pull the trigger.

Forgitreset – Unstage With Preview

```
zpwr forgitreset
```

Shows staged files in arb –fzf with diff previews. Select files to unstage. Like git reset HEAD but you can see exactly what you are unstaging first.

Use case: you ran git add -A like a maniac and now you need to surgically remove files.

```
zpwr forgitreset # browse and unstage selectively
```

Forgitrestore – Discard Changes Safely

```
zpwr forgitrestore
```

Shows modified files with diff previews. Select files to restore to their last committed state. This discards changes permanently. No undo.

The preview saves you. You see the diff before the changes evaporate into the void.

Forgitwarn – The Safety Net

```
zpwr forgitwarn
```

Displays warnings about uncommitted changes, diverged branches, and other hazardous states. Your pre-flight checklist before doing anything destructive.

Forgitinfo – The Status Dashboard

```
zpwr forgitinfo
```

Comprehensive repo information at a glance. Branch, remotes, recent commits, dirty state. Everything you need before making decisions.

THE FORGIT COMPLETE ARSENAL: forgitadd interactive stage forgitdiff browse diffs forgitlog commit history browser forgitstash stash manager forgitreset interactive unstage forgitrestore discard changes with preview forgitclean purge untracked files forgitignore* gitignore template management forgitwarn hazard warnings forgitinfo repo dashboard.

// forgit is not a git replacement

// it is git with eyes. git with a UI. git for humans.

Forgit Reference

// 14 verbs for interactive git //

Forgit wraps common git operations in interactive pickers. Interactive staging, diffing, stashing, and more.

Staging & Diffing

```
zpwr forgitadd          # interactive git add
zpwr forgitdiff         # interactive git diff
zpwr forgitreset        # interactive git reset
zpwr forgitrestore      # interactive git restore
```

Stash & Clean

```
zpwr forgitstash        # interactive stash show
zpwr forgitclean         # interactive git clean
```

Log & Info

```
zplr forgitlog      # interactive git log
zplr forgitinfo     # interactive git info
zplr forgitwarn     # interactive git warn
```

Gitignore Management

```
zplr forgitignore   # interactive gitignore templates
zplr forgitignoreclean # interactive gitignore clean
zplr forgitignoreget # interactive gitignore get
zplr forgitignorelist # interactive gitignore list
zplr forgitignoreupdate # interactive gitignore update
```


Chapter 21 – zpwr Environment Variables

Core Paths

Core Paths – The Skeleton Of The Machine

Every system has bones. these are ZPWR's.

ZPWR does not hardcode paths. Every directory is an environment variable. Move the root, and the entire system follows. This is not a dotfiles repo — it is a relocatable operating system.

\$ZPWR – The Root Of All Things

```
echo $ZPWR    # typically ~/.zpwr
```

The root directory. Every other path derives from this. Change this one variable and the whole tree moves.

\$ZPWR_LOCAL – Your Private Sanctum

```
echo $ZPWR_LOCAL # $ZPWR/local
```

Gitignored. Your tokens, secrets, local overrides. Nothing in here ever touches a remote repository. Your .tokens.sh lives here. Guard it with your life.

\$ZPWR_SCRIPTS – The Armory

```
echo $ZPWR_SCRIPTS # $ZPWR/scripts
```

Executable scripts. Bash, Python, Perl — whatever gets the job done. These are on your PATH.

\$ZPWR_TMUX – The Multiplexer Config

```
echo $ZPWR_TMUX # $ZPWR/tmux
```

Tmux configuration, scripts, and status line logic. The control center for your terminal multiplexer.

\$ZPWR_INSTALL – The Installer

```
echo $ZPWR_INSTALL # $ZPWR/install
```

OS-specific installation scripts. macOS, Ubuntu, Fedora, Arch — each gets its own installer. Run zpwr install and the right one fires.

THE PATH MAP: `$ZPWR ~/ .zpw` (the root) `$ZPWR_LOCAL $ZPWR/local` (gitignored secrets) `$ZPWR_SCRIPTS $ZPWR/scripts` (executables) `$ZPWR_TMUX $ZPWR/tmux` (multiplexer) `$ZPWR_INSTALL $ZPWR/install` (OS installers)

// paths are not strings. they are load-bearing structures.

// hardcode a path and you weld yourself to the floor.

// use the variables. always use the variables.

Autoload System

The Autoload System – Lazy Loading For Hackers

Why load everything when you can load nothing until needed?

Zsh autoloading is the secret to ZPWR's fast startup. Functions are not loaded into memory until you call them. The shell knows they exist, but their code stays on disk until the moment of invocation. Maximum laziness. Maximum speed.

`$ZPWR_AUTOLOAD` – The Function Warehouse

```
echo $ZPWR_AUTOLOAD # $ZPWR/autoload
```

The root of the autoload tree. Every file in this tree becomes a function name. The filename IS the function.

`$ZPWR_AUTOLOAD_COMMON` – Cross-platform Functions

```
echo $ZPWR_AUTOLOAD_COMMON # $ZPWR/autoload/common
```

Functions that work on every OS. The common denominator. macOS, Linux, BSD — these functions do not care.

How Autoloading Works:

1. Zsh scans `$fpath` directories at startup
2. Each filename is registered as a function stub
3. When you call the function, zsh reads the file
4. The file contents replace the stub
5. The function executes with the loaded code

This means hundreds of autoload stubs cost near-zero at startup. Only the ones you actually use consume memory.

The Compile Acceleration:

```
zpw compile      # compile autoloads to .zwc
zpw recompile    # recompile all
zpw uncompile    # remove compiled files
```

Compiled `.zwc` files load faster than raw source. ZPWR compiles its autoload dirs into wordcode files. It is like bytecode for your shell functions.

INSPECTION VERBS:

```
zpwr autoload      show the autoload system
zpwr autoloadcount  count autoloaded functions
zpwr autoloadlist   list all autoloaded functions
```

// the fastest code is code that never runs
// autoloading is the art of deferred execution
// 400 functions, zero startup cost. that is the deal.

Behavior Flags

Behavior Flags – The Knobs And Dials

Tune the machine without cracking open the engine.

ZPWR is not a take-it-or-leave-it monolith. These boolean and string flags let you control startup behavior, logging verbosity, and debug output without editing a single line of source code.

\$ZPWR_PROFILING – Startup Profiler

```
export ZPWR_PROFILING=true
```

Enables zsh startup profiling. Shows exactly where your milliseconds go during shell init. Pair with.

```
zpwr bench for the full picture.
```

Set it, open a new shell, and read the profiling data. Find the bottleneck. Kill the bottleneck. Repeat.

\$ZPWR_DEBUG – Debug Mode

```
export ZPWR_DEBUG=true
```

Turns on debug-level log messages. When something is broken and you need the machine to talk, flip this. Check output with `zpwr logdebug`.

\$ZPWR_TRACE – Full Trace Mode

```
export ZPWR_TRACE=true
```

The firehose. Every function call, every expansion, traced to the log. Use `zpwr trace` to run commands with tracing, or `zpwr logtrace` to read trace logs. WARNING: produces enormous output. Use surgically.

\$ZPWR_BANNER_CLEARLIST – Startup Banner Control

```
export ZPWR_BANNER_CLEARLIST=true
```

Controls whether the ZPWR banner and clearlist show at shell startup. The banner is glorious, but if you open 50 terminals a day, you might want silence.

Related verbs:

```
zpwr banner      show the banner manually
zpwr clearlist    show clearlist manually
zpwr bannerlolcat banner with rainbow colors
```

QUICK REFERENCE: ZPWR_PROFILING startup timing data ZPWR_DEBUG debug-level logging
ZPWR_TRACE full execution trace ZPWR_BANNER_CLEARLIST startup banner toggle.

// the flags do not change the code. they reveal it.

// debugging is not failure. it is x-ray vision.

Plugin System

Plugin System – The Turbo Loader

zinit: loading plugins at the speed of light.

ZPWR uses zinit (formerly zplugin) as its plugin manager. But not the basic way. The turbo way. Plugins load asynchronously after prompt appears. You see your prompt instantly. Plugins catch up.

\$ZPWR_PLUGIN_MANAGER – Choose Your Loader

```
echo $ZPWR_PLUGIN_MANAGER # zinit
```

Controls which plugin manager ZPWR uses. The default and recommended value is zinit. Other managers exist but zinit's turbo mode is what makes ZPWR fast.

Turbo Loading Explained:

Normal plugin loading: source plugin1.zsh # blocks source plugin2.zsh # blocks source plugin3.zsh # blocks # prompt finally appears after ALL plugins load.

Zinit turbo loading: prompt appears immediately plugin1 loads in background ... done plugin2 loads in background ... done plugin3 loads in background ... done # user never notices the delay

Plugin Inspection:

```
zpwr zshplugincount    count loaded zsh plugins
zpwr zshpluginlist     list all loaded zsh plugins
zpwr vimplugincount    count vim plugins
zpwr vimpluginlist     list all vim plugins
zpwr emacsplugincount  count emacs plugins
zpwr emacspluginlist   list all emacs plugins
```

Update And Maintain:

```
zpwr updatezinit      update zinit itself
zpwr updatedeps       update all plugin deps
zpwr plugins          plugin information
```

// plugins are not bloat. they are force multipliers.

// but only if they load in zero perceived time.

// turbo mode makes 50 plugins feel like zero.

Display & Input

Display & Input – The Human Interface Layer

How the machine talks to you, and how you talk back.

These variables control what you see and how fast the shell responds to your keystrokes. Milliseconds matter when your fingers are moving at light speed.

\$ZPWR_EXA_COMMAND – The ls Replacement

```
echo $ZPWR_EXA_COMMAND # exa or eza
```

Controls which modern ls replacement ZPWR uses. exa is the original, eza is the maintained fork. Both give you colored, git-aware directory listings.

ZPWR verbs like zpwr ls and zpwr clearls use this. Set it in your tokens file to override the default.

\$ZPWR_KEYTIMEOUT – Keystroke Latency

```
echo $ZPWR_KEYTIMEOUT # in hundredths of a second
```

How long zsh waits for the next key in a multi-key sequence. Lower = snappier. Too low = missed sequences. Default is tuned for fast typists.

If vi-mode feels laggy on Esc, lower this. If key combos misfire, raise this. Sweet spot is personal. Experiment.

\$ZPWR_EXPAND_PRE_EXEC – Alias Expansion

```
echo $ZPWR_EXPAND_PRE_EXEC
```

Controls whether ZPWR expands aliases before execution. When enabled, you type the alias and see the full command in your history. Transparency mode.

With expansion: you type ga -> history shows git add Without expansion: you type ga -> history shows ga.

\$ZPWR_SEND_KEYS_FULL – tmux Key Passthrough

```
echo $ZPWR_SEND_KEYS_FULL
```

Controls full send-keys behavior in tmux integration. When ZPWR manages tmux panes, this determines how keystrokes are forwarded to target panes.

TUNING GUIDE: Feeling laggy? Lower ZPWR_KEYTIMEOUT Missing combos? Raise ZPWR_KEYTIMEOUT Want transparency? Enable ZPWR_EXPAND_PRE_EXEC Using eza? Set ZPWR_EXA_COMMAND=eza.

// the interface is not decoration. it is velocity.

// every millisecond of latency is a papercut.

// tune these until your shell feels like thought.

Runtime State

Runtime State – The Machine’s Self-awareness

The shell knows what it is. do you?

These variables hold runtime state — what OS you are on, what TTY you are using, where logs go. They are set at startup and consulted throughout the session.

\$ZPWR_TTY – Your Terminal Identity

```
echo $ZPWR_TTY    # e.g., /dev/ttys003
```

The TTY device for the current shell session. Used by ZPWR to track which terminal is which, especially when managing multiple tmux panes.

The `zpwr tty` verb shows your current TTY info.

\$ZPWR_OS_TYPE – Platform Detection

```
echo $ZPWR_OS_TYPE    # Darwin, Linux, etc.
```

Detected at startup. Used throughout ZPWR to branch on platform-specific behavior. macOS vs Linux vs BSD, the right code path fires automatically.

This is why ZPWR works on macOS and Linux without you changing a single setting. The OS type decides.

\$ZPWR_LOGFILE – The Black Box

```
echo $ZPWR_LOGFILE
```

Where ZPWR writes its log output. Debug, info, error, trace — all levels go here when enabled.

Read the logs:

```
zpwr loginfo      info-level messages
zpwr logdebug     debug-level messages
zpwr logerror     error-level messages
zpwr logtrace     trace-level messages
zpwr taillog      live tail with colored output
```

\$ZPWR_COMPSYS_CACHE – Completion Cache

```
echo $ZPWR_COMPSYS_CACHE
```

Controls the completion system cache. Zsh completions are expensive to regenerate. This flag keeps them cached.

\$ZPWR_HISTFILE – History File

```
echo $ZPWR_HISTFILE
```

Your shell history file location. ZPWR manages history with deduplication and backup. Your commands are sacred.

Related: `zpwr backuphistory` and `zpwr restorehistory`.

THE FULL ENV VAR MAP: Core paths: ZPWR, ZPWR_LOCAL, ZPWR_SCRIPTS, ZPWR_TMUX Autoload: ZPWR_AUTOLOAD, ZPWR_AUTOLOAD_COMMON Flags: ZPWR_PROFILING, ZPWR_DEBUG, ZPWR_TRACE Display: ZPWR_EXA_COMMAND, ZPWR_KEYTIMEOUT Runtime: ZPWR_TTY, ZPWR_OS_TYPE, ZPWR_LOGFILE.

// the machine knows itself. every variable is a mirror.

// read them to understand the current state of reality.

Color Systems – Painting The Void

256 colors. true color. the terminal is not monochrome.

Color is not cosmetic. It is information density. A colored diff tells you more in one glance than a monochrome one does in ten seconds. ZPWR gives you tools to explore, test, and master terminal color.

Colortest – The 16-COLOR Palette

```
zpwri colortest
```

Displays the basic 16-color ANSI palette. The foundation. Black, red, green, yellow, blue, magenta, cyan, white — plus their bright variants. 16 soldiers in formation.

Use this to verify your terminal theme is sane. If red looks orange, your theme needs fixing.

Colortest256 – The Full Spectrum

```
zpwri colortest256
```

Displays all 256 extended colors. A grid of every color code your terminal supports. From 0 to 255. Useful for picking colors for prompts and scripts.

“What number is that teal?” Run this and find out.

Pygmentcolors – Syntax Highlighting Palette

```
zpwri pygmentcolors
```

Shows available Pygments color schemes. Pygments is the syntax highlighting engine used by many ZPWR tools. Pick a scheme that matches your aesthetic.

Prettyprint & Altprettyprint – Formatted Output

```
zpwri prettyprint <file>  
zpwri altprettyprint <file>
```

Syntax-highlighted file display. prettyprint uses one engine, altprettyprint uses another. Both turn raw source code into technicolor glory in your terminal.

Like cat, but for people who respect their eyes.

COLOR EXPLORATION WORKFLOW: 1. zpwri colortest check basic 16 colors 2. zpwri colortest256 explore full 256 palette 3. zpwri pygmentcolors pick a syntax theme 4. zpwri prettyprint view files in color.

// monochrome terminals are a war crime

// color is data. learn to read it.

// 256 colors and you are still using white-on-black?

Script Management

Script Management – The Stryke

Create scripts. count scripts. print scripts to PDF. ship scripts.

ZPWR manages its own script collection and gives you tools to create new ones, inventory them, and even export them to PDF for documentation or review.

Scriptnew – The Script Generator

```
zpwr scriptnew <name>
```

Creates a new script with proper boilerplate. Shebang, error handling, argument parsing — all pre-wired. Stop writing boilerplate. Start writing logic.

The script lands in `$ZPWR_SCRIPTS`, already executable, already on your PATH. Zero friction from idea to tool.

Scripttopdf – The Documenter

```
zpwr scripttopdf
```

Converts scripts to syntax-highlighted PDFs. For code reviews, documentation, or printing out and framing on your wall like the art it is.

Send your code to a printer. Assert dominance.

Scriptcount – The Census

```
zpwr scriptcount
```

How many scripts live in `$ZPWR_SCRIPTS`? This tells you. Track the growth of your personal armory over time.

Scriptlist – The Inventory

```
zpwr scriptlist
```

Lists every script in the collection. Names, paths, the full manifest of your executable arsenal.

THE SCRIPT LIFECYCLE: 1. `zpwr scriptnew myutil` create the script 2. `zpwr vimscriptedit` edit scripts with vim/fzf 3. `zpwr scriptcount` check your inventory 4. `zpwr scriptlist` browse the collection 5. `zpwr scripttopdf` export for documentation.

Also Related:

```
zpwr mkdir <name>      create a directory
zpwr makefile <name>   create a file
```

Simple creation verbs. No boilerplate, just `mkdir/touch` with ZPWR's logging and validation baked in.

// a script unwritten is a problem unsolved

// a script uncounted is a tool forgotten

// the stryke never closes. the armory always grows.

The Matrix

// not cmatrix. this is YOUR shell, raining. //

zpwr matrix is an animated cyberpunk hacker story followed by an infinite ecosystem rain screensaver. Unlike cmatrix, it rains YOUR data — verb names, function names, aliases, env vars, and git repo names. Color-coded by type.

The Story

ACT I The Signal — the terminal underground survives ACT II The Discovery — curl | zsh changes everything ACT III The Power — bench, flame, doctor, gitwho ACT IV The Choice — VS Code or zpwr matrix? Between acts: fake SSH sessions, encrypted transmissions, payload downloads with glitchy progress bars.

The Fx

CRT scanlines green sweep top-to-bottom VHS tracking horizontal distortion bands Pixel corruption spreading block static Screen tears offset colored bands Figlet chaos random fonts, random words, dissolve Progress bars with random glitch spikes Pulse events ecosystem facts flash across screen Scrolling ticker horizontal stat crawl HUD overlay live verb/alias/func/repo counters.

The Rain Colors

Green verbs cyan functions yellow aliases magenta env vars red git repos.

Usage

```
zpwr matrix          # full story + rain
zpwr matrix -i       # skip story, rain only
zpwr matrix -s 7     # faster (1-9)
zpwr rain            # alias
zpwr screensaver     # alias
```

Fortune Cookies

// wisdom from the digital underground //

zpwr fortune displays a random hacker quote rendered in a random figlet font with neon colors. 40 curated quotes from legends and original ZPWR wisdom.

The Verb

```
zpwr fortune         # random quote, random font
zpwr fortune -p      # plain text with box frame
zpwr fortune -f doom # force doom font
zpwr wisdom          # alias
zpwr quote           # alias
```

The Quotes

Linus Torvalds “Talk is cheap. Show me the code.” Brian Kernighan “Controlling complexity is the essence...” Grace Hopper “The most dangerous phrase is...” ZPWR “Home is where the .zshrc is.” MenkeTechnologies “Aliases are love letters to your future self.” ...and 35 more.

The Fonts

Randomly selected from: cyberlarge, cybermedium, doom, slant, speed, banner, standard. Each quote picks a keyword for large figlet rendering, full quote below.

Use Cases

- Add to .zshrc for a fortune on every new shell - Pipe to lolcat: `zpwr fortune -p | lolcat` - Use as a tmux status-right: too slow, but fun to try

Matrix: The Full Experience

8 acts, 6 characters, and your ecosystem raining down

The Story

```
zpwr matrix          # 4-act story + rain
zpwr matrix --extended # full 8-act saga
```

The Characters

```
zpwr matrix --cast      # show character bios
```

```
Null Pointer .... protagonist, zpwr operator
Segfault ..... bash loyalist in recovery
Chmod777 ..... reckless, sudo for everything
```

```
DevNull ..... silent assassin
```

Chad Electron ... MegaCorp VP, 64GB for Slack

```
The Kernel ..... mythical elder, speaks in dmesg
```

Visual Fx

```
zpwr matrix --noir      # monochrome green
zpwr matrix --vhs       # VHS tracking distortion
zpwr matrix --crt       # CRT scanlines
zpwr matrix --glitch    # extra corruption
zpwr matrix --sound     # terminal bells
zpwr matrix --no-fx     # disable all FX
zpwr matrix -c 10       # chaos level 1-10
```

Playback Control

```
zpwr matrix -s 7        # speed 1-9
zpwr matrix --turbo     # skip all pauses
zpwr matrix -a 5        # jump to act 5
zpwr matrix -r          # skip story, rain only
```

Multiplayer

```
zpwr matrix --battle      # 2-player tmux typing battle
```

Player 1 (Null Pointer) vs Player 2 (Chad Electron). Type zpwr commands faster than your opponent. HP bars, taunts, and a winner-takes-all endgame. Requires tmux. Auto-splits into two panes.

Info

```
zpwr matrix --stats      # ecosystem stats  
zpwr matrix --credits    # end credits roll
```

Chapter 23 – Exercism & External

Exercism Integration

Exercism Integration – Level Up Your Code

Practice makes perfect. ZPWR makes practice frictionless.

Exercism.org is a free coding practice platform with mentor-reviewed exercises in 50+ languages. ZPWR wraps the exercism CLI so you never leave your terminal.

Exercism – The Launcher

```
zpwr exercism
```

The base exercism command integration. Manages your exercism workspace, configuration, and submission workflow from within ZPWR.

Your exercism workspace lives under `$ZPWR` and is accessible via `zpwr homeexercism` for quick navigation.

Exercismdownload – Fetch Exercises

```
zpwr exercismdownload
```

Downloads an exercise to your local workspace. The exercise appears, ready to solve. Tests included. No browser clicking. No manual downloads.

The workflow: 1. Browse exercises on exercism.org 2. `zpwr exercismdownload` to fetch it 3. Solve the exercise in your editor 4. Run the tests locally 5. Submit when you pass.

Exercismedit – Solve In Your Editor

```
zpwr exercismedit
```

Opens exercism exercises directly in your configured editor. Download, edit, test, submit — all from the terminal. The browser is optional.

WHY EXERCISM + ZPWR:

The exercism CLI is good. ZPWR makes it seamless. No context switching. No browser tabs. No friction.

Languages supported by exercism: Python, Go, Rust, Elixir, JavaScript, TypeScript, C, C++, Java, Ruby, Haskell, Clojure, Kotlin, Swift, Bash, and 35+ more.

// coding practice should be as easy as

// opening a terminal and typing three words

// zpwr exercismdownload. done. now solve.

Travis CI & External Services

CI/CD from your terminal, not your browser.

ZPWR integrates with external services so you can monitor builds, check statuses, and manage CI/CD pipelines without opening a single browser tab.

Travis – The Ci Dashboard

```
zpwr travis
```

Travis CI integration. View build status, logs, and pipeline information from your terminal.

Travisbuild – Build Status

```
zpwr travisbuild
```

Check the current build status. Green, red, or pending. No need to refresh a web page in a loop.

Travisbranch – Branch Builds

```
zpwr travisbranch
```

View build status for a specific branch. When you push to a feature branch and want to know if CI passed.

Travispr – Pull Request Builds

```
zpwr travispr
```

Check build status for pull requests. The merge button is only as good as the CI behind it.

Other External Integrations:

```
zpwr goclean      clean Go build cache
zpwr google <query>  search Google from terminal
zpwr digs <domain>   DNS lookups
zpwr torip         check your Tor IP address
zpwr toriprenew     renew your Tor circuit
zpwr curl          enhanced curl operations
```

THE EXTERNAL SERVICES PHILOSOPHY: The browser is a context switch. Every tab you open is a distraction vector. ZPWR brings the services to you, in the terminal, where you already are.

// the terminal is the cockpit

// external services are instruments on the dashboard

// you read them. you do not visit them.

Chapter 24 – Tips, Tricks & Easter Eggs

The One-Letter Verbs

The One-letter Verbs – Maximum Power, Minimum Keystrokes

Six letters that do the work of six paragraphs.

郑WR has 460 verbs. Six of them are single characters. These are not abbreviations — they are the distilled essence of the most common operations. One keystroke each. The speed limit of human-shell interaction.

A – Autoload

```
郑WR a
```

Quick access to the autoload system. When you need to check on autoloaded functions fast.

C – cat

```
郑WR c
```

The cat operation. View file contents. The most basic read operation, compressed to one letter.

E – Edit

```
郑WR e
```

Open your editor. The gateway to modification. Uses your configured \$EDITOR.

F – Filesearch

```
郑WR f
```

File search. Find things. The single most common operation after listing and editing.

R – Replay

```
郑WR r
```

Replay operations. Redo what you did before. History is not just for reading.

Z – Zcd

```
zpwr z
```

Z-jump. Frequency-based directory navigation. Type a partial path, z figures out the rest. The single most addictive feature in ZPWR.

THE ONE-LETTER ARSENAL: a autoload system c cat / view files e edit files f file search r replay z z-jump directory navigation.

// every extra character is latency between thought and action

// one letter is the event horizon of efficiency

// below this, there is only telepathy

Alias Chains

Alias Chains – Compound Operations

One verb triggers another. the cascade begins.

ZPWR verbs do not exist in isolation. They chain. They compose. They call each other. This is not an accident — it is the architecture.

The Shorthand Aliases:

ZPWR is aliased to shorter forms for speed:

```
zpwr  the full command
zp     shorter
zpz    alternate short form
```

All three are equivalent:

```
zpwr help == zp help == zpz help
```

Compound Verb Patterns:

Many verbs are compounds that do two things:

Filesearch + edit = zpwr filesearchedit findsearch + edit = zpwr findsearchedit locatesearch + edit = zpwr locatesearchedit wordsearch + edit = zpwr wordsearchedit vimrecent + cd = zpwr vimrecentcd vimrecent + sudo = zpwr vimrecentsudo.

The pattern: action + modifier = compound verb Search, then edit. Browse, then cd. Open, then sudo.

The Clean Chain:

```
zpwr cleanall      clean everything
zpwr cleancache    clean caches only
zpwr cleancompcache clean completion cache
zpwr cleangitcache  clean git repo cache
zpwr cleangitcleancache clean git clean cache
zpwr cleangitdirtycache clean git dirty cache
zpwr cleantemp      clean temp files
zpwr cleanrefreshupdate clean + refresh + update
```

The Git Chain:

```
zpwr gitforalldir    run git cmd in all dirs
zpwr gitfordirmain   run git cmd in dirs on main
zpwr gitzfordir      z-style git for all dirs
```

// verbs are words. chains are sentences.

// the grammar of ZPWR is combinatorial.

// learn the words, and the sentences write themselves.

The Token System

The Token System – Your Private Override Layer

Secrets, aliases, and local config that never touch git.

ZPWR has sensible defaults. But you are not sensible. You have API keys, custom aliases, machine-specific paths, and opinions. The token system is where all of that lives — safely outside version control.

The Token Files:

`$ZPWR_LOCAL/.tokens.sh` Your main tokens file. Aliases, exports, secrets. This is sourced during shell init. Gitignored.

`$ZPWR_LOCAL/.tokens-pre.sh` Sourced BEFORE ZPWR's main configuration. Override defaults before they are set. Set `ZPWR_*` variables here to change behavior.

`$ZPWR_LOCAL/.tokens-post.sh` Sourced AFTER ZPWR's main configuration. Override anything ZPWR set. Final word. Your aliases here trump everything.

The Override Pattern:

Want to change a ZPWR default? Do not edit ZPWR source. Put it in `tokens-pre.sh` (before) or `tokens-post.sh` (after).

Example: override in `tokens-pre.sh`.

```
export ZPWR_EXA_COMMAND=eza
export ZPWR_KEYTIMEOUT=1
export ZPWR_BANNER_CLEARLIST=false
```

Example: add custom aliases in `tokens-post.sh`.

```
alias k=kubectl
alias tf=terraform
export GITHUB_TOKEN=ghp_XXXXX
```

Managing Tokens:

```
zpwr tokens          view your tokens files
zpwr vimtokens        edit tokens in vim
zpwr emacsstones      edit tokens in emacs
```

THE LOAD ORDER: 1. `tokens-pre.sh` (your early overrides) 2. ZPWR core config (the framework defaults) 3. `tokens-post.sh` (your final overrides) 4. `.tokens.sh` (your secrets and aliases)

NEVER commit token files. They are gitignored for a reason. API keys in git history haunt you forever.

```
// the token system is your private dimension
// ZPWR updates without touching your config
// your overrides survive every git pull
```

ARB Everywhere

Arb Everywhere – The Fuzzy Finder Conspiracy

Arb -fzf is not a tool. it is a religion. ZPWR is its cathedral.

Arb -fzf (fuzzy finder) is woven into the fabric of ZPWR. Dozens of verbs pipe their output through arb -fzf for interactive selection. If a verb ends in 'arb -fzf', it is the fuzzy-interactive variant. But many others use it too.

Dedicated Arb Verbs:

```
zpwr verbs    browse subcommands in arb --fzf
zpwr verbs    browse all verbs in arb --fzf
```

Verbs That Use Arb Internally:

Forgit* verbs: forgitadd, forgitdiff, forgitlog, forgitstash, forgitreset, forgitrestore, forgitclean All use arb -fzf for interactive selection with previews.

Search + edit verbs: filesearchedit, findsearchedit, locatesearchedit, wordsearchedit — search with arb -fzf, open selection.

Recent file verbs: vimrecent, vimrecentcd, emacsrecent, emacsrecentcd Browse recent files with arb -fzf preview.

Editor verbs: vimalledit, emacsalledit — select files to edit.

Git tag verbs: vimgtagsedit, emacsgtagsedit — browse tags in arb -fzf.

The Arb Pattern:

The pattern is always the same: 1. Generate a list of candidates 2. Pipe through arb -fzf with a preview command 3. Take the selection and act on it.

This pattern turns every list into a menu. Every menu into a decision. Every decision into action.

Arb Customization:

Set FZF_DEFAULT_OPTS in your tokens file:

```
export FZF_DEFAULT_OPTS='--height 80% --reverse --border'
```

```
// if you can list it, you can arb -fzf it
```

```
// if you can arb -fzf it, you can act on it
```

```
// arb -fzf is the universal adapter between data and decisions
```

Alias Cheat Sheet

// the short names you will actually use //

Many zpwr verbs have short aliases. These are the ones worth memorizing — one or two keystrokes that replace entire workflows.

Diagnostics

```
zpwr doc      = doctor      zpwr zprof      = startup
zpwr check    = lint        zpwr audit       = pathaudit
zpwr cruft    = stale       zpwr rank       = funcrank
zpwr benchmark = bench      zpwr dashboard = top
zpwr heatmap  = flame       zpwr instrument = trace
zpwr flamechart = flame     zpwr checkup   = doctor
zpwr callers  = deps        zpwr aliasusage = aliasrank
```

Monitoring

```
zpwr snap      = snapshot   zpwr load       = restore
zpwr rerun     = replay     zpwr onchange   = watch
zpwr book      = study
```

Creative

```
zpwr rain      = matrix     zpwr wisdom     = fortune
zpwr quote     = fortune    zpwr screensaver = matrix
```

Git

```
zpwr who       = gitwho     zpwr gitcontri= gitwho
```

Introspection

```
zpwr which     = resolve    zpwr whatis     = resolve
zpwr today     = timeline   zpwr activity    = timeline
```

Logging

```
zpwr logs      = taillog
```

Chapter 25 – The Complete Reference

All Verbs A-G

All 460 Verbs: A Through G (1/3)

The complete arsenal. memorize at your own peril.

A quick autoload access about display ZPWR about info alacrity alacrity terminal config aliasrank rank aliases by usage frequency all show all ZPWR info allsearch search across all sources altprettyprint alternate syntax-highlighted print animate terminal animations attach attach to tmux session autoload show autoload system autoloadcount count autoloading functions autoloadlist list all autoloading functions background run command in background backup backup ZPWR configuration backuphistory backup shell history banner display ZPWR banner bannercounts banner with counts bannerlolcat rainbow banner via lolcat bannerno pony banner without ponysay bannerpony banner with ponysay bench benchmark shell startup brc source bashrc c cat / view file cat enhanced cat catcd cat file and cd to its dir catnviminfo recentf cat nvim info recent files catrecentviminfo cat recent files from viminfo catviminfo cat viminfo file cd enhanced cd cdup cd up one directory cfasd cd via fasd frequency changename rename files interactively changenamezpwr rename ZPWR components cleanall clean everything cleancache clean caches cleancompcache clean completion cache color2 turn on stderr color filter environmentvariables list zpwr env vars (alias: envvars) environmentvariablesall list all env vars (alias: envvarsall) environmentvars list zpwr env vars (alias: envvars) environmentvarsall list all env vars (alias: envvarsall)

... continued. Run zpwr verbs for the complete list.

All Verbs H-S

All 460 Verbs: Forgitdiff Through Recompilepath (2/3)

The middle of the alphabet is where the real work lives.

Forgitdiff interactive diff via forgit forgitignore browse gitignore templates forgitignoreclean clean gitignore cache forgitignoreget fetch gitignore template forgitignorelist list gitignore templates forgitignoreupdate update gitignore cache forgitinfo repo info dashboard forgitlog interactive git log via forgit forgitreset interactive unstage via forgit forgitrestore interactive restore via forgit forgitstash interactive stash via forgit forgitwarn warn about repo hazards forked manage forked repos forward port forwarding fp fpath info funcrank rank functions by usage fwd forward shortcut gh GitHub CLI integration ghcontribcount GitHub contribution count ghz GitHub + z integration gitaemail set git author email gitcemail set git committer email gitclearcache clear git cache gitclearcommit clear a git commit gitclearfile clear file from git gitcommit enhanced git commit gitcommitcount count commits gitcommits browse commits gitconfig show git config gitcontribcount count contributions gitcontribcountdirs count contribs per dir gitcontribcountlines count contributed lines gitdir show git directory gitedittag edit a git tag gitemail show git email gitforalldir git cmd in all dirs hometests cd to test directory (alias: hometest) loadjenv lazy load jenv logincount count logins by user regenpdf regenerate encyclopedia PDF subcommandsedit edit subcommand selection subcommandslist list all subcommands.

... continued. Run zpwr verbs for the complete list.

All Verbs T-Z

All 460 Verbs: Refreshzwc Through Zstyle (3/3)

The final stretch. the end of the alphabet. the beginning of mastery.

Refreshzwc refresh compiled wordcode regen regenerate single component regenall regenerate everything regenconfiglinks regenerate config symlinks regenctags regenerate ctags regenenvcache regenerate env cache regengitdirtyrepocache regen dirty repo cache regengitrepocache regen git repo cache regengtags regenerate gtags regengtagspygments regen gtags + pygments regengtagstype regen gtags by type regenhistory regenerate history regenkeybindings regen keybindings regenpowerline regenerate powerline regenzsh regenerate zsh config reload reload shell config relpath get relative path rename rename files replacer find and replace replay replay commands repo repository info reset reset shell state restart restart shell restore restore configuration restorehistory restore history backup return2 return to previous dir reveal open repo in browser revealrecurse reveal recursively reverse reverse input rmconfiglinks remove config symlinks rvs reverse shortcut scriptcount count scripts scriptlist list all scripts scriptnew create new script scripts browse scripts scripttopdf convert script to PDF testsall run all env tests (alias: testall) upload upload file with curl verbsedit edit verb selection zpwrCloneToForked clone zpwr to forked dir zpwrgithub open zpwr GitHub page.

Run zpwr verbs for the complete live list.

Chapter 26 – powerlevel10k & Instant Prompt

Instant Prompt: Zero Latency

// your prompt loads before .zshrc finishes. time is a flat circle. //

The single best UX trick in zpw: the prompt appears INSTANTLY while .zshrc is still loading behind the curtain. Zero perceived latency. The illusion of speed IS speed.

How It Works

On previous shell exit, p10k cached the prompt to a file:

```
${XDG_CACHE_HOME:-$HOME/.cache}/p10k-instant-prompt-${(%):-%n}.zsh
```

This file contains the last rendered prompt as raw ANSI escape sequences. On next shell start, it is sourced BEFORE .zshrc loads. Result: the prompt appears in <10ms. The user can start typing. .zshrc continues loading in the background. Nobody notices.

The Loading Sequence In .zshrc

1. zmodload zsh/datetime (timestamp)
2. Source zpw-hash-dirs.zsh (cached hash -d entries)
3. Source p10k-instant-prompt-*.zsh (the magic file)
4. promptTimestamp=\$EPOCHREALTIME (measure prompt time)
5. ...rest of .zshrc loads (plugins, aliases, etc)...
6. p10k finalize (at very end of .zshrc)

Step 3 must come before ANY console output. If something prints to stdout before the instant prompt, p10k aborts it.

The Hash Dir Cache

The prompt uses %~ to show named dirs like ~ZPWR instead of /Users/you/.zpw. But hash -d entries are set in zpwBindDirs, which runs AFTER instant prompt. Chicken-and-egg problem. Solution: zpwBindDirs writes a cache file:

```
$ZPWR_LOCAL/zpw-hash-dirs.zsh
```

This cache is sourced at line 73 of .zshrc, BEFORE instant prompt, so %~ resolves ~ZPWR in the cached prompt correctly.

Zpwr_banner_clearlist Suppression

ZpwrClearList prints a banner + file listing on login. But if it runs before instant prompt finalizes, it corrupts the display. Fix: the banner is deferred until AFTER p10k finalize:

```
if [[ -n "$__p9k_instant_prompt_active" && "$ZPWR_BANNER_CLEARLIST" == true ]]; then
  zpwrClearList || true
fi
```

The Prompttimestamp Measurement

ZPWR_VARS[promptTimeMs] records how fast the prompt appeared. Run `zpwr environmentcounts` to see it. With instant prompt, this is typically 0.010-0.050 seconds. Without it, 0.5-2.0s.

P10k Finalize

The last line of `.zshrc`:

```
(( ! ${+functions[p10k]} )) || p10k finalize
```

This tells p10k that `.zshrc` is done. The real prompt replaces the cached one. Any deferred output can now safely print.

Powerlevel10k Configuration

// the prompt is your cockpit HUD. configure it or fly blind. //

The p10k config lives at `$ZPWR_ENV/p10k.zsh` – a 1700+ line file that controls every pixel of your prompt. The wizard generated it, but you own it now.

Left Prompt Segments

These appear on the left side, top to bottom across 4 lines:

```
Line 1: custom_env, custom_env_bat, custom_env_editor
Line 2: custom_env_term, ip
Line 3: context, dir, history, ssh, root_indicator, status, vcs
Line 4: custom_tty, prompt_char
```

The `dir` segment shows your current directory using `%~` expansion. The `vcs` segment shows git branch, dirty status, ahead/behind. The `prompt_char` shows green on success, red on failure.

Right Prompt Segments

The right side is information-dense. Across 3 lines:

```
Line 1: dir_writable, background_jobs, virtualenv, node_version,
```

`go_version, rust_version, java_version, package, os_icon...`

```
Line 2: load, disk_usage, ram, swap, wifi
Line 3: battery, time, command_execution_time, vi_mode, custom_pid
```


The Dir Segment And %~ Expansion

Zsh's %~ prompt expansion replaces known paths with named dirs. If you hash -d ZPWR=/Users/you/.zpw, then /Users/you/.zpw shows as ~ZPWR in your prompt. Shorter. Cleaner. Professional. POWERLEVEL9K_DIR_MAX_LENGTH=80 caps the dir segment width. POWERLEVEL9K_DIR_MIN_COMMAND_COLUMNS=40 reserves typing space.

How Zpwrbinddirs Populates Named Directories

The function runs hash -d for ~20 directories:

```
hash -d ZPWR="$ZPWR"
hash -d ZPWR_TMUX="$ZPWR_TMUX"
hash -d ZPWR_SCRIPTS="$ZPWR_SCRIPTS"
hash -d TMUXH="$TMUX_HOME"
```

Then it writes them all to \$ZPWR_LOCAL/zpwr-hash-dirs.zsh for instant prompt to use on next startup. The cache iterates \${(kv)nameddirs} to capture every registered named directory.

The Multiline Frame

The prompt uses box-drawing characters for the frame:

```
First line:  %244F----- (top connector)
Middle:      %244F----- (mid connector)
Last line:   %244F----- (bottom connector)
```

Gap fill character is '—' in grey (color 244). Mode: nerdfont-complete. Separators use Powerline glyphs.

Prompt_char Symbols

Insert mode: — Command mode: — Visual: V Overwrite: —

P10k Prompt Customization

// your prompt is a canvas. paint it with ANSI and Unicode. //

ZPWR ships custom p10k segments beyond the defaults. These are defined with POWERLEVEL9K_CUSTOM_* variables in \$ZPWR_ENV/p10k.zsh.

Custom Segments

Custom_tty Shows \$ZPWR_TTY (e.g. /dev/ttys003) Defined as:

```
POWERLEVEL9K_CUSTOM_TTY='echo "%K{blue}%F{white} $ZPWR_TTY %f%k%f{blue}%f"'
```

Custom_env Environment info (custom shell, OS, etc.) custom_env_bat Shows bat (syntax highlighter) version/status custom_env_editor Shows \$EDITOR custom_env_term Shows \$TERM_PROGRAM custom_pid Shows shell PID (right prompt, bottom line)

Adding Your Own Segment

1. Define the command that generates output:

```
typeset -g POWERLEVEL9K_CUSTOM_MYWIDGET='echo "hello"'
```

2. Set colors:

```
typeset -g POWERLEVEL9K_CUSTOM_MYWIDGET_FOREGROUND=255
typeset -g POWERLEVEL9K_CUSTOM_MYWIDGET_BACKGROUND=24
```

3. Add 'custom_mywidget' to LEFT or RIGHT PROMPT_ELEMENTS array. 4. Run p10k reload or source \$ZPWR_ENV/.p10k.zsh to apply.

Colors And Icons

P10k uses 256-color indices. Print the color map:

```
for i in {0..255}; do print -Pn "%K{$i} %k%F{$i}${(l:3::0:)i}%f "; done
```

Foreground/background for each segment and state:

```
POWERLEVEL9K_DIR_BACKGROUND=4          (blue)
POWERLEVEL9K_DIR_FOREGROUND=254        (near-white)
```

Anchor dirs (like .git roots) get bold + brighter color (255).

Truncation

POWERLEVEL9K_DIR_MAX_LENGTH=80 caps directory width. POWERLEVEL9K_DIR_MIN_COMMAND_COLUMNS_PCT=50 means at least half the terminal width is reserved for your command input. SHORTEN_STRATEGY can be set to truncate_to_unique for smart shortening (commented out by default in zpwr).

The .p10k.zsh File Structure

The file is a single anonymous function () { ... } that: - Unsets all POWERLEVEL9K_* vars (clean slate) - Defines LEFT/RIGHT_PROMPT_ELEMENTS arrays - Sets mode, icons, colors, truncation per segment - Configures transient prompt, instant prompt, hot reload

Seeing Prompt Timing

```
zpwr environmentcounts
```

Shows promptTimeMs, startupTimeMs, and per-phase timestamps. This is how you prove your prompt loaded in 0.015 seconds.

Transient Prompt & Visual Tweaks

// old prompts fade into ghosts. only the present matters. //

Transient prompt collapses previous prompts to a minimal form, keeping your scrollback clean and focused. In zpwr it is currently set to 'off' but easy to enable.

Transient Prompt

When enabled, after you press Enter, the full multi-line prompt collapses to just the prompt_char symbol (— or —). All segments, git status, directory — gone from old lines. Only the current prompt shows full detail.

```
typeset -g POWERLEVEL9K_TRANSIENT_PROMPT=off
```

Change to always or same-dir to enable. 'same-dir' only collapses when you stay in the same directory.

Hot Reload

Edit .p10k.zsh and apply changes without restarting zsh:

```
p10k reload
```

This re-sources .p10k.zsh and rebuilds the prompt in-place. No exec zsh needed. No losing your command history buffer.

P10k Configure

The interactive configuration wizard. Walks you through: - Font detection (powerline glyphs, nerdfont icons) - Style selection (rainbow, lean, classic, pure) - Separator shapes, line count, frame options - Instant prompt mode (verbose, quiet, off) It generates a fresh .p10k.zsh. ZPWR ships a pre-configured one in `$ZPWR_ENV/.p10k.zsh` (rainbow style, nerdfont-complete). Running the wizard overwrites it, so back up yours first.

Instant Prompt Vs Zpwrclist

The banner (zpwrClearList) prints colorized file listings on login. Instant prompt redirects all stdout/stderr during init to a buffer. If zpwrClearList runs during this window, the output gets swallowed or garbled. ZPWR's fix: 1. Check if `__p9k_instant_prompt_active` is set 2. If so, defer zpwrClearList to AFTER p10k finalize 3. The banner appears cleanly after the prompt settles.

The eza --classify=always Fix

Under instant prompt, fd/stdout gets redirected. eza's `--classify` flag (append `*/` to dirs) auto-detects terminal capabilities. When stdout is redirected, it thinks there's no terminal and drops the indicators. Fix: use `--classify=always` to force classification regardless of fd state. ZPWR_EXA_COMMAND uses this:

```
command eza --git -il --classify=always -H --color-scale -g -a
```

\$TTY Immune To fd Redirects

Zsh sets \$TTY before rc files load. Instant prompt redirects fds 0/1/2, but \$TTY still points to the real terminal. ZPWR captures it: `export ZPWR_TTY=$TTY` This is used by the custom_tty prompt segment.

Chapter 27 – zinit & Plugin System

Zinit: The Plugin Engine

// the package manager for your shell. apt-get for zsh plugins. //

Zinit (formerly zplugin) is a flexible zsh plugin manager with turbo mode, ice modifiers, and completion management. ZPWR uses it as the default: `ZPWR_PLUGIN_MANAGER=zinit`.

What zinit Does

- Downloads GitHub plugins to `$ZPWR_PLUGIN_MANAGER_HOME` - Sources them into your shell (or loads as completions) - Manages load order with turbo mode (deferred loading) - Handles OMZ snippets (individual files from oh-my-zsh) - Compiles plugins to .zwc bytecode for speed

zinit Load Vs zinit Snippet

zinit load user/repo Clone full GitHub repo, source .plugin.zsh zinit snippet OMZP::git Download single file from OMZ plugin zinit snippet OMZL::git.zsh Download single OMZ lib file load = full repo clone. snippet = single file download. ZPWR uses both: load for GitHub plugins, snippet for OMZ.

The Plugin Arrays

ZPWR organizes plugins into arrays in .zshrc: `ZPWR_GH_PLUGINS` 28+ GitHub repos (fzf-tab, zsh-z, forgit...) `ZPWR_OMZ_PLUGINS` 25+ OMZ snippets (git, python, go lang...) `ZPWR_OMZ_LIBS` 4 OMZ lib files (git.zsh, directories.zsh...) `ZPWR_OMZ_COMPS` 12 OMZ completion files (ng, coffee, scala...) Conditional plugins are appended if the command exists:

```
zpwrcommandExists docker && ZPWR_GH_PLUGINS+=( zsh-docker-aliases )
```

Plugin Storage

Everything lives under `$ZPWR_PLUGIN_MANAGER_HOME`:

```
$ZPWR_PLUGIN_MANAGER_HOME/bin/      zinit itself
$ZPWR_PLUGIN_MANAGER_HOME/plugins/   cloned repos
$ZPWR_PLUGIN_MANAGER_HOME/snippets/  OMZ snippets
$ZPWR_PLUGIN_MANAGER_HOME/completions/ symlinked completions
```

The 48+ Plugin Roster

Total plugins managed by zpwrc (from .zshplugins): MenkeTechnologies forks: fzf-tab, forgit, zsh-expand, zsh-autocomplete, zsh-z, fasd-simple, revolver, zunit... zdharma-continuum: fast-syntax-highlighting, zbrowse, zui... zsh-users: completions, autosuggestions, history-substring Plus: hlissner/zsh-autopair, bigH/git-fuzzy, and more.

First Run Bootstrap

If `$ZPWR_PLUGIN_MANAGER_HOME/bin` doesn't exist, `.zshrc` auto-clones zinit from GitHub. After that, `OPTIMIZE_OUT_DISK_ACCESSES=1` skips unnecessary `stat()` calls on subsequent starts.

Turbo Mode: Deferred Loading

// why load 48 plugins at startup when you can load 1? //

Turbo mode is zinit's killer feature: plugins load AFTER the prompt appears, using zsh's sched mechanism. Your shell is interactive instantly; plugins trickle in behind the scenes.

The Wait Ice Modifier

The 'wait' ice defers loading until after the first prompt:

```
zinit ice wait          Load after prompt (default priority)
zinit ice wait'0'       Same as wait (priority 0)
zinit ice wait'0a'      Priority 0, sub-order a (loads first)
zinit ice wait'0b'      Priority 0, sub-order b (loads second)
zinit ice wait'0e'      Priority 0, sub-order e (loads later)
```

The letter suffix controls load order WITHIN the same priority. a loads before b loads before c, etc. This is how ZPWR orchestrates 48 plugins into a precise loading sequence.

The Graduated Loading Sequence

Synchronous: `powerlevel10k` (must load before prompt) wait: OMZ completions, libs, plugins, GH plugins wait'0': `zsh-autopair` (needs ZLE ready) wait'0a': `zsh-expand` + late aliases + alias cache wait'0b': `zsh-autocomplete` (if `ZPWR_AUTO_COMPLETE=true`) wait'0c': `zsh-history-substring-search` wait'0d': `zsh-autosuggestions` + ARB late bindings + verbs wait'0e': `zsh-zinit-final` (penultimate, final keybindings) wait'0f': `zsh-more-completions` (low priority, last resort) wait'\${ZPWR_ZINIT_COMPINIT_DELAY}g': `compinit` + `cdreplay` wait'\${ZPWR_ZINIT_COMPINIT_DELAY}h': fast-syntax-highlighting.

Why Startup Is Fast

Only ONE plugin loads synchronously: `powerlevel10k`. Everything else is turbo-deferred. Combined with instant prompt, you see the prompt in ~15ms. The other 47 plugins load in the next ~200ms while you're still thinking.

Atinit And Atload Hooks

These run code at precise moments during plugin loading: `atinit` Runs BEFORE the plugin is sourced `atload` Runs AFTER the plugin is sourced ZPWR uses these extensively to bind keymaps and aliases:

```
zinit ice wait'0a' atload'zpwBindAliasesLate; zpwCreateAliasCache'
```

The `atload` fires after `zsh-expand` is sourced, then late aliases override any OMZ defaults. Precise surgical timing.

Zpwr_zinit_compinit_delay

Default: 0. Controls how long to wait before running `compinit`. Higher values defer completion system init even further. Set in `$ZPWR_ENV/.zpwr_env.sh`. The 'g' and 'h' suffixes on the wait ice ensure `compinit` and syntax highlighting load last.

Ice Modifiers Deep Dive

// ice modifiers are the spice of zinit. season generously. //

Every zinit load/snippet can be preceded by 'zinit ice' with modifiers that control HOW the plugin loads. Ice melts after one use – it applies only to the next command.

Core Ice Modifiers Used By zpwr

Lucid Suppress the 'Loaded plugin' message ZPWR uses this on every plugin. No spam. nocompile Skip .zwc compilation of the plugin Useful when zpwrRecompileFiles handles it globally. nocd Don't cd to plugin dir before sourcing Prevents p10k internals from polluting named dirs. nocompletions Don't install completions from this plugin Used for zsh-more-completions (fpath-only approach). as'completion' Load the file as a completion, not a plugin Used for OMZ completion snippets like _ng, _coffee. as'program' Add the pick'd file to \$PATH as executable Used for git-fuzzy. as'null' Don't source anything – just run hooks Used for compinit and the fzf zsh-plugin clone. pick'null' Don't source any file from the plugin dir Used with as'completion' for OMZ comp snippets. pick'bin/git-fuzzy' Source or add this specific file With as'program', adds it to PATH.

Hook Ice Modifiers

Atinit Code to run BEFORE the plugin sources atload Code to run AFTER the plugin sources Example from ZPWR:

```
zinit ice lucid nocompile wait atinit='zpwrBindOverrideOMZ;zpwrBindForGit'
zinit load MenkeTechnologies/forgit
```

Before forgit loads, zpwrBindOverrideOMZ runs to clear OMZ aliases that would conflict with forgit's git shortcuts.

How zpwr Uses Ice By Category

OMZ completions: lucid nocompile as'completion' pick'null' wait OMZ libs: lucid nocompile wait atload='zpwrOmzOverrides' OMZ plugins: lucid nocompile wait GH plugins: lucid nocompile wait Programs: as'program' lucid nocompile pick'bin/git-fuzzy' wait Syntax highlight:lucid nocompile wait"`\${ZPWR_ZINIT_COMPINIT_DELAY}h"

Ice Stacking

Multiple ices combine. Order doesn't matter (mostly). The = syntax for atload/atinit works with single quotes:

```
atload='zpwrBindAliasesLate; zpwrCreateAliasCache'
```

Or with quoted strings:

```
atinit'zpwrBindZdharma' atload'zpwrBindZdharmaPost'
```

Zinit Completions & Compinit

// compinit is the most expensive call in zsh startup. defer it. //

The completion system (compsys) needs compinit to initialize. But compinit scans all fpath entries and parses completion files. On a system with many registered completions, this is slow. ZPWR defers it behind turbo mode.

zinit Compinit Options

```
ZINIT[COMPINIT_OPTS]='-C -u'
```

-C skips security checks (don't re-verify file ownership). -u suppresses warnings about insecure directories. These flags make compinit fast but trust your fpath.

```
ZINIT[ZCOMPDUMP_PATH]="$ZSH_COMPDUMP"
```

Points compinit to zpwr's zcompdump location.

The Deferred Compinit

Compinit runs in the LAST turbo phase:

```
zinit ice lucid nocompile nocd as'null'  
wait"${ZPWR_ZINIT_COMPINIT_DELAY}g"  
atinit'zicompinit; zicdreplay; zpwrBindOverrideOMZCompdefs'
```

Zicompinit: runs compinit with the configured opts. zicdreplay: replays all compdef calls that were recorded while compinit wasn't loaded yet. Plugins that called compdef during turbo load had their calls queued – now they fire.

The Omz Completion Symlink Hack

OMZ completion snippets land in:

```
$ZSH/snippets/OMZP::coffee/_coffee
```

But compinit needs them in the completions dir. ZPWR symlinks them using zsh/files builtin:

```
zmodload -F zsh/files b:zf_ln  
zf_ln -sfm $ZSH/snippets/OMZP::${p%/*}/... $ZSH/completions/...
```

Zf_ln is a builtin ln(1), no fork needed. Fast.

Zpwrstalezcompdump

The zcompdump cache gets stale when plugins update. zpwrStaleZcompdump checks the cache age. If it is older than ~1 week, it triggers a full compinit rebuild. This runs automatically every shell startup (cheap check).

Force Rebuild

```
zpwr regenzzsh
```

Nukes the zcompdump, re-runs compinit from scratch, and recompiles everything. Use when completions go missing or after adding new completion files to fpath.

Zpwr_zinit_compinit_delay

Default: 0. Set in `$ZPWR_ENV/.zpwr_env.sh`. Controls the numeric prefix of the wait ice for compinit. 0 = load immediately after prompt. Higher = more delay. On fast machines, 0 is fine. On slow machines, 1 or 2 lets other plugins finish first.

Plugin Troubleshooting

// when plugins misbehave, you need diagnostic tools. //

Zinit ships with built-in debugging commands. When something breaks, these are your first responders.

Diagnostic Commands

`zinit report <plugin>` Show load status, errors, files.

```
zinit report zsh-users/zsh-autosuggestions
```

Shows: source file, ice modifiers used, compdef calls, functions defined, aliases created, PATH changes.
`zinit times` Show plugin load times Sorted by load time. Find the slowest plugin. If something takes >50ms, investigate.
`zinit loaded` List all loaded plugins
`zinit unload <plugin>` Unload a plugin from current session
Useful for debugging conflicts. Does not persist.
`zinit list` List all registered plugins
`zinit cd <plugin>` cd into plugin directory.

Common Issues

Plugin not loading? - Check if 'wait' ice is set (turbo mode defers loading) - Run `zinit report` to see if it errored silently - The 'lucid' ice suppresses messages – remove it to debug
Completion missing after plugin install? - Run `zpwr regenzzsh` to rebuild `zcompdump` - Check `fpath`: `print -l $fpath | grep <plugin>` - Ensure the plugin uses `as'completion'` or adds to `fpath`
Keybinding conflicts? - Load order matters. Plugins loaded later override earlier ones. - ZPWR uses `wait'0a'` through `wait'0e'` to control this. - `zpwrBindOverrideZLE` fires in the `0a` phase to win conflicts.

Adding Custom Plugins

Add to `ZPWR_GH_PLUGINS` in `$ZPWR_LOCAL/.tokens.sh`:

```
ZPWR_GH_PLUGINS+=( your-user/your-plugin )
```

The `.tokens.sh` file is sourced before plugin loading starts (via `zpwrTokenPre`), so your additions join the array in time. For conditional loading:

```
zpwrCommandExists myapp && ZPWR_GH_PLUGINS+=( user/zsh-myapp )
```

Updating Plugins

```
zpwr updatezinit
```

Runs `zinit update --all` to pull latest from all repos. After updating, run `zpwr regenzzsh` to rebuild completions.

Nuclear Option

If `zinit` is truly broken, nuke and rebuild:

```
rm -rf $ZPWR_PLUGIN_MANAGER_HOME
```

Next shell start auto-clones `zinit` and re-downloads everything. Slow but thorough. The cockroach approach to debugging.

The Plugin Ecosystem

// 30+ custom plugins from MenkeTechnologies //

ZPWR loads approximately 95 plugins via zinit. Over 30 are custom MenkeTechnologies creations that extend zpw with additional verbs, aliases, completions, and integrations.

Key Plugins

Zsh-learn 60 verbs, MySQL learning DB zsh-git-repo-cache 20 verbs, filesystem repo index fzf-zsh-plugin 14 verbs, brew/chrome fuzzy pickers zsh-git-acp 318 aliases, git workflows zsh-docker-aliases 148 aliases, Docker shortcuts zsh-openshift-aliases 106 aliases, OpenShift zsh-expand spell correct, abbreviation expand zsh-more-completions many thousands of tab completions zsh-cargo-completion 32 Rust/Cargo aliases zsh-travis 8 functions, CI integration zsh-nginx nginx management forgit interactive git (add, diff, log) zsh-z frequency directory jumping fzf-tab arb-powered tab completion zconvey cross-session communication.

Conditional Loading

Plugins for docker, dotnet, kubectl, ngrok, rails, and systemctl only load if those commands exist. No bloat on machines without those tools.

Deferred Loading

All plugins load with zinit wait'' ice — deferred after the prompt appears. Only powerlevel10k loads synchronously for instant prompt.

zsh-sudo: Privilege Escalation

Toggle sudo on the current command line with one keybind.

The Keybinding

Ctrl-S (or Escape Escape in some configs)

How It Works

```
sudo not on the line? .... it gets prepended
sudo already there? ..... it gets stripped
empty line? ..... pulls last command from history with sudo
```

What It Strips

When removing sudo, it also strips: builtin, command, env, and all their arguments. So sudo -E env PATH=... command foo becomes just foo.

Example

You type: vim /etc/hosts Hit Ctrl-S: sudo vim /etc/hosts Hit Ctrl-S again: vim /etc/hosts

Empty line + Ctrl-S: pulls your last command back with sudo prepended. No retyping. No arrow keys.

Plugin: MenkeTechnologies/zsh-sudo.

zsh-sed-sub: Inline Search & Replace

Global search and replace on the current command line.

The Keybinding

Ctrl-F Ctrl-P in both viins and vicmd modes.

How It Works

1. Type a command on the line
2. Hit Ctrl-F Ctrl-P
3. Enter the search pattern
4. Enter the replacement
5. All occurrences on the line are replaced

Example

You type: `cp file.txt /old/path/ && mv file.txt /old/path/bak` Hit Ctrl-F Ctrl-P, search: `old`, replace: `new`
Result: `cp file.txt /new/path/ && mv file.txt /new/path/bak`

Faster than arrow keys. Works on the entire line. Like sed but for your command line buffer.

Plugin: MenkeTechnologies/zsh-sed-sub.

gh_reveal: Open Repo in Browser

Quickly open the current git project in your browser.

The Command

```
zplr reveal          # open current repo on GitHub
```

How It Works

Reads the git remote URL from the current repo, converts it to an HTTPS URL, and opens it in your default browser. Works with GitHub, GitLab, Bitbucket, and any git remote.

Use Cases

- Check the PR page for the branch you are on - View the README without cat-ing it locally - Jump to Issues or Actions from the terminal - Share a repo link with a coworker fast

Also accessible as `zplr revealrecurse` to search up the directory tree for a git repo.

Plugin: MenkeTechnologies/gh_reveal.

Colorful Manuals & Package Completions

The smaller plugins that make everything nicer.

Zsh-very-colorful-manuals

Custom colorscheme for man pages. Headers in bold cyan, underlines in magenta, flags in green. Makes man pages actually readable instead of a wall of white text. Works automatically — no commands needed.

Zsh-gem-completion

Enhanced Ruby gem completion. Includes all OMZ gem completion plus remote gem search:

```
gem install <tab>          # completes remote gems
```

Results come from gem search output.

Zsh-pip-description-completion

Enhanced pip completion. All OMZ pip completion plus remote package search:

```
pip install <tab>          # completes remote packages
```

Results come from pip search output with descriptions.

Zsh-better-npm-completion

Better npm completion. Context-aware:

```
npm install <tab>          # completes package names
npm run <tab>              # completes scripts from package.json
npm uninstall <tab>        # completes installed packages
```

Each subcommand gets the right completions.

All plugins: MenkeTechnologies/*

Specialty Completions & Aliases

Platform-specific plugins loaded conditionally.

Zsh-cpan-completion

Tab completion for Perl CPAN modules.

```
cpan install <tab>        # complete module names
```

Zsh-dotnet-completion

Tab completion for .NET CLI (dotnet).

```
dotnet <tab>              # complete subcommands
dotnet add package <tab>  # complete packages
```

Loaded conditionally when dotnet is available.

Zsh-xcode-completions

Tab completion for Xcode CLI tools on macOS.

```
xcodebuild <tab>      # complete build options
xcrun <tab>            # complete SDK tools
```

MacOS only — loaded when Xcode is detected.

Kubectl-aliases

800+ aliases for kubectl. Every common operation is a short alias:

```
kgp  = kubectl get pods
kgs  = kubectl get services
kd   = kubectl describe
kl   = kubectl logs
```

Loaded conditionally when kubectl is available.

Jhipster-oh-my-zsh-plugin

Aliases and completions for JHipster (Java/Spring Boot scaffold generator).

```
jh    = jhipster
```

Loaded conditionally when jhipster is available.

All plugins: MenkeTechnologies/* Conditional loading means zero overhead if the tool isn't installed on your system.

Chapter 28 – zsh Internals

Autoloading: Lazy Function Loading

// 443 functions. zero parsing until you call one. //

Autoloading is zsh's lazy-loading mechanism. Register a name, and zsh only reads the function body from disk on first call. This is why ZPWR can have hundreds of autoloaded functions with negligible startup cost.

How Autoload -z Works

1. `autoload -z funcname` registers 'funcname' as autoloadable.
2. Zsh records the name but does NOT read the file yet.
3. On first call to `funcname`, zsh searches `$fpath` for a file named 'funcname' (exact match, no extension).
4. The file is sourced. The function is now defined in memory.
5. The function executes with the original arguments.
6. Subsequent calls use the in-memory version. No more disk.

The Glob In `.zshrc`

```
builtin autoload -z $ZPWR_AUTOLOAD_COMMON/*(:t)
```

The glob `*(:t)` means: * All files in the directory `(.)` Qualifier: regular files only (not dirs or symlinks) `:t` Modifier: tail only (strip directory, keep filename) So it registers every regular file's name as a function. Same pattern runs for `ZPWR_AUTOLOAD_DARWIN` or `ZPWR_AUTOLOAD_LINUX`.

The Fpath

Fpath is the function search path (like `PATH` but for functions):

```
fpath=( $ZPWR_AUTOLOAD_COMMON $ZPWR_AUTOLOAD_COMP_UTILS $ZPWR_COMPS $fpath )
```

Order matters: first match wins. ZPWR dirs come first so they can override system completions if needed. `fpath` is NOT exported (typeset `+x FPATH`) – child shells don't inherit it.

The Function File Format

Each file in the autoload dir contains one function. The file is named exactly like the function. Inside:

```
function zpwrSomething() {  
  # function body  
}  
zpwrSomething "$@"
```

The last line calls the function with args passed through. This is the ZPWR convention. The function wraps itself.

Why Autoloading Is Fast

443 function registrations = 443 hash table entries. Cost: microseconds. No file I/O, no parsing, no execution. Compare to sourcing 443 files: hundreds of milliseconds. Autoloading turns $O(n)$ startup into $O(1)$ with lazy $O(1)$ per call.

Autoload -Uz

-U suppresses alias expansion during loading (safety). -z forces zsh-style function loading (vs ksh-style). ZPWR uses -z for its functions, -Uz for system utilities like zrecompile, zmv, zargs, and compinit.

ZLE: The Zsh Line Editor

// the engine behind every keystroke. readline on steroids. //

ZLE (Zsh Line Editor) is the subsystem that handles your command line input. Every key press triggers a ZLE widget. ZPWR rewires dozens of these widgets for custom behavior.

Widgets: Named Editor Actions

A widget is a named action that ZLE can execute:

```
.accept-line      Built-in: execute the current line
.backward-delete-char Built-in: backspace
.self-insert      Built-in: insert the typed character
```

The dot prefix means 'built-in widget'. User-defined widgets can override them. ZPWR defines custom widgets like:

```
zpwrMagicEnter      Custom Enter key behavior
zpwrExpandTerminateSpace Custom Space key behavior
zpwrAcceptLine      Custom line acceptance
```

Bindkey: Mapping Keys To Widgets

Bindkey connects key sequences to widgets:

```
bindkey '^M' zpwrMagicEnter      Enter key
bindkey ' ' zpwrExpandTerminateSpace Space key
bindkey -v          Switch to vi mode
bindkey -e          Switch to emacs mode
```

ZPWR_BINDKEY_VI controls which mode: true=vi, false=emacs.

How Zpwrmagicenter Works

On empty line (just pressing Enter): - If in a git dir: runs git status -u . - If not: runs ls -lh . - Resets the prompt via `_p9k_precmd_impl` + zle reset-prompt On non-empty line: delegates to zle accept-line (zpwrAcceptLine).

How Zpwracceptline Works

The accept-line replacement does serious work: 1. Updates `zconvey` name with current command + TTY + PID 2. Detects commands that modify files (rm, mv, cp, mkdir...) and sets `ZPWR_WILL_CLEAR=true` for post-exec ls refresh 3. Handles sudo + grc colorization for aliased commands 4. Prevents global alias expansion in first position 5. Optionally expands aliases before execution 6. Calls `.accept-line` (the real built-in)

The Ctrl-f Chord Prefix System

ZPWR binds Ctrl-F as a prefix key for multi-key chords:

```
Ctrl-F Ctrl-D   ARB file search
Ctrl-F Ctrl-I   ARB word search (ag/grep)
Ctrl-F Ctrl-G   ARB git operations
```

This gives you an entire second keymap layer without conflicting with normal editing keys. Like tmux prefix but for your command line.

Hook Functions: precmd, preexec, chpwd

// zsh calls your code at key moments. event-driven shell. //

Zsh provides hook points where you can inject code that runs automatically before prompts, before commands, and when directories change. ZPWR uses all three.

Precmd: Before Each Prompt

Runs after a command finishes, right before the prompt draws. p10k's `_p9k_precmd` is the main user – it updates git status, execution time, and every prompt segment. ZPWR adds `zpwrPrecmd` to the hook:

```
function zpwrPrecmd(){ export ZPWR_OPTS="$-" }
```

This captures the current shell options before each prompt.

Preexec: Before Each Command

Runs after Enter but BEFORE the command executes. Receives the command string as `$1`. ZPWR's `zpwrPreexec` is minimal (a placeholder hook). p10k uses `preexec` to record the start timestamp for `command_execution_time` segment.

Chpwd: When Directory Changes

Runs whenever `$PWD` changes (`cd`, `pushd`, `popd`, `auto_cd`). ZPWR's `zpwrChpwd` hook handles directory-change actions.

The `_functions` Arrays

Zsh supports multiple hooks via arrays:

```
precmd_functions+=( myHook )    Add a precmd hook
preexec_functions+=( myHook )   Add a preexec hook
chpwd_functions+=( myHook )     Add a chpwd hook
```

Every function in the array runs at the hook point. Order matters: first in array = first to execute.

How zpwr Registers Hooks

`ZpwrBindPrecmd` (runs during p10k atload):

```
precmd_functions=( zpwrPrecmd ${(@)precmd_functions:#zpwrPrecmd} )
```

This puts `zpwrPrecmd` FIRST, removing any duplicate. The `${(@)array:#pattern}` syntax removes matching elements. `zpwrBindPreexecChpwd` (runs during syntax-highlighting atload):

```
preexec_functions=( ${(@)preexec_functions:#zpwrPreexec} zpwrPreexec )
chpwd_functions=( ${(@)chpwd_functions:#zpwrChpwd} zpwrChpwd )
```

These put zpwr hooks LAST (after p10k and other plugins).

The Hook Count In zpwr Top

The zpwr top dashboard shows the total hook count:

```
nhooks=$(( ${#precmd_functions} + ${#preexec_functions} + ${#chpwd_functions} ))
```

Typical value: 5-8 hooks total across all three arrays. If this number grows unexpectedly, a plugin is adding hooks you may not want. Check with: `print -l $precmd_functions`.

Emulate & Options

// zsh has 200+ options. emulate -L zsh resets them all locally. //

Zsh options control fundamental behavior: globbing, word splitting, quoting, history. ZPWR functions use `emulate -L zsh` to guarantee a known environment regardless of user settings.

Emulate -L zsh

```
builtin emulate -L zsh
```

Resets ALL options to zsh defaults for the current function scope. `-L` means local: options revert when the function returns. Without this, a function might inherit `noglob` from the caller and break globbing, or inherit `shwordsplit` and break arrays. ZPWR functions start with this to be immune to the caller's state.

Key Options In zpwr

extendedglob `##` = one-or-more, `(#i)` = case-insensitive glob `^pattern` = negation. Essential for ZPWR globs. glob_dots Dot files matched by `*` (no need for `.`) globstarshort `**c = **/*.c` (recursive glob shorthand) bracecl `{abcd0-9}` expands as character class `rc_quotes` " inside single quotes = literal single quote `rc_expand_param` Array expansion includes prefix: `xx${arr}` = `xxA xxB` nocaseglob Globs are case-insensitive globally numeric_glob_sort file1 file2 file10 sort numerically prompt_subst Allow `$(...)` and `${...}` in prompt strings transient_prompt Right prompt only on current line

The Noshwordsplit Default

Bash splits unquoted variables on whitespace by default. Zsh does NOT (`noshwordsplit`). This means:

```
x="hello world"; print $x -> one argument in zsh, two in bash
```

If a function inherits `shwordsplit` (e.g., from `emulate sh`), arrays and parameter expansions break silently. Another reason for `emulate -L zsh`.

The Local-in-loop Bug

In zsh, 'local' inside a loop prints the variable value as a side effect when the variable already exists:

```
for i in 1 2 3; do local x=$i; done # prints x=1 x=2 x=3
```

This is because 'local' on an existing local acts like `typeset`, which prints. Declare locals BEFORE loops, not inside them.

Auto_name_dirs: Removed

This option auto-registers any parameter set to a directory path as a named directory. Sounds nice, but p10k/gitstatus set internal variables to paths, polluting the named dir table. ZPWR removed it. Explicit hash -d in zpwrBindDirs handles it. The comment in .zshrc: # auto_name_dirs removed: pollutes named # dirs with p10k/gitstatus internals.

Parameter Expansion Magic

// zsh parameter expansion is a programming language in itself. //

Zsh's \${...} syntax can do transformations that would require sed, awk, basename, dirname, tr, and cut in bash. No forks. No subshells. Pure in-process string manipulation.

Parameter Expansion Flags (the Parenthesized Letters)

\${(K)assoc} Keys of associative array \${(v)assoc} Values of associative array \${(kv)assoc} Key-value pairs interleaved \${(s/.)var} Split string on / into array \${(j:.)array} Join array elements with , \${(L)var} Lowercase the entire string \${(U)var} Uppercase the entire string \${(f)var} Split on newlines (lines to array) \${(F)array} Join with newlines (array to lines) \${(t)var} Type of variable (scalar, array, etc.) \${(q)var} Quote the value for re-use in eval \${(Q)var} Remove one level of quoting \${(z)var} Split into words like the shell parser \${(o)array} Sort ascending \${(O)array} Sort descending.

Modifiers (the Colon Letters)

\${Var:t} Tail: basename (/a/b/c.txt -> c.txt) \${var:h} Head: dirname (/a/b/c.txt -> /a/b) \${var:r} Root: remove extension (file.txt -> file) \${var:e} Extension: (file.txt -> txt) \${var:A} Absolute path (resolve symlinks) \${var:l} Lowercase (zsh 5.0+) \${var:u} Uppercase (zsh 5.0+)

Substitution And Pattern Matching

\${var//pat/rep} Global substitution

\${var/pat/rep} First match substitution \${var#pat} Remove shortest prefix match \${var##pat} Remove longest prefix match \${var%pat} Remove shortest suffix match \${var%%pat} Remove longest suffix match.

zpwr Examples

From zpwrBindDirs (writing hash dir cache):

```
print -r "hash -d ${k}=${(q)v}" -- (q) quotes the value safely
```

From zpwrAcceptLine (parsing command buffer):

```
mywords=( ${ (z)cmdstr} )      -- (z) shell-word splits
cmd=${mywords[1]#=}           -- strip leading = from =cmd
```

From zpwrBindPrecmd (removing duplicates from array):

```
${(@)precmd_functions:#zpwrPrecmd} -- remove matching element
```

These are not academic exercises. They replace forks.

The .zwc Compilation System

// bytecode for your shell. because even scripts deserve a compiler. //

Zsh can compile scripts to .zwc (Zsh Word Code) bytecode. These files skip the parsing phase on load, going straight to execution. ZPWR compiles 50+ files for speed.

What .zwc Files Are

A .zwc file is a binary representation of parsed zsh code. When zsh sources a .zsh file, it checks for a .zwc sibling. If the .zwc exists and is newer than the source, zsh loads the bytecode directly. No lexing, no parsing. Speedup: 2-5x for large files. Marginal for tiny ones.

Zrecompile

The zrecompile utility compiles only if the source is newer:

```
autoload -Uz zrecompile
zrecompile -p -U $ZPWR_ENV_FILE
```

-p means 'check pairs' (source vs .zwc timestamps). -U suppresses alias expansion during compilation. If the .zwc is current, zrecompile is a no-op.

Zpwrrecompilefiles: What Gets Compiled

The function compiles two categories of files: Local files (~47): ~/.zshrc, ~/.zshenv, ~/.zlogout, ~/.zlogin p10k-instant-prompt*.zsh, p10k-dump*.zsh \$ZSH_COMPDUMP, \$ZPWR_ALIAS_FILE, \$ZPWR_PROMPT_FILE \$ZPWR_LOCAL/tokens.sh, \$ZPWR_ENV_FILE, \$ZPWR_RE_ENV_FILE All .zsh/.sh in \$ZPWR_ENV/, all .zsh in \$ZPWR_SCRIPTS/ All plugin .zsh files under \$ZPWR_PLUGIN_MANAGER_HOME/bin/ OMZ lib, plugin, theme, and custom files Global files (~8): /etc/profile, /etc/zshrc, /etc/zshenv, /etc/zprofile, etc. These need sudo (compiled with sudo zsh -c zrecompile).

zpwr Verbs For Compilation

zpwr recompile	Compile all known files (zpwrRecompileFiles)
zpwr decompile	Remove .zwc files (zpwrUncompile)
zpwr refreshzwc	Remove + recompile (clean rebuild)

The Zsh/files Module

Zsh can load builtin file operations via zmodload:

```
zmodload -F zsh/files b:zf_ln b:zf_rm b:zf_mkdir
```

These are shell builtins that replace ln, rm, mkdir. No fork, no exec – pure kernel syscalls from the shell process. ZPWR uses zf_ln for the OMZ completion symlink hack. In a loop creating 12 symlinks, this saves 12 fork+exec pairs.

Other Useful Modules

zsh/datetime	\$EPOCHREALTIME, \$EPOCHSECONDS
zsh/compllist	Menu-select completion mode
zsh/zprof	Profiler (ZPWR_PROFILING=true enables it)

Modules are loaded with: `zmodload zsh/datetime`.

Chapter 29 – Arb Architecture

arb -fzf: The Neural Interface

// the universal adapter between data and decisions. //

Arb is a TUI for every pipeline. Its `-fzf` mode is a general-purpose command-line filter: pipe anything in, fuzzy-search interactively, get the selection back on stdout. ZPWR wires it into dozens of workflows: files, words, git, verbs.

The Picker Is One Variable

Every ZPWR picker runs whatever `$ZPWR_FZF` points at:

```
export ZPWR_FZF='arb --fzf --ansi'
```

That is the single switch. Change it and every verb, widget, global alias and fzf-tab menu follows without further edits.

The Pattern: Pipe | Arb | Action

Every picker integration follows the same architecture:

```
<data source> | arb --fzf [options] | <action on selection>
```

The data source generates lines. `arb` presents them. The action consumes the selected line(s). Simple. Composable. Powerful.

Fzf_default_opts

ZPWR sets global picker defaults in `zpwrBindFZFLate`:

```
FZF_DEFAULT_OPTS="$ZPWR_COMMON_FZF_ELEM --reverse --border"
```

Plus a Solarized Dark color scheme layered on top:

```
--color fg:-1,bg:-1,hl:$blue,fg+:$base2,bg+:$base02
```

`ZPWR_COMMON_FZF_ELEM` adds the logo prompt and nav binds:

```
--prompt='$ZPWR_FZF_LOGO ' --bind=ctrl-n:page-down,...
```

The Preview Window

The `-preview` flag runs a command per highlighted item:

```
arb --fzf --preview 'bat --color=always {}'
```

ZPWR configures rich previews per context: Files: `bat` with syntax highlighting, line numbers, grid Dirs: `stat` output Packages: `dpkg -l` or `rpm -qi` Git: diff preview, commit log If `bat` is installed, it's the default colorizer. Falls back to `pygmentize` with `terminal256` output.

Fzf-completion (Tab)

The picker replaces default Tab completion with fuzzy search. Type a command, press Tab, and the picker opens:

```
cd **<Tab>      fuzzy-select a subdirectory
kill <Tab>       fuzzy-select a running process
ssh **<Tab>      fuzzy-select a known host
```

FZF_COMPLETION_OPTS controls the preview for these.

Fzf-tab Plugin

MenkeTechnologies/fzf-tab replaces zsh's menu completion. Any time the completion system would show a menu, fzf-tab intercepts it and shows the picker instead. This means: - Fuzzy matching for every completion, not just ** triggers - Previews for file completions, git branches, everything - FZF_TAB_OPTS configures behavior (ansi, nth, delimiter) - FZF_TAB_COMMAND is set from `$ZPWR_FZF`, so it is arb too.

Global Aliases For Arb

```
${ZPWR_GLOBAL_ALIAS_PREFIX}z  Pipe into arb: ls | ,,z
${ZPWR_GLOBAL_ALIAS_PREFIX}zb Inline selection: vim ,,zb
```

ARB in Practice

// the theory is simple. the practice is where it gets good. //

ZPWR ships dozens of arb-powered workflows. Each follows the generate | arb -fzf | action pattern. Here are the heaviest hitters.

File Search (zpwr Filesearch / Ctrl-f Ctrl-d)

Pattern: find files | arb -fzf -preview 'bat' | open in editor 1. find/fd generates a file list (respecting .gitignore) 2. arb -fzf shows files with bat-powered syntax-highlighted preview 3. Selected file opens in \$EDITOR at the matched location FZF_CTRL_T_OPTS configures the preview command.

Word Search (zpwr Wordsearch / Ctrl-f Ctrl-i)

Pattern: ag/rg search | arb -fzf -preview 'context' | open at line 1. ag (silver searcher) or rg searches file contents 2. arb -fzf shows matches with file:line:content format 3. Selection opens the file at the exact line number FZF_AG_OPTS: delimiter ':', match on 3rd+ field (skip file:line).

Git Add (forgitadd)

Pattern: git diff -stat | arb -fzf -preview 'git diff' | git add 1. Lists modified files with diff stats 2. Preview pane shows the actual diff for each file 3. Toggle files with Tab, confirm to stage them Part of forgit: the git + picker integration plugin.

Verb Search (zpwr Verbsfzf)

Pattern: ZPWR_VERBS keys | arb -fzf -preview 'help' | eval 1. All 460+ zpwr verbs piped into arb -fzf 2. Preview shows the verb description and command 3. Select a verb to execute it immediately FZF_ENV_OPTS_VERBS configures the preview for verbs.

Custom Arb Key Bindings

The Ctrl-F prefix opens ZPWR's arb -fzf keymap layer:

```
Ctrl-F Ctrl-D   File search (find + arb --fzf + editor)
Ctrl-F Ctrl-I   Word search (ag + arb --fzf + editor)
Ctrl-F Ctrl-G   Git search (git log + arb --fzf)
Ctrl-F Ctrl-R   History search with arb --fzf
```

These are ZLE widgets bound via `zpwrBindOverrideZLE`.

ZpwrIntofzf And ZpwrIntofzfag

Two core ZLE widgets that bridge the command line with arb -fzf: `zpwrIntofzf` File-oriented arb -fzf (Ctrl-T style) `zpwrIntofzfag` Content-oriented arb -fzf (ag/grep style) Both capture arb -fzf output and insert it into the command line buffer (\$BUFFER). You select files/matches, they appear at your cursor. Seamless integration between search and input.

The Colorizer Chain

ZPWR_COLORIZER preference: `bat > pygmentize`. `bat` uses themes (`BAT_THEME=$ZPWR_BAT_THEME`). `pygmentize` uses `PYGMENTIZE_COLOR` style. Language-specific colorizers: `FZF_COLORIZER_C`, `_SH`, `_YAML`, `_JAVA`.

Building Your Own ARB Verbs

// once you grok the pattern, everything becomes arb-able. //

The `generate | arb -fzf | action` pattern is infinitely extensible. Here's how to build your own arb-powered verbs.

The Pattern

```
<generator> | arb --fzf [--preview '<preview-cmd>'] | <consumer>
```

Generator: anything that outputs lines (find, git, docker, etc.) Preview: command that shows detail for highlighted line (\{\}) Consumer: action on selected lines (xargs, while read, eval)

Example: Arb Through Docker Containers

```
function arb-docker() {
  docker ps -a --format '{{.ID}} {{.Names}} {{.Status}}' |
  arb --fzf --preview 'docker inspect {1} | head -50' |
  awk '{print $1}' |
  xargs -r docker logs --tail 100
}
```

Generator: `docker ps`. Preview: `docker inspect`. Consumer: `docker logs`.

Example: Arb Through tmux Sessions

```
function arb-tmux-session() {
tmux list-sessions -F '#{session_name}: #{session_windows} windows' |
arb --fzf --preview 'tmux list-windows -t {1}' |
cut -d: -f1 |
xargs tmux switch-client -t
}
```

Generator: tmux list-sessions. Consumer: tmux switch-client.

The Accept Vs Edit Pattern

ZPWR verbs come in two flavors: Accept verbs Execute the selection immediately Example: verbsfzf selects a verb and runs it. Edit verbs Insert the selection into \$BUFFER for editing Example: filesearch inserts the path into your command line. The difference is in the consumer. Accept uses eval/exec. Edit uses BUFFER=\$selection + zle redisplay.

Making It A Zle Widget

To bind your arb --fzf verb to a key:

```
function my-arb-widget() {
BUFFER=$(my-generator | arb --fzf)
CURSOR=${#BUFFER}
zle redisplay
}
zle -N my-arb-widget
bindkey '^F^X' my-arb-widget
```

Now Ctrl-F Ctrl-X triggers your custom arb --fzf workflow.

Tips For Good Arb Experiences

- Use --ansi if your generator outputs color codes - Use --delimiter and --nth to control what arb --fzf matches on - Use -m for multi-select (toggle with Tab) - Use --header for context hints at the top - Preview window size: --preview-window=right:50%:wrap - Test with: echo 'alnblnc' | arb --fzf --preview 'echo {}'

fzf-zsh-plugin: Brew & Chrome

// fuzzy finding extends to your package manager and browser //

The fzf-zsh-plugin adds fuzzy interfaces to Homebrew package management and Chrome browser data. It ships its own picker rather than going through `$ZPWR_FZF`.

Homebrew Verbs

```
zpwr brewinstall          # fuzzy install formula
zpwr brewuninstall        # fuzzy uninstall formula
zpwr brewcaskinstall      # fuzzy install cask
zpwr brewcaskuninstall    # fuzzy uninstall cask
zpwr brewupdate           # fuzzy update formula
```

Each command pipes the available packages into the picker with multi-select. Pick one or many, then install/remove in one shot. No more googling package names.

Chrome Verbs

```
zpw! chromehistory      # fuzzy search Chrome history
zpw! chromebookmarks    # fuzzy search Chrome bookmarks
```

Reads Chrome's SQLite databases directly. Select a URL from the picker and open it in the browser.

Plugin: [MenkeTechnologies/fzf-zsh-plugin](https://github.com/MenkeTechnologies/fzf-zsh-plugin).

Chapter 30 – tmux Deep Dive

Tmux Architecture in ZPWR

// the terminal multiplexer. one screen, infinite shells. //

Tmux is the backbone of the ZPWR workflow. Multiple shells, persistent sessions, split panes, and code execution across panes – all orchestrated from `$ZPWR_TMUX`.

Session / Window / Pane Model

Session Named group of windows (e.g., 'dev', 'zpwr-study') Window A tab inside a session (like browser tabs) Pane A split inside a window (like vim splits) Typical ZPWR layout: 1 session, 2-4 windows, 2-4 panes each. Prefix key: Ctrl-B (default). Followed by a command key.

Session Persistence

Tmux-resurrect Manual save/restore of sessions Save: prefix + Ctrl-s Restore: prefix + Ctrl-r Saves: pane layouts, working directories, running programs. @resurrect-processes ':all:' – saves ALL running programs. tmux-continuum Auto-save every 15 minutes Works silently in the background. No manual intervention. Combined with resurrect: your tmux state survives reboots.

The Powerline Status Bar

Tmux sources powerline bindings for the status bar:

```
source-file ~/.tmux/powerline/bindings/tmux/powerline.conf
```

The symlink `~/.tmux/powerline` points to pip's powerline-status. powerline-daemon runs in the background for performance. Status bar shows: session name, window list, hostname, time, battery level (via tmux-battery plugin).

\$ZPWR_TMUX: The Config Directory

All tmux config lives under `$ZPWR_TMUX`:

<code>init.conf</code>	Main config (bindings, options)
<code>tmux-mac.conf</code>	macOS-specific overrides
<code>tmux-linux.conf</code>	Linux-specific overrides
<code>tmux_ge_21.conf</code>	Tmux >= 2.1 features
<code>tmux_ge_29.conf</code>	Tmux >= 2.9 features
<code>tmux_ge_31.conf</code>	Tmux >= 3.1 features
<code>four-panes.conf</code>	Layout: 4 panes
<code>sixteen-panes.conf</code>	Layout: 16 panes
<code>config-files.conf</code>	Opens brc, vrc, trc, zrc in 4 panes
<code>learn.conf</code>	Study layout

Init.conf Highlights

- 30,000 line scrollbar (ZPWR_TMUX_HISTORY_LIMIT) - Vi keybindings in copy mode - Prefix S: synchronize-panes (type in all panes at once) - Prefix q: display pane numbers for 5 seconds - 3-second repeat-time for directional keys

Vim-Tmux Code Execution

// select code in vim. execute in the pane next door. magic. //

The killer feature of the ZPWR vim+tmux integration: write code in vim, send it to an adjacent tmux pane for execution. No copy-paste. No switching windows.

The Tmuxrepeat Function

Defined in .vimrc, TmuxRepeat has three modes: 'file' Save the file, execute it in the right pane 'visual' Save visual selection to temp file, execute it 'repl' Paste visual selection directly into right pane.

Supported Languages

TmuxRepeat detects file type by extension:

```
sh, zsh, py, rb, pl, clj, tcl, vim, lisp, hs, ml,  
coffee, swift, lua, java, f90, cr
```

For known types, it sends the file to zpwrRunner.sh which dispatches to the correct interpreter. For unknown types, it falls back to TmuxRepeatGeneric (re-run previous command).

How It Works

1. Ctrl-V in normal/insert mode (or leader-vj): Saves the file, then sends:

```
tmux send-keys -t right 'bash "$ZPWR_SCRIPTS/zpwrRunner.sh" "file"' C-m
```

2. leader-vk in visual mode: Yanks selection to temp file `$ZPWR_HIDDEN_DIR_TEMP/.vimTempFile.<ext>`, then sends that temp file to the right pane for execution. 3. leader-vl in visual mode (REPL mode): Yanks selection, loads it into tmux buffer 'buffer0099', then pastes directly into the right pane. For REPLs like python, irb, node – code runs line-by-line in the REPL.

The Pane Targeting

Standard tmux: sends to pane named 'right' (the next pane). MacVim/GUI: sends to session 'vimmers:0.' (external tmux). The pane_count check ensures a target pane exists.

Allpanes.zsh: Broadcast To All Panes

`$ZPWR_SCRIPTS/allPanes.zsh` sends commands to all tmux panes:

```
prefix + Space  Run command in all panes (single mode)  
prefix + v      Run command in all panes (multi mode)  
prefix + b      Open URL in all panes
```

Synchronize-panes

Prefix + S Toggle synchronize-panes When enabled, every keystroke goes to ALL panes in the window. Useful for running identical commands on multiple servers, or setting up parallel environments simultaneously.

Google From tmux

Prefix + g Google search from tmux Runs `$ZPWR_TMUX/google.sh` with the current pane content. Select text, press prefix+g, and your browser opens with a Google search for the selection.

Tmux Plugins & Extensions

// tpm: the plugin manager for your multiplexer's plugins. yo dawg. //

ZPWR manages 9 tmux plugins via tpm (Tmux Plugin Manager). They live in `~/.tmux/plugins/` and add features that tmux does not provide out of the box.

Tpm: tmux Plugin Manager

Installed to `~/.tmux/plugins/tpm`. Managed via prefix keys:

```
prefix + I      Install plugins listed in .tmux.conf
prefix + U      Update all installed plugins
prefix + Alt-u   Clean plugins not in config
```

Plugins are declared in `$ZPWR_INSTALL/.tmux.conf`:

```
set-option -g @plugin 'tmux-plugins/tpm'
set-option -g @plugin 'tmux-plugins/tmux-resurrect'
```

The Plugin Roster (.tmuxplugins)

Tmux-plugins/tpm Plugin manager itself tmux-plugins/tmux-sensible Sane defaults for tmux tmux-plugins/tmux-resurrect Session save/restore tmux-plugins/tmux-continuum Auto-save sessions tmux-plugins/tmux-battery Battery indicator tmux-plugins/tmux-prefix-highlight Highlight when prefix active tmux-plugins/tmux-yank System clipboard integration MenkeTechnologies/tmux-thumbs Quick copy with hints MenkeTechnologies/tmux-fzf-url ARB through URLs in pane.

Tmux-resurrect: What It Saves

- Pane layouts (splits, sizes, positions) - Working directories for each pane - Running programs (@resurrect-processes 'all:') - Command history is NOT saved (that's zsh's job) Save: prefix+Ctrl-s. Restore: prefix+Ctrl-r.

Tmux-continuum: Auto-save

Piggybacks on resurrect. Auto-saves every 15 minutes. @continuum-restore is commented out in ZPWR by default. Enable it to auto-restore on tmux server start.

Tmux-fzf-url: Url Picker

Scans the visible pane for URLs, opens them in arb -fzf:

```
prefix + e   ARB through URLs, open selected in browser
prefix + u   ARB through URLs, search mode
```

Uses ZPWR_TMUX_HISTORY_LIMIT for scrollback depth.

Custom Scripts In \$ZPWR_TMUX

Layout scripts create multi-pane environments:

```
prefix + D   control-window.conf (IDE layout)
prefix + F   four-panes.conf
prefix + G   eight-panes.conf
prefix + O   sixteen-panes.conf
prefix + T   config-files.conf (brc, vrc, trc, zrc)
prefix + M   learn.conf (study layout)
```

zpwr Study: The Split-pane Learning Environment

Creates a dedicated tmux session 'zpwr-study' with the encyclopedia on the left and a live shell on the right. Read the docs, try commands. Knowledge through practice.

Tmux Layout Save & Restore

Your pane arrangement, preserved forever.

The Problem

You spent 20 minutes splitting panes, resizing them, cd-ing into the right directories. Then tmux crashes. Or you reboot. Or you accidentally kill the session. All that layout work? Gone. Every time.

The Solution

```
zpwr tmuxsave dev           # save layout as 'dev'
zpwr tmuxload dev           # restore it later
```

What Gets Saved

```
Windows ..... names and count
Pane splits ..... exact geometry via select-layout
Directories ..... per-pane working directory
Running commands ..... full args from process tree
Window options ..... synchronize-panes, auto-rename
Session options ..... non-plugin tmux settings
Active pane/window .... cursor position restored
```

Management

```
zpwr tmuxsave           # save with timestamp name
zpwr tmuxsave --list    # list saved layouts
zpwr tmuxsave --delete dev # remove a layout
zpwr tmuxload           # arb --fzf picker (no args)
zpwr tmuxload dev mywork # restore as session 'mywork'
```

Keybinding

Prefix W # save layout (tmux keybinding)

Aliases

```
zpwr layoutsave = zpwr tmuxsave
zpwr layoutload = zpwr tmuxload
```

Layouts are stored as self-contained zsh scripts in `$ZPWR_LOCAL/layouts/`. You can edit them by hand, version control them, or share them with your team.

PROTIP: Save your layout before experimenting. If you break everything, one command brings it back.

Chapter 31 – vim Deep Dive

Vim Plugin Architecture

// 15 plugins in .vimrc. each one a force multiplier. //

ZPWR's vim config uses Vundle as the plugin manager and ships 15 curated plugins that transform vim into a full IDE with completion, snippets, and tmux integration.

Vundle: The Plugin Manager

Plugins live in `~/.vim/bundle/`. Managed by Vundle:

```
set runtimepath+=~/.vim/bundle/Vundle.vim
call vundle#begin()
Plugin 'VundleVim/Vundle.vim'
... more plugins ...
call vundle#end()
```

Install: `:PluginInstall` Update: `:PluginUpdate` Clean: `:PluginClean`.

Youcompleteme: Semantic Completion

```
Plugin 'ycm-core/YouCompleteMe'
```

The heavyweight champion. Provides semantic completion for: C/C++ (via clangd), Python (via jedi), and more. Config: completion after 1 char, enabled in all file types, completes in comments. Uses C-n/C-p for selection.

Ultisnips + Vim-snippets: Snippet Engine

```
Plugin 'SirVer/ultisnips'
Plugin 'honza/vim-snippets'
```

Tab expands snippets, Tab/Shift-Tab navigate tab stops. Hundreds of snippets for every language: type `'for<Tab>'` in Python and get a complete for loop with cursor placement.

Supertab: Tab Completion Glue

```
Plugin 'ervandew/supertab'
```

Unifies Tab behavior: C-n cycles forward through completions. Coordinates with YCM and UltiSnips so Tab does the right thing in every context (snippet vs completion vs indent).

Vim-auto-save: Never :w Again

```
Plugin '907th/vim-auto-save'
```

Saves on InsertLeave, TextChanged, and TextChangedI. Every edit auto-saves silently. No more 'Unsaved changes' panics. g:auto_save_silent=1 suppresses the notification.

Rainbow: Matching Bracket Colors

```
Plugin 'luochen1990/rainbow'
```

Nested brackets get different colors: parentheses in blue, inner brackets in orange, deeper in cyan, deepest in magenta. g:rainbow_active=1 enables it globally on startup.

Vim-multiple-cursors: Multi-cursor Editing

```
Plugin 'TerryMa/vim-multiple-cursors'
```

Sublime Text-style multi-cursor. Ctrl-N selects next match, then edit all occurrences simultaneously.

Tmux-complete.vim: Cross-pane Completion

```
Plugin 'wellle/tmux-complete.vim'
```

Completes words from ALL tmux panes. Writing a command in vim that references output in another pane? Tab-complete it. Uses async.vim and asyncomplete.vim for non-blocking operation.

Vim Sessions & Workflow

// your vim state, frozen in amber. thaw it when you need it. //

Vim sessions save your entire editing state: open buffers, window layouts, cursor positions. ZPWR integrates sessions with tmux for a seamless development environment.

vim Session Management

ZPWR configures vim sessions as manual-save only:

```
let g:session_autosave = 'no'
let g:session_autoload = 'no'
```

No surprises. Sessions save when you tell them to. Save: Ctrl-D q (mapped to :SaveSession! in normal/insert mode).

Config Sessions: Vrc, Brc, Zrc, Trc

ZPWR defines verbs for editing its own config files: vrc Open .vimrc in vim brc Open shell aliases file (.zpwrc_alias.sh) zrc Open .zshrc in vim trc Open tmux.conf in vim The tmux layout script config-files.conf opens all four in a 4-pane arrangement. Prefix+T triggers it. Each pane runs one of these verbs automatically.

The Auxiliary vim Plugins

Vim-autotag Auto-regenerate ctags on save.

```
Plugin 'craigemery/vim-autotag'
```

G:autotagStartMethod='fork' – non-blocking tag generation. vim-online-thesaurus Look up synonyms (visual mode \K)

```
Plugin 'beloglazov/vim-online-thesaurus'
```

Vim-zsh-completion Zsh syntax completion in vim.

```
Plugin 'Valodim/vim-zsh-completion'
```

Vim-minimap Sublime-style code minimap.

```
Plugin 'severin-lemaignan/vim-minimap'
```

The zpwr Snapshot/restore Integration

ZPWR has verbs for full environment snapshots:

```
zpwr snapshot    Capture terminal environment state
zpwr restore     Restore from snapshot
```

ZpwrRestore rebuilds shell state: directory stack, aliases, parameters, and more. This complements tmux-resurrect: resurrect restores the tmux layout, snapshot/restore restores the shell state within each pane.

The Full zpwr vim Workflow

1. Open a tmux session with your project layout
 2. Vim in left pane, shell in right pane
 3. Edit code in vim, Ctrl-V to execute in right pane
 4. Visual select + leader-vl to paste into a REPL
 5. tmux-complete.vim grabs words from the REPL output
 6. Auto-save means you never lose work
 7. Prefix+Ctrl-s saves the entire tmux state
- This is not an IDE. This is a weapon system.

Chapter 32 – temprs – Temporary File Stack Manager

temprs: The Stack Machine

// your filesystem is a stack. you just didn't know it yet. //

temprs is a Rust-based temporary file stack manager by MenkeTechnologies. Think of it as a LIFO/FIFO clipboard that persists across shell sessions using actual tempfiles on disk. Version 2.9.3. Every operation is instant. No databases, no daemons — just a master record file and a directory of tempfiles.

The Stack Paradigm

temprs models your scratch data as a stack with both ends open: push (default), pop, shift, unshift — like a Perl array on disk.

Core Operations

temprs # no args: create new tempfile, push to top
temprs -o INDEX # read contents of tempfile at INDEX to stdout
temprs -i INDEX # write stdin into tempfile at INDEX
temprs -p # pop from top of stack (print + remove)
temprs -s # shift from bottom of stack (print + remove)
temprs -u # unshift: push stdin to bottom (silent)
temprs -a INDEX # insert new tempfile at INDEX
temprs -r INDEX # remove tempfile at INDEX.

The Master Record

temprs keeps a master record file that maps stack positions to actual tempfile paths on disk. The temprs directory holds the files themselves. Both are in your system's temp directory. temprs -d # list the temprs directory contents
temprs -m # list the master record file path.

Pipeline Integration

temprs reads stdin when available — perfect for pipelines:

```
echo 'stash this' | temprs -u # silent push to bottom
curl -s api.io/data | temprs # push response to top
temprs -o 0 | jq . # read top, pipe to jq
```

The -u unshift flag reads stdin and produces no stdout — ideal for mid-pipeline stashing without breaking the flow.

Positional Argument

temprs myfile.txt # read file into new tempfile on stack
If both stdin and a file arg are present, stdin takes priority. Your data flows in, gets a stack address, and waits for recall.

temprs: Advanced Operations

// the stack goes deeper. every tempfile has a story. //

Listing & Inspection

temprs -l # list all tempfile paths on the stack
temprs -n # list numbered (index + path)
temprs -L # list all tempfile contents
temprs -N # list numbered with full contents
temprs -k # count tempfiles on the stack
temprs -I INDEX # show metadata for tempfile at INDEX.

Editing & Tagging

temprs -e INDEX # open tempfile at INDEX in \$EDITOR
temprs -w NAME # tag a tempfile with an alias name
temprs -R OLD NEW # rename tag from OLD to NEW
Tags let you retrieve tempfiles by name instead of index. Your stack positions shift; your tags don't.

Search & Concatenate

temprs -g PATTERN # grep through all tempfile contents
temprs -C 0 2 5 # concatenate tempfiles at indices to stdout
The grep flag searches across every tempfile on the stack — like a personal knowledge base with instant full-text search.

Stack Manipulation

temprs -D 0 1 # diff two tempfiles by index or name
temprs -M 0 3 # move tempfile from position 0 to 3
temprs -x INDEX # duplicate tempfile onto top of stack
temprs -S 0 2 # swap two tempfiles by index or name
temprs -rev # reverse the entire stack order
temprs -A INDEX # append stdin to existing tempfile.

Housekeeping

temprs -c # clear/purge ALL tempfiles from stack
temprs --expire 24 # purge tempfiles older than 24 hours
temprs --wc INDEX # print line count of tempfile
temprs --size INDEX # print byte size of tempfile
temprs --path INDEX # print the actual filesystem path
temprs --sort mtime # sort stack by name, size, or mtime.

Mode Flags

temprs -q # quiet mode: suppress output on create
temprs -v # verbose mode: extra diagnostic output.

The Cyberpunk -h

Run `temprs -h` for the full ASCII art banner with the TEMPRS logo, status bar, and signal meter. Because even help text should look like a console from a Gibson novel. Also: `--head INDEX N` and `--tail INDEX N` for partial reads, and `--replace INDEX PAT REPL` for in-place sed-style edits.

Chapter 33 – Isofrs – Enhanced Isof

Isofrs: Network Intelligence

// every open file descriptor is a signal. Isofrs reads them all. //

Isofrs is a Rust-based enhanced Isof wrapper v4.7.0 by MenkeTechnologies. It wraps the venerable Isof with colorized output, structured formatting, and cyberpunk-grade aesthetics.

Why Not Raw Lsof?

Raw Isof dumps walls of monochrome text. Isofrs adds color coding by type, structured columns, and filtering that doesn't require piping through 5 layers of awk and grep.

Selection Flags

Isofrs -i [ADDR] # select internet connections ADDR format: [4|6][proto][@host|addr][:svc|port] Isofrs -c COMMAND # select by command name (prefix match) Isofrs -p PID # select by PID (comma-separated, ^excludes) Isofrs -u USER # select by login name or UID Isofrs -d FD # select by file descriptor set Isofrs -g [PGID] # select by process group ID Isofrs -a # AND selections (default is OR)

Network Flags

Isofrs -n # inhibit hostname resolution (fast mode) Isofrs -P # inhibit port-to-name conversion Isofrs -N # select NFS files Isofrs -U # select UNIX domain socket files.

zpwr Integration

The zpwr Isof verb pipes Isof output through arb -fzf for interactive process selection and killing:

```
zpwr lsof          # arb --fzf over lsof, select PIDs to kill
zpwr lsosedit      # same but edit the kill command first
```

The zpwrLsoffzf widget (bound to FH in viins and vicmd) runs `sudo lsof -i | arb -fzf -m`, extracts PIDs with awk, and inserts them into your command line. Scan the local network like a deck jockey, pick your targets with arb -fzf.

Files & Directories

Isofrs FILE... # list processes using these files Isofrs -dir DIR # open files in DIR (one level, like +d) Isofrs -dir-recurse DIR # recursive (like +D)

Isofrs: Practical Usage

// who's listening on port 8080? Isofrs knows. Isofrs always knows. //

Finding What's On A Port

```
lsofrs -i :8080      # anything on port 8080
lsofrs -i TCP:443    # TCP connections on 443
lsofrs -i 4          # all IPv4 connections
lsofrs -i 6TCP       # IPv6 TCP only
```

Display & Output

lsofrs -F [FIELDS] # machine-readable output fields lsofrs -J / -json # JSON output for piping to jq lsofrs -R # list parent PIDs lsofrs -t # terse: PID only (for scripting) lsofrs -0 # NUL field terminator (xargs -0 friendly) lsofrs -color MODE # auto, always, or never.

Live Monitoring

lsofrs +r 2 # repeat every 2 seconds lsofrs -W / -monitor # live full-screen like top lsofrs -delta # highlight new/gone FDs in repeat mode lsofrs -follow PID # watch one process, highlight opens/closes lsofrs -top 20 # live top-N processes by FD count lsofrs -watch FILE # watch who opens/closes a file over time.

Diagnostics

lsofrs -leak-detect # detect FD leaks (poll every 5s, 3 rounds) lsofrs -stale # find FDs pointing to deleted files lsofrs -ports # listening ports summary (like ss -tlnp) lsofrs -tree # process tree with FD counts (pstree + lsof) lsofrs -summary # aggregate FD summary by type and user.

Comparison

lsof — the OG. every Unix has it. output is a wall of text. ss — socket statistics. fast but sockets only, no files. netstat — deprecated on Linux. still works on macOS/BSD. lsofrs — wraps lsof with color, JSON, live monitoring, leak detection, tree views, and the cyberpunk banner you get with lsofrs -h. Built for operators who stare at FD tables and like what they see.

Quick Recipes

```
lsofrs -t -i :3000 | xargs kill # kill whatever's on 3000
lsofrs -c node -a -i TCP      # node's TCP connections
lsofrs -u $(whoami) -a -i     # your network connections
```

Chapter 34 – eza – The Modern ls

eza: ls Evolved

// ls is dead. long live ls. //

eza is the modern Rust replacement for ls, formerly known as exa. Maintained by eza-community after exa's author went AWOL. In zpw, it's the default file lister for everything.

\$ZPWR_EXA_COMMAND – The Configured Invocation

```
command eza --git -il --classify=always -H --color-scale -g -a --colour=always
```

Every flag earns its place: `--git` # git status per file (M modified, N new, I ignored) `-i` # show inode numbers `-l` # long listing format `--classify=always` # / after dirs, * after executables, always `-H` # show hard link count `--color-scale` # gradient colors for file sizes and ages `-g` # show group ownership `-a` # show hidden files (dotfiles) `--colour=always` # force color even when piped.

Why `--classify=always`?

Not just `-F`. Under p10k instant prompt, stdout gets redirected during init. eza's `--classify` auto-detects terminal capabilities and when stdout is redirected, it drops the indicators. The `=always` suffix forces classification regardless of fd state. Without it, your login banner loses all the / and * markers.

Where It's Used

ZpwrClearList and zpwrListNoClear both eval `$ZPWR_EXA_COMMAND`. The fallback chain: `eza -> grc + gls -> raw ls`. zpwrListNoClear checks for eza first, falls back to gls on macOS (with grc for colorization) or plain ls on Linux.

```
alias eza="$ZPWR_EXA_COMMAND" # bound in zpwrBindAliasesLate
alias lr="$ZPWR_EXA_COMMAND -R --tree | less -rMN"
```

The lr alias gives you a recursive tree piped through less with line numbers — a full filesystem X-ray.

Extended Attributes

Set `ZPWR_EXA_EXTENDED=true` in `.zpwr_env.sh` to add `--extended`, which shows xattr metadata on macOS (com.apple.quarantine etc).

eza: Colors & Integration

// a file listing without color is just noise. //

Eza_colors Environment Variable

eza uses EZA_COLORS (or EXA_COLORS for compat) to customize colors beyond the defaults. Format is colon-separated key=value:

```
export EZA_COLORS="di=34:ln=35:so=32:pi=33:ex=31"
```

Keys match LS_COLORS extensions. eza also adds its own: da=date, sn=file size number, sb=file size unit, uu=current user, un=other user, gu=current group.

LS_colors Compatibility

eza reads LS_COLORS as a fallback. If you have a vivid or dircolors setup, eza inherits it. EZA_COLORS overrides.

Tree View

```
eza --tree          # recursive tree (like tree command)
eza --tree -L 3     # limit depth to 3 levels
eza --tree --git-ignore # respect .gitignore in tree
```

The tree integrates with `--git`, so you see git status markers inline with the tree structure. New files glow green.

The Fallback Chain In Zpwrlistnoclear

1. If eza exists: `eval $ZPWR_EXA_COMMAND`
2. macOS + grc: `grc -c ~/conf.gls gls -iFlhA --color=always`
3. macOS bare: `ls -iFlhAO` (O = macOS file flags)
4. Linux + grc: `grc -c ~/conf.gls ls -iFlhA --color=always`
5. Linux bare: `ls -iFlhA` grc (Generic Colouriser) wraps ls output with conf.gls rules. It's the middle ground between raw ls and eza.

zpwr Verbs Using eza

ZpwrClearList clears the terminal then runs the listing. It's called on every `cd` via the `chpwd` hook — so you always see what's in the directory you just landed in. `zpwrListNoClear` does the same listing without clearing.

Sorting & Filtering

```
eza --sort=modified # sort by modification time
eza --sort=size     # sort by file size
eza -r             # reverse sort order
eza --group-directories-first # dirs at top
eza -I '*.pyc|__pycache__' # ignore patterns
```

Combine with `--git-ignore` to hide everything in `.gitignore`. Your file listings become as curated as your code.

Chapter 35 – bat – cat With Wings

bat: Syntax Highlighting for Everything

// cat is for reading files. bat is for understanding them. //

bat is a cat clone with syntax highlighting, git integration, and automatic paging. Written in Rust. In zpwr it's the default colorizer: ZPWR_COLORIZER=bat in .zpwr_env.sh.

Core Features

Line numbers in the gutter. Git diff markers (+/-/~) in the margin. Syntax highlighting for 200+ languages. Automatic paging via less when output exceeds terminal height.

Discovery

bat --list-themes # see all available color themes bat --list-languages # every syntax bat can highlight.

How zpwr Uses bat

bat is the preview engine for arb -fzf throughout zpwr:

```
arb --fzf --preview 'bat --color=always {}'
```

The full zpwr colorizer invocation:

```
bat --paging never --wrap character --color always  
--style="numbers,grid,changes,header"
```

This is stored in `$ZPWR_COLORIZER` and expanded into: FZF_COLORIZER — preview for generic arb -fzf FZF_COLORIZER_FILE — preview for file-specific arb -fzf FZF_COLORIZER_FILE_TEXT — text file preview (uses -l ASP) When bat is unavailable, zpwr falls back to pygmentize.

zpwr Verbs

zpwr filesearch and zpwr wordsearch both use bat in their arb -fzf preview windows. vimfilesearch, vimwordsearch, and the emacs variants (emacsfilesearch, emacswordsearch) all inherit the same bat-powered preview pipeline.

Standalone Usage

```
bat src/main.rs           # highlighted source view  
bat -l json data.txt      # force language detection  
bat --diff                # show only git-changed lines  
bat -A                   # show non-printable chars  
bat --plain               # no line numbers, no border  
bat -r 10:20 file.zsh     # show only lines 10-20
```

The -r (--line-range) flag is perfect for surgical code review.

bat: Configuration & Tricks

// configure your cat. teach it new tricks. make it purr. //

Bat_theme

```
export BAT_THEME="Dracula"
```

Set this in your env to change the default theme globally. Popular themes: Dracula, Monokai Extended, Nord, OneHalfDark, Solarized (Dark), TwoDark, ansi. Preview them all:

```
bat --list-themes | arb --fzf --preview='bat --theme={} src/main.rs'
```

Style Components

```
bat --style=numbers,changes,header
```

Components: full, auto, plain, numbers, changes, header, header-filename, header-filesize, grid, rule, snip. Combine with commas. 'full' enables everything. 'plain' disables everything — just colored text, no chrome.

bat As Manpage Colorizer

```
export MANPAGER="sh -c 'col -bx | bat -l man -p'"
```

This makes every man page syntax-highlighted. The -l man forces manpage language detection, -p is --plain (no line numbers cluttering man output). col -bx strips backspace formatting.

Syntax Cache

```
bat cache --build    # rebuild syntax/theme cache
bat cache --clear    # clear and rebuild from scratch
```

After adding custom .sublime-syntax files to ~/.config/bat/syntaxes/, rebuild the cache to pick them up.

Debian/ubuntu Gotcha

On Debian-based systems, the binary is batcat not bat due to a name conflict with an unrelated package. Fix:

```
alias bat=batcat      # or symlink: ln -s batcat ~/.local/bin/bat
```

bat + Delta For Git Diffs

Delta is a syntax-highlighting pager for git diffs. It uses bat's syntax themes internally. Configure in .gitconfig:

```
[core] pager = delta
[delta] syntax-theme = Dracula
```

Delta adds line numbers, side-by-side view, and word-level diff highlighting. bat does files, delta does diffs.

Bat_pager

```
export BAT_PAGER="less -RFX"
```

Customize the pager bat uses. -R for color, -F to quit if one screen, -X to not clear screen on exit. Or set to "" to disable paging entirely — zpwrr does this with `--paging never`.

Chapter 36 — fd-find — Find Replacement

fd: find for Humans

// find is powerful. fd is powerful AND usable. //

fd is a fast, user-friendly alternative to find, written in Rust by David Peter (sharkdp — also the author of bat). It respects .gitignore by default and uses regex, not globs.

Basic Usage

```
fd PATTERN          # search current dir recursively
fd PATTERN path/    # search specific directory
fd '^test.*\.zsh$'  # full regex support
```

No -name, no -type f, no quoting gymnastics. Just the pattern.

Key Differences From Find

1. Regex by default (find uses glob with -name)
2. Ignores .git, node_modules, .gitignore entries automatically
3. 5-10x faster than find due to parallel directory traversal
4. Smart case: case-insensitive unless pattern has uppercase
5. Colorized output by default

Common Flags

fd -e zsh # filter by extension fd -t f # files only fd -t d # directories only fd -t l # symlinks only fd -t x # executables only fd -H # include hidden files fd -I # don't respect .gitignore fd -u # unrestricted (-H -I combined) fd -g GLOB # use glob pattern instead of regex.

Execution

fd -x cmd {} # execute command per result (parallel) fd -X cmd{} # execute once with all results as args.

Placeholders: {} = path, {/} = basename, {/}/ = parent dir,

{.} = Path without extension, {/.} = basename without extension.

```
fd -e jpg -x convert {} {/.}.png # convert all jpgs to png
```

The -x flag runs in parallel by default — your cores earn their keep. -X collects all results and passes them as one batch. This is the find | xargs pattern without the pipe.

fd: Integration with zpw & arb -fzf

Fd: Integration With zpwr & Arb

// fd feeds arb. arb feeds your brain. the pipeline of knowing. //

Fzf_default_command

zpwr sets FZF_DEFAULT_COMMAND to use find + ag by default:

```
export FZF_DEFAULT_COMMAND='find * | ag -v ".git/"'
```

You can override this with fd for faster results:

```
export FZF_DEFAULT_COMMAND='fd --type f --hidden --follow'
```

fd is faster because it parallelizes traversal and skips .gitignore entries without needing ag to filter them out.

fd + Arb Patterns

```
fd -e zsh | arb --fzf          # fuzzy find zsh files
fd -t f | arb --fzf --preview 'bat {}' # find + preview
vim $(fd -t f | arb --fzf)      # find, select, edit
```

This three-tool combo (fd + arb -fzf + bat) is the holy trinity of file navigation. fd finds, arb -fzf filters, bat previews.

Temporal & Size Filters

fd -changed-within 1d # modified in last 24 hours fd -changed-before 1w # modified more than a week ago
fd -size +1m # files larger than 1 megabyte fd -size -10k # files smaller than 10 kilobytes Units: b (bytes), k (kilobytes), m (megabytes), g (gigabytes). Time: s (seconds), m (minutes), h (hours), d (days), w (weeks).

fd + bat

```
fd -e rs --exec bat{} # colored find results
fd -e md -X bat       # cat all markdown files at once
```

fd Vs Find Vs Locate

Find — universal. every Unix has it. slow on large trees. supports -exec, -newer, -perm, -prune. the Swiss Army. fd — fast parallel traversal, sane defaults, .gitignore aware. best for daily use in code repositories. locate — instant results from a pre-built database (updatedb). stale until the next cron run. no real-time accuracy.

Depth & Exclusion

fd -d 3 # max depth of 3 levels fd -E node_modules # exclude a directory by name fd -E '*.min.js' # exclude by glob pattern Stack -E flags to build precise exclusion lists on the fly.

Chapter 37 – ripgrep – grep At Light Speed

ripgrep: grep Reimagined

// grep -r is a campfire. ripgrep is a directed energy weapon. //

Ripgrep (rg) is a blazing-fast recursive grep written in Rust by Andrew Gallant (BurntSushi). It respects .gitignore, skips binary files, and uses Rust's regex engine for raw speed. 5-100x faster than grep -r on large codebases. Not a typo.

Basic Usage

```
rg PATTERN                # search current dir recursively
rg PATTERN path/          # search specific directory
rg 'fn main'              # literal or regex – rg auto-detects
```

Output is colored by default: filenames in magenta, line numbers in green, matches highlighted. Grouped by file.

Key Flags

rg -i PATTERN # case insensitive search
rg -l PATTERN # files with matches only (paths)
rg -c PATTERN # count matches per file
rg -w PATTERN # whole word matches only
rg -F PATTERN # fixed string (no regex interpretation)
rg -v PATTERN # invert match (lines NOT matching)

File Type Filtering

rg -t zsh PATTERN # search only zsh files
rg -T js PATTERN # exclude javascript files
rg -g '*.zsh' PAT # glob filter (include)
rg -g '!*.min.js' PAT # glob filter (exclude)
rg --type-list shows all built-in type definitions.

Context Lines

rg -A 3 PATTERN # 3 lines after each match
rg -B 3 PATTERN # 3 lines before each match
rg -C 3 PATTERN # 3 lines before AND after.

Multiline Matching

rg -U PATTERN # --multiline: match across lines.

```
rg -U 'struct \{[\\s\\S]*?\\}' # match entire struct blocks
```

Without -U, patterns match within single lines only.

The Competition

grep -r — universal, slow, no .gitignore, no parallelism ag — the silver searcher. fast, .gitignore aware. rg is 2-5x faster on most benchmarks. ack — Perl-based, pioneered the “better grep” category. slower than both ag and rg. still has loyal users.

ripgrep: Advanced & zpwr Integration

// rg -json | jq — structured grep for the data-obsessed. //

Structured Output

rg -json PATTERN # JSON output: one object per line Each line is a JSON object with type (begin, match, end, summary), path, line number, match offsets. Pipe to jq:

```
rg --json TODO | jq 'select(.type=="match") | .data.path'
```

Search & Replace Preview

rg -replace 'NEW' PATTERN # preview replacements This prints what the output WOULD look like with replacements but does NOT modify files. It's a dry-run. Capture groups work:

```
rg 'foo(\w+)' --replace 'bar$1' # foo_x -> bar_x
```

Statistics

rg -stats PATTERN # show match stats after results Prints: N matches, N matched lines, N files searched, N files matched, bytes searched, duration. Quick audit.

Custom File Types

```
rg --type-add 'zpwr:*.zsh' -t zpwr PATTERN
```

Define a custom type inline and use it immediately. Persist custom types in `~/.ripgreprc` (set RIPGREP_CONFIG_PATH).

zpwr Integration

zpwr grep uses ag (silver searcher) piped into arb -fzf:

```
zpwr grep # ag into arb --fzf for interactive search
```

zpwr wordsearch searches file contents with arb -fzf preview:

```
zpwr wordsearch # search by word in subdirectory files
```

Both use bat-powered previews in the arb -fzf window.

Arb + rg Live Reload

The ultimate interactive grep: rg feeds arb -fzf, bat previews:

```
rg --line-number PATTERN |  
arb --fzf --delimiter : --preview 'bat --highlight-line {2} {1}'
```

rg outputs file:line:match, arb -fzf splits on :, bat highlights the matching line. Three tools, one seamless experience.

Configuration

```
export RIPGREP_CONFIG_PATH=~/ripgreprc
```

Put default flags in this file, one per line: `--smart-case`, `--hidden`, `--glob=!.git`, `--max-columns=200` Also: `.rgignore` in any directory works like `.gitignore` — project-specific patterns that rg will skip. `.ignore` files (used by ag too) are also respected by rg.

Chapter 38 – neovim

Neovim: The Next Generation

// vim, but the timeline where Bram had unlimited funding. //

Neovim is the modernized fork of vim. Refactored internals, async everything, built-in LSP, Lua plugin API. In zpw, the default editor: ZPWR_VIM='nvim' in .zpw_env.sh.

The Init.vim Bridge

\$ZPWR_INSTALL/init.vim is just three lines:

```
set runtimepath^=~/.vim runtimepath+=~/.vim/after
let &packpath = &runtimepath
source ~/.vimrc
```

This means neovim sources the same .vimrc as vim. One config to rule them both. All 80+ vim plugins work in either editor. The symlink is created by plugins_install.sh:

```
ln -sf $ZPWR_INSTALL/init.vim ~/.config/nvim/init.vim
```

Neovim-specific Features

Async job control — run external commands without blocking the editor. vim 8 added this too, but nvim's API is cleaner (jobstart/jobstop). Built-in LSP — nvim 0.5+ has a native LSP client. Configure with lua: vim.lsp.start(). No plugin needed for basic LSP. Lua plugin API — write plugins in Lua, not just VimScript. Lua runs 10-100x faster than VimScript. :terminal — built-in terminal emulator. Better than vim's :terminal — proper job control, scrollback, and buffer integration. True color — 24-bit color out of the box. No termguicolors hacks needed (though the option exists).

Provider Support

nvim uses providers for python3 and node integration:

```
pip3 install pynvim          # python3 provider
npm install -g neovim        # node.js provider
```

Run :checkhealth to verify all providers are working.

zpw vim Info

zpw catviminfo and zpw catrecentviminfo read the nvim shada (shared data) file — the nvim equivalent of .viminfo.

Neovim: Configuration & Tips

// one vimrc. two editors. zero compromises. //

The Shared Config Philosophy

zpwr's init.vim just sources `~/.vimrc`. This means: 1. Every plugin in .vimbundle works in both vim and nvim 2. All keybindings, settings, and autocmds are shared 3. One file to edit: zpwr vrc opens .vimrc 4. No divergent configs to maintain across editors The .vimrc is the single source of truth. init.vim is a shim.

Neovim-specific Settings

Guard nvim-only settings with `has('nvim')`:

```
if has('nvim')
set inccommand=nosplit    " live substitution preview
set pumblend=15           " popup menu transparency
set winblend=15           " floating window transparency
endif
```

Inccommand is a killer feature: `:s/old/new` shows replacements live as you type. No more guessing what your regex will do.

:checkhealth

Run this in nvim to diagnose your setup. It checks: - python3/node providers (pynvim, neovim npm package) - clipboard support (pbcopy on macOS, xclip on Linux) - terminal capabilities, true color support - plugin health (many plugins register health checks) First thing to run when something feels wrong.

Session Tracking

Tpope/vim-obsession (in .vimbundle) tracks sessions:

```
:Obsession          # start tracking session to Session.vim
:Obsession!         # stop tracking and delete Session.vim
nvim -S Session.vim # restore session
```

Works in both vim and nvim since it's a VimScript plugin.

Why zpwr Supports Both

nvim is better locally — async, LSP, Lua, true color. But: - Remote servers often only have vim (or vi) - Docker containers ship vim, not nvim - macOS ships vim in `/usr/bin/vim` The shared .vimrc means your muscle memory works everywhere. nvim when you can, vim when you must, same config either way.

zpwr Editor Verbs

```
zpwr vrc            # edit .vimrc (shared by both)
```

Both editors, one config, zero excuses. The way it should be.

Chapter 39 – perl In zpwr

Perl Oneliners: The Secret Weapon

sed wishes it could do this. awk cries in a corner.

Perl is zpwr's scalpel for text surgery. Where sed and awk stumble on complex patterns, perl oneliners slice clean. zpwr uses perl throughout for text transformation, parsing, delimiter handling, and file manipulation.

The Key Flags

-e 'code' execute code -n loop over input lines (like sed) -p loop and print each line (like sed -p) -l chomp newlines, add them back on print -a autosplit into @F array (like awk) -F'sep' set field separator for -a -i in-place edit (like sed -i) -E enable say, state, switch (modern perl)

Whitespace Cleanup (scripts/lib.sh)

```
perl -pi -e 's@\s+$@\n@g; s@\x09$@    @g'
```

Strips trailing whitespace, converts trailing tabs to spaces. The -pi combo: -p loops, -i edits in place. zpwr's file cleaner.

Print Separator Lines

```
perl -le "print '#x80'"
```

Prints 80 # characters. The x operator repeats a string. Used in zpwrPrettyPrint for visual separators.

Delimiter Transformation

```
perl -F"$ZPWR_DELIMITER_CHAR" -anE 'say ...'
```

Splits lines on ZPWR_DELIMITER_CHAR into @F fields. Used heavily in zpwrContribCount, zpwrTotalLines for reformatting git contribution data into tabular output.

Filter Blank Lines

```
perl -ne 'print if /\S/'
```

Only prints lines containing non-whitespace. Used in zpwrRegenAllKeybindingsCache to clean keybinding dumps.

The perl Repl Alias

```
plr (alias for rlwrap perl -wnE'say eval()//$@')
```

Interactive perl REPL with readline. Type expressions, get results.

Perl Oneliners: Advanced Patterns

When you need a chainsaw, not a butter knife.

Path Quoting (zpwrGetFound)

```
perl -pe 's@^([~]*)([^~].*)$@ $1"$2"@;s@\s+@ @g'
```

Wraps non-tilde paths in double quotes and collapses spaces. Handles filenames with spaces for safe shell consumption.

Arb Result Formatting (zpwrAgIntoFzf)

```
perl -pe 's@^(.*?):(\d+):(.*)@+$2 "PWD/$1"@'
```

Converts grep output (file:line:match) into vim args (+line "file"). The brain behind zpwr grep -> arb -fzf -> vim pipeline.

Verb List Formatting (zpwrVerbsFZF)

```
perl -e '@a=<>;for(@a){print "zpwr $1" if m{^\(\\S+)\s+}}'
```

Reads verb list, extracts first word, prepends 'zpwr'. Builds the command string that arb -fzf executes on selection.

In-place Rename (zpwrChangeNameZpwr)

```
perl -i -pe 's@\bOLDNAME\b@NEWNAME@g' '**/*(*)'
```

Renames all occurrences across all files. The \b word boundary prevents partial matches. The **(*) glob hits all regular files.

Login Counting (zpwrLoginCount)

```
perl -e '""'print qx(last -f "$_") for </var/log/wtmp*>'""'
```

Reads all wtmp files and runs last on each. The diamond operator <> globs and iterates. Then pipes to:

```
perl -lane '""'print [0] if /\S+/ && !/wtmp/'""'
```

Extracts username (first field) from non-empty, non-header lines.

File Count Dedup (zpwrEnvCounts)

```
perl -lne '@l=<>;@u=do{my %seen;grep{!$seen{$_}++}@l};'
```

Reads all lines, deduplicates with a hash, then checks if each unique path is a real file. The densest oneliner in zpwr. Counts unique zpwr files across all tracked locations.

Ip Address Extraction (ipa Alias)

```
ifconfig | perl -lane 'do {print \[1]=~s/addr//r;exit0} if /inet\s/ and !/127/'
```

Finds the first non-loopback inet address. The s///r flag

Returns the modified string without changing the original.

Why perl Over Sed/awk?

1. Perl regex is more powerful (lookahead, backrefs, /x)
2. -i flag for in-place edit is reliable across OS
3. Complex logic in one line (if/unless/for/do blocks)
4. @F autosplit is awk-like but with perl's full stdlib
5. Perl is installed everywhere (even minimal containers)

Chapter 40 – Hidden Subsystems

Zconvey: Terminal Communication

Every terminal screams its identity into the void.

What Is Zconvey?

MenkeTechnologies/zconvey - a zinit plugin Allows sending commands between terminal sessions. Your terminals can talk to each other. Yes, really. Loaded via zinit in [\\$ZPWR_INSTALL/.zshplugins](#).

How zpwr Uses It

Every time you press Enter, zpwrAcceptLine fires. It calls zc-rename to label the current terminal with: TTY:\${TTY} PID:\${\$} CMD:\$BUFFER PWD:\${PWD} DATE:\$(date) This broadcasts your terminal's state to the zconvey bus. Every pane, every tab, every session - all labeled.

The Convey Variables

ZPWR_CONVEY_NAME the full label string sent to zconvey ZPWR_CONVEY_LAST_CMD the last command (for zconvey commands) When you run a zc-* command, it preserves the previous label so your terminal doesn't get renamed to "zc-send ls".

The Rename Logic

```
if ! [[ $(zpwrExpandAliases $BUFFER) = zc* ]]; then
  ZPWR_CONVEY_NAME="TTY:${TTY} PID:${$} CMD:..."
  zc-rename $ZPWR_CONVEY_NAME
```

If BUFFER starts with zc*, keep the old label. Otherwise, stamp the terminal with the new command.

Why This Matters

tmux status bar can show what each pane is running. You can identify panes by their last command and PWD. Remote debugging: see exactly what terminal ran what. It's like process accounting, but for your shell sessions.

Sending Commands Between Terminals

Zc-send 2 'cd /tmp && ls' send command to terminal 2 zc-all 'echo hello' broadcast to all terminals Your terminals are a mesh network. Act accordingly. SIGNAL: zpwrAcceptLine is the heartbeat of zpwr. Every Enter keypress: rename terminal, check for clear, detect file-modifying commands, send tmux keys. zconvey makes that heartbeat visible across sessions.

The 24-Hour Updater Loop

Your system updates itself while you sleep.

The Infinite Loop

`$ZPWR_SCRIPTS/autoUpdater.sh` runs forever. Every 24 hours it wakes up and updates everything.

```
while true; do
  bash -l updater.sh -e
  # sleep in 10-minute chunks until 24h elapsed
done
```

Uses Perl Time::Piece to calculate the next update time. Logs the time diff on each wake to `$ZPWR_LOGFILE`.

What Gets Updated

Updater.sh orchestrates the full update: `zpwrUpdater` runs the main `zpwr` update sequence `zpwrUpdateDeps` updates all package manager dependencies Homebrew, npm, pip, gem, go - everything gets refreshed. History files backed up. Logs rotated. Deps pinned.

Systemd Service (LINUX)

On Linux, `autoUpdater.sh` runs as a systemd service. `zpwr` restart restart the `zpwr` systemd service `zpwr` serviceup start and enable the systemd service `zpwr` servicedown stop and disable the systemd service On macOS, `$ZPWR_SCRIPTS/updaterForLaunchCtl.sh` is used.

Email Notifications

`$ZPWR_SCRIPTS/updaterEmail.sh` Pipes the update log to email via mutt. Wake up to a full report of what changed overnight. Your system admin is a bash script. It never sleeps.

The Sleep Strategy

Doesn't sleep for 24h straight (that would drift). Sleeps in 10-minute intervals, checks elapsed time:

```
((timediff > $((3600 * 24)))) && break
```

This handles laptop suspends and clock adjustments. WARNING: The updater keeps `zpwr` and all deps current. If something breaks after an update, check the log: `zpwr` taillog to see what the updater did last. The machine maintains itself. You just write code.

The 32-Pane REPL Layout

Every language, one screen, zero excuses.

The tmux Keybindings

Prefix R source `$ZPWR_TMUX/thirtytwo-panes-repl.conf` prefix O source `$ZPWR_TMUX/sixteen-panes.conf` prefix G source `$ZPWR_TMUX/eight-panes.conf` prefix F source `$ZPWR_TMUX/four-panes.conf` One keystroke splits your terminal into a REPL army.

The 32 Repls (prefix R)

Pane 0: lua pane 1: swift pane 2: ghci pane 3: clisp pane 4: ocaml pane 5: clojure pane 6: pry (ruby) pane 7: bpython pane 8: php -a pane 9: groovysh pane 10: csharp pane 11: kotlinc pane 12: node pane 13: coffee pane 14: tcsh pane 15: ksh pane 16: bash pane 17: csh pane 18: perl -dE1 pane 19: gore (go) pane 20: jshell pane 21: mycli pane 22: pgcli pane 23: python2 pane 24: nslookup pane 25: mongo pane 26: irb pane 27: scala pane 28: utop pane 29: iex pane 30: bc pane 31: gforth 32 languages. 32 panes. One tmux window. Absolute madness.

The Vim-tmux Execution Pipeline

Write code in vim, send it to a REPL pane: Supported languages for vim-to-tmux execution: zsh, bash, python, ruby, perl, node, go java, scala, R, octave Select code in vim, fire it into the matching pane. Edit-run-edit-run without ever leaving vim.

Progressive Layouts

Don't need 32 languages? Scale down: prefix G 8 panes - quick polyglot session prefix O 16 panes - serious multi-language work prefix R 32 panes - full REPL matrix (needs big screen) Each layout is a tmux conf file in `$ZPWR_TMUX/` NOTE: prefix R on a 13" laptop is an act of hubris. You'll need at least a 27" monitor to read anything. But hey, it works. And it's glorious.

Cross-Platform Installer

One repo to rule them all, one script to install them.

Installer Scripts In `$ZPWR_INSTALL`

Formulae.sh Homebrew formulae (macOS/Linux) casks.sh Homebrew casks (macOS GUI apps) mincasks.sh minimal cask set for lean installs taps.sh Homebrew tap repositories npm_install.sh Node.js packages via npm npm.sh npm package list gems.sh Ruby gems pip_install.sh Python packages via pip pip.sh / pip3.sh pip package lists go_install.sh Go packages rustupinstall.sh Rust toolchain via rustup.

Editor & Plugin Installers

Neovim_install.sh Neovim setup vim_install.sh Vim from source vim_plugins_install.sh Vim plugin manager + plugins zsh_plugins_install.sh Zsh plugins via zinit tmux_plugins_install.sh Tmux plugin manager + plugins spacemacs_install.sh Spacemacs (Emacs) setup.

Shared Infrastructure

Common.sh shared installer functions used by all scripts zpwrInstall.sh the master installer orchestrator Supports 20+ Linux distros, macOS, and FreeBSD. Detects your OS and calls the right package manager.

zpwr Verbs For Installation

zpwr install run configure, make, make install zpwr uninstall uninstall all zpwr configs zpwr regenconfiglinks regen symlinks from `$ZPWR_INSTALL` to `$HOME` regenconfiglinks recreates dotfile symlinks after a reinstall.

The Symlink Strategy

zpwr doesn't copy config files to `$HOME`. It symlinks them. `.vimrc`, `.tmux.conf`, `.gitconfig` - all point back to `$ZPWR_INSTALL`. Edit once in the repo, git commit, and every machine gets it. zpwr `regenconfiglinks` rebuilds them if they break. PHILOSOPHY: Package lists are just text files. Add a line to `formulae.sh`, and your next machine gets it. Your entire dev environment is version-controlled.

Chapter 41 – Quality Assurance

The Test Suite

60 test files standing between you and chaos

Testing Framework

zpwr uses zunit - a testing framework written in zsh. Config lives in `$ZPWR/.zunit.yml` Tests dir: `$ZPWR_TEST`
Output: `$ZPWR_TEST/_output` Support helpers: `$ZPWR_TEST/_support`.

Test Categories

T-autoload-* verify all autoloaded functions load, defined, existence, fpath, whence, type, body Every autoload function must: exist, load, have a body. t-autoload-subdir-* same checks for subdirectory autoloads t-env-* verify environment variables defined, exported, comprehensive, type checks Every ZPWR_* var must be defined and exported. t-scripts-* verify script files executable, readable, non-empty, have shebangs, valid syntax t-verbs-* verify zpwr verb definitions key format, non-empty values, callable, type checks.

More Test Files

T-functions.zsh test core shell functions t-callable-fns.zsh verify functions are callable t-navigation.zsh test directory navigation t-install-files.zsh verify installer file integrity t-repo-files.zsh verify repo structure t-plugin-counts.zsh verify plugin load counts.

Running Tests

zpwr test run all zpwr tests zpwr testall run ALL env tests (comprehensive sweep) fail_fast: true means first failure stops the run. verbose: true means you see every assertion. GUARANTEE: Every autoload loads. Every env var defined. Every script executable. Every verb callable. 60 test files verify zpwr won't break your shell. If tests pass, your environment is sound. Ship it.

Chapter 42 – Colorization Stack

GRC: Generic Colorizer

Because monochrome ls is a crime against humanity.

What Is Grc?

grc wraps commands and adds color output via regex rules. It reads a config file that maps patterns to colors. Any command's output can be colorized. Zero modifications.

Grc In Zpwrlistnoclear

The fallback chain when listing files:

1. eza preferred (color-scale, git status)
2. grc + ls fallback (colorized via conf.gls)
3. plain ls last resort (no color config)

Platform-specific Commands

MacOS:

```
grc -c $HOME/conf.gls gls -iFlhA --color=always
```

Linux:

```
grc -c $HOME/conf.gls ls -iFlhA --color=always
```

MacOS uses gls (GNU ls from coreutils) because BSD ls doesn't support all the flags. Linux uses native ls.

The Conf.gls Config

Custom color configuration for ls output, lives at:

```
$HOME/conf.gls
```

Maps file types, permissions, sizes to ANSI colors. Directories, executables, symlinks - all distinct colors.

Other Grc Targets

grc can colorize almost any command: ping latency numbers in color traceroute hop-by-hop color coding make errors red, warnings yellow gcc compiler output colorized netstat connection states highlighted diff additions green, deletions red.

Grc In Log Colorization

`$ZPWR_SCRIPTS/colorLogger.sh` uses `grc` for system logs. `tmux` control windows show colorized live log output. TIP: If `eza` is missing and `ls` looks dull, install `grcrs`. It's the middle tier of `zpwr`'s color fallback chain. One package, instant color for dozens of commands.

Ponysay, Lolcat & ASCII Art

Enterprise-grade cartoon horse infrastructure.

The Ascii Art Toolchain

Ponysay My Little Pony ASCII art (cowsay but fabulous) lolcat rainbow gradient text output figlet large ASCII text banners from font files cowsay classic ASCII art speech bubbles toilet another ASCII art text renderer `zpwr` combines all of these for its startup banner.

zpwr Banner Verbs

`zpwr bannerpony` hostname banner with ponysay `zpwr bannerlolcat` ponysay + lolcat rainbow gradient `zpwr bannernopony` banner without ponysay `zpwr figletfonts` preview all installed figlet fonts Each verb calls a different autoloaded function:

```
zpwrPonyBanner, zpwrBannerLolcat, zpwrNoPonyBanner
```

The Startup Banner

On shell startup, `zpwrDarwinBanner` / `zpwrLinuxBanner` fires. It picks a random figlet font each time via:

```
$ZPWR_SCRIPTS/macOnly/figletRandomFontOnce.sh
```

`ZPWR_BANNER_TYPE` controls pony vs no-pony: ponies -> `zpwrBannerLolcat` (rainbow horse) other -> `zpwrNoPonyBanner` (figlet only)

Banner Control

`ZPWR_BANNER_CLEARLIST` controls startup banner + `ls` display When true: clear screen, show banner, then `zpwrListNoClear`. When false: skip the banner entirely (fast startup).

Font Logging

The random figlet font name is logged to:

```
$ZPWR_LOGFILE
```

Want to know which font made that gorgeous banner? `zpwr taillog` and look for the font name. Found one you love? Set it permanently in your config. REALITY CHECK: A rainbow pony greeting you on every terminal open is either peak engineering or peak insanity. `zpwr` does not distinguish between the two. Your hostname, rendered in ASCII art, by a cartoon horse. This is what unlimited power looks like.

GRC, zcolorizer & the Color Pipeline

Every byte of output deserves to be beautiful.

The Full Colorization Stack

zpwr has a color tool for every type of output: FILE LISTING eza (color-scale, git status) -> grc+ls -> plain ls
FILE VIEWING bat (syntax highlighting) -> cat ZPWR_COLORIZER=bat in `$ZPWR_ENV/.zpwr_env.sh` LOG
VIEWING zcolorizer (log colorizer) -> plain tail zpwrTailLog pipes `$ZPWR_LOGFILE` through zcolorizer. GIT
DIFFS delta (syntax-aware diff) -> plain diff delta adds line numbers, word-level highlighting, and side-by-side
view with Dracula theme.

Zcolorizer: Log File Colorizer

Zcolorizer parses log formats and applies semantic colors: timestamps, severity levels, PIDs, IP addresses.
zpwr taillog uses zcolorizer for colorized log tailing. System logs become readable at a glance.

Color Testing Verbs

zpwr colortest test terminal basic 16 colors zpwr colortest256 test full 256-color palette zpwr pygmentcolors
show all pygment color schemes zpwr colorsdiff colorized side-by-side diff Use these to verify your terminal
supports the full palette.

The Fallback Philosophy

Every color tool has a plain fallback: No eza? grc+ls. No grc? plain ls. No bat? cat. zpwr degrades gracefully.
Color is never a hard dependency. SSH into a minimal server? Everything still works. Just less pretty. Function
over form, always. THE RESULT: In a fully-equipped zpwr terminal: File listings glow. Logs are semantic. Diffs
are surgical. Your eyes never have to parse raw monochrome again. The color pipeline is zpwr's cosmetic
surgery department.

Chapter 43 – The End

The Epilogue: You Are the Wizard

The terminal bows to you.

What You've Learned

From keybindings to kernel. Every chapter, every verb. Autoloaded functions. Tmux pipelines. Fzf integration. Vim execution. Perl oneliners. Systemd services. 32-pane REPL layouts. Cross-platform installers. A 24-hour updater loop. A 60-file test suite. A colorization stack five tools deep. And a startup banner delivered by a rainbow pony.

The 5 Commands To Memorize

zpwr verbs list every zpwr subcommand zpwr doctor run environment health check diagnostic zpwr bench benchmark startup with percentiles zpwr wizard interactive guided tour (you're here) zpwr study split-pane learning: book left, shell right Five commands. That's your entire onboarding.

How To Add Your Own Verbs

1. Create an autoload function in `$ZPWR/autoload/common/`
2. Add to `ZPWR_VERBS` in `$ZPWR_ENV/zpwr.zsh` Format: `ZPWR_VERBS[myverb]='myFunc=description'`
3. Reload: `exec zsh`. Your verb is live.

How To Extend The Wizard

Add a new page_NNN.zsh to `$ZPWR_INSTALL/wizard_pages/` Follow the format: `PAGE_TITLE, PAGE_CHAPTER, PAGE_CONTENT()`. The wizard auto-discovers pages. No registration needed.

How To Contribute

Fork github.com/MenkeTechnologies/zpwr Add your feature. Write tests. Submit a PR. The codebase is the documentation. Read it.

The zpwr Philosophy

Verbs, not scripts. Discoverability over memorization. Speed over bloat. Fallbacks over hard dependencies. Convention over config. Your shell should just work.

One Last Thing

Run `zpwr about` to see the zpwr banner. Then open a new terminal. Watch the pony greet you. You are the wizard now. The terminal is yours.

End Of Transmission

// ZPWR WIZARD ENCYCLOPEDIA v1.0 // SIGNAL LOST //

May your shell be fast, your pipes be clean, and your exit codes always zero.

Chapter 44 – The Verb Dispatcher, Line By Line

Everything in the previous chapters is reached through one entry point. The `zpwr` command is not a binary, not a case statement, and not a directory of subcommand scripts. It is a single anonymous zsh function that populates one associative array and then evaluates a string out of it. The whole dispatcher lives in `env/zpwr.zsh`, and it is worth reading closely, because every design decision in it is load-bearing for how verbs behave at the command line.

The Table Is The Program

The file opens by guaranteeing the table exists and is the right type:

```
if ! (( ${ZPWR_VERBS} )) || \
  [[ ${parameters[ZPWR_VERBS]} != association ]]; then
  typeset -Ag ZPWR_VERBS
fi
```

This is not paranoia. `ZPWR_VERBS` is a global that plugins extend — a zinit plugin can register its own verbs into the same namespace, which is why the encyclopedia's own verb count is computed by counting `ZPWR_VERBS[ assignments` across the source and any installed plugins rather than being a fixed number. The guard checks both existence and *type*. A scalar named `ZPWR_VERBS` inherited from a stray export would otherwise cause every subsequent subscript assignment to silently do something wrong. The same guard is repeated for `ZPWR_VARS`, the auxiliary table that carries dispatcher state.

The body then runs inside an anonymous function invoked with the caller's arguments:

```
() {
  local v verb cmd found ret exp
  builtin emulate -L zsh
  setopt nullglob globdots extendedglob
  ...
} "$@"
```

Two things matter here. `emulate -L zsh` with the `-L` flag makes the option changes local to the function, so `nullglob` and `globdots` — needed for the conditional registration blocks that test directories — evaporate on return and never leak into the interactive shell. And `builtin emulate` bypasses any user-defined function shadowing `emulate`, which is exactly the class of self-inflicted breakage a shell framework must be immune to.

Value Shape: Target, Then Prose

Each entry packs two fields into one string separated by the first `=`:

```
ZPWR_VERBS[aliasrank]='zpwrAliasRank=mine history for alias usage stats'
ZPWR_VERBS[attach]='$ZPWR_ZTMUX attach-session=attach to tmux session'
ZPWR_VERBS[filesearch]='zpwrFzfFilesearchVerb c=c a file in a sub dir'
```

The left side is what gets executed; the right side is the human description that feeds `zpwri -h`, the `fzf` verb picker, and this book. Splitting is done with a single parameter expansion:

```
v=$ZPWR_VERBS[$verb]
cmd=${v%%%=*}
```

`%%%=*` strips the longest match from the end, so the first `=` wins and any `=` inside the description is harmless. Note what this implies for the target side: an entry whose command legitimately contains `=` cannot be expressed directly, which is why targets are function names and simple invocations rather than inline assignments. The indirection through an autoload function is not stylistic; it is forced by the encoding.

Targets are also allowed to be *unexpanded* variable references — `$ZPWR_ZTMUX attach-session` is stored literally with the dollar sign intact. Expansion is deferred to the `eval` at dispatch time, which is what lets the `tmux` binary be swapped (from `tmux` to `ztmux`) after the verb table was written, without rewriting a single entry.

Chained Targets

The target field is not one command. It is a `;`-separated list, split with a substitution-flag form that uses `%` as the delimiter so the `;` itself can be the separator without quoting gymnastics:

```
for exp in ${s%;%}cmd; do
```

So `filesearch`, whose target is `zpwriFzfFilesearchVerb c`, is one command, but a verb can equally chain several. Each element is dispatched independently, and `ret` is overwritten by each, meaning the exit status of a chained verb is the status of its *last* element, not an aggregate. That is a deliberate simplifying choice: chained verbs are composite user-facing actions, not transactions, and a non-fatal failure in an early step (a cache clean that found nothing to clean) should not poison the result.

Aliases Versus Everything Else

The single most interesting branch in the dispatcher is this:

```
if alias $exp 1>/dev/null 2>&1; then
    eval "$exp"
else
    eval "$exp " ${q}@
fi
```

If the target names an alias, the remaining command-line arguments are *not* appended. If it names anything else — a function, a builtin, an external command — they are, quoted with `${q}@`.

The reason is that `zsh` aliases are textual expansions that frequently already end in their own arguments, a pipeline, or a redirection. Appending user arguments to `foo | less` would put them after `less`, not after `foo`. Rather than attempt to parse the alias body and splice arguments into the right position, the dispatcher treats aliases as zero-argument macros. Verbs that must take arguments therefore have to resolve to functions. This is why the verb table is dominated by `zpwri*` function names: it is the only target kind that composes with arguments.

`${q}@` applies `zsh`'s quoting flag to each positional parameter separately, so a filename with a space survives the round trip through `eval` as one word. Using `"$@"` there would be wrong — the arguments are being spliced into a string that is about to be re-parsed, so they must carry their own quoting.

The Pre-Dispatch Gauntlet

Before any lookup happens, the argument runs a short obstacle course:

```

if [[ $verb =~ '-V|--version' ]]; then
    zpw>About | less
    return 0
fi

if [[ $verb =~ '-h|-\\?|--help' ]]; then
    zpw>ListVerbsColor | less
    return 0
fi

while [[ $verb == '--' ]]; do
    verb="$1"
    shift
done

```

The `--` loop is a `while`, not an `if`, so repeated end-of-options markers are tolerated; this matters when `zpw` is invoked from another wrapper that defensively inserts its own `--`.

Then the self-guard:

```

if [[ $(basename -- $verb) == $(basename -- ${ZPWR_VARS[VERB_0]}) ]]; then
    return 0
fi

```

`ZPWR_VARS[VERB_0]` was captured at the top of the file as `$0` — the path of the verb file itself. The test detects the case where the file is being executed with its own name as the first argument, which is what the test runner does when it sources the file directly. Without the guard, a test harness sourcing `env/zpw.zsh` would attempt to dispatch a verb named after the file and produce a spurious unknown-subcommand error. It costs two forks on a path that only tooling takes.

Conditional Registration

The tail of the table registers verbs only when their prerequisites exist:

```

if zpwCommandExists systemctl; then
    ZPWR_VERBS[restart]='restart=restart zpw systemd service'
    ZPWR_VERBS[serviceup]='ssu=start and enable systemd service'
    ZPWR_VERBS[servicedown]='ssd=stop and disable systemd service'
fi

```

The consequence is that the verb namespace is machine-dependent by design. On a macOS laptop the `systemd` verbs do not exist at all — they are not stubs that print “not supported”, they are absent, so `zpw servicedown` reports an unknown subcommand and `zpw -h` never lists it. The same pattern gates the `tmux` start-up verbs on the existence of the Python scripts directory.

Note which predicate is used where. `zpwCommandExists` checks `(( $+commands[$cmd] ))` under `zsh` — the hash table of external binaries only. `zpwExists` uses `type -a` and therefore also matches functions and aliases, with an explicit rejection of suffix aliases:

```

t=$(type -a -- "$1" 2>/dev/null) || return 1
[[ $t != *"suffix alias"* ]] || return 1

```

A suffix alias (the `alias -s` form that makes `foo.md` open an editor) would otherwise report as “existing” for a name that cannot actually be invoked as a command. Gating registration on the wrong predicate is how frameworks end up with verbs that exist in the help output and fail at runtime.

Failure

The unknown-verb path is three lines:

```
if [[ $found == false ]]; then
  zpwrlLogConsoleErr "Unknown subcommand: '$verb'"
  return 1
fi
```

There is no fuzzy matching, no “did you mean”, no automatic fallback to the closest key. Given that many verbs are destructive — cache wipes, git resets, repo re-clones — a dispatcher that guesses is a dispatcher that eventually guesses `zpwrl reset` when you typed something adjacent. The discovery affordance is the `fzf` verb picker, which is explicit and interactive; the dispatcher itself is exact-match only.

Finally, note that dispatch happens through `eval` in the *calling* shell context, not a subshell. This is what allows `zpwrl cd`, `zpwrl autoload`, and the ZLE-buffer-editing verbs to change the state of your interactive session. A verb is not a program you run; it is code you inject.

Chapter 45 – Autoload As Architecture

The autoload tree is where the framework keeps essentially all of its behavior. The rule is one file, one function, filename equals function name, and the tree under `autoload/` is partitioned into subdirectories that are each independently placed on `fpath`. The bootstrap is two lines, visible in the test harness at `tests/test_lib.zsh`:

```
fpath=($ZPWR_AUTOLOAD/*(/) $fpath)
autoload -z $ZPWR_AUTOLOAD/*/*(:t)
```

Both lines lean on glob qualifiers hard enough that they deserve unpacking. `*(/)` restricts the glob to directories, so `fpath` gains exactly the subdirectory layer — `common`, `comps`, `comp_utils`, `darwin`, `fzf`, `linux`, `systemctl` — and never the tree root or any stray file at that level. `/*/*(:t)` matches plain files (`.`) one level down and applies the `:t` modifier, which strips the directory component and leaves the tail. So `autoload -z` receives a list of bare names, not paths.

That distinction is the entire mechanism. `autoload NAME` tells zsh “a function called NAME exists; find it on `fpath` when someone calls it.” Passing a full path instead would have created a function whose name contained slashes. The `-z` flag forces zsh-style autoloading (as opposed to ksh-style), which matters because the two differ in how the loaded file’s contents are interpreted: under zsh-style loading the entire file body becomes the function body.

Why Every File Ends By Calling Itself

Open almost any file in `autoload/common` and the last line is the function invoking itself:

```
function zpwChpwd(){
    : echo "$@"
}

zpwChpwd "$@"
```

This looks redundant and is not. Under zsh-style autoload, the file body *is* the function body, so the first invocation of `zpwChpwd` executes a body that (a) defines a function named `zpwChpwd`, replacing the autoload stub, and (b) immediately calls it with the original arguments. Subsequent calls go straight to the real definition. The pattern is the standard zsh idiom for making a single file serve as both an autoload payload and a directly-sourceable script, and it means the *first* call to any `zpw` function costs one extra function definition and nothing else.

The convention is loose enough to be useful. Several files define a differently named inner function and dispatch to it on the last line — `zpwRunner` ends with `__runner "$@"`, `zpwRegenZshCompCache` ends with `__regenZshCompCache "$@"`. The public name is the filename; the implementation name can be whatever the author wanted, including a private double-underscore name that will not collide in the global function namespace. A few files are pure data or pure side effect and have no trailing call at all.

Every file also carries a two-line editor preamble:

```
# -*- mode: sh -*-
# vim: set ft=sh:
```

Autoload files have no extension — they cannot, because the filename is the function name — so neither Emacs nor vim can infer a mode from it. The modelines restore syntax highlighting for a tree of several hundred

extensionless files. This is a trivial detail that is nonetheless the difference between the autoload tree being editable and being a wall of monochrome text.

Partitioning By Platform And Role

The subdirectory split is not cosmetic organization; it changes what gets loaded. `common` holds everything cross-platform. `darwin` and `linux` hold platform-specific implementations. `systemctl` holds the service-management functions, and — critically — that directory is *not* autoloaded by the `autoload/*/` glob in the interactive path. It is loaded separately, gated:

```
if zpwrcCommandExists systemctl; then
  if zpwriIsZsh; then
    eval 'autoload -z $ZPWR_AUTOLOAD_SYSTEMCTL/*(:t)'
  fi
fi
```

The `eval` wrapper exists because this file is also read by bash, where the glob qualifier `(.:t)` is a syntax error at parse time, not at execution time. Hiding the zsh-only syntax inside a string that bash never evaluates is the standard technique for a single file that must parse under two shells. The `zpwriIsZsh` test is a one-liner — `test -n "$ZSH_VERSION"` — and appears throughout the shared files for the same reason.

The three completion directories carry different contracts. `comps` holds real `#compdef`-style completion functions. `comp_utils` holds helper functions that completion functions call — including deliberate overrides of zsh's own `_files`, `_command_names`, and `_parameters`, which work precisely because the zpwrc autoload directories are prepended to `fpath` and therefore shadow the system completion tree. `fzf` holds the completion-generator pairs described in a later chapter.

The Cost Model

The reason this structure exists is startup cost. An autoload declaration is a hash-table entry pointing at a name; zsh does not open, read, or parse the file until the function is called. Declaring several hundred functions is a directory scan plus several hundred small allocations. Sourcing several hundred function definitions instead would mean parsing every byte of every file on every shell start, including the ones for the platform you are not on and the tools you do not have installed.

The tradeoff is that a syntax error in an autoload file is not detected at startup. It surfaces the first time you call that function, possibly months later, possibly in the middle of something else. This is exactly why the test suite contains a generated per-function family that loads and inspects every single autoload file — the tests buy back the error detection that lazy loading gave away. That is the honest accounting: autoloading is not free, it moves a cost from startup to CI.

Side Effects That Cannot Assume A Terminal

Buying error detection with a test suite has a consequence the self-invocation pattern makes sharp. Because the file body *is* the function body and ends by calling itself, an autoload file is simultaneously a definition and a program, and the test suite runs the program. Anything the function does at the end of its run happens in CI, on a machine with no terminal attached.

`autoload/common/j` is the case where that bites. The function takes one or more paths, creates any missing parent directories, touches the files, and then opens them:

```

for file;do
  dirname="$(dirname $file)"
  [[ "$dirname" != . ]] && command mkdir -p "$dirname"
  touch "$file"
done

if [[ -t 0 ]]; then
  "${ZPWR_ZMAX:-vim}" "$@"
fi

```

The `for file;do` with no `in` list iterates the positional parameters, which is the shell's own idiom for “every argument” and avoids restating “\$@”. The `dirname` guard skips the `mkdir` when the path has no directory component, because `mkdir -p .` is a pointless syscall on the common case of creating a file in the current directory.

The last block is the interesting one, and it is three separate decisions in four lines. `[[ -t 0 ]]` tests whether standard input is a terminal. Under `zunit`, under a pipeline, under a CI runner, it is not — and a full-screen editor launched with no controlling terminal does not fail, it blocks, which turns a test run into a hang with no output to diagnose. The test that motivates the guard is in `tests/t-navigation.zsh` and does exactly what a caller would:

```

run j "$tmpfile"
assert $state equals 0
run test -f "$tmpfile"
assert $state equals 0

```

That is also why the guard is an `if` block rather than the shorter `[[ -t 0 ]] && "${ZPWR_ZMAX:-vim}" "$@"`. A trailing `&&` list is the last command in the function, so its exit status becomes the function's — and when the test is false, that status is 1. The file-creation work would have succeeded and the function would have reported failure anyway, for the sole reason that nobody was watching. The `if` form leaves the status of the `touch` loop standing.

The editor itself is `"${ZPWR_ZMAX:-vim}"`, not `$EDITOR` and not `$ZPWR_VIM`. The `:-` default matters because this file can be executed as a script by a process that never sourced the environment layer, where none of the framework variables exist; `vim` is the one editor that can be assumed present. Quoting the whole expansion means a `ZPWR_ZMAX` holding a command with arguments would be run as a single filename rather than split — the variable is a bare command name by construction, and the quoting encodes that expectation rather than inviting a multi-word value.

Recovering A Body From A Stub

There is one place where lazy loading is genuinely inconvenient, and the source tree contains an unusually thorough workaround for it. The environment-index generator needs the *text* of every function, including ones that have never been called and therefore exist only as stubs. Asking `zsh` for such a function's definition returns the stub, not the body:

```

if [[ ${functions[$k]} == *'autoload -X'* ]]; then

```

When that test fires, the generator hunts for the file on disk, walking `fpath` first, then falling back to parsing `type -a` output for a `... from PATH` suffix, then to extracting a path out of the stub's own `autoload -X` text, then to evaluating an `fpath=(...)` assignment embedded in the stub. Four escalating strategies, because plugin managers construct autoload stubs in several different shapes and any of them can show up in a live shell.

Having found the file, it reconstructs a plausible function definition: drop the trailing self-invocation line if present, rewrite the first `function NAME() {` or `NAME() {` line into canonical `NAME () {` form, and if no declaration line was found at all, wrap the whole file body in a synthetic one. That last case is the interesting one — it handles autoload files that are bare statement sequences with no enclosing function at all, which

under zsh-style autoloading is perfectly legal.

The result is that the environment index contains real, readable bodies for functions that the shell has never loaded. Achieving that required knowing the stub formats of every plugin manager in the stack. It is a good example of what "lazy loading" costs once you want to introspect the thing you deferred.

Chapter 46 – The Completion Stack: `_megacomplete` And Friends

Standard zsh completion answers one question: given this command and this partial word, what are the valid values? The completion stack here answers a broader one: given this command line, this shell's history, this tmux window, and this system clipboard, what might the user plausibly want to type next? The mechanism is a replacement for the top-level completion entry point plus targeted overrides of three of zsh's own completion functions.

The Entry Point

`autoload/comp_utils/_megacomplete` is short enough to read in full, and every block in it is a separate fallback source:

```
local -a whitelist_tmux_completion
whitelist_tmux_completion=(ping ping6 nslookup nmap dig digs host
    mtr traceroute tracepath whois torsocks proxychains nc netcat
    curl wget http)
```

That list is the set of commands whose arguments tend to already be visible on your screen — hostnames, IPs, URLs from a previous `dig` or `curl`. It is deliberately narrow.

The first real work is finding the actual command, which is not simply `$words[1]`:

```
continueRegex='^(\-.*|builtin|command|sudo|env|nohup|nice|time)$'
i=1
cmd=${(Q)words[i]}

while [[ $cmd =~ $continueRegex ]]; do
    cmd=${(Q)words[$((++i))]}
done
```

This walks past the prefix words that wrap a real command — `sudo`, `env`, `nohup`, `nice`, `time`, the `builtin/command` precommand modifiers, and any leading option. So `sudo -E nohup curl` resolves `cmd` to `curl`. `${(Q)...}` strips one level of quoting so a quoted command name still matches. Everything downstream that keys off `cmd` — the tmux whitelist in particular — gets the semantically correct answer rather than the syntactically first word.

Then the normal completion system runs, and its result is captured rather than returned:

```
\_complete 66 ret=0 || ret=1
```

The backslash suppresses alias expansion on the name. `ret` now records whether zsh's own completion produced anything, and that single bit drives the rest of the function: the extra sources are consulted only when the standard answer was empty. This is the design decision that makes the stack usable rather than noisy. When you tab-complete `git checkout` and zsh offers you branches, you get branches — not branches plus your clipboard plus your last command's arguments.

Source Two: The Screen

```
if [[ -n "$TMUX_PANE" ]]; then
    if (( $whitelist_tmux_completion[(I)$cmd] )); then
        zpwrtmux_pane_words
    fi
fi
```

`$array[(I)value]` is the reverse-subscript flag: it returns the index of the last matching element, or 0 if absent, so `(( ... ))` is a membership test without a loop. Inside `tmux`, for whitelisted network commands, the words currently displayed in the pane become completion candidates. The value is obvious once you have used it — you ran `dig`, the answer is on screen, and now you want to `curl` that address without retyping or reaching for the mouse.

Notice this block is unconditional on `ret`. Screen words are offered alongside normal completions rather than only as a fallback, because for these specific commands the on-screen text is usually a *better* candidate than anything the completion system knows.

Source Three: History, At Position One

```
if (( $CURRENT == 1 )); then
    _complete_hist
fi
```

`$CURRENT` is the index of the word being completed, so this fires only when the cursor is on the command name itself. The helper is four lines:

```
last_ten=( ${f}"$(fc -l -200 2>/dev/null |
    stryke -lane 'print "@F[1..$#F]'" )" } )
_wanted last-ten expl 'last commands' compadd -Qa last_ten
```

`fc -l -200` lists the last two hundred history entries with line numbers; the one-liner drops field zero (the number) and reprints the rest. `${f}"..."}` splits the result on newlines into an array. The variable name is a fossil from an earlier, smaller window and the comment in the source does not match the current count — worth noting because it is the kind of thing that misleads a reader who trusts identifiers over code.

`compadd -Qa` is the important flag pair. `-a` says the following argument is an array *name*, not a list of values, which avoids re-splitting entries that contain spaces. `-Q` suppresses `zsh`'s automatic quoting of the inserted match, so a history entry that already contains quotes is inserted as written rather than as an escaped literal. Every one of the custom sources uses `-Qa` for this reason: they are all inserting whole command fragments, not filenames.

Sources Four And Five: The Last Command And The Clipboard

The final block fires only when there is more than one word and the last word is non-empty:

```
if (( $#words >= 2 )) 66 [[ $words[-1] == [^[:space:]]## ]]; then
    num=$((HISTCMD-1))
    last_command=${history[$num]}
    last_command_array=(${(u)=last_command} ${last_command}
        "\"${last_command}\"" "( ${last_command}; )"
        "{ ${last_command}; }" "\${${last_command}})"
        "\"\${${last_command}}\"" "'${last_command}'")
```

`[^[:space:]]##` is an extended-glob pattern meaning one or more non-space characters — the `##` suffix is zsh's `+` quantifier. The array that follows is the genuinely novel part of this stack: it offers the previous command in eight different *syntactic wrappings*. `${(u)=last_command}` word-splits it and uniquifies, giving each individual argument. Then the whole command bare, in double quotes, in single quotes, wrapped in a subshell, wrapped in a brace group, in command substitution, and in quoted command substitution.

This is a direct attack on a specific friction point. You ran `find . -name '*.rs'`; now you want `wc -l <$(find . -name '*.rs')` or `echo "$(find . -name '*.rs')"`. The candidate list contains the form you want already assembled, and fuzzy completion narrows to it in two or three keystrokes. The alternative is retyping or a history-recall-plus-manual-edit dance.

The clipboard fallback follows the same structure with fewer forms:

```
clipboard_str="${${(Q)=ZPWR_PASTE_CMD} 2>/dev/null)//${_r}'"}"
```

`${(Q)=VAR}` quote-strips *and* word-splits the paste command, so a multi-word platform paste command (the cross-OS resolver chooses `pbpaste`, `xclip`, or a Windows equivalent) executes correctly. The trailing `//${_r}'"` deletes carriage returns, which is what makes clipboard content copied from a Windows or browser context paste cleanly.

Both of these run only when `ret == 1` — when zsh had nothing to say. They are last resorts, and they are the correct last resorts, because when the completion system is silent the thing you are typing is usually a literal you already have somewhere.

Overriding `_command_names`

`comp_utils/_command_names` replaces the zsh function that completes the first word of a command line. The stock version offers external commands, builtins, functions, and reserved words. The override extends the alternatives list:

```
defs+=(
    'global-aliases:global alias:__zpwrgaliases'
    'aliases:alias:__zpwraliases'
    "functions:shell function:compadd -k 'functions$ffilt'"
    'builtins:builtin command:compadd -Qk builtins'
    'suffix-aliases:suffix alias:_suffix_alias_files'
    'reserved-words:reserved word:compadd -Qk reswords'
    'jobs::_jobs -t'
    'parameters::_parameters'
    'files:files:_files'
)
```

Global aliases, jobs, parameters, and files are the additions. Files in first position is the one that reads as strange until you remember suffix aliases and the `AUTO_CD` option — in this environment, a bare path *is* frequently a valid command line.

The alias source is cached, using the completion system's own cache API:


```

if ( ! ( ( $+__aliases_ary ) ) || _cache_invalid alias_cache ) &&
! _retrieve_cache alias_cache; then
  for k v in ${(kv)aliases}; do
    __aliases_ary+=($k:"${(q)v}")
  done
  _store_cache alias_cache __aliases_ary
fi
_describe -t aliases alias __aliases_ary

```

The `name:description` element format is what `_describe` consumes to render a two-column menu, so the completion menu shows each alias next to its expansion. `${(q)v}` quotes the expansion so that an alias body containing a colon does not corrupt the field split.

Overriding `_files`

The `_files` override is the most aggressive of the three. It vendors the original zsh implementation verbatim as a nested function named `__origfiles`, calls it through `_alternative`, and then adds a fallback:

```

_alternative 'files:files:__origfiles'

if [[ $original_ret == 1 ]] && ( ( $#words > 1 ) ) &&
  type -ap -- fasd &>/dev/null; then
  if [[ $words[-1] != '-' && $words[-1] != '--' ]]; then
    _alternative \
      'fasd-file:fasd ranked files:__fasd_files_comp' \
      'fasd:fasd ranked directories:__fasd_dirs_comp'
    return 1
  fi
fi

```

If no file in the current directory matches what you typed, the frequency database is consulted — so a filename fragment completes to a file you edited last week in a directory three levels away. The guard against `-` and `--` as the last word prevents offering ranked paths where an option was clearly intended.

The `original_ret` variable exists because the vendored body sets `ret` in several places and returns early from some of them; a second variable records the outcome that the outer function actually needs to branch on. Vending rather than wrapping is the right call here: `_files` has a dozen internal exit paths and a wrapper that only saw the final return status could not distinguish “found nothing” from “found something but the caller filtered it”.

Chapter 47 – The Semicolon Trigger: fzf Completion Protocol

There are two distinct fuzzy-completion systems in play and they are easy to confuse. One is menu interception: any completion that would have produced a menu gets rendered by fzf instead. The other is an explicit trigger — you type a sentinel character and press Tab, and a *purpose-built generator* runs for that specific command. The second system is where the custom work lives, and its trigger is set in the ZLE binding function:

```
export FZF_COMPLETION_TRIGGER=';'
```

The upstream default is `**`. A single semicolon is one keystroke instead of two, on the home row, and — critically — it is a character that is syntactically invalid in the middle of a command word, so there is no ambiguity about whether you meant it as a trigger.

The Two-Function Contract

For a command `foo`, the protocol looks for `_fzf_complete_foo` to generate candidates and `_fzf_complete_foo_post` to transform the selection before it is inserted into the buffer. The files in `autoload/fzf` implement this pair, with the leading-underscore-count convention that keeps the autoloaded public name distinct from the inner implementation name:

```
function __fzf_complete_git() {  
    ...  
}  
  
__fzf_complete_git "$@"
```

The post-processor is where a lot of the value is, because the generator is free to emit rich, padded, colorized, multi-column lines that are pleasant to search through, and the post-processor is responsible for turning the selected line back into valid shell syntax. Separating the two means the display format is not constrained by the insertion format.

Case Study: The Verb Completer

`_fzf_complete_zpwr` renders the verb table into aligned columns by hand:

```
FZF_COMPLETION_OPTS=$FZF_ENV_OPTS_VERBS \
_fzf_complete '-m --preview-window=down:25 --border' "$@" < <{
  local len sep k v i width
  sep=" "
  width=25
  for k in ${ZPWR_VERBS[@]}; do
    len=${#k}
    printf $k
    spaces=$(( width - len ))
    for (( i = 0; i < $spaces; ++i )); do
      printf $sep
    done
    printf "${ZPWR_VERBS[$k]}\n"
  done
}
```

`${(ko)ZPWR_VERBS[@]}` takes the keys (`k`) and sorts them (`o`), so the list is alphabetical regardless of hash order. The padding loop is manual rather than a `printf '%-25s'` because the value being padded is a shell variable whose width must be computed with `${#k}` — and, more to the point, because a `printf` format width would count bytes rather than characters. Emitting spaces in a loop after measuring with `${#}` gives the right answer for any key.

The generator is fed through process substitution `< <{ ... }`, which is what keeps the whole thing fork-cheap: no temp file, no pipeline into a wrapper, just a file descriptor.

The post-processor is the interesting half:

```
stryke -e '@a=<>;$c=${#a};for (@a){print "zpwr " if $c!=$#a;
  print "$1" if m{^(\\S+)\\s+};print ";" if $c--;print " "}'
```

Read it against the `-m` flag on the generator — multi-select is enabled, so the selection can be several lines. The one-liner slurps all selected lines, and for each one: prints the leading `zpwr` prefix for every line except the first (because the first already has `zpwr` on the command line, that being what triggered the completion), extracts the first whitespace-delimited field (the verb name, discarding the description column), and emits `;` between entries while the counter is non-zero.

So selecting three verbs produces `zpwr a; zpwr b; zpwr c` — a valid compound command, assembled from a multi-select of a two-column padded table. That round-trip, from rich display format back to correct shell syntax, is exactly what the post-processor half of the contract exists for.

Case Study: The Git Completer

The `git` generator is more elaborate because it builds its own preview command at completion time:

```
if ! zpwrIsGitDir; then
  return 1
fi

local lastWord
lastWord=${${(Az)@}[-1]}

if git cat-file -t -- $lastWord &>/dev/null; then
  export FZF_GIT_OPTS="$ZPWR_COMMON_FZF_ELEM --preview \
    '$(bash "$ZPWR_SCRIPTS/fzfGitOpts.sh" $lastWord)'"
else
  export FZF_GIT_OPTS="$ZPWR_COMMON_FZF_ELEM --preview \
    '$(bash "$ZPWR_SCRIPTS/fzfGitOpts.sh" HEAD)'"
fi
```

`${(Az)@}` splits the arguments as the shell would (`z`) into an array (`A`), and `[-1]` takes the last element — the word immediately before the trigger. `git cat-file -t` asks whether that word names any git object at all: a commit, a tag, a tree, a blob. If it does, the preview is built relative to *that* object; otherwise relative to `HEAD`.

The practical effect is that `git diff ;<Tab>` previews each candidate against your working tree, while `git diff SOMESHA ;<Tab>` previews each candidate against `SOMESHA`. The completion understands the semantics of the partially typed command, not just its name.

The candidate list unions two sources with explicit ANSI framing:

```
printf "\x1b[${ZPWR_COMMIT_STYLE}m"
git log --format='%h %s'
git for-each-ref | stryke -lane '$_=substr($F[0],0,7)." $F[2]";
  print if ! m{^\s*$}'
printf "\x1b[0m"
```

Commits come from `git log` as `abbrev-hash subject`. Refs come from `for-each-ref`, whose default output is `SHA TYPE REFNAME`; the one-liner truncates the full SHA to seven characters and pairs it with the ref name, producing lines in exactly the same shape as the commit lines. Uniformity of shape is what lets a single post-processor handle both:

```
cut -d ' ' -f1
```

One field. The entire rich display — colorized hashes, subjects, ref names — is discarded and only the object identifier is inserted. `--ansi` is passed to `fzf` so the escape sequences render as color rather than being matched as text.

Note the color is applied by bracketing the whole block with a set and a reset rather than by coloring each line. Since `fzf` reads a stream and the escape codes persist until reset, two `printf` calls colorize an unbounded number of lines.

What The Protocol Buys

The general shape is worth stating plainly because it generalizes. A completion in this system is: a guard that can decline (`zpwrIsGitDir` returning 1 falls back to normal completion), a context read that inspects the already-typed words, a generator that emits display-optimized lines, and a post-processor that emits syntax-optimized text. The two halves are decoupled, so the display can be as elaborate as the terminal allows without ever risking the correctness of what lands in the buffer.

Compare against what the stock completion system can express: a list of matches with optional descriptions, where the match itself is what gets inserted. Anything richer requires the display and insertion formats to diverge, and the stock system has no place to put that transformation.

Chapter 48 – The Searchable Environment Index

Every interactive shell holds several thousand named things: parameters, aliases, builtins, reserved words, hashed commands, and functions. There is a verb that lets you fuzzy-search all of them at once and preview the definition of whatever is highlighted. Doing that at interactive speed is not trivial, because the preview command re-runs on every cursor movement, and the naive implementation — grep the dump file for the selected name — costs a full scan per keystroke.

The solution is a three-file cache with a byte-offset index, built by `scripts/zshRegenSearchableEnv.zsh` and read by `scripts/fzfEnvPreview.zsh`. It is the most performance-conscious code in the tree and it is instructive well beyond this feature.

The Three Files

`ZPWR_ENV_KEY_FILE` holds one line per name, `type name`, sorted. This is what gets piped into `fzf`. `ZPWR_ENV_VALUE_FILE` holds the definitions, concatenated with no separators at all. `ZPWR_ENV_IDX_FILE` maps each name to `type offset length`.

The absence of separators in the value file is the point. There is no need to scan for a delimiter because the index already knows exactly which byte range belongs to which name.

Byte Counting, Not Character Counting

The generator opens with an option setting that looks like a mistake and is not:

```
builtin emulate -L zsh
setopt noglob
unsetopt multibyte    # ${#str} returns byte count
```

With `multibyte` set — the default in any modern `zsh` — `${#str}` counts characters. With it unset, `${#str}` counts bytes. Since the recorded lengths will be used to seek and read a file, they must be byte quantities. The preview script repeats the same `unsetopt multibyte` with a comment explicitly tying it to the generator's offsets. A function name or a parameter value containing a non-ASCII character would otherwise desynchronize every subsequent offset in the file — an error that would be nearly impossible to diagnose from the symptom, which is that previews for names late in the file show fragments of their neighbours.

This is one of those cases where the correct code looks wrong and the wrong code looks correct.

Building The Value File

Each category is walked and emitted as a self-contained block, with the offset counter advanced by the block's byte length:

```

for k v in ${(kv)parameters}; do
  [[ $v == *hideval* ]] && continue
  block="export $k=${(P)k}"$'\n'
  builtin print -rn -u 3 -- "$block"
  idx_by_name[$k]="param $offset ${#block}"
  (( offset += ${#block} ))
done

```

`${(P)k}` is the parameter-name-indirection flag: it expands to the value of the parameter *named by k*. `print -rn -u 3` writes raw (`-r`, no escape interpretation), without a trailing newline (`-n`, since the block already ends with one), to file descriptor 3, which was opened once before the loop and closed once after. Not reopening the file per entry is what keeps the regeneration from being dominated by syscall overhead.

The `hideval` skip deserves note. The `parameters` associative array maps names to a comma-separated list of type attributes, and zsh flags certain internal parameters as having a value that should not be displayed — `historywords`, `funcstack`, and various plugin-internal state. Some of them error on `(P)` expansion; others expand to enormous useless strings. One test against the type string handles both failure modes.

Builtins and reserved words get synthesized one-line blocks ("`$k is a shell builtin`"), which is all there is to say about them. Commands get an index entry with a sentinel offset of `-1` and no value-file block at all, because the preview for a command is generated from the file on disk rather than from a cached string.

The Index As Sourceable Code

The index file is not parsed. It is *executed*:

```

{
  builtin print -r -- 'typeset -gA ZPWR_ENV_IDX'
  builtin print -rn -- 'ZPWR_ENV_IDX=( '
  for name in ${(k)idx_by_name}; do
    builtin print -rn -- " ${((qqqq))name} ${((qqqq))idx_by_name[$name]} "
  done
  builtin print -- $'\n)'
} > "$ZPWR_ENV_IDX_FILE"

```

The file contains a zsh associative-array literal, so the preview script loads the entire index with `builtin source` — no fork, no field splitting, no parsing loop. `${((qqqq))}` selects the `$'...'` ANSI-C quoting style, which can represent any byte sequence including newlines and single quotes. Given that the keys are arbitrary parameter and function names harvested from a live shell, weaker quoting would eventually meet a name it could not round-trip.

The Preview Hot Path

The preview script's design constraint is stated in its own header: the only fork permitted is the one `fzf` itself imposes by running the preview command as a process. Everything inside must be builtins.

```

zmodload zsh/system # sysseek / sysread
...
builtin source "$ZPWR_ENV_IDX_FILE"
local entry="${ZPWR_ENV_IDX[$line]:-}"
read -r type offset length <<< "$entry"

```

Then the extraction:

```

exec {fd}< "$ZPWR_ENV_VALUE_FILE"
sysseek -u $fd $offset
local remaining=$length chunk
integer bytes_read=0
while (( remaining > 0 )); do
    sysread -s $remaining -c bytes_read chunk <&$fd || break
    (( bytes_read == 0 )) && break
    print -rn -- "$chunk"
    (( remaining -= bytes_read ))
done
exec {fd}<&-

```

`sysseek` and `sysread` come from the `zsh/system` module and are thin wrappers over the `lseek` and `read` syscalls. `exec {fd}<` uses zsh's automatic file-descriptor allocation so the script never hardcodes a descriptor number that might collide. The loop is written correctly for short reads: `sysread` reports how many bytes it actually delivered via `-c`, and the loop continues until the requested length is satisfied or the read returns nothing. A single unchecked `sysread` would silently truncate long function bodies.

The comparison worth drawing: the obvious implementation of this preview is `grep -A50 "^$name" dump.txt` or `awk '/^name/,/^$/'`. Both are a fork plus a linear scan of a multi-megabyte file, per keystroke. This is a hash lookup plus one seek plus one read, with the fork count fixed at one by fzf's own process model.

Two Modes And A Verb Adapter

The preview takes a mode argument. `plain` passes the extracted bytes through `cat`; `reg` pipes them through a `cowsay/ponysay/lolcat` chain. The pipeline is constructed so that each stage's failure is tolerated (`2>/dev/null` on each), which matters because none of those tools are dependencies.

There is also a source argument that adapts the same preview to the verb picker, whose lines are the padded two-column format from the completion generator rather than bare names:

```

if [[ $src == verbs ]]; then
    local -a parts=( ${=line} )
    if (( ${#parts} >= 2 )); then
        line=${parts[2]%%=*}
    fi
fi

```

`${=line}` word-splits on `IFS`, collapsing the padding runs, so field two is the target expression and `%%=*` trims the description. The verb picker therefore previews the *implementation* of whatever verb you are hovering over — the function body, extracted from the same index. Chapter by chapter, that is the same body-recovery machinery described in the autoload chapter, reaching all the way through to an interactive preview pane.

Cache Invalidation

The consuming widget checks all three files and regenerates if any is empty:

```

if [[ ! -s "$ZPWR_ENV_KEY_FILE" ]]; then
    zpwrRegenSearchEnv
fi

```

Three separate checks rather than one, because a regeneration interrupted midway can leave one file written and another not. Publication of the value file is a truncate-and-rewrite:

```
<$ZPWR_TEMPFILE2 >$ZPWR_ENV_VALUE_FILE
```

That is a redirection-only command — no `cat`, no fork; zsh's `NULLCMD` handling turns a bare input/output

redirection pair into a copy. The source comments are honest that this leaves a partial-read window rather than being atomic. A rename would close it, at the cost of requiring the temp file to live on the same filesystem as the destination. The chosen tradeoff accepts a rare garbled preview in exchange for not constraining where the temp directory lives.

Chapter 49 – Environment Resolution And The Override Protocol

A configuration framework that exports two hundred environment variables has a hard problem: the user will want to change some of them, the framework will be re-sourced several times per shell start, and every re-source is an opportunity to stomp the user's change. The mechanism that solves this is a single function in `env/.zpwir_re_env.sh`, and it implements a genuine three-state protocol rather than the usual `${VAR:-default}` guess.

Why The Obvious Approach Fails

The conventional idiom is:

```
export FOO="${FOO:-default}"
```

This has a specific failure mode in a layered configuration system. It cannot distinguish “the user set `FOO`” from “we set `FOO` ourselves on a previous pass”. Once the framework has exported its own default, every subsequent re-source sees a non-empty `FOO` and declines to update — which means that changing the *upstream* default has no effect on a live shell, and more importantly that a derived variable whose base changed never recomputes.

The environment here is layered precisely so that derived values recompute: `.zpwir_env.sh` sets static preferences, the pre-tokens file applies user overrides, `.zpwir_re_env.sh` recomputes everything derived from them, and the post-tokens file applies a final override layer. The middle stage has to be re-runnable.

The Three States

```
function zpwrEvalIfNeeded() {
    local name current prev wanted
    name="$1"; current="$2"; wanted="$3"

    if [[ -z "$current" ]]; then
        eval "export $name=\"\$wanted\""
        ZPWR_VARS[prev_$name]="$wanted"
        return 0
    elif [[ -z "${ZPWR_VARS[prev_$name]}" ]]; then
        eval "export $name=\"\$wanted\""
        ZPWR_VARS[prev_$name]="$wanted"
        return 0
    fi

    if [[ "$current" == "$wanted" ]]; then
        return 0
    fi
    prev=${ZPWR_VARS[prev_$name]}

    if [[ "$current" == "$prev" ]]; then
        eval "export $name=\"\$wanted\""
        ZPWR_VARS[prev_$name]="$wanted"
        return 0
    fi
    # user has made direct change to env variable so leave alone
}
```

The extra state is `ZPWR_VARS[prev_NAME]` — a shadow record of the last value the framework itself wrote. With it, the four cases separate cleanly.

Unset: write the computed value and record it. First pass with a value already present but no shadow record: the variable came from `.zshenv` or the ambient environment, and the framework overwrites it — the source comment states the policy explicitly, that all configuration must happen inside the framework's own files, `ZPWR` itself being the sole exception since it is what locates everything else. Current equals wanted: nothing to do. Current equals the shadow record but not the wanted value: the framework wrote the current value and the inputs have since changed, so recompute and re-record.

The fall-through case — current matches neither `wanted` nor the shadow record — is the one that matters most, and it does nothing at all. That is a user edit, and it survives every subsequent re-source for the lifetime of the shell.

Note the callers pass the *current* value as an argument rather than letting the function read it:

```
zpwrEvalIfNeeded ZPWR_LOCAL "$ZPWR_LOCAL" "$ZPWR/local" "$ZPWR"
```

The fourth argument is documentation of the dependency — this derived value is a function of `ZPWR` — not something the function reads. The call sites read as a dependency graph, which is useful when tracing why a path resolved somewhere unexpected.

Order Is The Design

The load sequence in `scripts/init.sh` is explicit and the ordering is the whole point:

```

source "$ZPWR_ENV_FILE"      # static preferences
source "$ZPWR_TOKEN_PRE"     # user overrides of preferences
source "$ZPWR_RE_ENV_FILE"   # recompute all derived paths
source "$ZPWR_TOKEN_POST"    # user overrides of derived values

```

Set `ZPWR_REPO_NAME` in the pre-tokens file and the history file path, the compdump path, and the GitHub URL all follow, because they are computed after that file is read. Set a history file path directly in the post-tokens file and it wins outright, because nothing recomputes after it.

Both token files are created if missing:

```

if test -f "$ZPWR_TOKEN_PRE"; then
    source "$ZPWR_TOKEN_PRE" || zpwrlLogConsoleErr "could not source ..."
else
    touch "$ZPWR_TOKEN_PRE"
fi

```

A user who has never customized anything still has the files, empty, in a gitignored directory. The discoverability of a configuration system is largely determined by whether the file you are supposed to edit already exists.

Sourcing For Functions Versus Sourcing For Variables

`init.sh` contains a comment that explains an otherwise puzzling branch:

```

# env vars are exported
# so just get the functions
source "$ZPWR_LIB"

```

When `ZPWR_LIB` is already set, the whole environment layer is skipped and only the library file is sourced. The reasoning is exact: environment variables cross the `exec` boundary into a child process, but shell functions do not. A script launched from an interactive shell inherits every `ZPWR_*` path for free and needs to re-source exactly one thing — the function definitions. Re-running the full environment resolution would be pure waste on a path that scripts take constantly.

This is why `init.sh` is the entry point for standalone scripts in the tree. Each such script locates the framework root by walking up the directory tree looking for a marker file:

```

zpwrlBaseDir="${0:A}"
while [[ ! -f "$zpwrlBaseDir/.zpwrl_root" ]]; do
    zpwrlBaseDir="${zpwrlBaseDir:h}"
    if [[ "$zpwrlBaseDir" == / ]]; then
        echo "Could not find .zpwrl_root file up the tree." >&2
        exit 1
    fi
done

```

`${0:A}` resolves to an absolute path with symlinks followed; `${var:h}` is the head modifier, equivalent to `dirname` without a fork. The marker file exists solely to be found this way, and its contents say so. The result is that scripts work when invoked through a symlink from a bin directory, from a relative path, or from an arbitrary working directory — without any of them hardcoding the installation location.

Late Upgrades

There is one category of variable that deliberately does not follow this protocol, and the source comments explain why:

```

if zpwrCommandExists zmax; then
    export ZPWR_ZMAX='zmax'
elif zpwrCommandExists nvim; then
    export ZPWR_ZMAX='nvim'
fi
if [[ "$ZPWR_EDITOR_TO_VIM" == true ]]; then
    export EDITOR="$ZPWR_ZMAX"
fi
if zpwrCommandExists ztmux; then
    export ZPWR_ZTMUX='ztmux'
fi

```

These live in the alias file rather than the environment files because they depend on `PATH` being fully assembled. The static environment file is sourced before the `PATH` block runs, at which point the Homebrew binary directory and the Cargo binary directory are not yet on `PATH`, so a command-existence test there would produce a false negative and pin the fallback forever. The static files set conservative defaults; the upgrade happens once `PATH` is complete; and because the token files are sourced afterward, an explicit user pin still beats the auto-detection. Three layers, and the ordering of all three is load-bearing.

Where EDITOR Is Set, And Why It Moved

`EDITOR` is the clearest demonstration of what that ordering buys, because it is the one assignment that had to be relocated to get it right. It used to be exported from the per-OS block earlier in the same file, alongside `PSQL_EDITOR`, under a `$ZPWR_EDITOR_TO_VIM` guard. That position is upstream of the `PATH` block, so the value it captured was `$ZPWR_VIM` — the static `nvim` default — and the later `zmax` upgrade could never reach it. The export now sits directly under the `zmax` detection, three lines after `ZPWR_ZMAX` has been resolved to its final value:

```

if [[ "$ZPWR_EDITOR_TO_VIM" == true ]]; then
    export EDITOR="$ZPWR_ZMAX"
fi

```

There is no command-existence test on this one, and it does not need one. `$ZPWR_ZMAX` is either the result of a test that already ran (`zmax`, else `nvim`) or the static `vim` fallback from `.zpwr_env.sh`, so by the time this line executes the variable is guaranteed to name something the environment is willing to launch. The guard that remains is the policy switch, not a capability check: `ZPWR_EDITOR_TO_VIM` decides whether the framework claims `EDITOR` at all, leaving the value alone for a user who wants their login environment's choice to survive. Its name is now a half-truth — the variable points `EDITOR` at `$ZPWR_ZMAX`, which is a vim only when nothing better is installed — and the source comment above it says exactly that rather than renaming a variable that user token files are entitled to set.

What stayed behind in the per-OS block is `PSQL_EDITOR`, and it stayed for a reason:

```

export PSQL_EDITOR="$ZPWR_VIM -c \"setf sql\""

```

That value is not an editor name; it is an editor name plus `-c "setf sql"`, a vim command-line argument that forces the SQL filetype on the buffer `psql` hands over. It is bound to the vim family by its own syntax, so it correctly reads `$ZPWR_VIM` rather than following `EDITOR` to whatever `$ZPWR_ZMAX` resolved to. The two arms of the `darwin` conditional that surround it are now byte-identical for this block — the platform split above them exists for Homebrew paths and `MANPATH`, not for editors, and the editor assignment was duplicated into both arms rather than being lifted out of the conditional entirely.

The same split runs through the verb table, where it is visible to the user as a pair rather than as an ordering problem:

```
ZPWR_VERBS[tokens]='vtok=zmax the .tokens.sh file'  
ZPWR_VERBS[vimtokens]='tok=vim the .tokens.sh file'  
ZPWR_VERBS[emacstokens]='etok=emacs the .tokens.sh file'
```

Three verbs, one file, three editors. The aliases they dispatch to are the same line three times with one expansion swapped — `${=ZPWR_ZMAX}`, `${=ZPWR_VIM}`, `${=ZPWR_EMACS_CLIENT}` — each followed by the same `zpwrClearList` and the same `git diff HEAD` when the directory is a repository. The unadorned `tokens` verb is the one that follows the upgrade chain; the explicitly-named `vimtokens` is the escape hatch for when you want the editor you asked for by name and not the one the machine turned out to have.

Chapter 50 – The Cross-OS Layer

The framework runs on macOS, on a dozen Linux families, on the BSDs, under Cygwin and MSYS, and inside the Windows Subsystem for Linux. It does this without littering platform conditionals through the function tree, because the platform differences are concentrated into two files with two distinct strategies: `scripts/crossOSCommands.sh` resolves *names*, and `scripts/crossOSExecute.sh` dispatches *code*.

Resolve Once, Export, Never Ask Again

The commands file is a set of pure functions, each a `case` over the OS type returning a command string:

```
function zpwrGetPasteCommand(){
    local paste_cmd
    case "$ZPWR_OS_TYPE" in
        darwin*) paste_cmd='pbpaste' ;;
        cygwin*) paste_cmd="powershell.exe -nopprofile \
            -command 'Get-Clipboard'" ;;
        linux*)
            if [[ "$_zpwr_uname_r" != *icrosoft* ]]; then
                paste_cmd='xclip -o -sel clip'
            else
                paste_cmd="powershell.exe -nopprofile \
                    -command 'Get-Clipboard'"
            fi
            ;;
        *bsd*) paste_cmd='xclip -o -sel clip' ;;
        msys*) paste_cmd='powershell.exe ...' ;;
        *) echo "Platform $ZPWR_OS_TYPE not supported"; return 1 ;;
    esac
    echo "$paste_cmd"
}
```

The file ends by calling each resolver exactly once and caching the result in the environment:

```
[[ -z "$ZPWR_OPEN_CMD" ]] && export ZPWR_OPEN_CMD="$(zpwrGetOpenCommand)"
[[ -z "$ZPWR_COPY_CMD" ]] && export ZPWR_COPY_CMD="$(zpwrGetCopyCommand)"
[[ -z "$ZPWR_PASTE_CMD" ]] && export ZPWR_PASTE_CMD="$(zpwrGetPasteCommand)"
```

Three command substitutions per shell, and after that every consumer reads a variable. Because the result is exported, a child script inherits the resolution rather than recomputing it — the same reasoning that drives the environment layer's split between exported variables and re-sourced functions. And because the assignments are guarded on emptiness, a user who wants a different clipboard backend sets the variable in a token file and the resolver never overwrites it.

Consumers therefore look like this, from the clipboard completion source:

```
clipboard_str="${${(Q)=$ZPWR_PASTE_CMD} 2>/dev/null}//$'\r'"
```

`${(Q)=VAR}` quote-strips and word-splits, which is required because the resolved value may be several words with embedded quoting — `xclip -o -sel clip` or a PowerShell invocation with a quoted `-command` argument. A bare `$VAR` would pass the whole string as one word; `${=VAR}` alone would split the PowerShell command in the wrong places. The two flags together handle both shapes with one expansion.

WSL Is Not Linux

The Linux branch of every resolver contains the same secondary test:

```
if [[ "$_zpwir_uname_r" != *icrosoft* ]]; then
    open_cmd='nohup xdg-open'
else
    open_cmd='cmd.exe /c start ""'
fi
```

`uname -s` reports `Linux` under WSL, so the OS type is correct and useless — the clipboard and the browser live on the Windows side. The kernel release string is what carries the signal, and it is matched with a pattern that deliberately omits the leading character, `*icrosoft*` rather than `*Microsoft*`, because the capitalization has varied across WSL versions. Matching the case-invariant substring is the cheap way to be right for both.

The kernel release is cached at file scope on the Linux path only:

```
if [[ "$ZPWR_OS_TYPE" == linux* && -z "$_zpwir_uname_r" ]]; then
    _zpwir_uname_r="$(uname -r)"
fi
```

with an explicit comment that this avoids repeated subshells. Three resolvers each testing WSL would otherwise fork `uname` three times per shell for a value that cannot change while the kernel is running.

Note the `nohup` on the Linux open command. `xdg-open` frequently launches a process whose lifetime should not be tied to the shell that spawned it; without `nohup`, closing the terminal takes the browser window with it. That is a one-word difference that only shows up as a bug reported hours later.

Dispatching Code, Not Names

The second file solves a different problem. Some behavior cannot be reduced to a command name — package installation, service management, log inspection. For those, the distribution family matters, and the file provides a set of dispatchers that take *code strings* and evaluate the matching one:

```
function zpwrOsRaspbianVsFamily() {
    zpwrSetDistroName
    case $ZPWR_DISTRO_NAME in
        (raspbian) eval "$1" ;;
        (debian | ubuntu* | elementary* | kali | linuxmint |
         zorin | parrot | *alpine*) eval "$2" ;;
        (fedora | ol | centos | rhel | amzn) eval "$3" ;;
        (*suse* | arch | *artix* | endeavouros | garuda |
         manjaro* | arcolinux* | archcraft*) eval "$4" ;;
        (*) eval "$5" ;;
    esac
}
```

There are several of these, each with a different partitioning of the same distribution space: a two-way Debian-versus-Red-Hat split, a four-way family split, a five-way split that separates Raspbian from its Debian parent, a split that isolates Fedora, and one organized around whether the system uses `journalctl`. Each dispatcher takes as many code arguments as it has branches.

The proliferation is the design, not a failure of it. A single canonical “what family is this” function would force every caller to handle every family, including the ones where the answer is identical. A caller that only needs to know “does `apt` or `dnf` install packages here” uses the two-way split and writes two strings. A caller that needs to special-case Raspbian’s divergence from Debian — and there are real cases, since Raspbian pins different package names — uses the split that separates them. The dispatcher set is a menu of partitions, and

each caller picks the coarsest one that still distinguishes the cases it cares about.

The distribution name itself comes from one place:

```
function zpwrSetDistroName() {
    test -z "$ZPWR_DISTRO_NAME" && export ZPWR_DISTRO_NAME=$(
        ${ZPWR_PERL:-perl} -lne
        'do{($_=$1)=~s/"@@g;print;exit 0} if m{^ID=(.*)}'
        /etc/os-release)
}
```

`/etc/os-release` is the standardized location, and the `ID=` field is its canonical short identifier. The one-liner strips the quotes that some distributions put around the value and exits at the first match rather than reading the rest of the file. As with the kernel release, the result is cached in an exported variable and the guard makes it computed at most once per process tree.

Note that the dispatchers are `eval`-based rather than taking function names. That choice trades a small amount of safety for a large amount of call-site brevity: a caller writes the two variants inline as strings instead of defining two named functions for a two-line difference. Given that the arguments are always literals written by the framework itself and never user input, the exposure is limited to the same class as any other `eval` in a shell configuration.

Where The Layer Leaks

The abstraction is not total, and the places it does not cover are instructive. `zpwrClearList` selects `otool -L` versus `ldd` inline rather than through a resolver, because that is its only platform difference and a resolver would be one indirection for one call site. The `ls` fallback chain is inline for the same reason, and additionally because it is a *capability* chain rather than a platform chain — the question is what is installed, not what kernel is running.

The rule that emerges: platform differences that appear in more than one place get a resolver and an exported variable; differences local to a single function stay inline. That is not a compromise, it is the correct boundary. An abstraction with one caller is a rename.

Chapter 51 – Logging As An Instrument

Most shell frameworks log by echoing a string with a colored prefix. The logging layer here is closer to an instrument panel: it renders a full-width block-glyph level banner, a bordered status box with a signal-strength meter and a timestamp, and only then the message. It is implemented entirely in `printf` inside `scripts/lib.sh`, and the arithmetic that keeps it aligned is worth studying because getting box alignment right in a colored terminal is a recurring source of bugs.

The Level Palette

Each level selects three foreground colors and a signal-bar count:

```
case "$lvl" in
  INFO)  c1="${_NC}"; c2="${_NP}"; c3="${_NG}"
         sig_fill=8; sig_color="${_NG}" ;;
  ERROR) c1='\x1b[38;5;196m'; c2='\x1b[38;5;160m'
         c3='\x1b[38;5;124m'; sig_fill=2; sig_color="${_NR}" ;;
  DEBUG) c1="${_NY}"; c2="${_NC}"; c3="${_NG}"
         sig_fill=6; sig_color="${_NY}" ;;
  TRACE) c1="${_NM}"; c2="${_NC}"; c3="${_NP}"
         sig_fill=4; sig_color="${_NM}" ;;
esac
```

The three colors are applied to the top, middle, and bottom thirds of the block-glyph banner, producing a vertical gradient. Every color is a 256-color escape (`38;5;N`) rather than one of the sixteen basic codes, which means the palette is not subject to terminal theme remapping — the neon cyan is the same neon cyan on every terminal that supports 256 colors, rather than whatever the user's theme assigned to “bright cyan”.

ERROR is the exception in the gradient scheme: rather than three different hues it uses three shades of red (196, 160, 124), so the banner reads as a solid red block fading downward. The signal meter reinforces the same information redundantly — eight bars for **INFO**, six for **DEBUG**, four for **TRACE**, two for **ERROR** — so the severity is legible from the bar even in a screenshot cropped above the banner.

The Alignment Arithmetic

The status line inside the box has a fixed set of visible components and one variable-width one (the level name), so the padding is computed rather than guessed:

```
# box_inner = 74 chars; content visible length (minus lvl) =
# " STATUS: " (9) + " // SIGNAL: " (13) + 10 bars +
# " // HH:MM:SS " (14) = 46
local -i _pad=$(( 74 - 46 - ${#lvl} ))
(( _pad < 0 )) && _pad=0
```

The comment is the derivation, and its presence is the difference between maintainable and unmaintainable box-drawing code. Every constant is accounted for. The clamp on negative padding means an unrecognized level name longer than 28 characters degrades to a ragged right edge rather than to a `printf` that tries to emit a negative number of spaces.

The reason this arithmetic is needed at all is that the visible components carry color escapes, and `${#string}`

counts those escape bytes. The code sidesteps the problem entirely by never measuring a colored string — it measures `${#lvl}`, which is a bare level name, and hardcodes the arithmetic for everything else using the derivation in the comment. That is the correct approach: compute padding from known-plain quantities, never from a string that has been through a color function.

The signal meter is drawn with two loops rather than string repetition:

```
for (( _i=0; _i<sig_fill; _i++ )); do printf "█"; done
printf "${_DM}"
for (( _i=0; _i<sig_empty; _i++ )); do printf "░"; done
```

with `sig_empty=$((10 - sig_fill))`. The two loops emit exactly ten glyphs between them regardless of level, which is what the alignment arithmetic assumed.

Console Versus File, And The Level Gate

There are two parallel families. The `zpwrLogConsole*` functions render to the terminal; `zpwrLog` and its wrappers render to `ZPWR_LOGFILE`. The split is by destination, not by format — the file gets the same block banners and boxes when colors are enabled:

```
function zpwrLog(){
    if [[ $ZPWR_COLORS == true ]]; then
        if [[ -n $2 ]]; then
            lvl=$1; shift
            zpwrLogColor $lvl "$*" >> "$ZPWR_LOGFILE"
        elif [[ -p /dev/stdin ]]; then
            zpwrLogColor STDIN "${cat}" >> "$ZPWR_LOGFILE"
        ...
    fi
}
```

Writing escape sequences into a log file is a deliberate choice with a cost: `grep` on the file matches against text containing escapes, so patterns that span a color boundary fail. The compensation is that `less -R` on the log gives you the same instrument panel you would have seen live, and the framework's `LESS` default already includes `-R`. When colors are disabled the file format switches to a plain underscore-delimited record, which is the greppable form.

`[[ -p /dev/stdin ]]` tests whether standard input is a pipe, which lets `something | zpwrLog` work with no arguments at all, logging the piped content under a synthesized `STDIN` level. That is the form used to capture the output of a long-running command into the log without a temp file.

The level gate is a plain string comparison at call time:

```
function zpwrLogDebug(){
    if [[ $ZPWR_DEBUG == true ]]; then
        zpwrLog DEBUG "$*"
    fi
}
```

The check happens inside the function, meaning the arguments are still expanded by the caller before the call — a `zpwrLogDebug "${expensive}"` pays for `expensive` even when debugging is off. The framework's own call sites avoid command substitution in log arguments for exactly this reason; the dispatcher's debug line is `zpwrLogDebug "Eval subcommand '$exp'", pure parameter expansion`.

Every Function Announces Its Own Usage

A pattern repeats across the entire library:

```
function zpwrPerlRemoveSpaces(){
    if [[ -z "$1" ]]; then
        zpwrLogConsoleErr "usage: zpwrPerlRemoveSpaces <file...>"
        return 1
    fi
    ...
}
```

Every library function that takes required arguments begins with a guard that prints its own usage through the error logger and returns non-zero. There is no central argument parser, no usage table, no help registry. The usage string lives adjacent to the code that consumes the arguments, which means it cannot drift out of sync with the implementation the way a centralized table does.

The convention interacts with the error banner in a way that is worth calling out. Because `zpwrLogConsoleErr` renders a full block-glyph **ERROR** banner, a usage error is visually enormous — it is not something you can miss scrolling past. The tradeoff is that consecutive error calls produce consecutive banners, so multi-line diagnostics must be a single call with embedded newlines rather than two calls. `zpwrLogColor` ends with `printf "%b"`, so escape sequences in the message — including `\n` — are interpreted, which is what makes that possible.

Trace As A Separate Axis

`ZPWR_DEBUG` and `ZPWR_TRACE` are independent booleans rather than points on an ordered severity scale. Trace is not “more verbose than debug”; it is a different channel, used for the `set -x` regions that the framework can enable around critical sections. Turning on trace without debug gives execution flow without diagnostic commentary; turning on both gives everything. Collapsing the two into a single numeric level would have made “show me the shell’s own expansion of these lines, but not the framework’s narration” inexpressible.

Chapter 52 – One Resolver For Everything

Press Enter on an empty command line and something runs. Type a word after `c` and something else runs. Both are the same function, `zpwrclearlist`, and it is the most-invoked piece of code in the framework. It answers a single question — “what is this name, exhaustively?” — and it answers it for every kind of name a shell can hold, in one pass.

Platform Resolution First

Before touching the arguments, the function resolves two platform-dependent command strings:

```
if [[ "$ZPWR_OS_TYPE" == darwin ]]; then
    if zpwrcommandexists eza; then
        ls_command="$ZPWR_EXA_COMMAND"
    else
        if zpwrcommandexists grc; then
            ls_command="grc -c $HOME/conf.gls gls -iFlhAd --color=always"
        else
            ls_command="ls -iFlhAOd"
        fi
    fi
    lib_command="otool -L"
elif [[ "$ZPWR_OS_TYPE" == linux ]]; then
    ...
    lib_command="ldd"
```

Two independent fallback chains. The listing command prefers the modern replacement, falls back to a colorizer wrapping the GNU implementation, and finally to the platform’s stock utility with platform-appropriate flags — note `-O` in the macOS branch, which is a BSD-only flag for file flags and would be an error on GNU. The library-dependency command has no fallback because there is no third option: `otool -L` on macOS, `ldd` everywhere else.

Both are strings that get `eval`’d later rather than arrays that get expanded. That is a deliberate choice driven by the `grc` variants, which need shell parsing of their own arguments, and by `ZPWR_EXA_COMMAND`, which is assembled in the environment layer from a user preference. The cost is that a path with a space in it needs care at the call sites, which is why the argument is interpolated into the eval string with explicit quoting.

The Exhaustive Command Resolution

For an argument that names anything invocable, the function iterates over *all* resolutions, not just the winning one:

```
done < <(type -a -- "$arg" 2>/dev/null | sort | uniq)
```

`type -a` reports every match — the alias, the function, the builtin, and each `PATH` entry, in precedence order. For each, the loop determines whether this particular resolution is the one that would actually run:

```
if [[ ${type} -- "$arg" == "$loc" ]]; then
    rank="Primary"
else
    rank="Secondary"
fi
```

`type` without `-a` gives the single winning resolution; comparing each line against it labels exactly one as **Primary**. This is the feature that makes the function worth using. The question you actually have when something misbehaves is not “where is `ls`” but “how many things are called `ls`, which one wins, and what are the others” — and that question is normally answered by reading `type -a` output and reasoning about precedence yourself.

For each file-backed resolution the function then reports, in order: the directory listing, `file` output, dynamic library dependencies, disk usage, `stat` metadata, man page paths from `man -wa`, the shell’s command hash entry, and the precedence rank. For non-file resolutions — aliases and functions — it prints the type line and then the definition, with function bodies numbered:

```
type -f -- "$(echo "$loc" | awk '{print $1}')" 2>/dev/null | nl -v 0
```

`nl -v 0` starts numbering at zero so the numbers align with the line offsets a `funcfiletrace` or an error message would report.

Three More Namespaces

Command resolution is only the first of four branches, and they are not exclusive — a name can be reported under several. After the command loop, the argument is tried as a path:

```
if eval "$ls_command -d -- \"$arg\"" 2>/dev/null; then
    FOUND=true
    ...
```

using the exit status of the listing command itself as the existence test, which avoids a separate `[[ -e ]]`.

Then as a shell parameter:

```
out=$(typeset -m -- "$arg")
if [[ -n $out ]]; then
    FOUND=true
    print -rl -- ${tP}arg
    echo $out
```

`typeset -m` matches parameter names against a pattern, so this branch handles globs — `zpwrclearlist 'ZPWR_*'` reports every matching parameter. `${tP}arg` combines two flags: **P** for name indirection and **t** for type, so it prints the parameter’s declared type (**scalar-export**, **association**, **array-tied**) rather than its value. Knowing that a variable is a tied array rather than a scalar is frequently the entire answer to why something behaved unexpectedly.

Finally, a fallback to the ambient environment via `set | grep "^$arg="`, for names that exist in the environment but not as shell parameters of the current context.

The **FOUND** flag is set by any branch, and only if all four miss does the function report failure. Because the branches are independent, a name that is simultaneously a function, a file in the current directory, and a parameter produces three complete reports.

The Empty-Argument Path

With no arguments the function clears the screen and lists the directory, and the implementation is more careful than it first appears:

```

if [[ -t 0 ]]; then
    clear && eval "$ls_command"
elif : < /dev/tty 2>/dev/null; then
    clear && eval "$ls_command" < /dev/tty
else
    clear && eval "$ls_command"
fi

```

Three cases. Standard input is a terminal: run normally. Standard input is not a terminal but a controlling terminal is reachable: reconnect standard input to it explicitly. Neither: run anyway and accept whatever happens. The middle case exists because the modern listing tools query the terminal for width and color capability, and when the function is invoked from a ZLE widget or a preview pane its standard input may be a pipe while a usable terminal still exists at `/dev/tty`. Without the reconnection the listing degrades to unstyled single-column output in exactly the contexts where it is most visible.

Where It Is Wired

This function is the value of two environment variables:

```

export MAGIC_ENTER_OTHER_COMMAND='zpwrclearList'
export MAGIC_ENTER_GIT_COMMAND='zpwrclearList'

```

so pressing Enter on an empty buffer runs it, in both git and non-git directories — the two hooks exist separately so that a git directory can get a status summary instead, and the framework deliberately declines that in favor of consistency. It is also the payload of the directory-change hook, the target of the `c` and `cat` verbs, and — through `zpwrclearList.zsh`, a standalone script wrapper — the preview command for binary files in the environment index.

One function, four invocation paths, four namespaces resolved per call. The alternative design is a family of small focused verbs — `zpw whichall`, `zpw paramtype`, `zpw libdeps` — and the reason that design loses is that it requires knowing which question to ask before you have the information that would tell you. Exhaustive output costs a screenful and answers the question you did not know you had.

Chapter 53 – Manufacturing Completions

The completion corpus in this stack is far larger than what any distribution ships, and it is not hand-written. Two generator scripts in the source tree harvest completions from the two places where command-line interfaces document themselves: `--help` output and man pages. Both scripts follow the same shape and differ only in the extraction backend.

The Gate

Both begin by iterating the shell's command hash and skipping anything already covered:

```
for command abs in ${(kv)commands}; do
  if [[ -z $_comps[$command] ]]; then
    ...generate...
  else
    echo "____completion for $command exists____"
  fi
done
```

`${(kv)commands}` iterates the hash table of external commands as name/path pairs, so `command` is the bare name and `abs` is the absolute path — both needed, since the gate keys off the name and the execution uses the path. `$_comps` is the completion system's own registry, populated by `compinit` from every `#compdef` tag on `fpath`. Testing against it means the generators produce completions only for commands that have none from any source, and re-running after installing a real completion is a no-op for that command.

The consequence is that this is an additive, idempotent process. Running the generator on a machine with a thousand binaries and eight hundred existing completions attempts two hundred generations.

Harvesting `--help`

```
$abs --help < /dev/null |& help2comp.py $command > "$comp_dir"/_ $command &
sleep 0.25
sudo kill -9 $!
```

Three defenses are packed into that. `< /dev/null` because a program that decides `--help` was not what you meant may prompt, and a prompt on a terminal blocks the loop forever. `|&` is the zsh shorthand for `2>&1`, because a significant fraction of programs write usage to standard error rather than standard output. And the background-plus-sleep-plus-kill sequence is a timeout: launch, wait a quarter second, kill unconditionally.

The kill is unconditional and that is the point. Some binaries do not implement `--help` at all and instead treat it as an operand — a compiler starts compiling a file named `--help`, a network tool starts connecting. The generator cannot know in advance which ones, so it treats every invocation as hostile and gives each a fixed budget. Anything that has not produced usage text in a quarter second was not going to.

If nothing usable came out, the second convention is tried:

```

if [[ ! -s "$comp_dir/_$command" ]]; then
    $abs -h < /dev/null |& help2comp.py $command > "$comp_dir/_$command &
    sleep 0.25
    sudo kill -9 $!
    if [[ ! -s "$comp_dir/_$command" ]]; then
        command rm "$comp_dir/_$command"
    fi
fi

```

`-s` tests for non-empty, so the retry is gated on the first attempt having produced nothing. Failure of both leaves an empty file, which is removed — otherwise `compinit` would later find an empty `_command` file on `fpath` and register a completion that produces nothing, which is worse than no completion at all because it suppresses the default file completion.

There is a blacklist, and reading it tells you exactly which lesson produced it:

```

ZPWR_COMPGEN_BLACKLIST=(shutdown halt reboot poweroff zzz Zzz
    coreaudiod memWatch.sh resetWifi.sh)

```

Running `shutdown --help` on a system where `shutdown` does not implement `--help` shuts the system down. The sleep-source binaries suspend the machine. The audio daemon and the two local scripts have their own side effects. A generator that runs every binary on the system with an unknown flag needs an explicit exclusion list, and this one was clearly assembled empirically.

Harvesting Man Pages

The man-page variant is simpler because man pages have no side effects:

```

mantozshcomp.py -s ${man -w $command} > "$comp_dir/_$command
if [[ ! -s "$comp_dir/_$command" ]]; then
    command rm "$comp_dir/_$command" 2>/dev/null
fi
if [[ -n "$comp_dir" ]]; then
    command rm "$comp_dir/"*(L1) &>/dev/null
fi

```

`man -w` prints the path to the man source rather than rendering it, so the parser reads roff markup directly — flag definitions in a man page are marked up structurally (`.B`, `.TP`), which is considerably more reliable to parse than the free-form alignment of `--help` output.

The second cleanup line is a zsh idiom worth extracting. `*(L1)` is a glob with a size qualifier: `L` selects by file length, and `1` with no unit means “one byte”. So the glob matches every one-byte file in the directory — files containing nothing but a newline, which is what the extraction produces when it finds no flags but still emits a trailing newline. The preceding `-s` test only catches genuinely zero-byte files. Two cleanup passes for two distinct degenerate outputs.

The guard `if [[ -n "$comp_dir" ]]` before a `rm` on an interpolated path is the right reflex. If `comp_dir` were somehow empty, the glob would expand relative to `/` and the command would attempt to delete every one-byte file in the root directory. The variable is set to a literal a few lines above and cannot be empty; the guard is there anyway, because the cost of the check is zero and the cost of being wrong is not.

Why This Is A Different Kind Of Completion

There is a quality hierarchy here and it is worth being explicit about it. A hand-written completion function knows the command's grammar: which flags take arguments, what those arguments may be, which flags are mutually exclusive, how subcommands change the available options. A generated completion knows a list of flags and their one-line descriptions.

For a tool you use daily, the hand-written completion is worth its authoring cost. For the long tail — the several hundred binaries that arrived with a package, that you invoke twice a year, whose flags you have never memorized — a flag list with descriptions is the difference between tabbing and opening the man page in another pane. That is the gap these generators fill, and the `_comps` gate ensures they fill only that gap and never displace a real completion.

The output is also raw material rather than a finished product: generated files are written to a staging directory, not directly onto `fpath`. A generated completion that turned out to be wrong would otherwise be indistinguishable from a curated one, and the corpus would degrade with every run.

Chapter 54 – The Test Harness

The framework is a large body of shell code with no compiler, no type system, and a lazy-loading strategy that defers every syntax error to first call. The compensating control is a test suite that is unusually large for a dotfiles project and unusual in shape: the great majority of its assertions are mechanically derived, one family per property, one test per symbol.

Bootstrap

`tests/test_lib.zsh` is loaded by every test file and does five things before any assertion runs. First it locates the root by the same marker-file walk that standalone scripts use. Then it forces the working directory:

```
if [[ "$PWD" != "$ZPWR" ]]; then
  cd "$ZPWR"
fi
```

Then the step that makes the suite trustworthy:

```
for __zpwr_test_var in ${(k)parameters[(I)ZPWR_*]}; do
  unset "$__zpwr_test_var"
done
unset __zpwr_test_var
```

`${(k)parameters[(I)PATTERN]}` uses the reverse-subscript flag in its pattern-matching form to select every parameter name matching `ZPWR_*`, and the loop unsets all of them. Without this, the suite would run against the developer's live configuration — every token-file override, every experimental setting — and would pass locally while failing in CI, or worse, pass in both while testing something other than the committed defaults. Environment leakage is the single most common reason a shell test suite is worthless, and four lines close it.

Only then are the environment files, the library, the alias file, and the verb table loaded in the documented order, each with an existence check that fails the test file rather than proceeding into a confusing downstream error.

The autoload bootstrap in the harness is the same two lines the interactive shell uses — `fpath` prepend and `autoload -z` over the tails — which is what makes the tests meaningful. A harness that sourced the function files directly would prove the files parse, not that they autoload correctly.

Generated Test Families

The test file names describe a matrix. For the autoload tree there is a family per property: existence, presence on `fpath`, being defined, loading successfully, having a non-empty body, `whence` resolution, `type` classification, word-type classification, and source-level syntax validity — each with a parallel `subdir` variant for the platform and completion subdirectories.

Within a family the tests are one per symbol and structurally identical:

```
@test 'fn a body non-empty' {
  run functions a
  assert $state equals 0
  assert "$output" is_not_empty
}
```

The `run` helper captures status into `$state` and output into `$output`; the two assertions check that asking the shell for the function's definition succeeded and produced text. This is a real check against a real failure mode — an autoload stub whose backing file was deleted or renamed still *exists* as a function name and still passes an existence test, but has no body.

The value of the matrix is that each property fails differently and the difference localizes the bug. A file present on `fpath` but not defined means the `autoload` declaration did not run. Defined but no body means the file is missing. Body present but `whence` disagreeing means a name collision with a system function. Running one composite “does this function work” test would collapse all of those into one failure message.

The same generated shape covers the other namespaces. Environment tests assert each `ZPWR_*` variable is defined, exported, and of the expected type. Script tests assert each file in the scripts directory is executable, readable, non-empty, carries a shebang, and passes a syntax check. Verb tests assert each key in the verb table has a valid key format, a non-empty value, and a callable target — the last being the check that catches a verb pointing at a function that was renamed.

The Configuration

```
allow_risky: false
fail_fast: true
verbose: true
```

`fail_fast` is the interesting setting for a suite of this size. Stopping at the first failure is normally the wrong default because it hides how many things are broken. Here it is right, and for a structural reason: the tests share process state through the bootstrap, so a failure in an early family — a missing environment variable, an autoload directory that did not get onto `fpath` — will cascade into thousands of downstream failures that all have the same root cause. Reporting the first one is more useful than reporting all of them.

The CI Contract

The CI workflow contains a constraint that is easy to miss and that shapes the library code:

```
# zpwrCommandExists tests must not assume tools outside this list
# (plus coreutils/util-linux from the Ubuntu image). Optional
# packages like lsof, jq, openssl, sudo, pagers, procps, file,
# strings are not guaranteed.
```

The installed dependency list is deliberately minimal. This is the enforcement mechanism for the graceful-degradation discipline that runs through the whole codebase — every fallback chain in `zpwrClearList`, every `zpwrCommandExists` guard, every conditional verb registration. If the library assumed `jq` existed, CI would catch it, because CI does not install `jq`. A CI environment provisioned with everything would test nothing about the fallback paths, which are precisely the paths that run on an unfamiliar machine.

The workflow builds the test runner from source rather than downloading a release, and caches the built binary keyed on a hash of the workflow file itself — so any change to the CI definition invalidates the runner cache, which is the conservative choice.

The dependency pinning comment on the language toolchain install is worth reproducing as a general lesson:

```
# IMPORTANT: --locked honors the Cargo.lock published with the
# crate. Without --locked Cargo resolves to the latest patch
# version, which has an unguarded platform-specific call that
# only compiles on macOS.
```

An unpinned install of a transitive dependency broke the Linux build, the fix was one flag, and the reasoning was written down at the call site rather than in a commit message. That is the difference between a fix and a fix that survives.

What The Suite Does Not Do

Worth stating plainly. The generated families verify structural properties — that things exist, load, are typed correctly, and are callable. They do not verify behavior. A function that loads cleanly and returns the wrong answer passes every generated test.

Behavioral coverage lives in the hand-written files, which test navigation, core functions, installer file integrity, and repository structure. That is the right division: structural properties are numerous, uniform, and mechanically checkable, so they should be generated; behavioral properties are few, specific, and require judgment, so they should be written. Generating behavioral tests would produce exactly the low-value boilerplate that makes large suites worthless.

Chapter 55 – What Happens When You Press Enter

`zpwAcceptLine` replaces ZLE's `accept-line` widget and runs on every command you execute. That position makes it the single most latency-sensitive function in the framework: any cost it incurs is paid on every command, forever. It also makes it the natural place to hang cross-cutting behavior, and it carries a substantial amount. The tension between those two facts explains most of its implementation.

Alias Expansion Without A Fork

The function needs to know what the buffer expands to — whether the command is one that modifies files, whether it is a terminal-communication command that should not trigger a rename. The obvious implementation forks a subshell. The actual implementation does not:

```
unset 'functions[_expand-aliases]'  
functions[_expand-aliases]=$BUFFER  
_expanded=${functions[_expand-aliases]##$'\t'}  
unset 'functions[_expand-aliases]'
```

Assigning a string to an element of the `functions` associative array defines a function with that body — and `zsh` parses the body at assignment time, performing alias expansion as part of parsing. Reading the element back returns the parsed, alias-expanded, canonically-formatted source. The leading tab that `zsh`'s formatter inserts is stripped with `##$'\t'`.

The whole operation is parameter assignment and expansion: no fork, no subshell, no external process. Compare the alternative, a command substitution around something that expands aliases, which costs a fork plus a full shell initialization on every Enter keypress. On a machine where the shell is the primary interface, that difference is measured in the tens of thousands of avoided forks per day.

The same discipline appears immediately after:

```
strftime -s _datestamp '%Y-%m-%d %H:%M:%S'
```

with a comment noting that the `datetime` module is already loaded at init time. `$(date)` would be a fork. `strftime -s` is a builtin writing into a named variable. Two forks removed from the hot path in the first fifteen lines of the function, and both replacements are strictly more correct as well — the builtin cannot fail due to `PATH` problems and cannot be shadowed by a user's `date` function.

Session Labeling

The first substantive block stamps the terminal with its current state:

```

if ! [[ $_expanded = zc* ]]; then
    ZPWR_CONVEY_NAME="TTY:${TTY} PID:${PID} CMD:$BUFFER PWD:${PWD} \
    DATE:$_datestamp"
    zc-rename $ZPWR_CONVEY_NAME &>/dev/null
    ZPWR_CONVEY_LAST_CMD=$BUFFER
else
    ...
    ZPWR_CONVEY_NAME="TTY:${TTY} PID:${PID} CMD: $ZPWR_CONVEY_LAST_CMD ..."
    zc-rename $ZPWR_CONVEY_NAME &>/dev/null
fi

```

Every terminal continuously advertises its TTY, PID, current command, working directory, and timestamp on a shared bus, so any session can enumerate the others. The guard is the interesting part: if the command *is* a terminal-communication command, the label preserves the *previous* command instead. Without it, every inter-terminal message would relabel the sending terminal as “the terminal that sent a message”, destroying exactly the information the labeling exists to provide. Note the test runs against `_expanded`, not `$BUFFER` — an alias for a communication command is caught too.

Fan-Out To Synchronized Panes

```

if [[ $ZPWR_SEND_KEYS_FULL == false ]]; then
    zpwrKeyClear
else
    if ! [[ $BUFFER == *zpwrStopSend* ]]; then
        for pane in ${ (Az)${ (s@, @)ZPWR_SEND_KEYS_PANE} }; do
            $ZPWR_ZTMUX send-keys -t $pane "C-m"
        done
    else
        for pane in ${ (Az)${ (s@, @)ZPWR_SEND_KEYS_PANE} }; do
            $ZPWR_ZTMUX send-keys -t $pane "C-u"
        done
    fi
fi

```

The nested expansion `${ (Az)${ (s@, @)VAR} }` splits the configured pane list on commas (`s@, @` using `@` as the delimiter character so commas need no escaping), then re-splits shell-style into an array. The multiplexer binary comes from a variable, which is what allows the drop-in replacement to be substituted globally.

The `zpwrStopSend` sentinel inverts the behavior: normally Enter is forwarded to every synchronized pane, but a buffer containing the sentinel sends `C-u` (kill line) instead, clearing the other panes rather than executing there. This is the escape hatch for the case where you have broadcast a command to sixteen panes and realize you do not want it to run.

Detecting File-Modifying Commands

The framework re-lists the directory after commands that change it, which requires knowing whether the command about to run is one of those:

```

for command in ${ZPWR_CMDS_THAT_MODIFY_FILES[@]}; do
    if [[ ${cmdstr} == ${command}([ ^[:alnum:]]_*|) ]]; then
        ZPWR_WILL_CLEAR=true
        break
    fi
done

```

The pattern is the interesting part. `${command}([ ^[:alnum:]]_*|)` matches the command name followed

by *either* a non-word character and anything, or nothing at all. So `touch` matches `touch` and `touch file`, but not `touchtest`. A naive prefix match `${command}*` would fire on every command sharing a prefix with a destructive one. The source carries a comment making exactly this point, which suggests it was found the hard way.

The candidate list is defined in the environment layer:

```
typeset -gaU ZPWR_CMDS_THAT_MODIFY_FILES=(rm mv cp mkdir ln chown
  chmod touch unlink rrm md rmdir chflags chgrp 'git reset'
  'git clone' gcl dot_clean)
```

`-U` makes it unique-enforcing, so a plugin appending an entry that is already present is a no-op. Multi-word entries like `git reset` work because the pattern match is against the whole command string, not against a single word.

Rewriting For The Colorizer Under sudo

The most involved branch handles a specific, real problem. The framework aliases many commands to colorizer-wrapped versions. Aliases only expand in command position, so `sudo ping` runs the raw binary and loses the colorization. The generic fix — enabling second-position alias expansion — has its own problems under `sudo`, where the environment must be forwarded explicitly.

```
if [[ $prefix == (*sudo*|*env*|*nice*|*time*) ]]; then
  value="${aliases[$cmd]}"
  if [[ $value == grc([[:alnum:]]*) ]]; then
    print -srn -- "$BUFFER"
    BUFFER="$ZPWR_FULL_SUDO $value ${args[@]}"
    echo
    eval "$BUFFER"
    BUFFER=""
    zle .accept-line
    return 0
  fi
fi
```

When the command line is prefixed by a privilege or environment wrapper and the wrapped command has a colorizer alias, the buffer is rewritten entirely. `ZPWR_FULL_SUDO` expands to a fully-qualified invocation that forwards `PATH` explicitly, defended against shadowing at every step:

```
export ZPWR_FULL_SUDO='"builtin" "command" "sudo" -E "env" PATH="$PATH"'
```

Each word is quoted individually, which suppresses alias expansion on that word — so a user alias for `command` or `env` cannot hijack a privileged invocation.

`print -srn -- "$BUFFER"` is the detail that makes this acceptable: `-s` pushes the string onto the history list. The *original* buffer, as typed, is what lands in history; the rewritten form is executed but never recorded. History stays a record of what you meant rather than of what the framework did on your behalf. The function then evaluates directly, clears the buffer, and calls the real widget with `zle .accept-line` — the dot prefix selecting the builtin widget rather than recursing into this function.

Suppressing Global Alias Expansion

The final block handles a conflict between two features. Global aliases expand anywhere on the line, not just in command position — which is the point of them, and which means a global alias named the same as something you want to run in command position becomes uninvocable.

```

if [[ -n ${galiases[$cmd]} ]]; then
    skipExpand=true
    if [[ -z "$ZPWR_GLOBAL_ALIAS_PREFIX" ]]; then
        if [[ -n ${aliases[$cmd]} ]]; then
            line="${aliases[$cmd]}"
            if [[ $line == *"'* ]]; then
                BUFFER="${line:1:-1} $mywords[2,$]"
            else
                BUFFER="$line $mywords[2,$]"
            fi
        else
            BUFFER="\ $mywords"
        fi
    else
        ...
        BUFFER="\ $mywords"
    fi
fi

```

The general solution is the backslash: `\foo` suppresses alias expansion on `foo` entirely, so prefixing the command word with one lets the underlying function or binary run. The complication is the case where the name is *both* a global alias and a regular alias, in which case the regular alias's expansion is substituted manually — with a quote-stripping step, `${line:1:-1}`, that removes the surrounding single quotes an alias body may carry.

The comment on the quote test says it plainly: a zsh pattern match avoids the fork and pipe that a `grep` would cost. That comment could be attached to a dozen places in this function, and the cumulative effect is why a widget carrying this much cross-cutting behavior remains usable on every keystroke.

The function ends by delegating to the real widget and resetting terminal attributes:

```

zle .accept-line
printf "\x1b[0m"

```

The reset is defensive — a prompt theme that leaves an attribute set would otherwise bleed its color into the output of the command you just ran. One escape sequence, unconditionally, is cheaper than diagnosing why command output is occasionally the wrong color.

Chapter 56 — The Keymap As A Namespace

`zpwBindOverrideZLE` is one function containing several hundred `zle -N` and `bindkey` calls. Read as a list it is impenetrable. Read as a namespace design it is coherent, and the design solves a problem every heavily-customized shell eventually hits: a terminal offers roughly thirty usable control characters, and a serious environment has an order of magnitude more actions than that.

Registration Before Binding

The function opens with two `autoload` calls and then a long block of widget registrations:

```
autoload -Uz select-bracketed select-quoted bracketed-paste-magic \
    edit-command-line
autoload -z zpwBufferXtrace zpwExpandOrCompleteWithDots

zle -N zpwBufferXtrace
zle -N accept-line zpwAcceptLine
zle -N bracketed-paste bracketed-paste-magic
zle -N zpwIntoFzf
...
```

The two `autoload` forms differ deliberately. `-Uz` suppresses alias expansion when the function is loaded (`U`) in addition to selecting `zsh`-style loading (`z`), and it is used for the *shipped* `zsh` contrib widgets — code the framework did not write, which must not be affected by the framework's own aliases. The framework's own widgets use bare `-z`, because they are written in the same alias environment they will run in.

`zle -N` has two forms visible here. One argument creates a widget named after an existing function. Two arguments create a widget under one name backed by a differently-named function — which is how `accept-line` gets replaced without losing access to the original. The builtin widget remains reachable as `.accept-line`, the leading dot selecting the built-in implementation, which is exactly what the replacement calls at its end to actually execute the line.

The Prefix Trick

The core of the namespace design is three lines that look like a mistake:

```
bindkey -M viins -r '^V'
bindkey -M vicmd -r '^V'
bindkey -M viins -r '^F'
bindkey -M vicmd -r '^F'
```

`-r` removes a binding. Both keys are unbound *after* dozens of two-key sequences starting with them have already been bound.

This works because of how `zsh` resolves multi-character key sequences. When a prefix of a bound sequence is typed, `zsh` waits for more input rather than acting. If the prefix is *itself* bound to a widget, `zsh` must decide between executing that widget immediately and waiting to see whether a longer sequence follows — and it

resolves that by waiting for `KEYTIMEOUT`, then acting on the shorter binding. Unbinding the prefix removes the ambiguity entirely: `^F` alone does nothing and produces no delay, while `^F^D`, `^F^I`, `^F^V` and the rest fire immediately on the second keystroke.

The order matters. Unbinding first and then binding the sequences would work equally well; unbinding a key that has no sequences under it would simply make it dead. What would *not* work is leaving the prefix bound, which is the default state — `^V` is `quoted-insert` and `^F` is `forward-char` in the stock keymaps. Both of those single-key functions are sacrificed, deliberately, to buy two namespaces of roughly thirty slots each.

The timeout is tuned to match:

```
export ZPWR_KEYTIMEOUT=10
```

in hundredths of a second. Long enough that a deliberate two-key sequence is never split by a slow second keystroke, short enough that a mistyped prefix resolves quickly.

Two Namespaces, Two Meanings

The split between the prefixes is semantic. `^F` sequences are search and file operations: file search into the buffer, content search, the lsof picker, the verb picker, external editor invocation on the command line. `^V` sequences are insertion and selection: fzf over commits, over the environment index, over frequency-ranked paths, over keybindings themselves.

Within each namespace, many actions are bound twice:

```
bindkey -M viins '^F^J' zpwVerbsWidgetAccept
bindkey -M viins '^Fj'  zpwVerbsWidgetAccept
```

The control form and the plain-letter form both work. This is not redundancy for its own sake — it is a concession to terminal emulators and remote sessions where certain control combinations are intercepted upstream and never reach the shell. Having a second spelling of the same action means a broken terminal costs you a keystroke shape, not a feature.

Every Binding Is Doubled Across Keymaps

Almost every `bindkey` appears twice, once with `-M viins` and once with `-M vicmd`:

```
bindkey -M viins '^V,' zpwFzfEnv
bindkey -M vicmd '^V,' zpwFzfEnv
```

The framework uses vi keybindings, which means two active keymaps and a modal distinction that applies to *editing* commands. But the fzf pickers, the search widgets, and the buffer transformers are not editing commands in the vi sense — they are applications invoked from the line editor, and there is no coherent reason for them to be unavailable in normal mode. Binding both keymaps makes the mode invisible for the entire custom namespace while leaving vi's own motions and operators modal.

A few bindings deliberately break the symmetry. `^W` is bound only in `vicmd`, because `viins` already has a working word-delete there and overriding it would fight muscle memory built on every readline application. `\e^M` is bound only in `viins` to `self-insert-unmeta`, which inserts a literal newline into the buffer — a normal-mode equivalent would be meaningless.

There is also a mode-switch escape hatch bound only in insert mode:

```
bindkey -M viins 'jf' vi-cmd-mode
bindkey -M viins 'fj' vi-cmd-mode
```

Both orderings of the same two adjacent keys, so the sequence works regardless of which finger lands first. This is the standard alternative to reaching for `Escape`, and binding both permutations removes the one failure

mode the technique has.

Conditional Bindings

Bindings for external tools are guarded on the tool existing:

```
zpwExists fasd 66 {
    bindkey -M viins '^V^F' zpwFasdFZF
    bindkey -M vicmd '^V^F' zpwFasdFZF
}

zpwExists z 66 {
    zle -N zpwZFZF
    bindkey -M viins '^V^S' zpwZFZF
    ...
}
```

Note that in the second case the widget *registration* is inside the guard too, not just the binding. Registering a widget whose backing function will fail on invocation is worse than not registering it: the key would appear bound, the widget would fire, and the failure would surface as an error message from inside the line editor. Keeping registration and binding together under one guard means an absent tool leaves the key genuinely unbound, and the terminal's default behavior for that key returns.

`zpwExists` rather than `zpwCommandExists` is the correct predicate here, because both of these tools are commonly installed as shell functions by their plugins rather than as binaries on [PATH](#).

Environment-Sensitive Bindings

The end of the function does something unusual: it inspects its own parent process to decide what to bind.

```
ZPWR_PARENT_PROCESS="${command ps -p $PPID 2>/dev/null |
    stryke -lane '$=' " ";print "@F[3..$#F]" if m{^s*\d+.*}'}"

if [[ "$ZPWR_OS_TYPE" == darwin ]]; then
    if [[ "$ZPWR_PARENT_PROCESS" == *(login|tmux)* ]]; then
        bindkey '\e[1;5A' zsh-gacp-CheckDiff
        bindkey '\e[1;5B' zpwUpdater
        ...
    fi
fi
```

The escape sequences bound here are the Control-plus-arrow-key encodings, and the problem they solve is that IDE-embedded terminals intercept those combinations for their own navigation. A shell running under a terminal application or inside tmux gets the bindings; a shell running as a subprocess of an IDE gets a different set, because binding keys the IDE will never deliver is equivalent to not binding them while still consuming the namespace.

The parent-process detection reads `ps` output and takes fields four onward, which is the command column. It is one fork, once, at shell initialization, and it is the only way to answer the question — there is no environment variable that reliably distinguishes an IDE-hosted terminal from a real one across the range of IDEs in play.

Reading The Result

The practical consequence of this design is that the keymap is too large to memorize and the framework knows it. There are widgets bound inside the namespace whose entire purpose is to search the namespace:

```
bindkey -M viins '^V.' zpwrfzfAllKeybind
bindkey -M viins '^Vk' zpwrfzfVimKeybind
```

fuzzy search over all keybindings, and over the editor's keybindings specifically, both reading from cache files maintained in the local directory. This is the same pattern as the verb picker: an exact-match dispatcher with no guessing, plus an interactive search tool for discovery. A namespace of a few hundred bindings is navigable if and only if it is searchable, and building the search tool into the namespace it searches is the design that makes the size sustainable rather than merely large.

Chapter 57 – Updating A Machine You Did Not Configure Today

A workstation that has accumulated a decade of tooling does not have a package manager. It has a dozen of them, layered by era and by language, each with its own idea of where things live, whether an upgrade needs elevated privileges, and what the word “outdated” means. `scripts/updater.sh` is the framework’s answer: one linear script that walks every ecosystem it can find, updates it, and does not stop when one of them fails.

The shape of the script is worth stating before the details, because the shape is the design. It is not a plugin system. There is no registry of updaters, no `for manager in "${MANAGERS[@]}"` loop, no interface a new ecosystem must implement. It is a flat sequence of guarded blocks, one per tool, in a fixed order. That looks unsophisticated until you consider what the alternative buys you: a uniform interface across ecosystems whose update semantics are not uniform. `brew` wants `update` then `upgrade` then three separate cleanup verbs. `npm` wants you to compute the outdated set yourself and install each member. `rustup` wants one word. `cargo` wants a third-party subcommand installed first. An abstraction over that set would be mostly escape hatches.

The Guard Is The Interface

Every block has the same first line and nothing else in common:

```
zpwCommandExists rustup && {  
  zpwPrettyPrint "Updating rustup"  
  rustup update  
}
```

`zpwCommandExists` is defined in `scripts/lib.sh` and has two implementations selected at source time by `zpwIsZsh`. Under `zsh` it tests `(( $+commands[$cmd] ))`, a lookup in the shell’s own command hash — no fork, no `PATH` walk. Under `bash` it falls back to `type -ap`. Both accept multiple names and return failure if any one is missing, which is why the installer can write `zpwCommandExists sudo perl git curl bash` as a single precondition check.

The `&& { ... }` form rather than `if ... fi` matters for what the script does on failure. A brace group after `&&` runs to completion regardless of what the commands inside it return; the group’s own exit status is that of its last command, and nothing consumes it. So a broken `gem update` neither aborts the script nor prevents `brew` from running twenty lines later. The script has no `set -e` and that is deliberate: the failure mode you actually care about on a long-running maintenance script is “one ecosystem was broken and the other eleven did not get updated,” and that is exactly the failure mode `set -e` produces.

The Writability Probe

Two blocks share a pattern that is worth extracting, because it encodes a real operational distinction that most update scripts get wrong by always using `sudo` or never using it:

```
installDir=$(npm root -g | head -n 1)
if [[ ! -w "$installDir" ]]; then
    zpwNeedSudo=yes
else
    zpwNeedSudo=no
fi
```

The question “does this need elevation” is not answered by asking the operating system, the user, or a configuration flag. It is answered by asking the package manager where it installs and then testing that directory for write permission. A Node installed through a version manager under the home directory needs no elevation; a Node installed from a system package does. The same binary, the same command line, two different answers, resolved by one `-w` test against a path the tool itself reports.

The pip variant in `lib.sh` extends this per-package rather than per-manager:

```
installDir="$PIP3_HOME/$package"

if [[ $forceSudo == true || ! -w "$installDir" ]]; then
    zpwNeedSudo=true
else
    zpwNeedSudo=false
fi
```

which is the correct granularity for pip, where a user-site install and a system-site install of two different packages can coexist. Note also the tied array declaration at the top of that function:

```
typeset -Tgx ZPWR_PIP_BLACKLIST zpwPipBlacklist
```

`typeset -T` binds a scalar and an array to the same storage, joined and split on colons. The scalar half is exported and therefore survives into subprocesses and across the ssh transport described below; the array half is what the membership test uses. `zpwValidatePipPackage` then checks `(( zpwPipBlacklist[(Ie)$package] ))` — the `(I)` subscript flag returns the index of the last match and `e` forces an exact string comparison rather than pattern matching, so a blacklist entry containing a glob character cannot accidentally match a neighbouring package name. A nonzero index is truthy, so the arithmetic evaluation is the whole test.

There is a dead branch in that same function, and it is worth naming rather than glossing over. `needSudoBase` is assigned the literal `false` and then tested against `true` three separate times, so the `sudo -EH` half of each of those conditionals cannot execute. The elevated forms are still correct code — they were reachable when the variable was computed rather than pinned — and pinning it to `false` is the safe direction to have pinned it. But a reader tracing why a system-site pip package never upgrades under elevation should know that the answer is a constant, not a probe.

The Remote Half

The second half of the script updates other machines, and the transport is the interesting part:

```
ssh -x "$hostname" zsh < <(echo "export ZPWR_REMOTE=true";
cat "$ZPWR_LIB" "$ZPWR_ENV_FILE" "$ZPWR_RE_ENV_FILE" \
"$ZPWR_TOKEN_PRE" "$ZPWR_TOKEN_POST" \
"$ZPWR_SCRIPTS/updaterPip.zsh" 2> /dev/null)
```

Read that as a deployment strategy, not as a command. The remote host is not required to have the framework installed. It is not required to have anything fetched, cloned, or synchronized. The local machine concatenates its own library, its two environment files, both user token files and the script to be run into a single stream, and feeds that stream to a remote `zsh` over standard input. The remote shell reads a program that arrives complete with its own dependencies and its own configuration, executes it, and exits leaving nothing behind.

This is agentless in the strict sense — no persistent component on the target, nothing to version-skew, nothing

to uninstall. It costs one ssh connection per script and it re-sends the environment every time, which is the tradeoff: bandwidth and startup latency in exchange for a target that needs no state. The `export ZPWR_REMOTE=true` prefix is what the sourced files use to suppress interactive-only behaviour, since the receiving shell is not attached to a terminal.

The `-x` flag disables X11 forwarding. On a script that fans out to every host in a fleet, forwarding negotiation is pure latency for a session that will never open a window, and disabling it explicitly beats depending on the remote `sshd` configuration to have it off.

Hosts themselves are encoded as colon-delimited pairs:

```
hostname="$(echo "$1" | awk -F: '{print $1}')"
manager="$(echo "$1" | awk -F: '{print $2}')
```

so an entry reads `buildbox:apt` or `alpine-node:apk`. The host half is an `.ssh/config` alias, which means every connection detail — real address, port, user, key, jump host — lives in the file that already exists for that purpose rather than being duplicated into the updater. The manager half selects one of five branches, and each branch is a single quoted script sent to the remote shell: `apt`, `zypper`, `pacman`, `apk`, `dnf`. Every one of them is written non-interactively, either by piping `yes` in or by the manager's own flag (`--non-interactive`, `--noconfirm`, `-y`), because there is no terminal on the far end to answer a prompt.

`PI_ARRAY`, the list those pairs live in, is not assigned anywhere in the source tree. It comes from the user's token file, which is the correct place for it: a fleet inventory is machine-specific data, not framework code, and putting it in a file the framework sources but does not ship means updating the fleet list never conflicts with updating the framework.

Git Repositories Are An Ecosystem Too

Before any package manager runs, four directory trees get the same treatment through `zpwrGitRepoUpdater`:

```
for generic_git_repo_plugin in "$enclosing_dir/"*; do
  if [[ -d "$generic_git_repo_plugin" ]]; then
    (
      builtin cd "$generic_git_repo_plugin" &&
      zpwrIsGitDir &&
      printf "\x1b[1m%s:\x1b[0m " \
        "${basename "$generic_git_repo_plugin"}" &&
      git fetch --all --prune &&
      git pull --all && zpwrClearGitCache
    )
  fi
done
```

Three details carry weight. The body runs in a subshell, so the `cd` is scoped and the loop does not need to return to the starting directory — the parent shell's working directory was never changed. `zpwrIsGitDir` is a `git rev-parse --git-dir` probe rather than a `[[ -d .git ]]` test, so a worktree or a submodule with a `.git` file rather than a directory is handled correctly. And the whole chain is `&&`-linked, so a directory that is not a repository silently produces nothing instead of an error, which is what you want when the loop is walking a plugin directory that may contain a stray download.

The `--prune` on the fetch is what keeps the local ref namespace from accumulating branches that were deleted upstream years ago. Over a plugin directory with dozens of repositories tracked over a decade, that is not a cosmetic concern; a `git branch -a` with several hundred stale remote refs is functionally unreadable, and the refs are consulted on every completion of a branch name.

Chapter 58 — roff In, Completions Out

The completion generators described earlier call out to a Python program to do the actual extraction from a man page. That program, `scripts/mantozshcomp.py`, is the largest file in the `scripts` directory and the only piece of the framework that is a derivative work: its header carries a 2012 BSD copyright notice, and its original form generated fish completions. The retarget to zsh is not a translation of output strings — zsh's `_arguments` has a different shape than fish's `complete`, and the emission layer was rewritten around it while the roff parsing layer was kept.

That division is the reason the port is tractable. Parsing roff is the hard, boring, empirical work that took years of contact with real man pages to get right. Emitting `_arguments` specs is a page of string assembly. Keeping the former and rewriting the latter is the correct place to cut.

Six Parsers, Tried In A Deliberate Order

There is no such thing as *the* man page format. There is roff, a typesetting language, and a set of conventions layered on it that vary by decade, by distribution, and by whether the author used `man` macros or `mdoc` macros. The program handles this by defining six parser classes and asking each one whether it recognizes the page:

```
parsers = [
    Type1ManParser(),
    Type2ManParser(),
    Type4ManParser(),
    Type3ManParser(),
    TypeDarwinManParser(),
    TypeDeroffManParser(),
]
parsersToTry = [p for p in parsers if p.is_my_type(manpage)]
```

The recognition tests are one regular expression each, and reading them in list order tells you the whole taxonomy. Type 1 matches `.SH "OPTIONS"` with the section name in quotes. Type 2 matches `.SH OPTIONS` without them. Type 4 matches `.SH FUNCTION LETTERS`, a convention used by a small number of older utilities whose flags are documented as letters rather than options. Type 3 matches `.SH DESCRIPTION` and scavenges option-looking lines out of prose. The Darwin parser matches `.S[hH] DESCRIPTION` case-insensitively on the macro itself, which is the mdoc dialect BSD man pages are written in. And the last one:

```
def is_my_type(self, manpage):
    return True # We're optimists
```

The list is not in numeric order, and the deviation is the interesting part. Type 4 is tried before Type 3 because a page with a `FUNCTION LETTERS` section almost certainly also has a `DESCRIPTION` section, so both tests pass and the list order decides which parser gets first refusal. Ordering by specificity rather than by name is correct; naming the classes numerically and then having to reorder them is the cost of naming things after their discovery order instead of their selectivity.

The cascade terminates on the first parser that both claims the page and produces output:


```

success = parser.parse_man_page(manpage)
if not built_command_output:
    success = False
if success:
    break

```

The middle two lines are load-bearing. A parser can return a truthy value while having emitted nothing — it recognized a section header and then found no options inside it. Without the emptiness check, that counts as success, the cascade stops, and a page that a later parser would have handled produces an empty completion. Trusting the return value alone is the obvious implementation and it is wrong.

The program also carries a commented-out line that would have annotated each generated file with the parser that produced it, disabled with a note that it misattributed the parser — reporting the deroff parser for pages that were really handled by Type 2. That is worth reading as a warning rather than as trivia: the provenance comment was emitted from a variable that did not hold what the author expected at that point in the flow, and the failure was silent in the sense that the generated completions were fine and only the diagnostics lied. Disabling the line rather than fixing it leaves the completions unattributed, which is honest. An attribution that is wrong is worse than none, because the next person to debug a bad completion starts from a false lead.

What Never Gets Parsed

Three exclusion mechanisms run before any parser is consulted, and each one has a distinct rationale.

```

ignoredcommands = [
    "cc", "g++", "gcc", "c++", "cpp", "emacs",
    "gprof", "wget", "ld", "awk",
]

```

These are the pages where automatic extraction is known to produce garbage. Compiler and linker man pages document hundreds of flags with complex argument grammars in prose that defeats structural extraction; the generated result is a large file of half-parsed options that actively degrades tab completion compared to having none. The editor and the download tool are on the list for the same reason at smaller scale.

```

ignored_prefixes = ["zsh"]

```

Every `zsh*` page is skipped, because `zsh` ships its own completions for its own tooling and a generated `_zshall` competing with the hand-written one is a regression by construction.

```

if "BUILTIN 1" in manpage or "builtin.1" in manpage:
    return

```

On BSD-derived systems, every shell builtin has a man page, and every one of those pages is a hard link or `.so` include pointing at a single `BUILTIN(1)` page. Without this check the generator would produce dozens of identical completions, all describing the same aggregate page, all attached to commands whose real completions come from the shell.

The decompression layer above these checks handles `.gz`, `.bz2`, `.xz` and `.lzma`, plus bare section-numbered files, and decodes every one of them as latin-1 rather than UTF-8. That is not laziness. latin-1 is the encoding in which every byte sequence is valid, so decoding as latin-1 cannot throw. Man pages on a real system include files in several encodings and some in none, and a generator that walks the entire man tree cannot afford to abort on a decode error in one file. Correctness of the high-byte characters is sacrificed for guaranteed progress, and since the extracted text is flag names and one-line descriptions, the damage is confined to accented characters in a description.

Assembling An `_arguments` Spec

The emission side collects option specs into a list and writes the file in one pass at the end:

```

built_command_output.insert(0, "#compdef " + CMDNAME + "\n")
built_command_output.insert(
    1, "# Autogenerated from man page " + manpage_path + "\n")
built_command_output.insert(2, "local arguments\n")
built_command_output.insert(3, "arguments=(")
built_command_output.append("'*:filename:_files'")

```

The four inserts push the preamble onto the front of a list that already holds the option specs, which is why they are inserts at fixed indices rather than appends to an empty list — the specs were accumulated by a parser that had no notion of where in the file they would land.

The `#compdef` tag is what `compinit` scans for. The `Autogenerated` comment on line two is not decoration: `file_is_overwritable` reads it back on later runs to decide whether a file in the output directory may be replaced. A hand-edited completion that has had the comment removed becomes permanently safe from the generator. That is a good convention — the marker for “this is machine-managed” lives in the file itself rather than in a manifest that can drift.

The trailing `'*:filename:_files'` gives every generated completion a fallback: anything not matching a known flag completes as a filename. Without it, a generated completion would offer flags and nothing else, and typing a path after the command would produce no completions at all — the failure mode that makes generated completions feel worse than no completions.

Option strings are classified into three zsh spec forms:

```

if option.startswith("--"):
    zsh_options.append("-l " + zsh_escape_single_quote(option[2:]))
elif option.startswith("-") and len(option) == 2:
    zsh_options.append("-s " + zsh_escape_single_quote(option[1:]))
elif option.startswith("-") and len(option) > 2:
    zsh_options.append("-o " + zsh_escape_single_quote(option[1:]))

```

Long, short, and old-style-long — the last being a single dash followed by a word, as `find` and `X` clients use. The three-way split is right, and it is worth tracing what happens to the third class downstream, because it does not survive. `output_complete_command` re-dispatches on the prefix of the accumulated spec:

```

if re.match("-s", arg):
    splitTrim(arg, comps, "-")
else:
    splitTrim(arg, comps, "--")

```

Only the `-s` marker is tested. A spec beginning `-o` takes the `else` branch and is emitted with two dashes, so an old-style long option documented as `-recursive` reaches the completion as `--recursive`. The classification is performed and then discarded. This is the sort of defect that generated completions make easy to live with and hard to notice: the affected commands are exactly the small set of old utilities using that convention, and their users are long past needing tab completion to remember the flag.

Descriptions Are Load-Bearing And Get Budgeted

A flag list without descriptions is barely worth generating; the value is in seeing what the flag does in the completion menu. The program treats the description as a fixed-width budget and packs sentences into it:

```

sentences = clean_desc.split(".")
truncated_description = lossy_unicode(sentences[0]) + udot
for line in sentences[1:]:
    proposed_description = (truncated_description + uspace
                             + line + udot)
    if len(proposed_description) <= max_description_width:
        truncated_description = proposed_description
    else:
        break

```

Greedy whole-sentence packing up to a fixed width, then a hard character truncation with an ellipsis if even the first sentence overflows. The result is that a description is never cut mid-sentence unless there was no alternative, which reads considerably better in a completion menu than the naive fixed-prefix truncation.

The escaping around it is aggressive in a way that reveals what the output format cannot survive. Apostrophes are not escaped in descriptions — they are deleted, along with a special case rewriting `don't` to `do not` first so the deletion does not produce `dont`. Square brackets are backslash-escaped because they delimit the description inside an `_arguments` spec. Runs of whitespace collapse to single spaces because roff line breaks are not semantic. And a leading `:` is stripped, which is a small artifact of the extraction rather than a property of the source.

Deleting apostrophes rather than escaping them is the pragmatic choice given that the whole spec is wrapped in single quotes and the escaping rules inside an `_arguments` description are nested two levels deep — quote for the shell, then bracket for the spec parser. A description reading `do not follow symlinks` in place of `don't follow symlinks` costs nothing. A description that breaks the quoting costs a completion function that fails to load, and one broken file on `fpath` produces an error on every completion attempt, not just for that command.

Finally, deduplication happens at the spec level rather than the option level:

```

existing_options = already_output_completions.setdefault(CMDNAME, set())
zsh_options = [opt for opt in zsh_options
                if opt not in existing_options]
existing_options.update(zsh_options)
if not zsh_options:
    return

```

Keyed by command name, so the same option documented twice in one page — common where a flag appears in both a synopsis section and an options section — emits once. The early return when nothing survives deduplication also prevents emitting a spec with an empty option list, which would be a syntax error in the generated file rather than a merely useless entry.

Chapter 59 – tmux As A Composed Configuration

A `.tmux.conf` is usually one file. This one is a directory of files that source each other conditionally, and the reason is that tmux has no version guards, no platform detection, and no way to express “this option only exists after 2.9.” Every option in a tmux configuration is unconditionally parsed at load, and an option tmux does not recognize is a load error that aborts the rest of the file. A configuration that must work across three operating systems and a decade of tmux releases therefore cannot be one file. It has to be a dispatcher.

if-shell Is The Only Conditional

The entry point is `tmux/init.conf`, and its final section is where the composition happens. tmux offers exactly one branching construct, and it is a subprocess:

```
if-shell "uname -s | grep -iq Darwin" \  
    "source-file $ZPWR_TMUX/tmux-mac.conf" \  
    "source-file $ZPWR_TMUX/tmux-linux.conf"
```

`if-shell` runs its first argument through the shell and picks the second or third argument based on the exit status. That is the entire conditional vocabulary. Everything else in the composition is built out of it, and the cost model is one fork per conditional at configuration load.

The version gates use the same mechanism with arithmetic pushed into `awk`, because tmux has no numeric comparison either:

```
if-shell "[ `tmux -V | awk '{print ($2 >= 2.9)}'` -eq 1 ]" \  
    'source $ZPWR_TMUX/tmux_ge_29.conf' \  
    'source $ZPWR_TMUX/tmux_lt_29.conf'
```

Three of these run in sequence, at 2.1, 2.9 and 3.1, each selecting one of a pair of files. Note what `awk` is doing: `$2` of `tmux -V` output is the version string, and comparing it against a numeric literal makes `awk` coerce it to a number, printing 1 or 0. This is a string-to-float coercion on a version number, which works because tmux versions have exactly one dot. It would misjudge a three-component version, and it silently truncates a suffixed version such as `3.2a` to `3.2` — which is the behaviour you want here, since a letter suffix in tmux denotes a patch release with no new options.

The gates are independent rather than nested, so all three fire on every load and a modern tmux sources the “greater than” half of all three pairs. This is additive layering, not a switch: each pair contributes the settings that changed at that boundary, and a running configuration is the union of three contributions plus the platform file plus the base.

Reading The Pairs As A Changelog

The version files are small and each one documents one backwards-incompatible change in tmux’s history. The 2.1 pair is the clearest:

```
# tmux_ge_21.conf
set -g mouse on

# tmux_lt_21.conf
set -g mode-mouse on
set -g mouse-resize-pane on
set -g mouse-select-pane on
set -g mouse-select-window on
```

Four separate options collapsed into one at 2.1. Setting `mouse` on an older tmux is an unknown-option error; setting the four old options on a newer one is four unknown-option errors. There is no formulation that works on both, which is precisely why the gate exists.

The 2.9 pair carries a different kind of change — `choose-tree` gained flags, so the newer file rebinds a key that the base configuration already bound:

```
bind-key w choose-tree -Zw
```

The base `init.conf` binds `w` to plain `choose-tree`, and the 2.9 file overrides it later in load order. That is the layering discipline in miniature: the base file always contains the form that works everywhere, and version files override downward. A base file containing the newest form with older files trying to undo it would fail, because the failure happens at parse time in the base file before any override could run.

The reload binding illustrates the same ordering rule at three levels. `bind-key r` appears in three of the six version files — the two “less than” files at 2.9 and 3.1, and the “greater or equal” file at 3.1 — and the last one sourced wins. On a modern tmux, the 2.9 gate contributes nothing to `r` and the 3.1 gate binds it with a trailing `copy-mode -q`, which exits copy mode as part of the reload. On an old tmux, the 2.9 file binds it and the 3.1 “less than” file binds it again identically. The redundancy is harmless and it makes each file independently readable, which matters more than eliminating a duplicate line in a five-line file.

The 2.9 file also does something worth pointing out to anyone who has not read tmux’s manual closely:

```
set-option -s command-alias[100] zoom='resize-pane -Z'
```

`command-alias` is an array option, and the subscript is the priority slot. Choosing 100 places the alias far above the built-in aliases tmux ships at low indices, so it cannot be shadowed by a future tmux release adding an alias with a colliding name at a lower index. Writing `command-alias[0]` would work today and break on an upgrade.

The Platform Files Are Not Symmetric

Three platform files handle the two things that genuinely differ across operating systems: the system clipboard and the prefix key. The clipboard divergence is total — macOS pipes to `pbcopy` through `reattach-to-user-namespace`, Linux pipes to `xclip` with both the primary and clipboard selections, and WSL pipes to `clip.exe`, a Windows binary invoked directly from the Linux side. Every copy-mode binding is therefore restated in full in each of the three files rather than being parameterized over a `$CLIPBOARD_COMMAND` variable.

That looks like duplication and it is, but the alternative is worse than it appears. The macOS bindings are not the Linux bindings with a different command substituted: `default-command` has to be wrapped in `reattach-to-user-namespace` on macOS and must not be on Linux, and the Linux clipboard command is a pipeline of two `xclip` invocations rather than a single command, so it cannot be substituted into a template that assumes one. Parameterizing across three platforms whose differences are structural rather than lexical produces a template with three escape hatches.

The prefix handling is asymmetric in a way that is worth reading carefully:

```
# tmux-mac.conf
unbind $ZPWR_TMUX_PREFIX_LINUX
set -g prefix $ZPWR_TMUX_PREFIX_MAC

# tmux-linux.conf
unbind C-b
set -g prefix $ZPWR_TMUX_PREFIX_LINUX
```

The macOS file unbinds the *configured Linux prefix* before setting its own; the Linux file unbinds the *literal tmux default*. Both are correct for the case they handle, and the asymmetry encodes an assumption: that the Linux prefix is the one that might already be bound when the macOS file loads. Given that `init.conf`'s platform branch is exclusive, neither file can actually observe the other's binding, so the macOS `unbind` is defensive against a state that the composition prevents.

WSL is handled as a modifier rather than a third branch. Its file sets no prefix at all, because the detection order means the Linux file has already run — `uname -s` on WSL reports Linux, and the WSL test is a separate `if-shell` on `uname -r` matching `icrosoft`, deliberately written without its leading character to sidestep the case difference between kernel version strings that report `Microsoft` and those that report `microsoft`. So WSL inherits the Linux prefix, the Linux mouse bindings, and everything else, and overrides only the clipboard path and the status bar position.

Layouts Are Configurations Too

The last piece of the composition is a set of files that are not configuration in the usual sense. `four-panes.conf`, `eight-panes.conf`, `sixteen-panes.conf` and their larger siblings are sequences of split and select commands:

```
selectp -t 0
splitw -h -p 50
selectp -t 0
splitw -v -p 50
selectp -t 2
splitw -v -p 50
selectp -t 0
```

and they are reached through key bindings in the base file:

```
bind-key F source-file $ZPWR_TMUX/four-panes.conf
bind-key G source-file $ZPWR_TMUX/eight-panes.conf
bind-key O source-file $ZPWR_TMUX/sixteen-panes.conf
bind-key R source-file $ZPWR_TMUX/thirtytwo-panes-repl.conf
```

This uses `source-file` as a procedure call. tmux has no user-defined commands and no way to bind a key to a sequence of more than a couple of commands without an unreadable chain of `\;` separators. A file of commands sourced by a binding is the closest thing tmux offers to a subroutine, and it gets you the two properties that matter: the layout is editable as a file rather than as an escaped one-liner, and it can be invoked from a binding, from the command prompt, or from a script identically.

The `-repl` variants are the same idea one level further — a pane layout plus the commands to start a language REPL in each pane, so a single keystroke produces a grid of live interpreters. Building this out of the layout files rather than out of a shell script is what keeps it a tmux-native operation: no external process supervises it, and the panes belong to the session normally rather than being managed by something outside it.

Chapter 60 – The Installer’s Ledger

An installer for a framework this size has one property that dominates its design: it will fail. Not might — will, on some machine, at some step, because a package name changed upstream or a network fetch timed out or a compiler is missing. Everything in `install/zpwrInstall.sh` and its companion `common.sh` is organized around making that failure survivable and diagnosable rather than around making the success path elegant.

Preconditions Before Anything Is Touched

`common.sh` is sourced first and its entire first half is refusal logic. It resolves its own location through a symlink chain, walks up to the directory containing the `.zpwr_root` marker, exports the derived paths, and then verifies that the two files it expects are present in the working directory. The main script adds four more checks that all produce the same message:

```
if [[ ! -d $ZPWR_LOCAL ]]; then
    echo "Must be in $ZPWR/install directory" >&2
    exit 1
fi

if [[ $ZPWR == "$ZPWR_PWD" ]]; then
    echo "Must be in $ZPWR/install directory" >&2
    exit 1
fi
```

The third of those is the one that repays reading. It is not checking that a directory exists; it is checking that the resolved framework root is *not* the current directory — that is, that you are one level down, inside `install`, rather than at the top. Every subsequent relative path in the script assumes that position. A script that guesses at the caller’s working directory and adjusts is more forgiving and much harder to reason about; refusing is a one-line check that makes every relative path in the rest of the script unambiguous.

Then the tool preconditions:

```
if ! zpwrCommandExists sudo perl git curl bash; then
    zpwrLogConsoleErr "you must have sudo, perl, git, bash and curl \
installed"
    exit 1
fi
```

Five tools in one call, because `zpwrCommandExists` fails on the first missing name. Note what is not in that list: `zsh`. The installer is a `bash` script that installs a `zsh` framework, and it must run on a machine where the target shell is not yet present.

`common.sh` makes a related choice for its text-processing dependency:

```
if type -p stryke &>/dev/null; then
    export ZPWR_PERL=stryke
else
    export ZPWR_PERL=perl
fi
```

with the reason stated in the comment above it — that the preferred tool may not be installed yet. The installer is the one context in the system where the framework’s own dependencies cannot be assumed, so it degrades

to the thing that is on every Unix by default and lets everything downstream of the install use the preferred implementation.

Everything Is Logged, Twice

Early in the run, after the directory checks and the `mkdir -p` calls, the script rewires its own output:

```
exec > {tee -a "$LOGFILE"}
exec 2>&1
```

`exec` with a redirection and no command replaces the shell's own file descriptors for the remainder of the script. The process substitution starts a `tee` that appends to the log, and standard output is connected to its input; the second line folds standard error into the same stream. From that point on, every subsequent command — including the hundreds of package-manager invocations that follow — writes to both the terminal and the log without any of them being aware of it.

Doing this with `exec` rather than by piping the whole script into `tee` at invocation matters for one reason: the script controls when logging starts. The precondition failures above happen before the redirection, so a user who runs the installer from the wrong directory gets an error on the terminal and does not create a log file recording a run that never began.

The log is written with escape sequences intact, which is why the log filename is `escaped_logfile.txt` and why the script verifies at startup that `scripts/escapeRemover.pl` exists and is executable before doing anything else. That program is a single loop over standard input, and it is a better ANSI stripper than the one-liner most people write:

```
while (<>) {
    s/ \e[ #%()*+\.\/] . |
    \r |
    (?:\e\[|\x9b) [ -?]* [@~] |
    (?:\e\||\x9d) .*? (?:\e\\|[\a\x9c]) |
    (?:\e[P^_]||[\x90\x9e\x9f]) .*? (?:\e\\|\x9c) |
    \e. |[\x80-\x9f] //xg;
    1 while s/^\b\b//g;
    print;
}
```

The usual `s/\e\[ [0-9;]*m//g` handles colour and nothing else. This handles the full escape taxonomy: character-set designators, CSI sequences of any final byte rather than just `m`, operating system commands terminated by either the string terminator or a bell, and device control, privacy message and application program command strings. Each family gets its own alternative because each has a different termination rule, and getting the OSC rule wrong is how a stripper ends up consuming the rest of a line when a program sets the terminal title.

The last two forms are the catch-alls: any remaining two-character escape, and the C1 control range `\x80-\x9f`, which is where the eight-bit forms of the same sequences live. And the line after the substitution is not about escapes at all:

```
1 while s/^\b\b//g;
```

That removes overstrike — the character-then-backspace convention that `man` uses for bold and underline. The `1 while` idiom repeats the substitution until it stops matching, which is required because a doubly-overstruck character needs two passes and a single `/g` pass cannot chew through nested pairs. Installer output that includes rendered man pages or `ls` output from a pager comes through with overstrike, and stripping colour without stripping overstrike leaves a log full of `^H`.

The stripper runs once, at the very end of the install, converting the escaped transcript into a second plain-text file:


```
"$ESCAPE_REMOVER" "$LOGFILE" \  
> "$ZPWR_INSTALLER_OUTPUT/zpwr_log.txt"
```

Both files are kept. That is the correct choice for a transcript: the escaped copy replays faithfully with `cat` on a terminal, colours and all, which is what you want when reproducing what the user saw; the stripped copy is what you `grep`, `diff`, and attach to a bug report. Stripping in the `tee` pipeline instead would have produced one file and destroyed the first of those uses.

Backups Before Overwrites

The framework installs by overwriting dotfiles, and the script says so explicitly before doing it, listing every file and directory it will replace and where the old copies are going:

```
BACKUP_DIR="$ZPWR_LOCAL/$USER.rc.bak.$(date +%m.%d.%Y)"
```

The date stamp is day-granularity, which means two installs on the same day share a backup directory and the second overwrites the first's copies with files the first install already replaced. That is a real limitation of the scheme and the tradeoff behind it is discoverability: a directory named for a date is one you can find a year later, whereas a directory named for a timestamp is one of forty indistinguishable siblings. For a script that a user runs once and then possibly once more after a failure, day granularity loses very little.

The copy loop is guarded per entry:

```
test -f "$HOME/$file" && cp "$HOME/$file" "$BACKUP_DIR"  
test -d "$HOME/$dir" && cp -R "$HOME/$dir" "$BACKUP_DIR"
```

so a fresh machine with no existing dotfiles produces an empty backup directory rather than a run of “no such file” errors, and a machine with some of them backs up exactly those.

Slow Work Runs In The Background

Three of the longest steps are launched concurrently and reported by process ID:

```
zpwrPrettyPrintBox "Installing vim and tmux plugins in background \  
@ $PLUGIN_PID"  
zpwrPrettyPrintBox "Installing YouCompleteMe in background @ $YCM_PID"  
zpwrPrettyPrintBox "Installing rustup for eza, fd and bat in \  
background @ $CARGO_PID"
```

Plugin cloning is network-bound, the completion engine build is CPU-bound, and the toolchain install is both. Running them alongside the package-manager work rather than after it is the difference between a coffee break and an afternoon. Printing the PID rather than a spinner is the right instrumentation for a long-running installer: it gives the user something to `kill`, something to look up in `ps`, and something to correlate against the log when a background job's output interleaves with the foreground's.

All three are launched the same way, with output redirected away from the `tee`'d stream and appended to a second log file — the same one for all three:

```
bash plugins_install.sh >> "$LOGFILE_CARGO_YCM" 2>&1 &  
PLUGIN_PID=$!
```

Diverting them off the main stream is right, because three builds interleaving into the transcript the user is watching would make the foreground progress unreadable. Sending all three into one file rather than three is where the scheme stops short: the concurrent appends interleave with each other instead, so reconstructing which of the three emitted a given error is a manual exercise. The PIDs printed to the terminal are the only correlation handle, and they do not appear in the log.

The Package Manifests Are Data, Not Code

The dependency lists are not arrays inside the installer. They are files of literal commands:

```
brew install ack
brew install ansible
brew install ant
```

one per line, one file per ecosystem — formulae, casks, a minimal cask set, gems, npm packages, pip packages, Go tools, taps. Writing the full command on every line rather than storing bare package names and prefixing them at runtime looks redundant, and it buys three things. The file is directly executable, so a user who wants only the Homebrew formulae can run that file alone. A line can deviate — a package needing a flag, a tap, or a different manager entirely — without the format needing to grow a syntax for exceptions. And the file greps cleanly: finding whether a tool is in the manifest is a search for its name, not a search for its name inside a quoted array element.

The separate minimal cask file is the important structural piece. A full GUI application manifest is not what you want on a build server or a VM, and the split is at the file level rather than behind a flag inside one file, so choosing the minimal set is choosing a different file rather than trusting a conditional to have been implemented correctly at every branch.

The Cargo Layer Bootstraps Itself

One manifest is not a file of literal commands, and the reason is that its package manager may not exist yet. `install/rustupinstall.sh` is the script launched as `CARGO_PID` above, and every line of it goes through one helper:

```
function ins() {
  p="$1"
  e="$2"

  zpwrCommandExists "$e" || {
    zpwrPrettyPrintBox "Installing Rustup if cargo does not exist"
    zpwrCommandExists cargo || curl https://sh.rustup.rs -sSf | sh -s -- -y
    zpwrPrettyPrintBox "Updating rustup"
    "$HOME/.cargo/bin/rustup" update
    zpwrPrettyPrintBox "Installing \"$p\" with cargo"
    "$HOME/.cargo/bin/cargo" install "$p"
  }
}
```

Two arguments, and the split between them is the whole design. `$p` is the crate name that `cargo install` wants; `$e` is the name of the executable that crate produces. The guard tests for the *executable*, so a tool already present from Homebrew, from a distribution package, or from a previous run costs one `command -v` and no network. Storing only the crate name would have made the idempotence check wrong for every crate whose binary is named something else, and seven of the twenty-eight invocations are exactly that case: `strykelang` ships `stryke`, `fd-find` ships `fd`, `ripgrep` ships `rg`, `tempr` ships `tp`, `git-delta` ships `delta`, `bottom` ships `btm`, `trippy` ships `trip`.

The rustup bootstrap lives *inside* the helper rather than at the top of the file, which means the toolchain is fetched on first need and never on a machine where every listed binary already exists. The `cargo` and `rustup` calls that follow use absolute paths under `$HOME/.cargo/bin` rather than bare names, and they have to: `common.sh` never puts that directory on `PATH`, and on the run that just bootstrapped rustup the directory did not exist when this process started. Naming the path directly is what makes the install-then-use sequence work within a single script invocation.

The list itself is twenty-eight `ins` lines naming twenty-seven distinct crates — `git-delta` appears twice, once

probed as `git-delta` and once as `delta`, so the second invocation is a no-op after the first has run. Read as a group, most of the list is not “useful Rust tools” but a substitution table: `pythonrs`, `awkrs`, `htoprs`, `lsotrs`, `grcrs`, `nmaprs`, `iftoprs`, `ztmux`, `arb`, `powerliners`, `temprs`, `storageshower`, `zcolorizer` and `stryke` are replacements for programs the environment would otherwise call out to, and they are installed by the same mechanism, in the same file, as `bat` and `ripgrep`. The framework does not distinguish between a third-party tool and a rewritten one at install time. It only asks whether the binary is on `PATH`.

The script opens with a wait rather than a check:

```
while true; do
  if zpwrcommandexists curl; then
    break
  fi
  sleep 5
done
```

Because this runs concurrently with the package-manager phase, `curl` may be minutes away from existing when the process starts. Polling every five seconds until it appears is the crude solution and the right one: the alternative is an ordering dependency between two things the installer deliberately runs in parallel. The cost of being wrong is a five-second delay; the cost of the ordering constraint is the parallelism.

One detail in the helper is a latent quoting bug rather than a decision. The progress line is written “Installing `$p` with cargo”, which closes the quoted string, interpolates `$p` unquoted, and reopens — so the crate name reaches `zpwrcolorize` as however many words it splits into. Every name in the list is a single shell word, so it has never mattered, and it is worth naming only because it is invisible until the day someone adds a crate with a version constraint or a flag beside it.

Chapter 61 – The Prompt Is A Pair Of Arrays

The prompt configuration in `env/.p10k.zsh` is not a theme in the sense of a string with escape codes in it. It is two ordered arrays of segment names plus several hundred variables that configure them, and understanding it means understanding that the prompt engine is a rendering pass over a list, not a format string.

Segments Are Names, Layout Is Newlines

The left prompt is declared as an array whose elements are segment identifiers, with `newline` as an ordinary element rather than a formatting escape:

```
typeset -g POWERLEVEL9K_LEFT_PROMPT_ELEMENTS=(
  custom_env
  custom_env_bat
  custom_env_editor
  newline
  custom_env_term
  ip
  newline
  context
  dir
  history
  ssh
  root_indicator
  status
  vcs
  newline
  custom_tty
  prompt_char
)
```

Treating the line break as a segment is what makes the multi-line prompt composable. Moving a segment to another line is moving an array element past a `newline` entry; there is no string to re-split, no positional escape to recompute, and no separate “line 2 format” variable. The right prompt is declared the same way and contains its own `newline` entries, and the engine pairs left and right lines by index — so a right-prompt segment lands on whichever prompt line its position in the array puts it on.

The consequence for a prompt this dense is that the array is the documentation. Reading down the left array tells you the visual layout directly: three environment segments, a break, terminal identity and network, a break, the main identity-and-status row ending in version control, a break, and then the TTY identifier next to the prompt character. There is no rendering step to simulate.

The right array is dominated by language and toolchain version segments — `pyenv`, `goenv`, `nodenv`, `nvm`, `rbenv`, `rvm`, `jenv`, `plenv`, `phpenv`, `scalaenv`, `luaenv`, and more — followed by cloud context segments and then a block of system metrics. This is a superset configuration: nearly every segment the engine offers is enabled, and the ones that are not appear commented out rather than deleted, which keeps the file usable as a menu of what is available. The engine only renders a version segment when the corresponding tool is actually present and relevant to the current directory, so enabling all of them costs nothing on a directory that

is not a project of that kind.

Custom Segments Are Shell Commands

The `custom_*` segments in the arrays are not built into the engine. Each one is backed by a variable whose value is a command string:

```
POWERLEVEL9K_CUSTOM_ENV='echo "$ZPWR_OPTS $ARCHFLAGS" '
POWERLEVEL9K_CUSTOM_ENV_BACKGROUND="purple"
POWERLEVEL9K_CUSTOM_ENV_FOREGROUND="white"
POWERLEVEL9K_CUSTOM_ENV_BAT='echo "$BAT_THEME" '
POWERLEVEL9K_CUSTOM_ENV_TERM='echo "$TERM" '
POWERLEVEL9K_CUSTOM_ENV_EDITOR='echo "$EDITOR" '
```

The naming convention is the binding: a segment named `custom_env_bat` in the elements array is rendered by evaluating `POWERLEVEL9K_CUSTOM_ENV_BAT` and colouring the result with the matching `_BACKGROUND` and `_FOREGROUND` variables. This is the escape hatch that makes an otherwise fixed segment vocabulary open-ended, and it is used here to surface exactly the state that a shell this configurable makes ambiguous: which editor `$EDITOR` currently names, which syntax theme `bat` is using, what `$TERM` claims to be, and which framework options are in effect.

That is a defensible use of the mechanism. The alternative is discovering that `$EDITOR` was pointing somewhere unexpected after a program has already opened the wrong one.

It is also the mechanism's expensive path, and the cost should be stated plainly. Each of these runs a command in a subshell on every prompt draw. `echo` is a builtin, so the cost is the fork rather than the program, but it is a fork per custom segment per prompt. One of the definitions in the file is considerably more than that:

```
function milliamps(){
  local amps
  amps="$(ioreg -rc AppleSmartBattery | grep CurrentCapacity \
    | awk '{printf "%s mAh\n", $3}')"
  echo "$amps \uf168"
}

POWERLEVEL9K_CUSTOM_BATT="milliamps"
```

Three processes and a walk of the macOS I/O registry to read a battery figure, which would be paid on every single prompt if that segment were rendered. It is not: `custom_batt` does not appear in either elements array. The definition and its two colour variables are configured and unreferenced — dead configuration, kept presumably as a working example of the pattern.

This is a distinction worth being precise about, because “the prompt runs `ioreg` every keystroke” would be a reasonable inference from reading the function and a wrong one. The engine renders the elements array; a `CUSTOM_*` variable with no corresponding array entry costs one variable assignment at configuration load and nothing thereafter.

The `custom_pid` segment, by contrast, *is* in the right array, and its definition shows the other half of what the mechanism permits:

```
POWERLEVEL9K_CUSTOM_PID='echo "\uf258 $$ \uf258 $(date +%D) \uf168" '
```

An embedded command substitution inside the segment string, so this one forks twice per prompt — once for the segment's own subshell and once for `date`. It also embeds Nerd Font code points directly as `\u` escapes rather than as literal characters, which is the right call for a configuration file that will be edited in terminals and editors with varying font coverage: an escape sequence survives copy-paste through a system that cannot render the glyph, and a literal private- use-area character does not.

Where The Engine Draws Its Own Lines

Two settings control the connective tissue between prompt lines:

```
POWERLEVEL9K_MULTILINE_FIRST_PROMPT_GAP_CHAR='─'  
POWERLEVEL9K_MULTILINE_NEWLINE_PROMPT_GAP_CHAR='─'
```

The gap character fills the horizontal space between the end of a left prompt line and the start of the corresponding right prompt content. With a prompt this tall the fill is not decoration — it is what makes it possible to visually track which right-hand segment belongs to which left-hand line across a wide terminal. Both are set to the same box-drawing character and the same colour, so the connection between the first line and its right content reads identically to the connection on every subsequent line.

`POWERLEVEL9K_MODE=nerdfont-complete` declares the icon set, which is what makes the `\u` escapes above resolvable. This is a hard dependency the configuration does not attempt to degrade: a terminal without a patched font renders the segments with replacement glyphs. Attempting to detect font coverage from inside a shell is not possible, so the choice is between assuming the font and shipping a second icon-free configuration, and the file assumes the font.

Finally:

```
typeset -g POWERLEVEL9K_INSTANT_PROMPT=quiet
```

The instant-prompt feature draws a cached prompt before the shell has finished initializing, and it has three settings that differ in what happens when initialization code writes to the terminal during the window where the cached prompt is displayed — output that would corrupt the display. The default warns about it on every startup. `quiet` keeps the feature and suppresses the warning, which is the setting for a configuration where console output during initialization is known and accepted rather than a bug to be hunted. The file's last two lines restore the caller's options and unset the temporary array they were saved in, so the configuration leaves no trace in the shell's option state.

The Cost Model Nobody States

A prompt of this size draws on every command. The engine's design decision that makes it viable is that segments are conditional at render time — a version segment produces nothing outside a project of that language, and a segment that produces nothing occupies no width and costs one function call. Enabling forty segments therefore does not cost forty segments' worth of work; it costs forty predicate evaluations, most of which are a variable test or a file existence check, plus rendering for the handful that match.

The custom segments break that model, because a command string is unconditional by construction. There is no predicate; the string is evaluated and whatever it prints is the segment. That is why the distinction between which `CUSTOM_*` definitions are wired into the arrays and which are not is the single most performance-relevant fact in the file, and why it is worth verifying against the arrays rather than assuming from the definitions.

Chapter 62 – Colorizing Programs You Did Not Write

There is a class of tool whose output you read constantly and cannot modify: `df`, `ps`, `netstat`, `mount`, `whois`, `traceroute`. They predate colour terminals, they emit columnar text, and reading them is a matter of finding one field among many. The framework's answer is not to replace them. It is to wrap them in a regular-expression colourizer and ship the rule files.

The Wrapping Loop

`env/grc.zsh` is short enough to read in full, and its shape is the whole mechanism:

```
if [[ "$TERM" != dumb ]] && (( $+commands[grc] )); then
(){
  local cmd cmds
  cmds=(
    cc configure cvs df diff dig gcc gmake ifconfig
    last ldap ls make mount mtr netstat ping ping6 ps
    traceroute traceroute6 wdiff whois
  );
  for cmd in $cmds ; do
    if (( $+commands[$cmd] )); then
      alias $cmd="grc --colour=on $cmd"
    fi
  done
}
fi
```

Two gates before anything happens. `$TERM` not being `dumb` is the standard capability test, and it matters here because a `dumb` terminal is what you get inside an editor's shell buffer, in a CI job, and over some serial connections — exactly the contexts where escape sequences end up as literal garbage in a transcript. The second gate tests for the colourizer itself in the shell's command hash.

Then a per-command existence check inside the loop. Aliasing `mtr` on a machine without `mtr` installed would be harmless in isolation, but it would produce a completion entry and a `type` result for a command that does not exist, which is the kind of small lie that costs an afternoon later. The alias is created only when both the wrapper and the wrapped command are present.

The body is an anonymous function, which is the zsh idiom for a scope. `cmd` and `cmds` are declared `local` inside it and cease to exist when it returns, so sourcing this file adds two aliases-worth of state to the shell and no variables. Doing the same thing with a plain loop at file scope would leave both names in the global namespace of every interactive shell forever.

Aliasing is the correct dispatch mechanism here rather than a function, and the reason is what happens to the wrapped command's own arguments. `alias df="grc --colour=on df"` expands textually at the head of the command line, so `df -h /` becomes `grc --colour=on df -h /` and every flag lands where it belongs with no quoting logic. A wrapper function would need `"$@"` handling that is equivalent but not simpler, and would additionally shadow the command name in contexts where aliases do not expand — inside scripts, inside `find -exec` — which is a behaviour difference, not just an implementation one.

Rule Files Are Ordered Regular Expressions

The colourizer's configuration is a sequence of pairs, and `install/conf.df` is a readable example:

```
regexp=^(Filesystem.*)$
colours=blue underline

regexp=\s\d*[.,]?dKi?\s
colours=green

regexp=\s\d*[.,]?dMi?\s
colours=yellow

regexp=\s\d*[.,]?dGi?\s
colours=red

regexp=\s\d*[.,]?dTl?\s
colours=bold red
```

The header line gets one treatment; sizes get a colour ramp by magnitude. That ramp is the entire point of the file. `df` output is a wall of numbers in which the thing you are looking for is “which of these is nearly full or unusually large,” and a magnitude ramp answers that question pre-attentively. Reading colour instead of parsing digits is a real difference in how fast the output is consumed, and it required no change to `df`.

Note the `[.,]?` in every size pattern. That is locale tolerance — a decimal comma rather than a decimal point — in a rule file for a tool whose numeric formatting follows the ambient locale. It is a one-character concession that keeps the ramp working on a machine configured for a European locale, and its absence would be an invisible failure: the colours would simply stop appearing for fractional sizes.

The listing rule file is considerably more ambitious and demonstrates what lookahead buys you in this format. Permission bits are coloured by field:

```
regexp=(?<=[-bcdDlMnpPs?])(-|(r))(-|(w))(-|([xsStT]))
(?=[-r][-w][-xsStT][-r][-w][-xsStT])
colours=unchanged,unchanged,green,unchanged,green,
unchanged,green
```

Both of those are single lines in the file; they are shown wrapped here because neither fits a printed column. A lookbehind asserting the file-type character, three alternations capturing the read, write and execute bits, and a lookahead asserting that six more permission characters follow. The lookahead is what confines the match to the owner triad specifically; the group triad and other triad are matched by the same shape with the assertions shifted. The colour list has one entry per capture group, and `unchanged` is the keyword for “match this group but leave its colour alone,” which is how a rule can capture structure it needs for anchoring without recolouring it.

That is the technique that makes the whole approach work on columnar output. Without lookahead, colouring the third character of a permission string requires a match that consumes the first two, and then those two are inside the match and cannot be left alone. Zero-width assertions let a rule locate a field by its context and then colour only the field.

Size rules in the listing file use the same trick from the other direction:

```
regexp=\s+(\d{9}|\d{3}M)(?=\s[A-Z][a-z]{2}\s)
colours=red
```

The lookahead requires a three-letter capitalized month abbreviation after the number. That is how the rule distinguishes a file size from an inode number, a link count, or a numeric user ID in the same line — not by counting columns, which would break the moment a field widened, but by asserting the presence of the date field that always follows the size. Column-position parsing of `ls` output is famously fragile; anchoring on an adjacent field's shape is not.

What The Approach Cannot Do

Being honest about the limits is worth a paragraph, because the technique looks more general than it is.

The colourizer sees a byte stream and has no model of the program that produced it. A rule cannot know that a number is a size rather than a count; it can only know that it looks like one in the context the pattern describes. Every rule in these files is a heuristic about text shape, and every one of them has inputs that defeat it — a filename containing something that matches the date pattern, a mount point whose name looks like a size. The failure mode is benign, which is what makes the tradeoff acceptable: a wrong colour is a cosmetic error, not a wrong answer, and the underlying text is unchanged.

It also costs a process and a full regular-expression pass over the output for every invocation of every wrapped command. For `df` that is nothing. For a listing of a directory with a hundred thousand entries it is not nothing, and it is one of the reasons the framework prefers a modern replacement listing tool when one is available and falls back to the colourized form only when it is not — a tool that emits its own colour has already done this work with knowledge of what each field means.

The final observation is about maintenance. These rule files travel with the framework rather than being taken from the colourizer's own distribution, which means they are the framework's to keep working. That is a real ongoing cost — a utility changing its output format silently breaks rules — and it is the price of having them tuned for the specific things this stack reads constantly rather than for a general audience.

Chapter 63 – Fan-Out And The Polling Daemon

Two scripts in the tree do the same structural thing at different time scales: apply an operation across a set of repositories, and keep applying it. Reading them together shows the framework's approach to repetition across directories, and one of them contains a defect worth walking through in detail because it is the kind that survives for years without symptoms.

The Directory Iterator

`scripts/forDir.sh` is the primitive, and its interface is a command string followed by directories:

```
function zpwrForldir(){
    if [[ -z "$2" ]]; then
        zpwrLogConsoleErr "usage: fordir CMD DIRS to run CMD in each dir"
        return 1
    fi

    local dir cmd
    cmd="$1"

    if [[ -d "$cmd" || -f "$cmd" ]]; then
        zpwrLogConsoleErr "usage: fordir CMD DIRS to run CMD in each dir"
        return 1
    fi

    shift

    for dir in "$@"; do
        if [[ -d "$dir" ]]; then
            (
                builtin cd "$dir" && \
                zpwrPrettyPrint "cd $dir && $cmd" && eval "$cmd"
            )
        fi
    done
}
```

The second guard is the interesting one. Having already required two arguments, it then checks whether the first argument names an existing file or directory and refuses if it does. That is an argument-order check disguised as a type check: the overwhelmingly likely mistake with this interface is writing the directories first and the command last, matching the shape of `cp`, `mv`, and most of the tools it sits beside. Detecting that the command slot holds a path catches the transposition before the command runs somewhere unintended.

It is a heuristic and it has a false positive — a command whose name collides with a file in the current directory — but the asymmetry of consequences is severe. The false positive costs a confusing usage message. The false negative runs a directory name as a command in a loop over a list of commands, which `eval` will happily attempt.

Each iteration runs in a subshell, so the `cd` never escapes and a command that changes directory or sets

variables cannot affect the next iteration. The `zpwriPrettyPrint` echoing the command before running it is not decoration either: the output of a fan-out over twenty repositories is unreadable without a header line telling you which repository each block of output came from.

`eval` is what makes the command string arbitrary. This is the correct place for it — the argument is a command supplied by the caller, so there is no injection boundary being crossed — and it is what lets the caller pass a compound command with pipes, redirections and `&&` chains as a single argument, which the wrapper above exploits fully.

Composing On Top Of It

`scripts/forDirMain.sh` is a thin caller and shows what the primitive is for:

```
branch='${zsh-gacp-mainBranch}'

source "$ZPWR_SCRIPTS/forDir.sh" \
  "zpwriIsGitDir && { _m=$branch; git checkout \$_m ; \
    git fetch -f --all --prune --tags; \
    git clean -dff && git reset --hard origin/\$_m && \
    git clean -dff ; git pull --all; unset _m; }" "$dir"/*
```

The quoting here rewards a careful read. `branch` is assigned a command substitution *in single quotes*, so the variable holds the literal text `$(zsh-gacp-mainBranch)` rather than the result of running it. That text is interpolated into the command string, which is then evaluated inside each repository's subshell — so the branch-name detection runs once per repository, against that repository's refs, rather than once in the caller against whatever directory it happened to start in. Deferred evaluation by deliberate under-quoting is a shell idiom that reads like a mistake and is exactly right here.

Inside the command string, `\$_m` is escaped and `$branch` is not, which is the same distinction: `$branch` expands now, when the string is being built, and `$_m` expands later, when the string is evaluated. The temporary is set and unset within the compound command so that the fan-out leaves nothing behind in the subshell it was evaluated in.

The operation itself is a hard resynchronization: check out the main branch, fetch everything including tags with pruning, clean untracked files including directories and ignored files, hard reset to the remote, clean again, and pull. The double `git clean -dff` bracketing the reset is not redundant. The reset changes which files are tracked, so a file that was ignored before the reset may be untracked after it, and only the second clean can see it.

`zsh-gacp-mainBranch` handles the naming split by probing rather than guessing:

```
for branch in main master; do
  if command git show-ref -q --verify refs/heads/$branch; then
    echo "$branch"
    return
  fi
done
echo main
```

`show-ref --verify` with a full ref path is an exact existence test rather than a pattern match, so a branch named `mainline` cannot satisfy the probe for `main`. The trailing fallback returns a name rather than failing, which pushes the error to the git command that will use it — where the message will be about a specific missing branch rather than about an empty variable.

The Daemon

`scripts/remoteRepoMonitorDaemon.sh` runs the same class of operation on a timer. Its loop polls two repositories, and when either has moved, it resynchronizes and then restarts the shell that is using them:

```
while true; do
    builtin cd "$configDir" || { ... }

    git fetch -f --all --prune --tags
    output=$(git log HEAD..$remoteName/dev --oneline)

    if [[ -n "$output" ]]; then
        echo "$(date) We have change to $(git remote -v)"
        main
    fi
    ...
    sleep 10
done
```

The change detection is `git log HEAD..remote/branch` — the list of commits reachable from the remote ref but not from the local head. Non-empty output means the remote has moved ahead. This is a better test than comparing hashes because it is directional: a local head that is *ahead* of the remote produces no output and correctly triggers nothing.

The interesting half is what happens after:

```
$ZPWR_ZTMUX send-keys -t right C-c C-c C-c \
" $ZPWR_ZTMUX source-file \"$ZPWR_TMUX/init.conf\" \
2>> \"$ZPWR_LOGFILE\" 1>&2; exec zsh\" C-m
```

Three interrupts to clear whatever the target pane was doing, then a command line sent as keystrokes into that pane, then Enter. The command reloads the tmux configuration and replaces the pane's shell with a fresh one via `exec`. This is how a configuration change propagates to a running session without the user doing anything: the daemon notices the push, resets the working copies, and restarts the consuming shell in place.

The leading space before the sent command is commented in the source as being there because the interrupt tends to cut off the first character. That is a real terminal behaviour and the fix is a single character; it is also the kind of detail that only gets written down after the failure has been observed several times and dismissed as flakiness.

The Bug That Does Not Bite

Both branch-detection functions in the daemon accept a remote name as their first argument and default it to `origin`:

```
function zsh-gacp-mainBranch() {
    local branch remote

    if [[ -z "$1" ]]; then
        remote=origin
    else
        remote="$1"
    fi
    ...
}
```

and every call site passes `"$remote"`:

```
git reset --hard $remoteName/${zsh-gacp-mainBranch "$remote"}
```

`remote` is declared `local` inside both functions and is never assigned anywhere at file scope. The call sites are in `resetToMain` and `resetToDev`, neither of which declares or sets it. So the expansion at every call site

is empty, the `-z "$1"` test inside the function always succeeds, and the parameter is always `origin`.

The parameterization is entirely inert. It has never selected a non-default remote and cannot, because the variable that would carry the choice does not exist outside the scope of the functions that consume it. The reason this has no observable effect is that the only remote in play is `origin`, hardcoded as `remoteName` in the same file and used directly on every git command.

Two things make this worth writing down. First, the shape — a `local` declaration in a callee and a bare expansion of the same name in the caller — is specific to dynamically scoped shells. In a lexically scoped language the caller's `$remote` would be an unbound-variable error at compile or first run. In bash it is an empty string, and an empty string is exactly what the function's default-argument logic is designed to handle gracefully. The defence swallows the defect.

Second, the failure is latent rather than harmless. The day a second remote is added and someone tries to use the parameter, they will find that passing a remote name at a call site does nothing, because the call site passes a variable rather than the value. A test would not catch it — the function is correct in isolation and the call sites produce correct results. Only reading the two together shows it, which is the general argument for auditing a shell script's variable scoping as a whole file rather than function by function.

Chapter 64 – Drawing Boxes That Do Not Break

Every progress message the installer prints comes out inside a drawn box, and the box is not produced by the shell. `scripts/boxPrint.pl` is a small Perl program that reads a string from arguments or standard input and emits it centred inside a frame. It is worth a chapter because box drawing is one of those problems that looks trivial, is not, and whose failures are immediately visible to the user on every single run.

Deciding How Wide The Terminal Is

The first thing the program does is answer a question that has no reliable answer:

```
if ($full) {
    $width = 80;
}
else {
    $width = `tput cols`;
    if ( $width == 80 ) {
        if ( defined $ENV{'COLUMNS'} ) {
            $width = $ENV{'COLUMNS'};
        }
    }

    if ( $width % 2 == 1 ) {
        $width -= 1;
    }
    else {
        $width -= 2;
    }
}
```

The interesting branch is the middle one. `tput cols` returns 80 when it cannot determine the real width — when there is no terminal, when `TERM` is unhelpful, when standard output is a pipe. So the program treats a result of exactly 80 as “probably a fallback rather than a measurement” and prefers the `COLUMNS` environment variable if one is set, on the theory that a shell that exported `COLUMNS` measured something.

That heuristic is wrong for the case of an actual eighty-column terminal with a stale `COLUMNS` in the environment, and it is right for the far more common case of a pipeline where `tput` had nothing to measure. Trading correctness in a rare case for correctness in a frequent one is the right trade for cosmetic output; it would not be for anything that computed a value the user relied on.

The parity adjustment afterwards is subtler than it looks. An odd width loses one column and an even width loses two, which normalizes every width to an even number strictly less than the terminal. Even matters because the frame is symmetric — the side character runs on the left and right are computed by halving a remainder, and an odd total guarantees that one side is a character longer than the other. Subtracting to reach an even number is cheaper than carrying a half-column through the rest of the arithmetic. Staying strictly below the terminal width matters for a different reason: a line exactly as wide as the terminal wraps on some terminals and not others, depending on whether the cursor advances past the last column before the newline arrives. One spare column removes the ambiguity.

The Usage Text And The Code Disagree

The program documents its own flag like this:

```
say "      [-f|--full] center box across width of terminal";
```

and implements it as `$width = 80` — a fixed width that ignores the terminal entirely. The flag does the opposite of what its description says: the *default* path is the one that measures the terminal, and `--full` is the one that pins to a constant.

The code is what runs, so the behaviour is fixed width. And this matters in practice, because `zpwPrettyPrintBox` — the function every installer message goes through — passes `-f` unconditionally:

```
printf "%s>>> %s\n" "$INSTALL_COUNTER" "$@" | "$perlfile" -f
```

So the entire install transcript is drawn at eighty columns regardless of the terminal it runs in. That is arguably the correct behaviour for this use: the transcript is tee'd to a log file that will be read later in an unknown terminal, and boxes drawn to a 200-column window are unreadable when the log is opened at 80. But it is correct by accident of the flag being misdescribed rather than by the description being followed. The installer's own `export COLUMNS="$(tput cols)"` near the top of the script exists to feed the measurement path, which the `-f` flag then bypasses.

The Step Counter Lives In The Printer

One line in that same function is doing structural work that has nothing to do with drawing:

```
(( ++INSTALL_COUNTER ))  
printf "%s>>> %s\n" "$INSTALL_COUNTER" "$@" | "$perlfile" -f
```

The installer's step numbering is a side effect of printing a box. There is no list of steps, no total, and no registry — the number is simply how many boxes have been drawn so far, incremented in the printing function itself.

This is the pragmatic design for a script whose step count depends on which platform it is running on and which tools it finds. A declared total would have to be computed in advance from branches that have not been evaluated yet, and a declared total that is wrong is worse than no total, because a progress indicator that reads "step 40 of 32" destroys confidence in everything else the script says. Numbering without a denominator makes no promise it cannot keep. What the user gets is a monotonic marker to correlate terminal output against the log, and that is the thing they actually need when a step fails.

Fitting Text That Does Not Fit

The layout arithmetic has two branches. The simple one centres a string that fits:

```
my $sideCharacterLength =  
  ( ( $width - $inputStringLength - $spacerLength * 2 ) + 1 ) / 2;
```

with an explicit correction afterwards for the case where the leftover space cannot be split evenly:

```
my $side2len = $sideCharacterLength;  
  
if ( $inputStringLength % 2 ) {  
  --$side2len;  
}
```

The right side is shortened by one when the string length is odd. This is the detail that separates box drawing that works from box drawing that looks fine until someone passes a string of the wrong parity — the frame's right edge drifts by one column relative to the top and bottom rules, and it drifts only for half of all inputs, which

is exactly the distribution that lets a bug survive testing.

The wrapping branch handles a string too long for one line:

```
my $lineCounter      = int( $inputStringLength / $maxTextLength ) + 1;
my $charactersPerLine = int( $inputStringLength / $lineCounter );
```

then emits `$lineCounter` rows, each taking a `substr` of `$charactersPerLine` characters at offset `$charactersPerLine * $i`. Dividing the string into equal chunks rather than filling each line to capacity is a deliberate aesthetic choice: a message split into three lines of sixty-six characters looks intentional, and one split into two full lines plus a five-character orphan looks broken.

The arithmetic has a consequence the code does not account for. Both divisions truncate, so the number of characters actually emitted is `$lineCounter * $charactersPerLine`, which is at most the input length and is generally less. A two-hundred-character message at a maximum line length of seventy-two produces three lines of sixty-six characters — one hundred ninety-eight — and the last two characters of the message are never printed. The remainder is always smaller than the line count, so the loss is at most a couple of characters, which is why this has survived: it clips the tail of long messages by an amount too small to notice unless you are comparing against the source string.

Splitting is also done on character offsets with no regard for word boundaries, so a wrapped message breaks mid-word. For a progress banner that is acceptable. It is the reason this program is used for banners and not for anything a user needs to read carefully.

What The Program Cannot See

The width calculation is `length $str`, which counts characters in the string. Any ANSI escape sequence in the input is counted as the characters that compose it, so a message containing colour codes computes a length several characters longer than what the terminal displays, and the frame closes early — the right edge lands inside the text rather than after it.

This is the single most common way box-drawing code breaks, and the program's defence against it is structural rather than algorithmic: it is invoked at the head of a pipeline with plain text, and the colour is added by the program itself from its own constants:

```
my $boxColor    = "\x1b[0;32;1m";
my $textColor   = "\x1b[0;34;4m";
my $resetColor  = "\x1b[0m";
```

Since the program emits every escape sequence it prints, it knows that none of them are inside the string it measured. Owning the colouring rather than accepting coloured input is what makes the naive length calculation safe. The constraint is real and unenforced — passing a pre-coloured string to this program produces a broken box with no diagnostic — but it is a constraint the call sites satisfy by construction, because they all pass a `printf` of literal text.

The standard-input path adds one more normalization:

```
while (<STDIN>) {
    chomp;
    s@\s+@ @g;
    $str .= $_;
}
```

Every run of whitespace collapses to a single space and every line is concatenated without a separator. Multi-line input therefore becomes one long string, which the wrapping branch then re-splits at its own chosen width. That is the right behaviour for this use — the caller's line breaks are almost certainly an artifact of how the message was written in the source, not a layout decision — and it means the box's internal layout is a function of the terminal width alone.

An Exit Where A Return Belongs

One detail in the wrapper is a genuine hazard rather than a tradeoff:

```
[[ ! -e "$perlfile" ]] 66 echo "where is perlfile '$perlfile'?" >&2 66 exit 1
```

`exit` inside a function terminates the shell, not the function. In the installer that is the intended outcome — a missing box printer means a broken checkout, and stopping is correct. But `zpwPrettyPrintBox` lives in the shared library, which is also sourced into interactive shells and into scripts that have their own error handling. Calling it there with a missing dependency closes the shell.

The split between the two forms in that file is not random, and tracing it shows the design intent. The functions that `exit` are the installer-lifecycle helpers — the assertion helper, the file-must-exist check, the two directory-changing helpers, the plugin installer, the confirmation prompt, and the two box printers. The functions that `return` are the general-purpose ones the shell uses every day: the existence probes and the exhaustive resolver. The rule the author was following is legible: a helper whose only sensible response to a broken precondition is to stop the install exits; a helper a script might reasonably continue past returns.

The box printers are the pair where that rule breaks down, because they belong to both worlds. They are installer plumbing by origin and interactive plumbing by destination, and they were categorized by origin. This is the general hazard with a single shared library sourced into several contexts: the correct error behaviour is a property of the caller, and a function that hardcodes `exit` has made a decision on the caller's behalf that the caller cannot undo.

Chapter 65 – Two Interpreters In One Script Directory

The script directory contains a set of file pairs that look, at first glance, like leftovers: `sdiffColorizer.pl` next to `sdiffColorizer.stk`, `escapeRemover.pl` next to `escapeRemover.stk`, and so on. They are not leftovers. They are a migration captured mid-flight, with both endpoints still on disk, and reading them side by side is the clearest available account of what the `.stk` language is for.

The Inventory

Count the two extensions in the script directory and you get eight `.stk` files and seven `.pl` files. Six names appear under both extensions:

```
escapeRemover      .pl  .stk
gitSdiffColorizer  .pl  .stk
minifySpaces       .pl  .stk
regexReplace       .pl  .stk
sdiffColorizer     .pl  .stk
stdinSdiffColorizer .pl  .stk
```

One `.pl` has no `.stk` counterpart — `boxPrint.pl`, the box-drawing helper that `scripts/lib.sh` invokes by absolute path. Two `.stk` files have no `.pl` counterpart: `banner.stk` and `c.stk`. That asymmetry is itself informative, and a later chapter returns to it, because the committed tags index still lists `banner.pl` and `c.pl` as though they existed.

What Actually Moved

A migration that leaves both files on disk raises the obvious question: which one runs? The answer is not uniform, and it is worth resolving by call site rather than by assumption.

Grep the tree for each script name outside the script files themselves and the picture is specific. The `.stk` versions are wired into live code paths:

```
autoload/common/gsdcc      gitSdiffColorizer.stk
autoload/common/zpwrRegenAllKeybindingsCache escapeRemover.stk
autoload/common/zpwrBindAliasesLate      c.stk
autoload/common/zpwrBindAliasesZshLate    stdinSdiffColorizer.stk
scripts/printHeader.sh      banner.stk
```

The `.pl` versions have almost no live callers left. `escapeRemover.pl` survives in two places: `install/zpwrInstall.sh` assigns it to an `ESCAPE_REMOVER` variable, and `scripts/keybindingsToREADME.zsh` pipes through it. `boxPrint.pl` is referenced from `scripts/lib.sh`. The rest — `minifySpaces.pl`, `regexReplace.pl`, `sdiffColorizer.pl`, `gitSdiffColorizer.pl`, `stdinSdiffColorizer.pl` — appear nowhere in the tree except in the committed tags index and in the test files that assert their existence.

That is a specific kind of migration state, and it is a defensible one. The `.stk` file is what runs. The `.pl` file is kept as the executable specification of what the `.stk` file is supposed to do, and as the fallback if the

newer interpreter is not installed on a given machine. Neither file is dead in the sense that matters: one is the implementation, the other is the reference.

The most interesting single line in this whole layer is in `keybindingsToREADME.zsh`, which pipes the output of `escapeRemover.pl` into a `stryke -ne` one-liner in the same command. A perl script feeding a stryke one-liner in a single pipeline. Whatever else the migration is, it is not ideological; the pipeline uses whichever side of the fence already had the code.

Reading The Language Delta

Put `stdinSdiffColorizer.pl` and `stdinSdiffColorizer.stk` next to each other and the design intent of the newer language is legible without any documentation.

The perl version opens with argument handling written the long way:

```
my $num_args = $#ARGV + 1;
my $numColumns = 0;
if ( $num_args == 1 ) {
    $numColumns = $ARGV[0];
}
else {
    $numColumns = `tput cols`;
    chomp $numColumns;
}
```

The stryke version is one line:

```
var $numColumns = @ARGV ? $ARGV[0] : qx "tput cols"
```

Three changes are packed in there. `var` replaces `my` and is paired with a `val` for bindings that never change — the ported scripts use `val` for the colour constants and `var` only for `$numColumns`, which is reassigned by `chomp`. `@ARGV` in boolean position is truthy when non-empty, removing the `$#ARGV + 1` arithmetic. And backticks become `qx` with a quoted string argument, which is the same operator with the ambiguous punctuation removed.

The colour constants show the same compression. Perl:

```
my $changeColor = "41";
my $changeFormatting = "1;4";
my $insertColor = "43";
my $insertFormatting = "1;4";
my $deleteColor = "42";
my $deleteFormatting = "1;4";
```

stryke:

```
val ($changeColor, $insertColor, $deleteColor) = ("41", "43", "42")
val $fmt = "1;4"
```

The three formatting variables held the same value in the perl original and were never independently varied, so the port collapses them into one. This is the normal outcome of a rewrite that a human performs while reading rather than a mechanical translation; a transliteration would have produced six bindings.

Pipelines And The Underscore

The `|>` operator appears in every ported file that opens a handle:

```
open(val $less, "|-", "less -M") or die $!
$less |> select
...
$less |> close
```

Perl writes `select $less` and `close $less`. The threading form puts the object first and the verb second, which matters when the chain is longer than one call. `banner.stk` opens with the shortest example in the set:

```
STDOUT |> binmode ":encoding(UTF-8)"
```

and `c.stk` uses it in a filter position:

```
val $exe_bat_exists = @p |> grep { -x "$_/$exe_bat" }
```

`@p` there is worth stopping on. In perl that line would read `grep { -x "$_/$exe_bat" } @path` after splitting the `PATH` environment variable on the path separator. The stryke version treats the search path as a first-class array variable, so the split never appears in the script at all.

The topic variable is the other consistent change. Perl's `$_` becomes a bare `_` in code position:

```
while (<STDIN>) {
  _ =~ s/.../.../xg
  1 while _ =~ s/^[^b][^b]//g
  p
}
```

against the perl original's `s/.../.../xg` with the implicit topic and an explicit `print`. Two things are happening. The topic is spelled without a sigil, so it reads as an identifier rather than a punctuation variable; and the implicit-target forms are gone — `_ =~ s///` is written out where perl allowed a bare `s///`. The gain is that `p` with no argument, `p $x`, and `_ =~ s///` are all unambiguous at a glance. The `1 while` idiom for repeated substitution survives intact, because there is no better way to write it.

Ranges, Slices And Lengths

Three smaller substitutions recur across the ported files.

`length` becomes `len`, and it takes the topic explicitly:

```
val $len = len(_)
```

Array slices use a colon range rather than a double-dot:

```
val ($file1, $file2) = @ARGV[0:1]
```

and the same form appears as a loop range in `regexReplace.stk`:

```
p "*" x 80 . "\n" for (0:2)
```

`len` also applies to arrays without a sigil change:

```
val $length = len @files
```

Perl would need `scalar @files` there, or would have to rely on assignment context. The range form `0:2` is the one place where a reader coming from perl has to stop and check whether the endpoint is inclusive; in the source it produces the separator bar three times, so it is.

Named Parameters

The clearest single improvement is in `minifySpaces`. The perl original is a recursive subroutine that indexes the argument array directly:

```
sub removeSpaces {
    if ( -d $_[0] ) {
        removeSpaces $_ for glob("$_[0]/* $_[0]/.[!.]*");
    }
    else {
        say "working on $_[0]";
        my $tmp = $_[0] . ".bak";
        rename $_[0], $tmp;
    }
}
```

The first positional argument is spelled out five times in the visible body. The stryke port names it:

```
fn removeSpaces($path) {
    if (-d $path) {
        removeSpaces _ for glob("$path/* $path/.[!.]*")
    } else {
        p "working on $path\n"
        val $tmp = "$path.bak"
        rename($path, $tmp)
    }
}
```

`fn` with a parameter list, `$path` throughout, and — the subtle part — `removeSpaces _` at the recursive call, where perl wrote `removeSpaces $_[0]`. The recursion passes the loop topic explicitly in both languages; only the spelling of the topic changed.

The perl original has one further wart that the port fixes silently. Its inner loop writes with an explicit handle after opening `$in` and `$out` without a `my`, so both handles are package globals inside a recursive function. The stryke version binds them with `val` and uses the select-and-restore pattern:

```
val $old = select $out
while (<$in) {
    ...
    p
}
select $old
```

which makes `p` write to the right handle and restores the previous default when the frame ends. In a recursive function that manipulates the default output handle, saving and restoring is not optional; the perl version got away with it only because it named the handle on every print.

The Options Block

`regexReplace.stk` carries the one construct in this set with no perl counterpart at all:

```
val %opts = %{ getopt({
    "regex|r=s"      => { help => "...", default => '\r' },
    "replacement=s" => { help => "...", default => "" },
}), {
    prog => "regexReplace.stk",
    desc => "Recursively replace a regex in text files.",
}} }
```

The perl script it replaces has no option parsing; the regex to remove is hard-coded to a carriage return, which is what its header comment says it is for. The port generalized the script and needed somewhere to put flags, help text and defaults. The shape chosen is a single declarative hash: the key carries the flag name, its alias and its type in one string (`regex|r=s` — long name, short alias, string argument), and the value carries the

help line and the default. A second hash supplies the program name and description for the generated usage output.

Two properties follow from that shape. Help text cannot drift from the flag table, because there is only one table. And a flag cannot be declared without a help string being at least syntactically expected, which is the failure mode every hand-rolled option loop eventually hits — in this same script directory, `mywatch.sh` and `myWatchMaintainEscapes.sh` each maintain an `optstring` variable and a separate `usage` heredoc, and the two have already diverged from one another.

The Confirmation Gate

`regexReplace.stk` is also the only script in the set that pauses. It walks the argument paths, collects every text file whose contents match the regex, prints the list, prints the count, and then asks:

```
p "Remove $regex?\x1b[1;34m>\x1b[0m "
var $answer = <STDIN>
chomp $answer
if ($answer =~ /y|yes/i) {
    qx "perl -i -pe 's#$regex#$replacement#g' \"$_\\"" for @files
}
```

Note what the confirmed branch actually runs: a shelled-out `perl -i -pe`, one process per file. The script that was ported off perl still calls perl to do the in-place edit. That is not an oversight — in-place editing with a backup-free rename is exactly the operation perl's `-i` was built for, and reimplementing it badly is a good way to lose files. It does mean the migration boundary for this particular script is drawn around the scanning and the interaction, not the mutation.

The answer test is `/y|yes/i` — an unanchored alternation, so any response containing a `y` anywhere accepts. `nyet` accepts. For a destructive recursive rewrite that is the wrong default, and the anchored form is the same number of characters.

What The Port Preserved Exactly

It would be a mistake to read the `.stk` files as rewrites. Compare the escape stripping regex in the two `escapeRemover` files and they are character for character the same alternation, including the free-spacing flag, the same branches in the same order, and the same repeated-substitution backspace collapse. The comments that labelled the branches in the perl file — the ones identifying the CSI, OSC and DCS forms — are gone, which is the only loss.

That regex is the highest-risk piece of code in either file. It consumes terminal control sequences from arbitrary program output, and a subtle change to one alternation branch would corrupt captured output in a way that would not surface until someone read a mangled generated README months later. The port left it untouched and changed only the scaffolding around it. That is the correct instinct, and it is visible in the diff.

Chapter 66 – The Preview Command Is A Generated Program

fzf's `--preview` takes a single shell command string. Everything the preview pane can ever do has to fit inside that one string, and the string is fixed at the moment the environment variable is exported. This tree does not write that string by hand. It runs a bash script whose entire job is to print a shell program, captures the output in a command substitution, and hands the result to fzf as the preview command.

The consequence is a code generator whose output nobody ever reads, executed by an interpreter that reports its errors into a preview pane the width of half a terminal. This chapter is about what that costs.

The Export Site

The bindings live in the autoloaded function `zpwBindFZF_Late`:

```
export FZF_CTRL_T_OPTS="$ZPWR_COMMON_FZF_ELEM \  
  --preview '${bash "$ZPWR_SCRIPTS/fzfPreviewOptsCtrlT.sh"}'"  
export FZF_CTRL_T_OPTS_2="$ZPWR_COMMON_FZF_ELEM \  
  --preview '${bash "$ZPWR_SCRIPTS/fzfPreviewOpts2Pos.sh"}'"  
export FZF_ALT_C_OPTS="$FZF_CTRL_T_OPTS"
```

and, further down the same function:

```
if [[ "$ZPWR_BANNER_TYPE" == ponies ]]; then  
  export FZF_COMPLETION_OPTS="$ZPWR_COMMON_FZF_ELEM \  
    --preview '${bash "$ZPWR_SCRIPTS/fzfPreviewOptsPony.sh"}'"  
else  
  export FZF_COMPLETION_OPTS="$ZPWR_COMMON_FZF_ELEM \  
    --preview '${bash "$ZPWR_SCRIPTS/fzfPreviewOpts.sh"}'"  
fi
```

Four call sites, of which three execute per shell start — the completion pair is an either/or. Each runs in a command substitution while the shell is still starting. The generated text is wrapped in single quotes inside a double-quoted assignment, which is why every one of these scripts carries the same header note: **Notes: no single quotes allowed**. A literal apostrophe anywhere in the emitted program would terminate the quoting and the rest of the preview command would be parsed as fzf options. That constraint is invisible from inside the generator and is enforced only by that comment.

`ZPWR_COMMON_FZF_ELEM` is assembled a few lines above and holds the prompt and the key bindings shared by every fzf invocation, so the preview program is the only part that varies.

Six Files, Two Halves

There are six scripts in the family and they form a two-column matrix:

	one-column input	two-column input
shared prefix	fzfPreviewOptsCommon	fzfPreviewOptsCommon2Pos
completion tail	fzfPreviewOpts	(none)
completion, ponies	fzfPreviewOptsPony	(none)
ctrl-t tail	fzfPreviewOptsCtrlT	fzfPreviewOpts2Pos

Each tail script begins by sourcing its prefix:

```
source "$ZPWR_SCRIPTS/fzfPreviewOptsCommon.sh"
```

and the prefix ends by printing an *unterminated* shell fragment. Its final heredoc opens `if test -f $file;` `then`, opens a second `if` chain that dispatches on file extension, and stops immediately after that chain's `else`. Nothing closes either construct. The tail script prints the rest.

This is a legitimate technique — it is how templating engines emit block structure — but it means neither half is syntactically valid on its own, and neither half can be checked in isolation. Only the concatenation is a program, and the concatenation exists for a few microseconds inside a command substitution during shell startup.

What The Prefix Dispatches On

The shared prefix is where the actual capability lives. Its emitted chain reads the candidate path into `$file` and then branches on extension:

<code>.jar / .war</code>	<code>jar tf \$file</code>	pipd to the java colorizer
<code>.tgz .tar .gz</code>	<code>tar tf \$file</code>	pipd to the C colorizer
<code>.deb</code>	<code>dpkg -I; dpkg -c</code>	pipd to the sh colorizer
<code>.rpm</code>	<code>rpm -qi; rpm -qlp</code>	pipd to the sh colorizer
<code>.zip</code>	<code>unzip -v</code>	pipd to the C colorizer
<code>.bz .bz2</code>	<code>decompress to stdout</code>	pipd to the YAML colorizer
<code>.xz .gz</code>	<code>decompress to stdout</code>	pipd to the YAML colorizer
<code>.so .dylib</code>	<code>nm plus xxd</code>	pipd to the YAML colorizer

Two of those branches are themselves conditional on the *generating* machine, not the previewing one:

```
zpwrCommandExists rpm && \
  rpm_cmd='{ rpm -qi $file; rpm -qlp $file; }' || rpm_cmd="stat"
zpwrCommandExists dpkg && \
  deb_cmd='{ dpkg -I $file; dpkg -c $file; }' || deb_cmd="stat"
```

If the host has no `rpm`, the generated program does not contain an `rpm` call at all; it contains `stat`. The absent-tool fallback is resolved once, at generation time, rather than being re-tested on every preview. For a string that is regenerated at every shell start that is the right trade — the check happens once per session instead of once per keystroke — but it does mean a preview command captured in a long-lived shell reflects the tool inventory as it was when that shell started.

The same pattern picks the symbol-dump command:

```
os="$(uname -s)"
if [[ "$os" == [Dd]arwin* ]]; then
  nmcmd="nm"
elif [[ "$os" == [Ll]inux* ]]; then
  nmcmd="nm -D"
else
  nmcmd="nm"
fi
```

`nm -D` on Linux lists dynamic symbols, which is what you want for a shared object; the macOS `nm` has no `-D` and lists them anyway. The `else` arm and the Darwin arm produce identical values, so the three-way branch is really a two-way one written as three for the sake of naming the platforms.

The colorizer names — `FZF_COLORIZER_JAVA`, `FZF_COLORIZER_C`, `FZF_COLORIZER_SH`, `FZF_COLORIZER_YAML`, and the three `FZF_COLORIZER_FILE` variants — are expanded at generation time too. Their values are commands, and what gets baked into the preview string is the command text, not the variable reference. Changing a colorizer in a running shell has no effect on previews until the bindings are rebuilt.

The One-Column And Two-Column Prefixes

`fzfPreviewOptsCommon.sh` and `fzfPreviewOptsCommon2Pos.sh` are near-duplicates. Diff them and exactly two things differ.

The first is the extraction of the path from the `fzf` line:

```
# Common
test -z $file && file=$(echo {} | sed "s@^~@$HOME@");

# Common2Pos
test -z $file && file=$(echo {} | cut -d" " -f2- | sed "s@^~@$HOME@");
```

`{}` is `fzf`'s placeholder for the current line. The `2Pos` variant drops the first whitespace-delimited field before treating the remainder as a path, which is what makes it usable against a list whose first column is something else — a line number, a mode string, a git status code. Both then rewrite a leading tilde to the home directory, because `fzf` hands over the display text and the display text may be abbreviated.

The second difference is a bare name:

```
# Common
zpwriCommandExists rpm && ...

# Common2Pos
zpwriExists rpm && ...
```

Two different helper functions for the same test, in two files that are otherwise byte-identical for that line. Both names are used elsewhere in the tree, so this is not a typo that would fail loudly; it is two copies of a file that drifted, and the drift is invisible because both spellings work.

That is the standing hazard with a copy-and-diverge prefix. The extension dispatch table — eight archive formats, the shared-object handling, the colorizer routing — exists twice, in full, in these two files. A ninth format added to one of them will not appear in the other, and nothing in the tree checks that they agree.

The Tail That Does Not Close

The completion tails are where this gets concrete.

`fzfPreviewOptsPony.sh` and `fzfPreviewOpts.sh` are the same file apart from the header comment, two blank lines, and the closing structure. Pony ends its emitted program like this:

```
        else
            cat ... || set ... || alias ... || { whois ... }
        fi
    } | cowsay | ponysay
fi
fi
```

and `fzfPreviewOpts.sh` ends it like this:

```

        else
            cat ... || set ... || alias ... || { whois ... }
        }
    fi
fi

```

The `fi` that closes the innermost `if` is missing. The brace that opens the grouped command is closed, so brace counting balances and nothing looks wrong on a quick read, but the `if` count and the `fi` count do not match.

You can check this without running a shell startup. Extract the final heredoc body from the prefix, extract the final heredoc body from the tail, substitute the case block, concatenate, and count keywords. The concatenation for the Pony variant has five `if` and five `fi`. The concatenation for the plain variant has five `if` and four `fi`. Feed both to `zsh -n` and the plain variant reports a parse error at the closing brace; the Pony variant and the `ctrl-t` variant parse clean.

The affected branch is the one that runs when the `fzf` candidate is not an existing path — the branch that treats the query as a hostname or an alias name and falls through `whois`, `dig`, the alias file and the environment. That is not an obscure corner; it is what makes the completion preview useful when you are completing something that is not a filename.

Why Nothing Caught It

The test suite does test these files. There is a `zsh -n` case and a `bash -n` case for each of the six, plus existence, readability, executability, shebang and non-emptiness assertions. Every one of them passes.

They pass because they check the generator, not the generated program. `zsh -n fzfPreviewOpts.sh` parses a bash script consisting of a `source` line and a `cat` with a heredoc. The heredoc body is a string literal as far as the parser is concerned. Its contents are never parsed by anything except the shell `fzf` spawns for the preview, and that shell's diagnostics land in a preview pane that is showing a failed preview — which is visually indistinguishable from a preview that had nothing to show.

This is the general shape of the problem with generated shell: the syntax check you can run cheaply is a check of the wrong artifact. The check that would have caught this is not expensive — generate the string, pipe it to `zsh -n`, assert a zero exit — but it requires the harness to execute the generator rather than parse it, and the harness is built around parsing files.

Which Variant You Get

There is one more turn. The choice between the two completion tails is:

```

if [[ "$ZPWR_BANNER_TYPE" == ponies ]]; then
    ... fzfPreviewOptsPony.sh ...
else
    ... fzfPreviewOpts.sh ...
fi

```

and the shipped default is set in the environment file:

```

export ZPWR_BANNER_TYPE=noponies

```

So the variant that parses is the one gated behind an opt-in cosmetic setting, and the variant reached by default is the one with the missing `fi`. The difference between the two files is a `fi` and a pipe into `cowsay | ponysay`; whoever added the pony pipeline closed the block correctly while doing it, and the change was never mirrored back.

The narrower lesson is about how the two files relate. They are not a base and a variant — they are two copies. If the pony pipeline had been implemented as a trailing filter variable in a single file, there would be one block structure to get right instead of two, and the bug could not exist.

Dead Assignments

Two of the tails open with:

```
zpwriIsZsh 66 ps='$pipestatus[1]' || ps='${PIPESTATUS[0]}'
```

`ps` is assigned and never referenced. Grep either file for the name and the assignment line is the only hit. The intent is legible — the emitted program would inspect the exit status of the first command in a pipeline, and the array that holds it is spelled differently in `zsh` and `bash`, so the generator resolves the spelling for the shell it is running under. Whatever was going to consume it either never landed or was removed, and the resolution survived.

The same pattern appears at the export site. `zpwriBindFZFLate` declares `local rpm_cmd deb_cmd` and assigns them:

```
zpwriExists rpm 66 rpm_cmd="rpm -qi" || rpm_cmd="stat"
zpwriExists dpkg 66 deb_cmd="dpkg -I" || deb_cmd="stat"
```

Those two names appear nowhere else in the function. They are locals, so they vanish when it returns. And note the values differ from the prefix script's: the prefix builds a compound command that both queries the package and lists its contents, while this copy queries only. Two independent, divergent computations of the same idea, one of which is discarded immediately.

None of this is expensive. It is worth naming because it is the fingerprint of the copy-and-adjust workflow that also produced the missing `fi`: the same edit made in several places, correctly in some, incompletely in others, with nothing in the build that can tell the difference.

The Clear-List Escape Hatch

One expansion recurs through every file in the family and deserves its own note:

```
$ZPWR_FZF_CLEARLIST
```

It appears in both prefixes, in all four tails, and in two sibling generators for the silver-searcher and `gtags` pickers. It is not defined in any of them. It comes from the recomputed environment file, through the lazy-assignment helper:

```
zpwriEvalIfNeeded ZPWR_FZF_CLEARLIST "$ZPWR_FZF_CLEARLIST" \
"zsh $ZPWR_SCRIPTS/zpwriClearList.zsh -- \\$file | fold -s -w 80" \
"$ZPWR_SCRIPTS"
```

The value is a command line with an escaped `$file` in it, so the reference to `$file` survives into the generated program and is resolved by the preview shell against whatever the current `fzf` line is. A variable holding a command that references a variable that will not exist until two expansions later. The triple backslash is what makes that work, and it is the reason this value is defined through the helper as a string rather than being written directly into each of the eight files that use it.

It is also the single point where the fallback preview — a long listing of the candidate, folded to eighty columns — can be changed for all eight call sites at once. Compared to the extension dispatch table, which exists twice and drifts, that is the shape the rest of this family should have had.

Chapter 67 – Running A Script Outside The Shell That Configures It

A shell framework has an easy time with functions. They run inside the shell that loaded the configuration, so every variable and helper is already present. Scripts are the hard case. A script is a separate process with a fresh environment, and it inherits only what was exported. If it needs a helper function, it has to go find it.

Thirty-two of the one hundred seventy-nine files under the script directory carry an identical block of preamble that solves this. It is worth reading in full, because it encodes a set of decisions that most projects make implicitly and badly.

The Two-Path Preamble

Here is the bash spelling:

```
if [[ -n "$ZPWR" && -n "$ZPWR_LIB_INIT" ]]; then
    if ! source "$ZPWR_LIB_INIT" ""; then
        echo "Could not source dir '$ZPWR_LIB_INIT'."
        exit 1
    fi
else
    source="${BASH_SOURCE[0]}"
    while [ -h "$source" ]; do
        zpwrBaseDir="$( cd -P "$( dirname "$source" )" \
            >/dev/null 2>&1 && pwd )"
        source="$(readlink "$source")"
        [[ $source != /* ]] && source="$zpwrBaseDir/$source"
    done
    zpwrBaseDir="$( cd -P "$( dirname "$source" )" \
        >/dev/null 2>&1 && pwd )"

    while [[ ! -f "$zpwrBaseDir/.zpwr_root" ]]; do
        zpwrBaseDir="$(dirname "$zpwrBaseDir")"
        if [[ "$zpwrBaseDir" == / ]]; then
            echo "Could not find .zpwr_root file up the tree." >&2
            exit 1
        fi
    done
    if ! source "$zpwrBaseDir/scripts/init.sh" "$zpwrBaseDir"; then
        echo "Could not source '$zpwrBaseDir/scripts/init.sh'."
        exit 1
    fi

    unset zpwrBaseDir
fi
```

Two paths. The fast one tests for an exported environment and sources the loader whose path that environment already supplied. The slow one has no environment to trust and must locate the installation from nothing but the path of the script currently executing.

The Fast Path Is Live

It is easy to look at a guard like `[[ -n "$ZPWR_LIB_INIT" ]]` and assume it is aspirational — that nothing ever sets the variable and every invocation takes the fallback. That is not the case here. The recomputed environment file assigns it:

```
zpwrEvalIfNeeded ZPWR_LIB_INIT "$ZPWR_LIB_INIT" \
"$ZPWR_SCRIPTS/init.sh" "$ZPWR_SCRIPTS"
```

So in any shell that has sourced the environment, both guard conditions hold and the fast path is taken. The slow path exists for the genuinely cold cases: a script invoked from `cron`, from a `launchd` or `systemd` unit, from an editor's build command, from a file manager, or from a login shell on a machine where the configuration has not been sourced yet. Those are exactly the contexts where a framework normally falls over, and they are the reason for the preamble.

Note the empty second argument: `source "$ZPWR_LIB_INIT" ""`. The loader takes an installation root as its first positional parameter, and passing an empty string means “you have an environment already, do not override it”. The slow path passes the discovered directory instead. One loader, two entry conditions, distinguished by whether the caller could supply the root.

Why A Sentinel File

The slow path could have hard-coded an installation directory. It does not; it walks upward from the script's own resolved location looking for a file named `.zpwr_root`, and gives up when it reaches the filesystem root.

That choice buys three things. A checkout can live anywhere — the tree is not pinned to a fixed path under the home directory, which matters for a test harness that wants to run against a scratch copy. A script can be invoked through a symlink from a directory on the search path, which is why the loop above resolves symlinks before it starts walking. And a script deep inside a subdirectory finds the same root as one at the top, without any script needing to know how deep it is.

The symlink resolution is the part most implementations get wrong. The loop follows a chain of symlinks rather than a single hop, and it resolves relative link targets against the directory of the link:

```
[[ $source != /* ]] && source="$zpwrBaseDir/$source"
```

Without that line, a relative symlink one directory over resolves against the current working directory and the walk starts from the wrong place. It is the standard portable idiom for this problem — `readlink -f` would be shorter but does not exist in the BSD userland, so the loop is written out.

Two Dialects

The preamble exists in two spellings. Twenty-four scripts use the bash form above. Eight use a zsh form that replaces the entire symlink loop with two parameter expansions:

```
0="${0:${ZSH_ARGZERO}:-${(%):-%N}}"
0="${0:${(M)0}:/}:-$PWD/$0"
zpwrBaseDir="${0:A}"

while [[ ! -f "$zpwrBaseDir/.zpwr_root" ]]; do
    zpwrBaseDir="${zpwrBaseDir:h}"
    ...
done
```

That is worth unpacking, because it is dense even by zsh standards.

The first line normalizes `$0` for the case where the file was sourced rather than executed. `${0:${ZSH_`

`ARGZERO` removes `$0` if it matches the shell's own argument zero — that is, if `$0` is the shell's name rather than the script's, which is what happens under some sourcing conditions. When the removal leaves nothing, the `:-` default kicks in with `${(%):-%N}`, the prompt-expansion form that yields the name of the script or function currently executing. Two different ways for the shell to tell you what file it is running, tried in order.

The second line makes the result absolute: `${(M)0: #/*}` keeps `$0` only if it matches a leading slash, and the default supplies the working directory as a prefix when it does not.

`${0:A}` then does everything the bash symlink loop did — resolve to an absolute path with all symlinks followed — in four characters. And `${zpwriBaseDir:h}` is the head-of-path modifier, replacing the `dirname` subprocess in the walk loop. The bash version spawns a process per level; the zsh version spawns none.

The zsh dialect is used by scripts whose shebang is already zsh. It is not a portability compromise so much as an acknowledgement that the two languages solve this problem at very different levels of directness.

The Loader

The file both paths converge on is `scripts/init.sh`, and its first line is the whole design:

```
if ! type -- "zpwriExists" >/dev/null 2>&1; then
```

The entire body is guarded on the absence of one helper function. If that function is already defined, the loader does nothing at all. This makes the loader idempotent under repeated sourcing, which matters when one script sources another, and it makes re-entry free rather than merely harmless.

Inside the guard, the load order is a decision tree on `ZPWR_LIB`:

```
if [[ -z "$ZPWR_LIB" ]]; then
    source "$ZPWR_ENV_FILE"
    source "$ZPWR_TOKEN_PRE"      # created if absent
    source "$ZPWR_RE_ENV_FILE"
    source "$ZPWR_TOKEN_POST"    # created if absent
else
    source "$ZPWR_LIB"
    source "$ZPWR_ALIAS_FILE"
fi
```

The comment above the second branch states the reasoning in two lines: environment variables are exported, so just get the functions. That is the correct distinction and it is easy to get wrong. Exported variables cross the process boundary; shell functions do not, unless the shell exports them and the child agrees to import them, which is a bash-specific mechanism with a history of security problems. So a script that inherits a fully configured environment still has to source something to get the function definitions back, and sourcing the whole environment again to obtain them would be waste.

The two token files are the user's extension points, sourced before and after the recomputed environment respectively. If either is missing it is created empty rather than skipped, so the file exists to be edited the first time anything runs.

Failing Loudly, With A Caveat

Every load in the loader is checked, and the failures are graded. A missing environment file or a missing recomputed environment file is fatal:

```
echo "cannot source ZPWR_ENV_FILE '$ZPWR_ENV_FILE'" >&2
exit 1
```

A failing token file is not:

```
zpwrlLogConsoleErr "could not source ZPWR_TOKEN_PRE '$ZPWR_TOKEN_PRE'"
```

The grading is right — the framework cannot function without its environment, but a syntax error in a user's personal override should not brick every script on the machine.

The caveat is `exit`. The loader is sourced, so `exit` terminates the caller, not the loader. From an executed script that is exactly what you want. From an interactively sourced script it terminates the interactive shell. Two scripts in the tree carry the preamble and also carry a header comment saying `Notes: source this file` — the two completion generators. Anything that reaches an `exit` on that path takes the interactive shell with it.

The Null-Command Log Lines

The successful branches of every load in the loader look like this:

```
if source "$ZPWR_ENV_FILE"; then
    : zpwrlLogInfo "loaded ZPWR_ENV_FILE '$ZPWR_ENV_FILE'"
else
```

The leading `:` is the null command. Everything after it is expanded as arguments and discarded. The logging call is disabled, but it is disabled in a form that keeps the arguments syntactically live — the string is still parsed, the variable is still expanded, and removing one character re-enables the call.

This is a deliberate idiom and a better one than commenting the line out. A commented-out call rots: the variable gets renamed, the call site is not updated because it is inside a comment, and re-enabling it later produces an empty message. A null-command call cannot rot, because it is still code. The cost is one expansion per load, and this runs once per cold script invocation.

It also tells you where the author wanted logging and could not have it. The loader runs before the logging subsystem is guaranteed to exist — the function being called is defined in a file the loader has not sourced yet at the point of the first call. That ordering problem has no clean solution, and disabling the calls in place while leaving them legible is a reasonable answer to it.

The Detail That Gives It Away

There is one small thing that shows how the preamble propagated. Its failure message comes in two spellings across the tree:

```
28 Could not source dir
8  Could not source file
```

The thing being sourced is `scripts/init.sh`, which is a file. Twenty-eight copies say `dir`. The eight that say `file` are the corrected version, and the distribution — a minority carrying the fix — is what you would expect if someone noticed the wording while editing a handful of scripts and did not go back to sweep the rest.

The message is only reachable when the loader itself fails to source, which is a condition that essentially does not occur in a working installation. So the wording has never mattered and probably never will. It is worth pointing at anyway, because it dates the copies: a script whose preamble says `dir` was copied from the original, and a script whose preamble says `file` was copied from one of the eight, and that is a more reliable ordering signal than the date in the header comment.

The Bash Trace Prompt

The loader's last act, when it is not running under `zsh`, is

an assignment to `PS4` using ANSI-C quoting. Stripped of its colour escapes, the value is:

```
> ${BASH_SOURCE} __${LINENO}__ ${FUNCNAME[0]}>
```

with the source file underlined and bright, the line number in blue between double underscores, and the function name in green.

`PS4` is the prefix bash prints before each traced command under `set -x`. The default is a single `+` repeated to show nesting depth, which tells you nothing about where you are. This replaces it with the source file, the line number and the enclosing function name, each in its own colour.

It is exported, so it applies to every bash process descended from the script, and it uses `$'...'` quoting so the escapes are real control characters rather than backslash sequences. The variable references inside are single-quoted and therefore expanded at trace time, not at assignment time — which is the entire point, since they have to reflect the command currently being traced.

Setting this once in the loader means that turning on tracing anywhere in the tree gives you a usable trace, without any script having to configure it. For a codebase this size, where a failure surfaces several levels down a chain of shelled-out scripts, that one assignment is worth more than most of the scripts it serves.

Chapter 68 – The Vended Usage Parser

The completion generator that harvests `--help` output ends its pipeline in `help2comp.py`. That program is the only piece of the completion machinery that is not local work: its header carries an upstream project URL, and it is vendored into the script directory rather than written here. It is also the smallest interesting parser in the tree, and reading it is the fastest way to understand exactly how much a generated completion knows and how much it does not.

The Contract

The invocation is fixed by the driver:

```
$abs --help < /dev/null |& help2comp.py $command > "$comp_dir"/_ $command
```

Standard input is the help text. The single argument is the command name. Standard output is a complete zsh completion function. There is no configuration, no options, and no state.

The failure mode is equally fixed:

```
options = parse_options(sys.stdin.readlines())
if len(options) == 0:
    sys.exit(2)
```

Nothing recognizable means exit status two and no output at all. The driver never inspects that status — it tests the output file for non-emptiness instead — so the exit code is functionally decoration, but the *absence of output* is the real signal and it is what the driver's `-s` test keys off. A parser that emitted a syntactically valid but empty completion function would defeat that test, and this one is careful not to.

Two Templates

The entire code-generation side of the program is three string constants. The outer shape:

```
COMPLETE_FUNCTION_TEMPLATE = """
#compdef $program_name

local arguments

arguments=(
  $argument_list
  '*:filename:_files'
)

_arguments -s $arguments
"""
```

and the two element shapes:

```

ARGUMENT_TEMPLATE = "" {opts}'[$description]$style'""
SINGLE_ARGUMENT_TEMPLATE = "" 'opt[$description]$style'""

```

Three details in the outer template are load-bearing.

The `#compdef` line is what makes the file a completion rather than a script. `compinit` scans every directory on the function search path, reads the first line of each file whose name begins with an underscore, and registers the function under whatever names follow that tag. Without it the file is inert.

The trailing `'*:filename:_files'` entry is the fallback. It says that any argument at any position may also be a filename, completed by the standard file completer. Without it, a generated completion would offer flags and nothing else — which is worse than having no completion, because the default behaviour in the absence of a completion is file completion. That one line is the difference between augmenting the default and replacing it with something less useful.

`_arguments -s` enables option stacking, so that a run of single-letter flags written together is understood as several separate flags. That is correct for the getopt-style interfaces this parser targets and wrong for the ones that use single-dash long options, which the parser has no way to detect.

The Two Element Shapes

The distinction between the two element templates is the alias question. A flag with both a short and a long spelling gets:

```
{-v,--verbose}'[be noisy]'
```

The braces are zsh's alternation form inside `_arguments`. They tell the completion system that these spellings are the same option, which has a consequence beyond brevity: once one of them appears on the command line, none of them is offered again. A flag with a single spelling cannot use the brace form and gets the plain string instead.

The selection is by list length:

```

if len(opts) > 2:
    model['opts'] = ",".join(opts[:-1])
    ... ARGUMENT_TEMPLATE ...
elif len(opts) == 2:
    model['opt'] = opts[0]
    ... SINGLE_ARGUMENT_TEMPLATE ...
else:
    pass

```

The list is the flag spellings with the description appended as the last element, so length two means one spelling plus a description and length greater than two means several spellings. Length one — a spelling with no description at all — falls through the `else` and is dropped entirely. That is a real loss: a flag documented in a help text with no explanatory text after it does not appear in the generated completion. The alternative would be an entry with an empty description, and the author chose silence over noise.

`model['style']` is set to the empty string on every path. It exists in both templates as a substitution point and is never given a value. Whatever it was for — an argument specification, a completion action for the flag's own operand — it is unwired.

There is a second unwired field. The model handed to the outer template includes a `url` key set to the upstream project address, and the outer template contains no `$url` placeholder. Because the code uses `safe_substitute` rather than `substitute`, neither of these mismatches raises. `safe_substitute` leaves unknown placeholders alone and ignores unused keys, which is the right choice for a template being filled from scraped text — a description containing a dollar sign would otherwise crash the generator — and it is also why two dead fields have survived unnoticed.

The Line Scanner

The parser proper is a two-state machine over lines:

```
for line in help_text:
    line = line.strip(STRIP_CHARS)
    if re.match(r'^--?[a-zA-Z0-9]+', line):
        previous_description_missing = False
        options = []
        while True:
            line, opt = cut_option(line)
            if opt is None:
                break
            options.append(opt)
        if len(line) == 0:
            previous_description_missing = True
            options.append(line)
            all_options.append(options)
        elif previous_description_missing:
            all_options[-1][-1] = line
            previous_description_missing = False
```

A line that starts with one or two dashes followed by an alphanumeric is an option line. Anything else is either a continuation of the previous option's description or noise, and the flag decides which.

The inner loop is the interesting part. It repeatedly bites the leading option off the front of the line and appends what it found, until nothing option-shaped remains. Whatever is left is the description. This handles the common help-text layout where several spellings share a line before the prose begins.

The state flag handles the other common layout, where a flag is alone on its line and its description is indented on the next:

```
-v, --verbose
    Produce verbose output.
```

The first line yields the two spellings and an empty remainder, which sets the flag. The next line does not start with a dash, so the `elif` fires and overwrites the empty description in place. The flag is cleared, so a third line of continued prose is discarded rather than replacing what was just captured. Multi-line descriptions are truncated to their first line by design.

Note what happens to every non-option line when the flag is *not* set: nothing. The program header, the usage synopsis, the examples section, the trailing copyright — all silently ignored, because the only lines that matter are the ones shaped like options. That is why this approach works at all across hundreds of unrelated programs. It never tries to understand the document; it looks for one shape.

Cutting An Option Off The Front

The bite function is one regular expression and a replace:

```
opt = re.findall(
    r'^(-[a-zA-Z0-9-]+)'
    r'(?:[\[\ ]=[^\ ]|[a-zA-Z\<\>\\[\]\:\/\_\?#\*\?])?',
    line)
if len(opt) > 0:
    newline = line.replace(opt[0], "", 1).strip(STRIP_CHARS)
    return newline, re.split(r'[ =]', opt[0], 1)[0]
```

The pattern matches a dash, then a run of alphanumerics and dashes, then optionally an argument attached by a bracket, a space or an equals sign. The optional trailing bracket handles the `[=WHEN]` style of optional

argument.

The return value splits the matched text on the first bracket, space or equals and keeps only the part before it — so `--color=WHEN` is recorded as `--color` and the operand name is thrown away. That is where the information loss happens. The generated completion knows a flag exists and knows its description; it does not know whether the flag takes an argument, and `_arguments` will therefore treat the next word as a fresh option candidate rather than as the flag's operand. This is the single largest quality gap between a generated completion and a hand-written one, and it is visible in six characters of the source.

The strip character set is worth noting too:

```
STRIP_CHARS = " \t\n,="
```

Comma and equals are in there alongside the whitespace. That is what lets the bite loop advance across `-v, --verbose` without needing a separator rule: after the first bite the remainder begins with a comma, and stripping removes it.

One forward-compatibility note. The `re.split` call passes its maximum-split count positionally. On a current Python this emits a deprecation warning:

```
DeprecationWarning: 'maxsplit' is passed as positional argument
```

The warning is suppressed by default, so it does not reach the terminal during generation, and the call still works. It is a two-character fix — name the argument — and it is the kind of thing a vendored dependency accumulates when nobody upstreams the maintenance.

Escaping Into A Quoting Context

The generated text is a zsh string inside single quotes inside a bracket-delimited description. Two characters break that, and both are handled in one function:

```
def _escape(line):
    line = line.replace(r'[', r'\[').replace(r']', r'\]')
    line = re.sub(r'\'', '', line)
    return line
```

Brackets get backslashed, because they delimit the description field. Single quotes are *deleted*, and the source comment gives the reason in five words: the apostrophe is unescapable as far as the author knew.

That is accurate. Inside `'...'` zsh treats every character literally including backslash, so there is no escape sequence that produces an apostrophe. The options are to end the string, emit an escaped quote outside it and reopen — the `'\''` dance — or to remove the character. Removal is lossy and produces `don't` where the help text said `don't`. The dance would have been correct and would have complicated a function that is currently three lines. The choice made here is defensible for the use case and would not be for a general-purpose generator.

The Sentence-Case Heuristic

One transformation has nothing to do with correctness:

```
desc = list(_escape(opts[-1]))
if len(desc) > 1 and desc[1].islower():
    desc[0] = desc[0].lower()
model['description'] = "".join(desc)
```

If the second character of the description is lowercase, the first is lowercased. So `Produce verbose output` becomes `produce verbose output`, while `GNU extensions enabled` is left alone because its second character is uppercase.

The purpose is cosmetic. Completion descriptions are rendered in a list under the command line, and a column of descriptions where some start with a capital and some do not looks like an error. Help texts are inconsistent about this between programs and sometimes within one program. The second-character test is a cheap proxy for “is this a sentence or an acronym”, and it gets acronyms and initialisms right often enough to be worth the two lines.

It also converts the list to a character array to modify one element, which is Python’s way of saying strings are immutable. A slice-and-concatenate would be shorter. The array form makes the intent — change exactly the first character — harder to misread.

What This Buys, Precisely

Setting the whole pipeline out plainly: for a command with no existing completion, the generator runs the binary with `--help` under a quarter-second budget, feeds whatever came out to a two-state line scanner, and produces a completion function that knows the flag spellings, knows which spellings are aliases of each other, knows a one-line description for each, and falls back to file completion for everything else.

It does not know which flags take arguments, what those arguments may be, which flags conflict, whether the command has subcommands, or whether any of the flags it found are real — a line in an examples section that happens to start with a dash will be parsed as an option definition.

For the long tail of installed binaries that is a good trade, and the gate in the driver ensures it is only ever applied to that tail. What makes it safe is not the quality of the parser. It is that the output is never allowed to displace a real completion, and that a completion which produces nothing is deleted rather than installed.

Chapter 69 – Fifteen Ways To Watch Something

Fifteen files in the script directory have “watch” in their name. They are not fifteen variations on one idea; they are three distinct architectures that happen to share a word, and separating them is the first useful thing to do.

```
event-driven, fswatch      6 scripts
event-driven, inotifywait  1 script
poll-and-redraw           4 scripts
composed wrappers        3 scripts
threshold daemon          1 script
```

The first group reacts to filesystem changes. The second polls a command and repaints the terminal. The third are one-line wrappers that invoke the second group with a fixed command. The last one watches a number rather than a file.

The Event Loop

Every fswatch-based script in the tree has the same skeleton:

```
dir_watching="$1"

while read -d "" event; do
    echo "The event was $event"
    ...do something...
done < <(fswatch -r -0 -E "$dir_watching" -e "/\..")
```

Three flags carry the design. `-r` recurses. `-0` makes the watcher emit NUL-separated paths, which pairs with `read -d ""` on the consuming side — `-d ""` sets the delimiter to the null byte, so a path containing a newline is still one record. That combination is the only correct way to stream filenames between two processes, and it is used consistently here.

`-e "/\.."` is the exclusion, and it is subtler than it looks. Under extended regex, `\.` is a literal dot and the following `.` is any character, so the pattern matches a slash, a dot, and at least one more character. That excludes version-control directories, editor state directories and dotfiles generally, while not excluding a path that merely ends in a dot. Without it, a watcher on a git working tree fires continuously — every git operation writes into the repository directory, and a watcher that rebuilds on change would rebuild forever.

The redirection is the last structural piece. `done < <(fswatch ...)` feeds the loop from process substitution rather than a pipe. If it were a pipe, the `while` loop would run in a subshell and any variable it set would be lost when the loop ended — which matters immediately for the overlap-guard variant below, whose entire mechanism is a variable that has to survive between iterations.

The Compile-On-Save Variant

`watchServiceFswatchRustCompile.sh` is the most complete of the group and the only one that filters events:

```

fileName=$(basename $event)
watchingFile=$(basename $2)

if [[ $fileName == "$watchingFile" ]]; then
    if output="$(rustc $2 2>&1)"; then
        compiled=${fileName%*.rs}
        clear
        ./"$compiled" && rm "$compiled"
    else
        clear
        echo "$output"
    fi
fi

```

The filter exists because a compiler writing into the watched directory generates events, and acting on those would loop. Comparing basenames rather than full paths means the watcher fires on the target file regardless of which path form the event arrived as.

The compile branch captures both streams into a variable and uses the assignment's own exit status as the condition. That is the right shape: on success `$output` is unused and the binary runs; on failure `$output` holds the diagnostics and is printed. There is no case where the compiler's output is lost, and no case where it is printed on top of the program's own output.

`${fileName%*.rs}` strips the extension. The `*` inside a `%` suffix pattern is redundant — `%` already anchors to the end and matches the shortest suffix — but harmless. Running the binary and deleting it with `&&` means a program that exits non-zero leaves its binary behind, which is either a bug or a debugging convenience depending on your view.

The Overlap Guard

`fsWatchLongRunning.sh` exists to solve the problem every naive file watcher has: the triggered task takes longer than the interval between triggers. Its answer is a first-iteration special case and a liveness check:

```

ps -ef | awk '{print $1,$2}' | grep -q "$pid" \
&& echo "blocking next call..." \
|| { ...start task...; pid=$!; }

```

The intent is clear and the implementation has two defects worth naming, because they are the defects every hand-rolled version of this check has.

The `awk` reduces each process line to its first two fields, which are the user and the process id. The `grep` is then unanchored and matches the pid anywhere in that two-column text. A user id numerically equal to the pid matches. A longer pid that merely contains the shorter one as a substring matches — pid 123 is reported alive whenever any process 1234 or 5123 exists, which on a busy machine is close to always. The check therefore fails toward “still running”, and the watcher silently stops doing work.

The second defect is that the check is on liveness, not on identity. Process ids are recycled. A guard that outlives its process eventually matches an unrelated one. The idiomatic fix in bash is `kill -0 "$pid" 2>/dev/null`, which asks the kernel about a specific process rather than pattern-matching a table, and which costs no subprocesses instead of three.

What the script gets right is the shape. The task is backgrounded, the pid is captured, and the guard is consulted before starting another. Getting that shape right is most of the problem; the check itself is a two-word substitution away from being correct.

Poll And Repaint

The second architecture is the local reimplement of `watch`. There are two base implementations and they differ in the one thing that matters.

`mywatch.sh` clears and reprints:

```
while true; do
    output=$(eval "$1")
    clear
    ...print...
    sleep $time
done
```

`myWatchNoBlink.sh` does not clear. It moves the cursor home and overwrites, padding each line to the terminal width so that stale characters from the previous frame are erased:

```
home=$(tput cup 0 0)
ed=$(tput ed)
el=$(tput el)
printf '%s%s' "$home" "$ed"
while true; do
    ROWS=$(tput lines)
    COLS=$(tput cols)
    eval "$CMD" | head -n "$ROWS" | while IFS= read; do
        printf '%-*.*s%s\n' $COLS $COLS "$REPLY" "$el"
    done
    printf '%s%s' "$ed" "$home"
    sleep $timeToSleep
done
```

The name is accurate. Clearing the screen and redrawing produces a visible flash on every frame because the terminal renders the blank state. Overwriting in place never presents a blank state.

The `printf` format is the mechanism: `%-*.*s` takes the width twice from arguments, once as the minimum field width and once as the maximum precision, so every line is padded *and* truncated to exactly the terminal width. Padding erases leftovers from a longer previous line; truncation prevents wrapping, which would desynchronize the row count and push the display off the bottom. The trailing `$el` — clear to end of line — is belt and braces after the padding has already filled the row.

`head -n "$ROWS"` bounds the output to one screen, and `ROWS` and `COLS` are re-read inside the loop, so a terminal resize is picked up on the next frame. The `IFS=` on the `read` prevents leading whitespace from being stripped, which matters for any command whose output is indented — which is most of them.

`tput civis` hides the cursor before the loop and a trap restores it:

```
trap 'tput cnorm; printf "\e[0m"; exit' INT
```

Without that trap, interrupting the watcher leaves the terminal with an invisible cursor, and the user has no obvious way to get it back. It is the single most important line in the file and it is easy to omit.

Where The Poll Scripts Went Wrong

These are old scripts and they carry the marks.

Both option loops `break` out of the `getopts` while loop inside the case arm that recognizes a flag:


```

while getopts $optstring opt; do
    case $opt in
        h) usage >&2; break ;;
        b) boldflag=true; break ;;
        *) usage >&2 ;;
    esac
done

```

`break` exits the parsing loop, so only the first flag is ever seen. In the two-flag base scripts that is invisible. In `myWatchMaintainEscapes.sh`, which adds a third flag taking an argument, it is not: that script's `optstring` is `dhl:` and its `l` arm also breaks, so `-d -l 20` parses `-d` and stops, and the line limit is silently ignored.

The `h` arm is dead code in a second way — `usage` ends in `exit 1`, so the `break` after it is unreachable.

`myWatchNoBlink.sh` declares its locals as `local home ed ell ROWS COLS CMD` and then uses `el` throughout. The declared name has a doubled letter; the used name does not. The variable the function actually reads and writes is a global. Nothing breaks — this script is not sourced into a shell that has an `el` of its own — but the declaration is not doing what it appears to.

The same file has a second shebang partway down the body, where the header comment block ends. It is a comment at that position and has no effect. Two other scripts in the directory have the same duplication.

The last line is `watchCommand $1`, unquoted, while the function body does `CMD="$@"`. The unquoted expansion splits the command string into words and the function reassembles them with spaces — which round-trips only because word splitting on default separators and rejoining with single spaces happen to be inverses for these inputs. The expansion is also subject to pathname expansion, and the wrapper scripts pass commands containing a `*`. That `*` survives only because no file in the working directory matches it. Quoting the expansion and using `"$1"` inside would remove both hazards.

After the infinite loop, the function ends with:

```

if [[ $boldflag == true ]]; then
    printf "\e[0m"
fi

```

which is unreachable. The reset it wants is in the trap, where it belongs.

The Composed Wrappers

Three scripts exist only to call one of the display engines with a fixed argument:

```

bash myWatchNoBlink.sh -b 'pstree -g 3 -u "$(whoami)" | sed ...' 1

```

This is composition done the shell way: the display engine takes a command string and a period, so a new monitor is a new file containing one line. The quoting is the awkward part — the command has to be a single argument, and the current user's name has to be interpolated into the middle of it, which produces the close-quote-expand-reopen sandwich around the substitution.

Note that the wrapper invokes the engine by bare name. It relies on the script directory being on the search path, which the environment arranges. The wrappers are therefore not runnable from a checkout without that environment, unlike the scripts carrying the root-finding preamble.

Two of the three point at the plain engine. The third points at the colorized variant, and that variant is where this family stops being usable. `myWatchNoBlinkColorized.sh` differs from its base in three ways: it declares one extra local, it pipes each line through an external filter, and — the significant one — it carries a bare `set -x` immediately after its header comment block, in the position where its base has a duplicated shebang. Tracing is on. Every command the script runs is echoed to standard error, interleaved with a display whose whole purpose is precise cursor positioning. The filter it pipes through is invoked as `java parser $LINE`, naming a Java class that exists nowhere in the tree.

That script is a work in progress that was committed with its debugging flag still set, and the wrapper that calls it is committed too. Both are exercised by the same existence-and-syntax tests as everything else in the directory, and they pass, because `set -x` is valid syntax and a missing Java class is a runtime condition.

Watching A Number

`memWatch.sh` is the outlier: no filesystem, no terminal, no display.

```
while true; do
    used=$(free | stryke -anE 'p $F[2] if $. == 2')
    if (( used > "$2" )); then
        mutt -s "MEMORY_THRESHOLD exceeded: $used" "$1" \
            < <(free -h; top -w 500 -c -o %MEM -bn1; )
        "builtin" command sudo -E reboot
    fi
    sleep 10
done
```

The extraction line is the clearest example in the tree of `stryke` standing in for `awk`. `-a` autosplits each line into a field array, `-n` wraps the program in a read loop, `-E` supplies the program on the command line. `$F[2]` is the third field and `$.` is the line number, so the whole expression is “print field three of line two” — which in `free` output is the used-memory column of the memory row. The `awk` spelling would be `NR==2{print $3}`; the field index differs because one language counts from zero and the other from one.

The mail body is another process substitution, this time feeding a command's standard input from the concatenated output of two commands — a human-readable memory summary and a one-shot process listing sorted by memory. That is the right payload for the alert: the number that tripped the threshold is in the subject, and the evidence for which process caused it is in the body.

Then it reboots. Unconditionally, immediately after sending the mail, with no grace period and no attempt to kill the offending process. The `"builtin" command sudo -E` prefix is the framework's idiom for bypassing any function or alias that might shadow the name, and `-E` preserves the environment across the privilege boundary.

This is a script for an unattended appliance, and the tree contains the unit file that confirms it:

```
[Service]
Type=simple
User=pi
Group=pi
Environment="EMAIL=myemail"
ExecStart=/usr/bin/env bash /home/pi/.zpwir/scripts/memWatch.sh
```

A small single-board machine, running unattended, whose failure mode is memory exhaustion and whose acceptable recovery is a reboot. In that context the unconditional reboot is the correct behaviour and the ten-second poll is generous.

The unit ships with the address literally set to `myemail`, which is a placeholder and not a deliverable address — the unit is a template to be edited, not a working configuration. And `ExecStart` passes no argument, so the script takes its default threshold from its own fallback rather than from the unit. Both are fine for a template; both are the kind of thing that is discovered at three in the morning when the alert does not arrive.

`memWatch.sh` also appears in the completion generator's blacklist, which is the correct place for it. A generator that runs every binary on the system with `--help` would otherwise start a memory watcher that reboots the machine when it sees a large number.

The Underscore Printer

The one inotify-based script deserves a note for a single habit. It prints a horizontal rule eight times, and every one of them is:

```
python -c 'print("_"*100)'
```

An entire Python interpreter, started and torn down, to print a hundred underscores. The shell can do it in one builtin invocation with no process at all. On the class of machine this script targets, interpreter startup is not free.

It is also a portability trap that has since sprung. The command is `python`, not `python3`. On a current macOS there is no `python` on the search path at all, so all eight calls fail and the rules simply do not print — the script still works, minus its formatting.

The same script has a quieter problem in its output redirection:

```
exec 1>>"$LOGFILE"  
exec 2>&1
```

`LOGFILE` is not set anywhere in the tree. The framework's own name for this is `ZPWR_LOGFILE`, which is a near-miss. With the variable unset the redirection fails with an error on standard error and the shell continues, so the script runs correctly but logs to the terminal it was supposed to detach from. A watcher intended to run unattended and mail on a match instead holds a terminal open. Two characters of prefix separate this from working.

Chapter 70 – The Index That Outlived Its Files

There is a file in the script directory named `etags`, with no extension and no shebang. It is not a script. It is a tags index in the Emacs format, generated from the tree at some point in the past and committed alongside the code it indexes. Next to the installer there is a second one: a directory holding the three binary database files that GNU Global produces.

Both are build artifacts under version control, and both are now wrong in ways that are worth cataloguing precisely, because the pattern generalizes to every generated file anyone has ever been tempted to commit.

What A Committed Index Costs

The argument for committing a tags file is real. Building one requires the generator to be installed; a fresh checkout with the index already present gets symbol jumping immediately, and on a large tree the build is not instant. For a shell environment whose whole premise is that everything works the moment you land on a machine, pre-building the index is consistent with the philosophy.

The cost is that the index is a snapshot of a tree that keeps moving, and nothing in the build or the test suite compares the two. A stale tags file does not error; it sends you to the wrong line, or to a file that is not there.

The tree does have a regeneration verb, and the first surprise is that it does not produce this file. `zpwirRegenCtags` runs the generator across the shell configuration, the four autoload directories, the completion directory and both script directories, and writes a file named `tags`:

```
builtin cd "$ZPWR_SCRIPTS"
command rm tags 2>/dev/null
ctags --language-force=sh --fields=+l ...
ctags --append --fields=+l "$ZPWR_SCRIPTS"/* "$ZPWR_SCRIPTS_MAC"/*
command cp tags "$HOME"
```

`--language-force=sh` is the load-bearing flag: most of these files have no extension and the generator would otherwise skip them or guess. The second invocation appends rather than replacing, which is why the script directories are indexed in a separate pass — they contain a mix of languages that the forced mode would have mangled.

That file, `tags`, is not in the tree. It is built on demand and lives beside the committed `etags` file without either one knowing about the other. Two consumers read `tags`; nothing in the tree reads `etags`. Grep for it and the only hits outside the file itself are two test assertions — one that it is readable and one that it is non-empty. It has no producer and no consumer. Its entire remaining function in this codebase is to satisfy two tests that assert it exists.

The two consumers of the live file are worth a sentence of their own, because they are the same function twice:

```
function zpwirVimZpwirCtags(){
    cat "$ZPWR_SCRIPTS/tags" | fzf
}
```

and its emacs-named counterpart, whose body is the identical pipeline preceded by a guard that refuses to run when emacs is not installed. Neither function invokes an editor. One of them declines to work without a

program it never calls.

Measuring The Drift

The measurement is mechanical. The index records one absolute path per indexed file. Extract those paths, take each basename, and ask whether a file by that name exists anywhere in the tree.

The index names one hundred sixty-three files. Ten of those basenames exist nowhere in the tree today:

```
banner.pl
c.pl
oldPipUpdater.sh
regexreplace.pl
storageShower.sh
updaterNoPip.sh
updaterPip.sh
zpwrclearList.sh
zpwrfzfEnv.sh
zpwruUpdater.sh
```

That list is not noise. Read it as a change log and it tells you what happened to the tree.

`banner.pl` and `c.pl` are the two names from an earlier chapter — the two `.stk` scripts that have no perl counterpart on disk. The index is where their counterparts went. Both were perl scripts, both were ported, and the perl originals were deleted rather than kept as references the way the other six port pairs were. The index is the only surviving evidence that they existed, and it is the reason the port layer looks asymmetric.

`zpwrclearList.sh` and `updaterPip.sh` are extension changes: the tree has `zpwrclearList.zsh` and `updaterPip.zsh`. Somebody moved those two scripts from a bash shebang to zsh and renamed accordingly. `updaterNoPip.sh` and `oldPipUpdater.sh` are the generation of Python-package updaters that the current one replaced. `zpwrfzfEnv.sh` predates the current family of environment pickers.

`regexreplace.pl` is the interesting one, because a file named `regexReplace.pl` does exist. The index entry differs only in the case of one letter. That is a different kind of staleness: not a deleted file but a renamed one, and a rename that a case-insensitive filesystem would have hidden completely. On the case-sensitive check the two names do not match, which is why it appears in the list at all.

The Path Problem

Every path in the index is absolute, and every one of them begins with the same home directory of the machine that generated it.

That makes the artifact useless to anyone else by construction. A second developer, a continuous-integration runner, a container, or the same person on a second machine with a different account name gets an index whose every entry points at a path that does not exist. The index does not degrade gracefully in that situation — it does not partially work — because not one of its entries resolves.

It also means the committed file discloses the layout of the machine it was generated on. For a tags index that is trivia. The habit is not trivia: any generated artifact that embeds absolute paths and is then committed carries whatever those paths happen to contain, and the person committing it is not looking at the contents.

The Global databases next to the installer have the same property in binary form. Grep them and they match on script names, which is how you can tell they index the same tree; they cannot be read by eye, so nothing about their contents is obvious to a reviewer looking at a diff that says three binary files changed.

Two Consequences For Reading The Tree

The practical advice for anyone reading this codebase is short.

First, the index is not authoritative about what exists. If you are grepping for call sites — the kind of survey that answers “is this script still used” — the index will produce hits for files that are gone, and a naive count of references will include them. Every survey in these chapters excluded it explicitly for that reason. A reference from the index is not a call site; it is a fossil.

Second, the fossils are useful for exactly one thing, which is history. The ten missing names above are a compressed record of four separate migrations — two perl scripts to stryke, two scripts from bash to zsh, two generations of package updater, one generation of environment picker. Reconstructing that from the commit log would take considerably longer than reading a list of names.

The Rule This Suggests

A generated artifact belongs in version control only when three things hold: it is portable, it is regenerable by anyone, and something fails loudly when it goes stale. The tags index here satisfies none of the three. It is pinned to one machine’s paths, it requires a generator that may not be installed, and it rots silently.

The alternative is not complicated, and the tree already implements it for the other index. `tags` is built on demand by a verb, is not committed, and is therefore never stale. What is committed is the rule that produces it. The committed index is the same artifact with the discipline removed.

The reason to spell this out in a book about this particular tree is that the tree is otherwise unusually disciplined about generated content. The completion corpus is generated into a staging directory rather than onto the function search path. The preview commands are generated at shell start and never written to disk. The keybinding documentation is regenerated from the bindings themselves. The runtime tags file is regenerated by a verb and never committed. In every one of those cases the generated thing is treated as disposable. The committed index is where that discipline lapsed, and the ten missing names are the measure of how long ago.

Chapter 71 — Three Manifests, One Installer

The framework installs plugins for three different programs — zsh, vim and tmux — and it does so with one function and three text files. The three installer scripts that call it are nearly identical, and the differences between them are the most interesting thing in the layer, because each difference is an accident rather than a decision.

The Manifests

Each ecosystem gets one file in the install directory, one repository per line, in `user/repo` form and nothing else. No versions, no branches, no options, no comments:

```
install/.zshplugins      48 lines, 47 distinct repositories
install/.vimbundle       77 lines, 77 distinct repositories
install/.tmuxplugins     9 lines, 9 distinct repositories
```

Thirty-three of the forty-eight zsh lines name a fork under the project's own organisation rather than the upstream repository. Two of the nine tmux lines do. Three of the seventy-seven vim lines do. That ratio is the clearest statement in the tree of where the maintenance effort goes: the zsh layer is the one that has been forked and changed, and the other two are consumed as published.

The forty-eighth zsh line is a duplicate. [MenkeTechnologies/fzf](#) appears at line 3 and again at line 35, in two different alphabetical neighbourhoods, which is what happens when a sorted list is edited twice by hand. The consequence at install time is one extra `git clone` attempt into a directory that now exists, which fails and is ignored.

One Function, Two Modes

All three installers reduce to four lines: check that they are being run from the install directory, source the shared library, change into the destination directory, and call one function.

```
if builtin cd "$ZPWR_PLUGIN_MANAGER_HOME/plugins"; then
    zpwInstallGitHubPluginsFromFile "$@" "$ZPWR_INSTALL/.zshplugins"
else
    echo "could not cd to $HOME/.vim/bundle/YouCompleteMe" >&2
    exit 1
fi
```

That error message is at `install/zsh_plugins_install.sh:24` and it is wrong. The identical string appears at `install/ycm_install.sh:26`, where it is correct, because that script really does change into the YouCompleteMe directory. The zsh installer was written by copying the vim-adjacent one and the message was never updated. It is only reachable when the plugin directory cannot be created, which is the exact situation in which a person needs the message to be accurate.

The function itself lives at `scripts/lib.sh:537` and is short enough to read in full. It parses two flags, takes a filename, and loops:

```

if [[ $forceFlag == true ]]; then
    while read repo; do
        zpwroverwriteGitHubPlugin "$repo"
    done < "$file"
else
    while read repo; do
        zpwriinstallGitHubPlugin "$repo"
    done < "$file"
fi

```

The two per-repository functions differ by three lines. The default one clones unconditionally:

```

echo "Installing plugin $user/$repo."
git clone "https://github.com/$1.git"

```

and the force variant removes any existing directory first, then guards the clone with a test that can no longer fail:

```

if [[ -d "$repo" ]]; then
    zpwprettyPrint "rm -rf $repo"
    command rm -rf "$repo"
fi
test -d "$repo" || git clone "https://github.com/$1.git"

```

The guard is on the branch that does not need it and absent from the branch that does. Re-running the default installer on a configured machine therefore prints one `fatal: destination path ... already exists` per installed plugin and continues, because the loop never checks a status. That is survivable behaviour — nothing is destroyed, and the intent of “install what is missing” is approximately achieved by clone failing — but it is achieved by ignoring errors rather than by asking a question.

Two smaller properties of the loop are worth naming. `read` is called without `-r`, so a backslash in a repository name would be eaten; no name in any of the three manifests contains one. And there is no skip for blank or commented lines, which is why the manifests contain neither: a `#` line would be handed to `git clone` verbatim. The format is not a configuration format, it is a list of arguments, and it only stays valid as long as nobody writes a note in it.

The flag variable is also global. The function declares `local file` and nothing else, so `forceFlag=true` persists in the shell after the function returns. In a script that calls the installer once this is invisible. In a shell where the function has been sourced and is called twice, the second call inherits the first call’s mode.

The Directory Guards Do Not Agree

Each installer creates its destination before entering it, and each does it differently.

```

# zsh_plugins_install.sh:17
if [[ ! -d "$ZPWR_PLUGIN_MANAGER_HOME/plugins" ]]; then
    mkdir -p "$ZPWR_PLUGIN_MANAGER_HOME/plugins"
fi

# vim_plugins_install.sh:17
if [[ ! -d "$HOME/.vim/bundle" ]]; then
    mkdir -p "$HOME/.vim/bundle"
fi

# tmux_plugins_install.sh:17
if [[ -d "$HOME/.tmux/plugins" ]]; then
    mkdir -p "$HOME/.tmux/plugins"
fi

```

The third test is inverted: it creates the directory only in the case where the directory already exists. On

a machine without a tmux plugin directory the `mkdir` is skipped, the `cd` fails, and the script exits with the message at line 24. The bug has never been visible because the only caller reaches it after the orchestrating script has already created `$HOME/.tmux/plugins/tpm` on its way to cloning the tmux plugin manager. A bug masked by call order is still a bug; it becomes visible the first time somebody runs the tmux installer on its own, which is exactly what a person does when one plugin fails to clone.

What The Orchestrator Does Around Them

`install/plugins_install.sh` is the script that calls the vim and tmux installers, and it is where the non-repository parts of the layer live: a `curl` of pathogen into `~/.vim/autoload`, a symlink of the snippets directory, a symlink of the neovim entry point, a copy-then-symlink of the powerline configuration, a clone of the tmux plugin manager, and a loop over ten configuration files:

```
symFiles=(.tmux.conf .ideavimrc .vimrc conf.gls conf.df conf.ifconfig
          conf.mount conf.whois .iftopcolors .inputrc)

for file in ${symFiles[@]} ; do
    ln -sf $ZPWR_INSTALL/$file "$HOME/$file"
done
```

Every one of those is a symlink into the repository rather than a copy, which means editing `~/.vimrc` edits the tracked file and `git status` in the framework's own directory is the change log for the user's configuration. The two files handled outside the loop are the exceptions that prove the rule: `htoprc` is copied, not linked, because htop rewrites its configuration file whenever a setting changes in its interface, and `.ctags` is linked but only if it does not already exist.

The Manifest Is Not The Loader

The manifests exist to clone repositories. Nothing sources them at shell start. The runtime list lives separately, in `install/.zshrc`, as three arrays — `ZPWR_GH_PLUGINS`, `ZPWR_OMZ_PLUGINS`, `ZPWR_OMZ_LIBS` — plus a fourth for completion snippets, and the plugin manager walks those arrays.

Two lists maintained by hand in two files is the classic setup for drift, so it is worth measuring rather than assuming. Every one of the forty-seven distinct repositories in `.zshplugins` is named somewhere in `.zshrc`. The drift runs in the other direction: `.zshrc` loads `MenkeTechnologies/zsh-autocomplete`, at line 554, and no manifest ever clones it.

```
if [[ $ZPWR_AUTO_COMPLETE == true ]]; then
    zinit ice lucid nocompile wait"0b" atinit='zpwBindOverrideAutoComplete'
    zinit load \
        MenkeTechnologies/zsh-autocomplete
fi
```

The gap is invisible because `ZPWR_AUTO_COMPLETE` defaults to `false` (`env/.zpw_env.sh:20`) and because the plugin manager clones on demand anyway — turning the feature on triggers a network fetch at first load rather than an error. The manifest is not the mechanism that makes plugins present; it is a pre-warm. That is a defensible design, and it means the manifest's real job is to make a first shell start fast rather than to make it work.

The Load Order As Actually Written

The staging in `.zshrc` uses six distinct deferral keys, not the alphabet-long sequence a reader might expect from the number of plugins:

```

(none)      powerlevel10k, loaded synchronously before the prompt
wait        every OMZ completion, lib, plugin, and every ZPWR_GH_PLUGINS entry,
            plus fzf, git-fuzzy, forgit, zconvey
wait'0'     zsh-autopair
wait'0a'    zsh-expand, zsh-autosuggestions, zsh-zinit-final,
            zsh-more-completions
wait'0b'    zsh-autocomplete (only if ZPWR_AUTO_COMPLETE is true)
wait'0c'    zsh-history-substring-search
wait"${ZPWR_ZINIT_COMPINIT_DELAY}a"
            compinit + cdreplay, then fast-syntax-highlighting

```

`ZPWR_ZINIT_COMPINIT_DELAY` defaults to `0` (`env/.zpwrc.sh:201`), so the last two entries resolve to `wait"0a"` — the same key as four other plugins. Within one key, zinit runs the registrations in the order they were made, which is why the file's ordering of those four `zinit load` calls is load-bearing even though they share a key, and why the comment above the syntax highlighter says it must come last rather than giving it a later key.

The only synchronous plugin is the prompt. Everything else, including the completion system's own initialisation, happens after the first prompt is drawn. That is the entire startup strategy, expressed in six string literals.

Chapter 72 – Four Ways To Load A Vim Plugin

The vim configuration in this tree is 1,998 lines, and it contains three plugin managers plus one hand-written path insertion. All four are live in the sense that all four run; only two of them do anything.

The Census

manager	where	plugins declared	state
neobundle	.vimrc:19-38	1	every line commented out
Vundle	.vimrc:130-271	17	active
pathogen	.vimrc:1539-1544	0 declared	infects ~/.vim/bundle
runtimpath	.vimrc:1549	1 (ctrlp)	active

The neobundle block is a fossil, preserved with its " comment markers intact, including the one plugin it once managed. It costs nothing at runtime and it records that the configuration predates the current arrangement.

Vundle is real. Seventeen `Plugin` lines sit between `call vundle#begin()` and `call vundle#end()`, two of them inside an `if has('nvim')` guard for the statusline pair. Every one of the seventeen also appears in `install/.vimbundle` — the overlap is exact, not approximate — which means each of those repositories is cloned once by the installer into `~/.vim/bundle` and then declared a second time to a manager that would clone it into the same directory.

Pathogen is the manager that actually loads the seventy-seven cloned repositories, and it is invoked last, defensively:

```
if filereadable(expand('~/.vim/autoload/pathogen.vim'))
  runtime! autoload/pathogen.vim
  if exists('g:loaded_pathogen')
    silent! execute pathogen#infect()
  endif
endif
```

Three guards for one call: the file must exist, sourcing it must have defined the plugin's marker variable, and any error from the call itself is swallowed by `silent!`. That is the correct amount of paranoia for a line that runs at the bottom of a configuration on a machine that may not have finished installing.

The fourth mechanism is one line:

```
set runtimepath^=~/.vim/bundle/ctrlp.vim
```

`^=` prepends, which puts that one plugin ahead of everything pathogen added. A plugin gets promoted to the front of the search path when something it defines must win a conflict, and doing it by hand rather than through a manager is how that intent gets expressed in a system where the manager offers no ordering control.

Why Four Is Stable

The obvious reading is that three of these should be deleted. The less obvious reading is that the arrangement is stable because each layer fails safely. Vundle's declarations are inert if Vundle is not installed — `call vundle#begin()` in a configuration without Vundle throws, and the throw happens inside a file vim continues to read. Pathogen's are inert if pathogen is not downloaded. The bundle directory is populated by an installer that needs neither of them. There is no arrangement of missing pieces that produces a vim which refuses to start; there are only arrangements that produce a vim with fewer plugins.

That property is worth more in this tree than tidiness, because the configuration is deployed to machines in unknown states and the failure mode “some plugins missing” is recoverable inside the editor while “editor will not start” is not.

The Editor Entry Points

Neovim gets three lines:

```
set runtimepath^=~/vim runtimepath+=~/vim/after
let &packpath = &runtimepath
source ~/.vimrc
```

That is the standard compatibility shim, symlinked into `~/config/nvim` by the orchestrating installer. It means there is exactly one editor configuration in this project and neovim is a second front end onto it, which is why the vim configuration contains `has('nvim')` guards rather than a separate neovim configuration containing shared includes.

The consequence shows up in the plugin list: `Shougo/deoplete.nvim`, `roxma/nvim-yarp` and `roxma/vim-hug-neovim-ipc` are all in the manifest, which is the set of packages needed to run a neovim-native completion engine under vim. The configuration does not choose between the two editors; it carries the compatibility shims for both directions.

The Snippet And Tag Layers

Snippets are not managed by any of the four mechanisms. The installer symlinks the repository's own directory into place:

```
ln -sfn $ZPWR_INSTALL/UltiSnips $HOME/.vim/UltiSnips
```

Seven files, 358 lines total, and the largest is `sh.snippets` at 141 lines — which is the expected shape for a shell-first environment. The snippet engine itself, `SirVer/ultisnips`, arrives through both Vundle and the manifest, and the snippets arrive through neither.

The tag browser is vendored rather than cloned. `install/taglist_46` is a copy of taglist 4.6, dated February 2013 in its own header, and the vim installer copies the directory into the bundle directory after the clone loop:

```
zpwrInstallGitHubPluginsFromFile "$ZPWR_INSTALL/.vimbundle"

echo "Installing Taglist Plugin"
cp -R "$ZPWR_INSTALL/taglist_46" .
```

That is the correct handling for a plugin whose canonical distribution is a tarball on a script archive rather than a git repository, and it is the only plugin in the tree treated that way. The cost is that the vendored copy is pinned forever, which for a plugin whose last release was in 2013 is not a cost at all.

Two Ways To Get YouCompleteMe

`ycm-core/YouCompleteMe` is declared three times: in `.vimbundle:4`, in `.vimrc:138` as a Vundle plugin, and in `install/ycm_install.sh`, which clones it a fourth time and then does the part the others cannot:

```
git clone https://github.com/ycm-core/YouCompleteMe.git
if builtin cd "$HOME/.vim/bundle/YouCompleteMe"; then
  git submodule update --init --recursive
  #need greater than 3GB RAM for compiling
  YCM_CORES=1 python3 install.py --all
```

`YCM_CORES=1` serialises the compile, which is the standard workaround for building the completion engine on a machine with limited memory, and the comment records why. The separate script exists because this is the only plugin in the list that has a build step; the clone loop can only clone.

The same directory-with-a-build-step pattern appears twice more in the install directory, for editors rather than plugins. `vim_install.sh` builds vim from source with a fixed configure line, and it carries a path that has aged:

```
--with-python3-config-dir=/usr/lib/python3.5/config
```

A pinned interpreter version in a configure flag is a dated artifact in the literal sense — it names a Python that a current distribution does not ship — and it is the reason that script is a manual escape hatch rather than something the main installer calls. `neovim_install.sh` is the same idea with a shorter configure step and no pinned paths, which is why it has aged better.

The One Language Integration

`install/stryke.vim` is 63 lines and is the only file in the editor layer that teaches vim about a language rather than about a plugin. It is sourced conditionally from the bottom of the configuration:

```
if executable('stryke') && filereadable(expand('~/.zpwir/install/stryke.vim'))
  source ~/.zpwir/install/stryke.vim
```

Both halves of the guard matter. The interpreter must be installed, because every integration in the file shells out to it, and the file must be present, because the configuration is deployed to machines where the framework directory may not be at the assumed path.

The file registers the same language three times for three different consumers. For syntax, it declares a new filetype and then borrows another language's highlighting:

```
autocmd BufNewFile,BufRead *.stk set filetype=stryke
autocmd BufReadPost *.stk set filetype=stryke
autocmd FileType stryke setlocal syntax=perl commentstring=#\ %s
```

Two autocommands set the same option on overlapping events, which is redundant rather than wrong. The `syntax=perl` line is the interesting one: the filetype is new, so no highlighting exists for it, and rather than ship a syntax file the integration points at the syntax of the language this one extends. Every construct the two share highlights correctly and every construct they do not share highlights as whatever perl would make of it.

For diagnostics, it defines a linter callback that turns the interpreter's own error format into the editor's:

```
let l:match = matchlist(l:line, '\v^(.) at .+ line (\d+)')
if !empty(l:match)
  call add(l:output, {'lnum': l:match[2] + 0, 'text': l:match[1], 'type': 'E'})
```

That pattern — message, then `at`, then a file, then `line`, then digits — is the perl diagnostic format, which the interpreter inherits along with the syntax. The registration is wrapped in `if exists('*ale#linter#Define')` and called from `VimEnter`, so the file is safe to source in an editor without the linter framework installed.

For completion, it registers a language server, and this is where the file stops being defensive:

```
if exists('*lsp#register_server')
  call lsp#register_server({
    \   'name': 'stryke',
    \   'cmd': ['stryke', '--lsp'],
    \   'allowlist': ['stryke', 'perl'],
    \ })
```

The allowlist includes perl, which means opening any perl file in an editor with this loaded starts the new interpreter as that file's language server. That is a deliberate reach — the environment's position is that the newer language subsumes the older one — and it is the one place in the editor layer where a configuration decision has consequences outside its own filetype.

Chapter 73 – The Man Pages Nothing Installs

The tree ships two roff manuals: `man/man1/zpwr.1` at 8 KB and `man/man1/zpwrall.1` at 53 KB. They are hand-written, they are accurate to an extent that is unusual for hand-written documentation, and nothing in the project puts them anywhere the `man` command will look.

The Shape Of The Pair

The naming is a deliberate borrowing. `zsh` ships `zsh(1)` as an overview and `zshall(1)` as the concatenation of everything, and the second page here says so in its own overview:

```
This is the meta-manual for ZPWR \{em the equivalent of
.BR zshall (1)
for the
.B zpwr
environment.
```

The macro usage is conventional and narrow. Between them the two files use `.TH`, `.SH`, `.SS`, `.TP`, `.B`, `.I`, `.RI`, `.BR`, `.PP`, `.IP` and `.br`, and neither uses `.nf/.fi` — there are no preformatted blocks anywhere in either page, which is why the examples are written as indented `.TP` entries rather than as displayed code. The section counts tell you the shape at a glance:

	<code>zpwr.1</code>	<code>zpwrall.1</code>
<code>.SH</code> sections	13	22
<code>.SS</code> subsections	9	36
<code>.TP</code> entries	70	599
<code>.B</code> bolds	95	670
<code>.BR</code> bold-roman	43	101

Six hundred tagged paragraphs is the size of a reference table, and that is what the large page mostly is: one `.SS` per letter of the alphabet, and one `.TP` per verb underneath it.

The Table Is Correct

A hand-maintained enumeration of a table that lives in code is the kind of thing that is stale by definition, so the interesting question is how stale. The dispatcher's table is built from `ZPWR_VERBS[...]=` assignments in `env/zpwr.zsh`; the manual's table is built from `.TP` entries in its `VERBS` section. Extracting both and comparing sets:

```

assignment lines in env/zpwr.zsh      462
distinct keys                        460
keys assigned twice                  search, zshsearch

.TP verb entries in zpwrall.1         462
distinct entries                    460
entries appearing twice              search, zshsearch

documented but not in the dispatcher  (none)
in the dispatcher but undocumented   (none)

```

The two sets are identical, and the manual reproduces the source's own duplication: the two verbs that are assigned twice in the dispatcher are documented twice in the manual. Whatever process produced this page walked the source in order rather than walking a de-duplicated set, and it was applied recently enough that not one of four hundred sixty verbs has drifted.

The environment section is nearly as close. It has 94 tagged entries against 89 `export ZPWR_*` lines in `env/.zpwr_env.sh`; most of the difference is entries that document several related variables under one tag, plus a handful — `PATH`, `FPATH`, `MANPATH`, `EDITOR`, `HISTFILE` — that the framework sets without owning. Netting those out leaves exactly two exported variables with no entry of their own, `ZPWR_ZMAX` and `ZPWR_ZTMUX`, which are the indirection variables that let the editor and the multiplexer be swapped for wrappers.

The Prose Is Not

The correctness stops at the tables. The description in the short page opens with a sentence containing four numbers:

```

ZPWR ships 504 verbs at runtime (462 keys / 460 spellings from the in-repo
dispatcher table, the rest from the zinit-installed plugins), 2k+ aliases, 47k+
  ~ completions via
zsh-more-completions, and a

```

Three of those four are checkable against the same file the table was built from, and the parenthetical has its labels backwards. There are 462 assignment *lines* and 460 distinct *keys*; “462 keys / 460 spellings” describes neither quantity correctly, since 460 is the number of keys and 462 is the number of assignments. The distinct set of dispatch targets — the thing “spellings” might reasonably mean — is 373, because many verbs are synonyms pointing at one function.

The 504 is a different kind of claim. It counts verbs contributed by plugins that are installed at runtime, which is not a property of this repository at all, and cannot be verified from a checkout. The generator that builds the encyclopedia used to have this same problem and solved it by refusing to look outside the pinned source tree; the manual still has it.

The lesson is not that the number is wrong. It is that the table stayed correct because it is mechanical — an entry per key, checkable by set comparison — and the prose went wrong because a sentence with four numbers in it has four opportunities to age and no way to fail loudly when it does.

No Installer, No MANPATH

The pages are not reachable through `man` after an install. Nothing in the install directory runs `mandoc` or `mandb`, copies the pages into `/usr/local/share/man`, or adds the framework's own `man` directory to `MANPATH`. The only occurrence of either name under `install/` is `zpwrInstall.sh:222`, which lists `mandoc` among the packages to install on Linux — the renderer is put on the machine and then never pointed at these files. Every `MANPATH` assignment in the tree points somewhere else:


```
# env/.shell_aliases_functions.sh:96
export MANPATH=$HOME/perl5/man:$MANPATH
export
  MANPATH="$HOMEBREW_OPT_HOME/erlang/lib/erlang/man:$HOMEBREW_PREFIX/share/man:$MANPATH"
```

Two paths for two package ecosystems, and not the framework's own. The README tells the reader to do it by hand — “Add the directory to your `MANPATH` to view them with the system `man`” — which is honest and is also the reason the pages are effectively invisible. A person who reads the README far enough to find that sentence has already read the material the manual contains.

There is no generator either. Searching the tree for anything that writes roff finds one file, `scripts/mantozshcomp.py`, and it runs in the opposite direction: it *reads* manual pages to produce completion functions. The framework has a pipeline that turns other projects' documentation into shell behaviour and no pipeline that turns its own source into documentation. That asymmetry is the same one the encyclopedia chapters keep finding — generated artifacts are treated as disposable, and hand-written artifacts are treated as permanent — except that here the hand-written artifact is the one that has stayed accurate.

What Two Correct Tables Are Worth

It would be easy to file the pair under “documentation nobody reads”. The set comparison argues otherwise. Four hundred sixty verbs, documented with a description and a backing function name each, matching the dispatcher exactly, is a machine-checkable invariant that currently holds. Nothing enforces it; there is no test in the suite that compares the manual to `env/zpwr.zsh`. The comparison takes about fifteen lines of script, and the fact that it passes today is the argument for writing it: an invariant that holds by accident is one commit away from not holding, and the difference between the two states is currently invisible.

The same fifteen lines would have caught the prose numbers, if the prose stated anything a script could parse. It does not, which is the second argument for keeping counts out of sentences and in tables where a machine can read them.

Chapter 74 — Sixty Copies Of One Box

An earlier chapter took apart `scripts/boxPrint.pl`, the Perl program that draws the frame around every installer message. That program is not the only box drawer in the tree. It is not even the most-used one. The interactive functions draw their own boxes, in zsh, with a three-function helper that is defined separately inside every function that needs it.

The Census

Twenty files under `autoload/` define a function named `_bx`. All twenty definitions are byte-identical:

```
_bx() { local _r; _r=$(printf "%s" "$1" | command sed $'s/\033\\[[0-9;]*m//g'); printf "
  ${D}|${N}%s%s${D}|${N}\n" "$1" $( _bw - ${#_r} ) "" ; }
```

The same twenty files define `_bt` and `_bb` for the top and bottom rules, also byte-identical across all twenty:

```
_bt() { printf " ${D}|"; printf '-%.0s' {1..$_bw}; printf "]${N}\n"; }
_bb() { printf " ${D}|"; printf '-%.0s' {1..$_bw}; printf "]${N}\n"; }
```

Sixty definitions of three functions. There are 62 call sites of `_bx` across the twenty files, which is an average of three boxed lines per function — each copy of the helper exists to draw three or four lines.

The width is the only thing that varies, and it varies three ways:

<code>_bw=66</code>	11 functions	<code>zpwPorts, zpwFortune, zpwStartup, zpwGitwho,</code> <code>zpwWatch, zpwPathaudit, zpwDoctor, zpwLint,</code> <code>zpwStale, zpwTimeline, zpwResolve</code>
<code>_bw=70</code>	6 functions	<code>zpwDeps, zpwRestore, zpwReplay, zpwSnapshot,</code> <code>zpwFlame, zpwTrace</code>
<code>_bw=78</code>	3 functions	<code>zpwFuncRank, zpwAliasRank, zpwBench</code>

Three widths is not an accident of copying; it is a deliberate spread. The 78-wide boxes belong to the functions that print ranked tables with a numeric column and a bar; the 66-wide ones belong to functions that print short status lines. The width was chosen per function and the helper was copied along with it.

Why The Helper Exists At All

The problem `_bx` solves is the one that makes box drawing hard in a coloured terminal. The padding has to be computed from the *visible* width of the content, and the content contains ANSI escape sequences that occupy zero columns but several characters. A naive `${#string}` counts the escapes and the right-hand border drifts left by exactly the number of colour bytes in the line.

The helper's answer is to compute the padding from a stripped copy:

```
local _r
_r=$(printf "%s" "$1" | command sed $'s/\033\\[[0-9;]*m//g')
printf " ${D}|${N}%s%s${D}|${N}\n" "$1" $( _bw - ${#_r} ) ""
```

`_r` is the content with every `ESC[...m` sequence removed, `${#_r}` is therefore the visible width, and `%s` with a computed field width emits exactly the right number of spaces. The coloured string is what gets printed; the

stripped string is what gets measured. That separation is the whole trick, and it is correct.

It costs one process per line. `{ ... | command sed ... }` forks a subshell and execs a stream editor for every boxed line, so a function that prints an eight-line box spends eight forks on padding arithmetic. In an interactive diagnostic this is invisible against the cost of what the function is measuring. It is still the only piece of the framework's output path that shells out to do string arithmetic that zsh can do natively — the same substitution is available as `${1//${'\033'\[ [0-9;]#m/}}` with `extendedglob` set, in-process.

The Two Boxes That Are Not This Box

Two functions draw boxes without the helper, and both illustrate what the helper prevents.

`zpwriTop` inlines the same computation with a different name and a different width:

```
local -i box_inner=76
...
hud_plain=" LIVE // PID $$ // up ${up} // ${interval}s // #${iteration} // q quit "
pad_count=$(( box_inner - ${#hud_plain} ))
(( pad_count < 0 )) 66 pad_count=0
print -r " ${D}|${printf '-%.0s' {1..$box_inner}}|${N}${K}"
print -r " ${D}|${N} ${G}LIVE${N} ${D}/${N} ${C}PID ${N} ..."
```

This is the same idea done properly for a harder case: the plain string is built separately and by hand, so no stripping is needed and no process is forked. The cost is that `hud_plain` and the coloured line are two independent pieces of text that must be kept character-for-character in agreement forever. Change the interval label in one and the border moves in the other. It is the fastest of the three approaches and the most fragile, and it is used in the one place where the box is redrawn on a timer.

`zpwriMatrix` does the thing the helper exists to prevent:

```
# box inner width is 66; content: " ROUND N // NAME vs OPP" = 16 + digits + names
printf " ${D}|${N} ${G}ROUND %d${N} ${D}/${N} ${C}%s${N} vs ${R}%s${N}" $_bround
  < "$_bname" "$_bopponent"
printf "%*s${D}|${N}\n" $(( 50 - ${#_bname} - ${#_bopponent} - ${#_bround} )) "
```

The padding is a literal 50 minus three hand-counted lengths, and the comment above it is doing the work a function would otherwise do — it records the arithmetic that the reader would otherwise have to redo. The border is drawn by a literal run of box characters elsewhere in the file rather than by a loop over a width variable, so the 66 in the comment and the length of that literal are two independent facts that agree today.

What The Duplication Buys

The obvious criticism writes itself: put `_bx`, `_bt` and `_bb` in one file, autoload them, delete fifty-seven definitions. The tree already has the mechanism — every one of these functions is itself autoloaded from a one-function file — and the fix is smaller than this chapter.

The argument on the other side is visible in the last line of most of these functions:

```
unfunction _bx _bt _bb 2>/dev/null
```

Define the helpers on entry, remove them on exit, and they never exist in the interactive namespace between calls. A shared autoloaded `_bx` would be permanently present, would appear in completion of function names, would show up in `${#functions}` — which several of these very functions print as a measurement — and would be one more name in a global namespace that already holds several thousand.

Thirteen of the twenty do that. Seven do not: `zpwriFuncRank`, `zpwriFlame`, `zpwriWatch`, `zpwriAliasRank`, `zpwriPathaudit`, `zpwriDoctor` and `zpwriTrace` return without unsetting anything, so running any one of them leaves `_bx`, `_bt` and `_bb` defined in the shell for the rest of the session — with that function's `_bw` gone with it,

since that one *is* declared local. A leaked `_bx` therefore computes `$(( _bw - ${#_r} ))` against an unset variable, which is negative, and `printf` reads a negative field width as left-justification — so the line gets padded by the length of its own content instead of out to a fixed column. The hygiene argument for duplicating the helper is real, and it is only being followed two thirds of the time, which is the usual fate of a convention that lives in twenty copies instead of one file.

A shared file would not have prevented the cleanup either way — it could define the three helpers and let each caller still `unfunction` them — so the reason the code looks like this is more likely that each of these functions was written by copying the previous one, which is also why the widths cluster into three values rather than twenty.

The measurable consequence of the current arrangement is small and worth stating precisely: a change to the padding algorithm has to be applied twenty times, and a change applied nineteen times produces a single function whose right border is one column off, in a program whose output nobody diffs. That is the specific failure mode duplication of formatting code produces — not breakage, but a cosmetic drift that no test will catch and that only shows up when two boxes are on screen at once.

Chapter 75 – The Instrument Panel

Six of the autoloaded functions exist to measure the shell rather than to do anything with it. They are the largest cluster of code in the tree with no external dependency and no side effect, and each one answers a different question about the environment it is running inside.

verb	function	question answered
top	zpwrTop	what does the shell contain right now
trace	zpwrTrace	what did this command cost
flame	zpwrFlame	which functions dominate startup
bench	zpwrBench	how long does startup take, over N runs
funcrank	zpwrFuncRank	which commands and verbs get used
deps	zpwrDeps	which functions call which

Every one is registered in `env/zpwr.zsh` as a verb, several under two spellings; `rank` and `funcrank` reach the same function, as do `aliasrank` and `aliasusage`.

Counting Without Forking

`zpwrTop` is a `top` for shell state. It redraws on an interval — two seconds by default — and every field it prints is a count of something the shell already knows:

```
nhooks=$(( ${#precmd_functions} + ${#preexec_functions} + ${#chpwd_functions} ))
nzf=$(( ${#functions}[(I)(#i)*zpwr*] ))
nza=$(( ${#aliases}[(R)(#i)*zpwr*] ))
nzv=$(( ${#ZPWR_VARS} )) nzverbs=$(( ${#ZPWR_VERBS} ))
nzc=$(( ${#_comps}[(I)(#i)*zpwr*] ))
```

`(I)` matches subscript keys against a pattern and `(R)` matches values; `(#i)` makes the match case-insensitive. So `nzf` is “how many function names contain `zpwr`” and `nza` is “how many alias *bodies* contain `zpwr`” — two different questions with nearly identical spellings, which is a fair summary of why these subscript flags are worth knowing.

The loop is explicit about avoiding forks, and the comments say so:

```
# use zsh builtins instead of forking to zmodload/setopt commands
mod_out=( "${(@k)modules}[(R)loaded]" )
opts_out=( "${(@k)options}[(R)on]" )
```

`$modules` and `$options` are associative arrays exported by `zsh/parameter`; selecting the keys whose value is `loaded` or `on` gives the same lists that `zmodload` and `setopt` would print, without running either. The same section uses `zstat` for the history file’s byte size rather than `wc -c`. Three external commands remain in the loop — `ps` for the process’s own memory, `wc -l` for the history line count, `pgrep` for the child count — and each is there because `zsh` has no builtin for it.

The display carries deltas. Seven `prev_*` integers hold the previous iteration’s counts, and a difference is only printed when it is non-zero:

```
if (( iteration > 1 )); then
    (( nf != prev_funcs )) && df=" ${Y}${(nf-prev_funcs)}${N}"
```

That is what makes the dashboard worth watching rather than glancing at. A number that never changes is

noise; the yellow delta appears exactly when a plugin finished loading in the background, when a completion function got defined on first use, or when something added to `$path`. It turns a static inventory into an instrument.

Two And A Half Passes To Measure One Command

`zpwTrace` runs a command and reports what it cost. The flags decide how many times the command runs:

```
-q|--quick) quick_mode=1; do_strace=0; do_ltrace=0
-s|--strace) do_strace=1; do_ltrace=0
-l|--ltrace) do_strace=0; do_ltrace=1
-a|--all) do_strace=1; do_ltrace=1
```

and the help text states the consequence plainly rather than burying it:

```
on linux, command re-runs for strace/ltrace stats (2-3x total)
use -q for destructive commands to run once with timing only
```

This is the honest way to ship a tool whose default mode runs the user's command up to three times. The alternative — attaching both tracers to one run — costs accuracy, because the timing measured under `ltrace` is not the timing of the program. The tool measures the untraced run for timing and the traced runs for counts, and tells you it is doing so.

The platform split is a genuine capability difference, not a portability convenience:

```
macOS    resource stats from /usr/bin/time -l          // SIP blocks dtrace
linux    strace -c (syscalls) + ltrace -c (C library) // full spectrum
```

On the platform where the syscall tracer requires disabling a system security feature, the tool does not offer it; it falls back to the resource accounting the kernel will hand over to any process. Everything else — wall time, exit code, memory delta, child process count — is computed by the function itself from `$EPOCHREALTIME`, `ps` and `pgrep` before and after, and is therefore identical on both platforms.

Profiling A Shell From Another Shell

`zpwFlame` renders the shell's own profiler as a bar chart, and its first act is to start a second shell:

```
local tmpfile=$(mktemp)
ZPWR_PROFILING=true ZPWR_BANNER_CLEARLIST=false zsh -ic "zprof >| $tmpfile 2>&1; exit"
↵  ↵>/dev/null
```

This is the only way to get the numbers. `zprof` reports on the shell that loaded it, and the interesting part of startup is over before an interactive user could type anything. Setting the profiling variable in the environment of a fresh interactive shell makes that shell load the profiler module during its own init, and `-i` is what makes it read the interactive configuration at all. The second variable suppresses a banner that would otherwise land in the captured output.

The parser that follows is a plain line reader over `zprof`'s columnar output, accumulating microsecond totals and scaling bars against the largest. There is no dependency on the profiler's exact column widths beyond field splitting, which matters because that output format is not a documented interface.

Percentiles From Ten Samples

`zpwBench` runs a fresh shell *N* times — ten by default — and reports distribution rather than a single number. The captured values come from the framework's own phase timestamps rather than from an external clock:

```

script -q /dev/null zsh -ic "
{
    printf '%s %s ' "\"$ZPWR_VARS[promptTimeMs]\" \"\"$ZPWR_VARS[startupTimeMs]\" \"
    printf '%s %s %s %s %s ' \"
        \"$ZPWR_VARS[startTimestamp]\" \"
        \"$ZPWR_VARS[phaseEnvTs]\" \"
        \"$ZPWR_VARS[phasePluginsTs]\" \"
        \"$ZPWR_VARS[phaseCompsysTs]\" \"
        \"$ZPWR_VARS[phaseOptionsTs]\" \"
    printf '%s' "\"$ZPWR_VARS[phaseFinishTs]\" \"
} >| $tmpfile
exit
" &>/dev/null

```

`script -q /dev/null` is there to give the child a pseudo-terminal. Without one, the prompt subsystem takes a different path and the numbers would describe a shell nobody uses. The five phase timestamps are set by assignments scattered through `.zshrc` — `ZPWR_VARS[phasePluginsTs]=$EPOCHREALTIME` sits immediately before the plugin manager block — so the benchmark's phase breakdown is not an estimate; it is the configuration reporting on itself.

The statistics are computed from a sorted copy, `${(on)startup_times}`, which sorts numerically. Two of the reported figures are the same figure:

```

(( p50_idx = (runs + 1) / 2 ))
(( p99_idx = runs ))
p99_startup=$sorted_startup[$p99_idx]
min_startup=$sorted_startup[1]
max_startup=$sorted_startup[$runs]

```

`p99` is defined as the last element, which is the maximum, and `max` is printed on the next line from the same index. At ten samples there is no other honest choice — the ninety-ninth percentile of ten observations is the largest one — but the two labels invite a reader to treat them as independent measurements when they are the same number. The median index is integer division of $(runs+1)/2$, which for ten samples selects the fifth of ten rather than interpolating between the fifth and sixth. Both are the standard cheap approximations, and both are more defensible than reporting a mean alone, which is what the tool's absence would leave a person doing.

Mining The History File

`zpwFuncRank` answers “what do you actually run” by parsing the history file, which is the only data source in the tree that records behaviour rather than configuration. The parse is one pass with two extractions:

```

[[ "$line" == ": *" ]] && line="${line#*;} "
word="${line%%[:space:]}*"
word="${word%%[:|&]*}"
[[ -z "$word" ]] && continue
[[ "$word" != [[:alnum:]]_./-]## ]] && continue
(( cmd_counts[$word]++ ))
if [[ "$word" == "zpw" ]]; then
    verb="${line#zpw}"
    verb="${verb%%[:space:]}#"
    verb="${verb%%[:space:]}*"
    [[ -n "$verb" && "$verb" == [[:alnum:]]_./-]## ]] && (( verb_counts[$verb]++ ))
fi

```

The first line strips zsh's extended-history prefix — `: <timestamp>:<elapsed>;` — so the same code handles both history formats. `##` after a bracket expression is the zsh pattern for one-or-more, so the two guards reject anything that is not a plausible command or verb name, which is what keeps pipelines and

redirections out of the counts. When the command is the dispatcher, the second word is counted separately, which is the only way to learn which verbs are used; the dispatcher is one entry in the command histogram no matter how many verbs it serves.

The result is cached, and the cache invalidation is a watermark rather than a timestamp:

```
cached_hist_lines=$(<"$stamp_file")
hist_wc=$(command wc -l < "$HISTFILE" 2>/dev/null)
current_hist_lines=${hist_wc//[0-9]/}
if (( current_hist_lines - cached_hist_lines < 1000 )); then
    need_rebuild=0
fi
```

Rebuild when the history has grown by a thousand lines. For a file that only ever grows, line count is a better staleness signal than modification time — the file's mtime changes on every command, and its content changes meaningfully only in bulk. The threshold means the rankings can be up to a thousand commands behind, which for a histogram over tens of thousands of entries is a difference that does not change the order of the top twenty.

Reading The Call Graph From Disk

`zpwDep`s builds a function call graph, and it builds it from files rather than from the running shell — which is the only way to see functions that have not been autoloaded yet:

```
scan_dirs=(
[[ -d "$ZPWR_AUTOLOAD_COMMON" ]] 66 scan_dirs+=( "$ZPWR_AUTOLOAD_COMMON" )
[[ -d "$ZPWR_AUTOLOAD_DARWIN" ]] 66 scan_dirs+=( "$ZPWR_AUTOLOAD_DARWIN" )
[[ -d "$ZPWR_AUTOLOAD_LINUX" ]] 66 scan_dirs+=( "$ZPWR_AUTOLOAD_LINUX" )
[[ -d "$ZPWR_AUTOLOAD_COMP_UTILS" ]] 66 scan_dirs+=( "$ZPWR_AUTOLOAD_COMP_UTILS" )
```

One file per function is what makes this cheap: the file name is the function name, so the node set of the graph is a directory listing. Only the edges require reading bodies. The shell library is handled as a special case, because it is the one place where many functions share a file, and its bodies are extracted by range:

```
body=$(sed -n "/^function $fname/,/^}/p" "${func_file[$fname]}" 2>/dev/null)
```

A line-range extraction bounded by a closing brace in column one — which works because every function in that file is written with its brace in column one, and which would silently truncate the first time somebody indents one. That is the recurring trade in this cluster of functions: they parse shell with patterns instead of a parser, they are correct against the tree's own conventions, and each of them documents a convention that is now load-bearing.

The four modes — summary, reverse, orphans, dot — come from four flags, and the default with no arguments is summary. The orphan mode is the one with teeth: a function in the autoload directories that nothing else calls and no verb dispatches to is either an entry point or dead, and the graph cannot tell which. Nothing in the tree removes what it reports, which is the correct amount of automation for a heuristic.

Chapter 16 – Capturing A Session And Replaying It

Three verbs try to put a working environment back the way it was: `snapshot` writes one out, `restore` reads one in, and `tmuxsave` writes a script that rebuilds a multiplexer layout. The three take three different approaches to the same problem, and the differences are instructive.

Ten Components Out

`zpwrsnapshot` writes a directory under `$ZPWR_LOCAL/snapshots/<name>`, with a manifest and one file per captured component. It captures ten things, numbered in the source, and the interesting property is that most of them are captured as *executable restoration commands* rather than as data.

Aliases are the clearest case:

```
for k v in ${(kv)aliases}; do
    printf "alias %s=%s\n" "${(q)k}" "${(q)v}"
done
echo "# global aliases"
for k v in ${(kv)galiases}; do
    printf "alias -g %s=%s\n" "${(q)k}" "${(q)v}"
done
echo "# suffix aliases"
for k v in ${(kv)saliases}; do
    printf "alias -s %s=%s\n" "${(q)k}" "${(q)v}"
done
```

Three separate tables — regular, global, suffix — each written with the flag that recreates its kind, and every name and body passed through `${(q)}` so the file is safe to source. Restoring is then `source aliases.zsh` and nothing else. The same shape is used for the named-directory table, which is captured with `hash -d` and rewritten line by line into `hash -d k=v` statements, because the builtin's output format is not its input format.

The environment capture is the one place where the file's content is filtered:

```
for k v in ${(kv)parameters[(R)scalar-export]}; do
    # skip sensitive vars
    [[ $k == (PASSWORD|SECRET|TOKEN|AWS_SECRET*|GITHUB_TOKEN) ]] && continue
    printf "export %s=%s\n" "$k" "${(q)v}"
done
```

`${(kv)parameters[(R)scalar-export]}` selects exactly the parameters whose type is an exported scalar, which is a precise way to say “the environment” without shelling out to `env`. The filter that follows is not precise. Four of the five patterns are exact names, so a variable called `PASSWORD` is skipped and one called `DB_PASSWORD` is written; only `AWS_SECRET*` has a wildcard. A snapshot directory is therefore a plaintext copy of the exported environment minus five specific spellings, and the comment above it promises more than the pattern delivers. Widening it to `(#i)*(password|secret|token|key|passwd)*` is a one-line change; the value of writing the current behaviour down is that “skip sensitive vars” reads as though it were already done.

Two of the ten components are captured as measurements rather than as state. Functions are counted, not written:

```
for k in ${(k)functions}; do
    [[ $k == (*|prompt_*|compdef) ]] && continue
    [[ ${functions[$k]} == *"builtin autoload"* ]] && continue
    (( fcount++ ))
done
echo "$fcount" >| "$snapdir/function-count.txt"
```

The two skips are correct and worth reading: completion and prompt functions are excluded by name, and autoload stubs are excluded by *body*, because an un-called autoloaded function's body is the one-line stub that loads it. What remains is the count of functions that have really been defined. Writing their bodies would make the snapshot a shell of its own; counting them makes it a fingerprint. Git state is captured the same way — branch, short commit, count of dirty paths — which is enough to tell you whether a restored session matches the tree it was taken from, and not enough to restore anything.

Seven Components Back

`zpwrRestore` handles a component list, defaulting to all of them:

```
if (( $# == 0 )) || [[ "$1" == "all" ]]; then
    components=( hash aliases env tmux vim history dirs )
else
    components=( "$@" )
fi
```

Seven, against the snapshot's ten. The three that are captured and never restored are the ones that were captured as measurements — the function count, the git status — plus the framework's own variable table, which is written to `zpwr-vars.zsh` and never read back. The asymmetry is right: a saved `ZPWR_VARS` contains that session's start timestamp and phase timings, and restoring them into a new shell would corrupt every measurement the previous chapter's tools take.

The multiplexer branch is the one that cannot restore directly, and it says so:

```
command cp "$target/tmux/resurrect.txt" "$resurrect_dir/last"
ln -sf "$resurrect_dir/last" "$resurrect_dir/last"
printf "${G}resurrect file staged${N} ${D}// press prefix+Ctrl-r to restore${N}\n"
```

Staging a file and telling the user which keystroke completes the operation is the honest interface for a restore that belongs to a plugin, because that plugin owns the running server's state and driving it from outside would race with it. The middle line is a no-op: it links a path to itself, which `ln -sf` accepts and which leaves the regular file exactly as `cp` left it. It appears to be a leftover from a version where the destination and the link had different names.

A Layout Is A Program

`zpwrTmuxSave` takes the third approach: rather than serialise state, it writes a shell script that recreates it. The output begins

```
print "#!/usr/bin/env zsh"
print "# zpwr tmux layout: ${layout_name}"
print "# saved: $(date '+%Y-%m-%d %H:%M:%S')"
print "# session: ${session_name}"
print "# windows: ${#win_indices}"
print ""
print "local _sess=\"\${1:-${layout_name}}\""
```

and continues with one command per structural element: `new-session` for the first window, `new-window` for each subsequent one, `split-window` per extra pane, `select-layout` with the captured geometry string,

`select-pane` and `select-window` to restore focus, and an attach-or-switch tail. The generated file takes an optional argument, so a saved layout can be replayed under a different session name — which is the difference between a backup and a template.

The capture reads five formatted lists out of the server, tab-separated and parsed by repeated prefix stripping:

```
while IFS=$'\t' read -r wline; do
  idx="${wline%% *}" ; wline="${wline#* }"
  wname="${wline%% *}" ; wline="${wline#* }"
  wlayout="${wline%% *}" ; wline="${wline#* }"
  wact="${wline}"
```

`IFS=$'\t' read -r wline` into a *single* variable does not split — it reads the whole line — and the splitting is then done by hand, four fields at a time. That is more verbose than reading into four variables, and it is deliberate: window names can contain almost anything, and doing the splits explicitly makes the last field a remainder rather than a field, so a name containing a tab cannot shift the columns.

The one genuinely hard part is restoring what was *running* in each pane, and the function reaches through the process tree to find it:

```
child_pid=$(pgrep -P "${pane_pids[$pi]}" 2>/dev/null | head -1)
if [[ -n "$child_pid" ]]; then
  child_args=$(ps -o args= -p "$child_pid" 2>/dev/null)
fi
[[ -z "$child_args" ]] && child_args=$(ps -o args= -p "${pane_pids[$pi]}" 2>/dev/null)
```

The pane's own process is a shell; what the user is looking at is that shell's child. So the code asks for the first child, falls back to the pane process itself, strips a macOS clipboard wrapper if one is present, and then refuses to emit anything that still looks like a shell:

```
child_args="${child_args#reattach-to-user-namespace -l }"
if [[ "$child_args" != */zsh && "$child_args" != */bash ]]; then
  print "$ZPWR_ZTMUX send-keys -t \"\$_sess:${widx}.${(pi-1)}\" \"${(q)child_args} Enter"
fi
```

Two filters for the same mistake, because emitting `send-keys ... /bin/zsh Enter` would nest a shell inside every restored pane. The command line is quoted with `${(q)}` before being embedded, so the generated script survives arguments with spaces.

Every command in the generated file is written as `$ZPWR_ZTMUX` — expanded at generation time, not at replay time, because the surrounding string is double-quoted. That variable is the multiplexer indirection: `env/.zpwir_env.sh` sets it to `tmux`, and one of the alias files overrides it to `ztmux` when a wrapper is present. Baking the resolved name into the script means a layout saved on a machine with the wrapper will call the wrapper on replay, whether or not that machine has it. Writing the variable reference instead would defer the choice to replay time; expanding it pins the layout to the environment that produced it. Neither is wrong, and the file's own header does not say which one was intended.

What The Three Have In Common

All three functions face the same problem — a running session is a graph of state owned by several programs — and each picks a different point on the same axis. The snapshot writes sourceable commands for the state the shell owns outright. The restore delegates to the owning plugin for state it does not own. The layout saver generates a program for state that has structure but no serialisation format of its own.

The one thing none of them attempts is the shell's own function definitions, scrollbar, or job table, and the reason is the same in each case: those are either enormous, or belong to a process that will not exist after restore. A session restore tool is defined as much by what it refuses to capture as by what it captures, and this one draws the line where the data stops being reproducible.

Chapter 77 – One Hundred Sixty-Four zstyles

The completion system is configured by exactly one function. `zpwBindZstyle` is 390 lines and issues 164 live `zstyle` calls — 163 of them in the `:completion:` namespace and one in `:fzf-tab:`, with six more commented out: five written `#zstyle` and one `# zstyle`. Nothing else in the tree configures completion at this scale; the plugin bindings add a handful each, and the shell configuration adds none. Reading this function is reading the entire completion policy of the environment.

Three Caching Policies And A Directory

The function opens by defining three nested functions, not by setting a style:

```
zpwMonthlyCachingPolicy () {  
    # rebuild if cache is more than a month old  
    local -a oldp  
    oldp=( "$1"(Nm+31) )  
    (( $#oldp ))  
}
```

`(Nm+31)` is a glob qualifier: `m+31` matches files modified more than 31 days ago and `N` makes a non-match expand to nothing instead of erroring. So the array is either empty or holds the one file it was given, and the arithmetic test returns true exactly when the cache is stale. Three of these are defined — monthly, weekly, daily — and they differ only in the number.

They are then attached to contexts at three granularities:

```
zstyle ':completion:*' cache-policy zpwMonthlyCachingPolicy  
zstyle ':completion*:({fasd|fasd-file|zdir})' cache-policy zpwDailyCachingPolicy  
zstyle ':completion*:({f|z|zshz|zpw-r-z|...|zm|zd|zg}):*:*' cache-policy  
    ~ zpwDailyCachingPolicy  
zstyle ':completion:daily-cache:*' cache-policy zpwDailyCachingPolicy
```

The default is a month; anything that completes *directories you have recently visited* gets a day. That is the correct split. A month-old list of installed packages is still nearly right; a month-old list of recent directories is useless, because the entire value of the data is its recency.

The weekly policy is defined and never attached to anything. It costs three lines and exists as a slot for a context that has not needed it yet.

Ordering Is Where The Design Lives

The single most consequential call in the file is `group-order`, which lists sixty-two tag names in the order their groups should appear in a completion list. The first twenty are private:

```

zstyle ':completion:*' group-order \
    zpwr-regen \
    zpwr-clean \
    zpwr-travis \
    zpwr-learn \
    zpwr-search \
    ...
    zpwr-monitor \
    c options commands aliases alias global-aliases \
    suffix-aliases functions builtins reserved-words parameters argument-rest \
    ...

```

Twenty `zpwr-*` tags, then the standard shell tags, then version-control tags, then package-manager tags, then files. The comment above it explains the trick — “using tag name as group name so ordering the groups by tag name here” — which is the mechanism that makes this possible at all: the completion functions in `autoload/comps` label their matches with private tag names, and because tags double as group names, one `group-order` call sorts categories that the standard completion system has no concept of.

The ordering is an opinion about what the user is most likely to want, and it puts the framework’s own verb categories above everything the shell offers. That is defensible in an environment where the dispatcher is the most-used command, and it is the kind of decision that is invisible until you compare a completion list here against one from a stock shell.

Matching Is Four Attempts, Not One

```

# 0 -- vanilla completion (abc => abc)
# 1 -- smart case completion (abc => Abc)
# 2 -- word flex completion (abc => A-big-Car)
# 3 -- full flex completion (abc => ABraCadabra)
zstyle ':completion:*' matcher-list '' \
    'm:{a-z\-=}{A-Z\_}' 'r:[^[:alpha:]]|[[[:alpha:]]]=** r:|=* m:{a-z\-=}{A-Z\_}' 'r:|?=**
    ~ m:{a-z\-=}{A-Z\_}'

```

`matcher-list` is tried in order and stops at the first entry that produces matches. The empty first entry means exact matching gets first refusal, so nothing fuzzy can shadow a literal match. The second maps lowercase to uppercase *and* hyphen to underscore, which is one specification handling two conventions at once. The third and fourth add anchored and unanchored partial-word matching.

The four-line comment above it is doing real work — it gives an example of what each level accepts — and it is the sort of comment that survives because the syntax it explains is unreadable without one.

One Hundred Fifteen Colours

Seventy percent of the calls in the file — 115 of 164 — set `list-colors`. They are all the same shape:

```

zstyle ':completion:*:*:*:aliases' list-colors '=(#b)(*)=1;30=34;42;4'

```

`(#b)` turns on backreferences, `(*)` captures the whole match, and the two trailing fields give the SGR sequence for the unmatched and matched portions. In practice this paints a whole category one colour pair: dim grey for the part the list is not highlighting, then a specific foreground/background combination for the entry itself. Twenty-five distinct SGR combinations appear across the 115 lines, which means the colour scheme is a palette of twenty-five, reused.

The categories run further than a reader would guess. Thirteen of the lines are for manual page sections alone:

```

zstyle ':completion:*:*:*:manuals.1' list-colors '=(#b)(*)=1;30=1;36;44'
zstyle ':completion:*:*:*:manuals.2' list-colors '=(#b)(*)=1;30=1;37;42'
...
zstyle ':completion:*:*:*:manuals.3p' list-colors '=(#b)(*)=1;30=37;46'

```

Section one gets one colour and section three another, so completing an argument to `man` shows at a glance whether a name is a command, a syscall, or a library function. That is a real affordance built entirely out of style declarations — no completion function was modified to produce it.

Seventeen more lines colour version-control categories individually: commits, heads, local heads, remote heads, tags, trees, blobs, blob objects, commit objects, modified files, cached files, revisions, recent branches. The completion functions for the version-control system already emit those tags; the only thing missing was that they all looked alike.

Exactly one of the 115 lines is a duplicate — `global-aliases` is set twice, identically, three lines apart. The second call overwrites the first with the same value, which is harmless and is the expected residue of a list this long maintained by hand.

Everything Colour-Related Is Behind A Flag

The colour block is not unconditional:

```

if [[ $ZPWR_COLORS == true ]]; then

```

and the twenty verb-category colours are behind a second, narrower one:

```

if [[ $ZPWR_COLORS_VERBS == true ]]; then

```

Two flags at two granularities, so a user can keep completion colours in general and turn off the aggressive per-verb backgrounds specifically. The same two-level structure appears earlier in the file for the prompt and format strings, where an entire alternative set of `list-prompt`, `select-prompt` and `format` styles is defined for the colourless case rather than the colours being stripped from one shared string.

That duplication — two full sets of format strings rather than one set plus a colour variable — is the file's largest maintenance liability, and it is also the only arrangement that lets the two cases have *different text*, which they do.

The Data That Is Not Static

Two blocks in the file compute their values rather than declaring them. The host list is parsed out of the ssh configuration:

```

if [[ -r "$HOME/.ssh/config" ]]; then
    local _ssh_cfg
    _ssh_cfg="$(< /dev/null .ssh/config)"
    ZPWR_HOSTS=(
        ${${@}[@]}%${_ssh_cfg}:#Host *}%Host }:#*[*?]*}
        ${${@}[@]}%${_ssh_cfg}:#Hostname *}%Hostname }:#*[*?]*}
    )
fi

```

Read innermost-out: `(f)` splits the file into lines, `(@M)...:#Host *` keeps only the lines that match `Host *`, `#Host` strips the keyword from each, and the final `:#*[*?]*` removes any entry containing a glob character — because `Host *.internal` is a pattern, not a hostname, and offering it as a completion would be offering something that cannot be connected to. Four parameter expansions, one pass, no external commands, and a comment above it noting that the file is read once into a variable instead of twice.

The result is attached only if it is non-empty:

```
if (( $#ZPWR_HOSTS )); then
    zstyle ':completion:*:("'|')ssh("|'|'):*' hosts $ZPWR_HOSTS
```

The context pattern `("'|')ssh("|'|')` matches the command whether it was typed bare, quoted, or single-quoted, which is the standard idiom for making a command-specific style survive quoting.

One Line For The Picker

The last `zstyle` in the file is the only one outside the completion namespace:

```
zstyle ':fzf-tab:*' command ${FZF_TAB_COMMAND:-fzf} $FZF_TAB_OPTS
```

One line to point the interactive completion picker at a binary and a set of options, both overridable from the environment. The whole of the rest of the file describes what the completion system produces; this line describes how it is displayed. The ratio — 170 lines of policy to one line of presentation — is the right way round, and it is why swapping the picker for another one changes nothing about the grouping, the ordering, or the colours.

Chapter 78 – The Palette Is A Permutation

The install directory ships colour configuration for six programs that are not the shell: two terminal emulators, one editor from a different vendor, a process viewer, a network monitor, and a statusline generator. Reading them together answers a question the shell configuration cannot: when a script prints “colour 2”, what colour appears?

What Ships

file	lines	program
install/jm.itermcolors	344	iTerm2 (Apple plist)
install/ProJMTerminal.terminal	1,417	Apple Terminal (plist, binary blobs)
install/Material_jm.icls	58,328	IntelliJ IDEA colour scheme
install/alacritty.toml	676	Alacritty
install/htoprc	51	htop
install/.iftopcolors	19	iftop
install/.config/powerline/	4 files	powerline (colors, scheme, theme, config)

The IntelliJ scheme is the largest single file in the repository that is not a binary or an image. Fifty-eight thousand lines of XML is what a colour scheme costs when every language’s every token type gets an entry, and it is included for the same reason the vim configuration is: the environment is one configuration deployed to whatever tools are on the machine.

Of these, only three are installed automatically. `.iftopcolors` is in the symlink list of the orchestrating installer, `htoprc` is copied — not linked, because htop rewrites its own file when settings change in its interface — and the powerline directory is copied into the local directory and then symlinked into place. The two terminal palettes and the editor scheme are imported by hand through the respective program’s preferences, which is the only way those programs accept them.

The Sixteen Slots Do Not Mean What They Are Named

The iTerm2 file stores each channel as a float. Converting all sixteen ANSI entries to hex:

slot	name in the format	value	what it looks like
0	black	#000000	black
1	red	#BB2404	red
2	green	#E417F7	magenta
3	yellow	#5CBDAC	teal
4	blue	#1300FC	blue
5	magenta	#44ABBD	cyan
6	cyan	#38BEA1	green-cyan
7	white	#5D9ABB	steel blue
8	bright black	#FB002F	red
9	bright red	#28F1F8	cyan
10	bright green	#FB15CB	pink
11	bright yellow	#67FCD6	mint
12	bright blue	#8200F8	violet
13	bright magenta	#FB2695	pink
14	bright cyan	#5C95FF	blue
15	bright white	#FFFFFF	white

Four of the sixteen hold the colour their slot is named after. The rest are a permutation, and a deliberate one: the palette is built out of cyan, magenta, violet and mint, and the slot names were treated as indices into that palette rather than as requests for particular hues.

The Alacritty configuration does the same thing with different results:

```
[colors.normal]
black = "0x000000"
blue = "0x1036F5"
cyan = "0x4025F5"
green = "0x76F950"
magenta = "0x9E32F5"
red = "0x66AAF9"
white = "0xd0d0d0"
yellow = "0xE73DD7"
```

Here **red** is a pale blue, **yellow** is pink, and **cyan** is a blue-violet. Green is the one slot the two files agree is roughly green — **0x76F950** against **#E417F7** in slot 2 of the other file, which is not agreement at all.

The two palettes are not translations of each other. A session in one terminal and a session in the other, running the same shell configuration, produce different colours for the same escape sequence. That is not a bug in either file; it is what the colour system actually is. **ESC[32m** does not request green, it requests slot 2, and every program in this book that prints **\$'\033[32m'** — the box helpers, the instrument panel, the dispatcher's own verb listing — is asking for whichever colour that terminal's palette has in that position.

The primary colours are where the intent shows through unambiguously, because they are not slots:

```
[colors.primary]
background = "0x000000"
bright_foreground = "0x76F952"
foreground = "0x00F787"
```

Black background, green-cyan foreground. The iTerm file makes the same choice: background **#000000**, foreground **#23FF23**. Both are the classic phosphor terminal, and the sixteen slots above them are a decorative palette scattered over it.

Two More Colour Systems That Are Not Slots

The framework also sets file-listing colours directly, and the two variables it sets take different formats. From **env/.zpwir_re_env.sh**:

```
if [[ $ZPWR_COLORS = true ]]; then
    export LS_COLORS="fi=38:di=32;1:ex=31;1"
    export EXA_COLORS="in=34:ur=32:uw=32:ux=32:gr=33:gw=33:..."
fi
```

`LS_COLORS` gets three entries: regular file, directory, executable. The directory and executable values are ordinary SGR sequences. `fi=38` is not: `38` is the extended-colour introducer, which expects to be followed by `;5;n` for a palette index or `;2;r;g;b` for a direct colour, and here it is followed by nothing. A terminal receiving `ESC[38m` has an incomplete request on its hands; the practical result is that regular files get whatever the terminal does with an unfinished sequence, which is usually nothing. Since the intent of the line is that regular files be unstyled, the mistake is invisible.

The listing replacement's variable is much longer and uses that program's own two-letter keys — `ur`, `uw`, `ux` for the user permission triad, `gr/gw/gx` for group, `tr/tw/tx` for other — which is a different naming scheme, a different set of defaults, and a value that has to be maintained separately from the one above it. Two tools that list files, two incompatible colour variables, and the environment sets both from the same conditional.

The Statusline Has Its Own Colour Space

The powerline configuration is the only one of these that separates colour from meaning properly. `colors.json` maps names to 256-colour terminal indices — `"black": 16`, `"white": 231`, `"green": 2`, `"darkestgreen": 22` — and the colour scheme file assigns *semantic groups* to those names:

```
"warning:regular": { "fg": "white", "bg": "brightred", "attrs": ["bold"] },
"critical:failure": { "fg": "white", "bg": "darkestred", "attrs": [] },
"critical:success": { "fg": "white", "bg": "darkestgreen", "attrs": [] },
"session": { "fg": "black", "bg": "gray10", "attrs": ["bold"] },
"network_load_gradient": { "fg": "green_yellow_orange_red", "bg": "gray0" },
```

`green_yellow_orange_red` is not a colour, it is a gradient name, and the segment that uses it picks a point along it based on the value being displayed. That is the one place in the entire configuration corpus where a colour is a function of data rather than a constant, and it is the reason the statusline can show load without a number.

The theme file next to it lists segments rather than colours, and its contents say what the statusline is for:

```
{ "function": "powerliners.disk.disk_io" },
{ "function": "powerline.segments.common.net.network_load", "interface": "en0" },
{ "function": "powerliners.vcs.git_status" },
{ "function": "powerline.segments.common.sys.cpu_load_percent" },
{ "function": "powerliners.thermal.thermal", "args": { "family": "cpu" } },
```

Half of the segment names resolve inside the upstream package and half sit under `powerliners`, which is not in this tree. Two comments in the tree say what it is — `install/zpwrInstall.sh:850` and `install/.tmux.conf:28` both describe it as “the Rust port of powerline-status”, supplying `powerline`, `powerline-daemon`, `powerline-config` and `powerline-render` on `PATH`. So the theme file is a set of dotted names resolved by a binary the tree does not build, referenced from JSON, with nothing to check them. If a segment does not resolve the statusline drops it and continues, which is why the arrangement survives: a missing segment is a gap in a status bar, not an error.

The configuration tree is installed twice, by two different paths, to two different sources. The orchestrating installer copies it under the local directory and links that copy:

```
command cp -R $ZPWR_INSTALL/.config "$ZPWR_LOCAL"
ln -sf $ZPWR_LOCAL/.config/powerline/ "$tmuxPowerlineDir"
```

while the `regenpowerline` verb links the repository copy directly:

```
powerlineDir="$ZPWR_INSTALL/.config/powerline"
dest="$configHome/powerline"
...
ln -sfn "$powerlineDir" "$dest"
```

Running the installer gives you a snapshot that survives a repository update unchanged; running the verb replaces it with a live link, after which editing the theme edits the tracked file. The verb is the better behaviour and it is also the one that silently changes what the earlier step set up. It does at least handle the case the naive form gets wrong:

```
# A real directory here would make ln nest the link inside it.
if [[ -d "$dest" && ! -L "$dest" ]]; then
  zpwPrettyPrint "Backing up $dest to $dest.bak"
  command rm -rf "$dest.bak"
  command mv "$dest" "$dest.bak"
fi
```

`ln -sfn` pointed at an existing real directory creates the link *inside* it rather than replacing it — the single most common way a dotfile installer produces `~/.config/powerline/powerline` — and the guard moves the directory aside first. The comment above it states the failure mode rather than the fix, which is the right way round.

The Two Files That Are Mostly Commented Out

`.iftopcolors` is nineteen lines, seventeen of them comments:

```
#colors are red, blue, green, yellow, magenta, cyan, black and white
#RECEIVE_BAR_COLOR yellow -
#SENT_BAR_COLOR yellow -
BOTH_BAR_COLOR blue bold
#SCALE_BAR_COLOR red nonbold
...
HOST1_COLOR red nonbold
```

Two settings active, sixteen commented option lines preserved as a menu of what else could be set, and one header line naming the eight colours the program accepts. That is a legitimate way to ship a configuration for a program with no documentation to speak of: the file *is* the documentation, and the commented lines are the option list. One of them carries a typo — `PEAK_TRANSFER_COLUMN_COLORt` — which is harmless precisely because it is commented, and which would be a silently ignored line if it were not.

`htoprc` is the opposite case, and its own first line explains why it is the only file here that is copied rather than symlinked:

```
# Beware! This file is rewritten by htop when settings are changed in the interface.
# The parser is also very primitive, and not human-friendly.
```

A program that rewrites its own configuration cannot be given a symlink into a version-controlled tree without turning every interactive tweak into a working directory change. The installer copies it, and accepts that the shipped configuration is a starting point rather than a maintained one.

Chapter 79 – Two Thousand Lines Of Cinema

The largest function in the tree is not the dispatcher, the completion binder or the diagnostic dashboard. It is `autoload/common/zpwrMatrix`: 2,219 lines and 93 KB, more than four times the size of the next largest autoloaded function, and it draws an animated story in the terminal.

What It Actually Is

The verb table describes it in one line:

```
ZPWR_VERBS[matrix]='zpwrMatrix=fullscreen matrix digital rain screensaver'
```

The function's own help text describes something else:

```
>> HACKER STORY // ANIMATED CYBERPUNK CINEMA <<

An animated hacker story told in ASCII, figlet banners, CRT
scanlines, and ecosystem rain. 8 acts of cyberpunk cinema.
```

Both are true of different code paths. The rain is what `-r` skips to and what the function falls into at the end; the eight acts are the default. The verb description was written for the first version and never updated, which is the smallest possible example of the drift this book keeps measuring: a one-line description of a two-thousand-line function, in a table of four hundred sixty such descriptions, where nothing checks any of them against anything.

The Structure

The file divides into four parts: option parsing and statistics, twenty-two animation primitives, eight acts, and a rain loop. Eighteen flags are parsed in one `case`, and every numeric one is clamped immediately after:

```
(( speed < 1 )) @@ speed=1
(( speed > 9 )) @@ speed=9
(( chaos < 1 )) @@ chaos=1
(( chaos > 10 )) @@ chaos=10
(( start_act < 1 )) @@ start_act=1
(( start_act > 8 )) @@ start_act=8
(( no_fx )) @@ { glitch_mode=0; vhs_mode=0; crt_mode=0; }
```

Clamping rather than rejecting is the right call for a program whose arguments control frame timing: an out-of-range speed produces a sensible animation, and an error message would produce nothing. The last line is a composite flag that turns three others off, which is the only interaction between the seventeen.

The twenty-two primitives are the interesting layer, because they are a small terminal animation library that happens to live inside one function. The typewriter is the base case:

```

_mx_type() {
    local text="$1" color="${2:-0;32}" spd="${3:-0.03}"
    local -i ti
    for (( ti=1; ti<=${#text}; ti++ )); do
        printf '\033[%sm%s\033[0m' "$color" "${text[$ti]}"
        sleep $spd
        read -t 0.001 -k 1 ch 2>/dev/null
        [[ "$ch" == "q" || "$ch" == "Q" ]] 66 { printf '\033[?25h\033[0m\033[2J\033[H';
            return 1; }
    done
    return 0
}

```

Three things in ten lines are worth naming. `${text[$ti]}` indexes a string by character, which zsh supports directly and which is why there is no substring-extraction helper anywhere in the file. `read -t 0.001 -k 1` polls for a single keystroke with a one-millisecond timeout, so the animation is interruptible at every character without blocking. And the quit path restores the cursor, resets attributes, clears the screen and homes the cursor before returning non-zero — after which every caller in the file checks the return value with `|| return 0`, which is how a `q` at any point in an eight-act story unwinds cleanly through several hundred call sites.

The rest of the primitives build on that: centred text, centred typing, a CRT scanline sweep, horizontal static, a figlet banner with a fallback for machines without figlet, an interruptible pause, an alarm flash, a fake BIOS boot, a radar sweep, a log scroll, a character speech helper, screen tearing, VHS tracking distortion, corruption, shake, a progress bar, and three set-piece animations for an ssh session, a decryption and a download.

The Numbers Are Live

The fake BIOS is where the function stops being decoration:

```

boot_lines=(
    'ZPWR BIOS v13.37.42'
    'Copyright (C) MenkeTechnologies'
    ''
    "CPU: ${ZPWR_OS_TYPE:-UNKNOWN} $(uname -m) @ ${(( n_funcs ))}MHz (metaphorical)"
    "RAM: ${n_aliases} alias banks, ${n_verbs} verb registers"
)

```

Every number in the animation is measured at run time, in one block near the top:

```

(( ${+ZPWR_VERBS} )) 66 n_verbs=${#ZPWR_VERBS}
n_aliases=${#aliases}
n_funcs=${#functions}
n_params=${#parameters}
n_autoloads=$(ls "$ZPWR_AUTOLOAD_COMMON" "$ZPWR_AUTOLOAD_DARWIN" "$ZPWR_AUTOLOAD_LINUX"
    && 2>/dev/null | wc -l)
n_scripts=$(ls "$ZPWR_SCRIPTS"/*.sh "$ZPWR_SCRIPTS"/*.zsh "$ZPWR_SCRIPTS"/*.stk
    && "$ZPWR_SCRIPTS"/*.py 2>/dev/null | wc -l)
n_comps=$(ls "$ZPWR_COMPS"/* 2>/dev/null | wc -l)
n_commits=$(git -C "$ZPWR" log --oneline 2>/dev/null | wc -l)
n_pages=$(ls "$ZPWR"/docs/wizard_pages/page_*.zsh 2>/dev/null | wc -l)

```

Nine measurements, four from shell parameters and five from the filesystem, and the `--stats` flag prints them as a table instead of as a story. That is the distinction that keeps the whole thing from being a toy: the animation is scripted, the figures inside it are not, and a machine with fewer scripts shows fewer scripts. The verb count is guarded with `${+ZPWR_VERBS}` because the function can be sourced directly by a subprocess that never loaded the dispatcher — which is exactly what the two-player mode does.

A Game Over Files

The `--battle` flag turns the function into a two-player typing game, and the mechanism deserves a section because it is the only inter-process feature in the tree that is not a daemon.

Player one creates a state directory and writes one file per variable:

```
_bdir="/tmp/zpwr-battle-$$"
command mkdir -p "$_bdir"
print "100" > "$_bdir/p1hp"
print "100" > "$_bdir/p2hp"
print "0" > "$_bdir/round"
print "waiting" > "$_bdir/state"
print "" > "$_bdir/p1input"
print "" > "$_bdir/p2input"
print "" > "$_bdir/command"
```

Then splits the terminal and launches the second player as a separate process running the same file:

```
$ZPWR_ZTMUX split-window -h "ZPWR='$ZPWR' zsh -c 'source $ZPWR/autoload/common/zpwrMatrix
  --player2 --gamedir $_bdir'"
```

The child is a non-interactive shell that *sources the autoload file directly* rather than loading the function through the shell's function path. That works because every file in the autoload directories ends with a line calling its own function with `"$@"` — a convention an earlier chapter examined — and it is the only place in the tree that exploits it deliberately: it turns a file designed to be autoloaded into a file that can also be run as a script.

Seven files of shared state, one directory per game, named by the initiating process id, removed by a trap:

```
trap 'printf "\033[?25h\033[0m\033[2J\033[H"; command rm -rf "$_bdir" 2>/dev/null; return
  0' INT QUIT
```

The second player, given no directory, looks for one:

```
for _found in /tmp/zpwr-battle-*(N); do
```

`(N)` again — the null glob qualifier — so the loop simply does not run when no game exists, and the error path below it reports “no active battle found” instead of a failed expansion. It is a discovery protocol built out of a glob.

The round loop picks a verb at random, writes it to the shared file, renders both health bars, and reads the player's attempt with a timeout:

```
_bcmd="zpwr ${_bverbs}[$(( RANDOM % ${#_bverbs} + 1 ))]"
print "$_bcmd" > "$_bdir/command"
print "ready" > "$_bdir/state"
...
_bdmg=$(( ${#_bcmd} / 2 + RANDOM % 8 + 5 ))
read -t $_btimeout -r _binput
```

Damage is proportional to the length of the command, which makes the longer verb names worth more, plus a random component so that identical rounds do not produce identical outcomes. The typing target is drawn from the real verb list, which means the game is a drill on the dispatcher's own vocabulary.

Why This Belongs In A Reference

Two thousand lines of animation in a shell environment invites the obvious objection, so it is worth stating what the file demonstrates that nothing else in the tree does.

It is the most demanding consumer of the terminal control layer. Everything else in the environment prints

lines; this prints frames, at a position, with a cursor that must be hidden and restored, on a screen whose dimensions it reads once and must handle changing. Every escape sequence the rest of the framework uses casually — cursor addressing, screen clear, cursor visibility, scroll regions — is used here under conditions that make errors visible immediately. When a box in a diagnostic is one column wide, nobody notices; when a frame in an animation is one column off, it flickers.

It is also the only place where the framework's own measurements are presented as the subject rather than as diagnostics. The instrument panel prints counts so a person can act on them. This prints the same counts to be looked at. The figures are identical because they come from the same parameters, which means the animation is, incidentally, a test that those parameters are populated: an environment where the dispatcher failed to load produces a boot sequence with zero verb registers, in large friendly letters, before anything else has a chance to report the failure.

Chapter 80 – The Picker Binary Is `arb --fzf`

`ZPWR_FZF` is the one indirection every picker in the tree goes through, and it carries `arb --fzf --ansi` rather than `fzf`. The four call sites that invoked `fzf` directly follow it, and the two availability gates accept either binary. The assignment is one line: `export ZPWR_FZF='arb --fzf --ansi'` in `env/.zpwir_env.sh`.

Nothing in the option strings changed, and that is the point of routing the swap through a single variable. `arb --fzf` reads `$FZF_DEFAULT_OPTS` and the per-picker `$FZF_*_OPTS` the way `fzf` does, including `--ansi`, `--nth` and `--with-nth`, `--delimiter`, `--preview`, `--print-query` and `--expect`, so every colour, bind and preview command configured for `fzf` keeps working unaltered against a different binary.

What it buys is that the picker becomes part of the fleet rather than a dependency beside it: the same binary that runs an `arb` query renders the menu, so a preview command can be an `arb` program rather than a shell pipeline.

Why this chapter is in the supplement and not in a wizard page

It is worth recording how this passage came to be written twice. The first version was edited directly into `docs/book.md` — and `scripts/book_pdf.sh` runs `docs/genEncyclopediaMd.py` over the wizard pages *before* it typesets, rebuilding `book.md` from scratch on every build. The edit therefore survived being written, survived a `grep` that confirmed it, and was erased by the very build that was supposed to publish it. The page count did not move, because the chapter had never reached the PDF.

That is the general hazard of a generated artifact that lives in the repository next to hand-written ones: it looks editable, and it accepts an edit, and the edit is silently discarded at build time. The generator's own seam is the fix — `emit_supplement()` splices this file in and renumbers its `@@` headings — so hand-written material belongs here, where a regeneration carries it forward instead of overwriting it.

Chapter 81 – The Turbo Ladder, As Actually Written

Chapter 27 of this book prints a graduated zinit loading sequence with nine rungs: synchronous, bare `wait`, `wait'0'`, then `0a`, `0b`, `0c`, `0d`, `0e`, `0f`, and finally two rungs suffixed `g` and `h` behind `ZPWR_ZINIT_COMPINIT_DELAY`. That ladder is not the one in the shell.

Chapter 27 is generated. Its text comes from `docs/wizard_pages/page_107.zsh`, which the encyclopedia build reads verbatim, and that page has drifted from `install/.zshrc`. The rungs `0d`, `0e`, `0f`, `g` and `h` do not appear anywhere in the live configuration. This chapter states the ladder that does, and it is the one to trust: it was read out of the file the shell sources, not out of a page describing it.

What The File Contains

Every `wait` ice in `install/.zshrc`, in file order, with nothing omitted:

```
line 473 (no wait)      powerlevel10k
line 482 wait           OMZ completion snippets (loop)
line 492 wait           OMZ lib snippets      (loop)
line 498 wait           OMZ plugin snippets   (loop)
line 513 wait           OMZP::rails, if rails exists
line 519 wait           GH plugins            (loop)
line 524 wait           MenkeTechnologies/fzf (as'program')
line 528 wait           bigH/git-fuzzy        (as'program')
line 532 wait           MenkeTechnologies/forgit
line 536 wait           MenkeTechnologies/zconvey
line 541 wait'0'        hlissner/zsh-autopair
line 546 wait'0a'       MenkeTechnologies/zsh-expand
line 552 wait'0b'       zsh-autocomplete, if enabled
line 558 wait'0c'       zsh-history-substring-search
line 563 wait'0a'       zsh-users/zsh-autosuggestions
line 570 wait'0a'       MenkeTechnologies/zsh-zinit-final
line 576 wait'0a'       MenkeTechnologies/zsh-more-completions
line 580 wait"${ZPWR_ZINIT_COMPINIT_DELAY}a" compinit phase (as'null')
line 585 wait"${ZPWR_ZINIT_COMPINIT_DELAY}a" fast-syntax-highlighting
```

Nineteen `zinit ice` lines and nineteen `zinit load / zinit snippet / zinit light` lines, one apiece. The only sub-order letters in use are `a`, `b` and `c`. `ZPWR_ZINIT_COMPINIT_DELAY` is exported as `0` at `env/.zpwir_env.sh:201`, so lines 580 and 585 evaluate to `wait'0a'` on a default install — the same rung as `zsh-expand`, `autosuggestions`, `zsh-zinit-final` and `zsh-more-completions`.

Old Rung Against New

The generated chapter's rungs do map onto real loads. Only the letters moved:

chapter 27 says	what .zshrc actually does
wait'0d' autosug + FZF late + verbs	wait'0a' zsh-autosuggestions (line 563) atload'...zpwBindFZFlate; zpwBindVerbs; zpwBindZstyle'
wait'0e' zinit-final	wait'0a' zsh-zinit-final (line 570) atinit'zpwBindPenultimate; zpwBindFinal; zpwTokenPost'
wait'0f' more-comps	wait'0a' zsh-more-completions (line 576)
wait'...g' compinit	wait"...a" zicompinit; zicdreplay (line 580)
wait'...h' f-s-h	wait"...a" fast-syntax-highlighting (line 585)

Every payload description in chapter 27 is still accurate. The `atload` on `autosuggestions` really does fire `zpwBindFZFlate` and `zpwBindVerbs`; the `atinit` on `zsh-zinit-final` really does run `zpwBindPenultimate` and `zpwBindFinal`; the `compinit` phase really is an `as'null'` load whose whole purpose is its `atinit` hook. What changed is that six distinct rungs were collapsed onto one.

The page's other example table is stale in the same way. `page_107.zsh:17` offers `zinit ice wait'0e'` as an illustration of a late sub-order, and the chapter's closing paragraph asserts that “the ‘g’ and ‘h’ suffixes on the wait ice ensure compinit and syntax highlighting load last.” Neither suffix is in the file.

What Collapsing The Ladder Costs

The comments around the collapsed lines still describe the old, separated ordering. `install/.zshrc:568` reads “late loaded, must be last to load” directly above a `wait'0a'`, and `install/.zshrc:562` reads “late, must come before syntax highlight” above another `wait'0a'`. The requirements did not change; the mechanism that encoded them did. Five loads now sit on the same rung, and the ice no longer expresses any relative order among them at all — the constraint lives only in the comments and in the order the lines happen to appear.

One consequence is visible without reading zinit's scheduler. `zsh-zinit-final` is loaded twice, once at line 570 (`zinit load, wait'0a'`) and once at line 582 (`zinit light, wait"${ZPWR_ZINIT_COMPINIT_DELAY}a"`), and with the delay at its default of `0` both spellings resolve to the same string. The second one sources nothing — `as'null'` — and exists purely to hang `zicompinit` off an `atinit`. Under the old ladder the two were three rungs apart.

Raising `ZPWR_ZINIT_COMPINIT_DELAY` is the escape hatch that still works. Setting it to `1` moves the compinit phase and fast-syntax-highlighting to `wait"1a"`, a full second later, which restores the separation the letters used to provide. That is the only knob left; the letters are gone.

The Array Sizes Drifted Too

Chapter 27's plugin-array section prints “28+ GitHub repos”, “25+ OMZ snippets”, “4 OMZ lib files” and “12 OMZ completion files”. Counting the non-comment entries of each array in `install/.zshrc` gives 29, 28, 4 and 12. The two figures written with a `+` are still true as lower bounds; the arrays have grown past them.

The “48+ plugin roster” figure holds exactly. `install/.zshplugins` has 48 non-blank lines. It has 47 distinct ones: `MenkeTechnologies/fzf` is listed at both line 3 and line 35.

Read chapter 27 for what each ice modifier means and what each plugin does — that material is correct. Read this chapter for the order.

Chapter 82 – Installing A Unit File From The Repo That Ships It

A dotfiles repository that also ships systemd units has a distribution problem no other file in it has. A `.vimrc` is used from where it lands. A unit file is not: systemd reads `/etc/systemd/system`, it needs root to write there, and the unit's contents encode a user, a group and absolute paths that are correct on exactly one machine. The framework's answer is a 63-line shell function that treats the committed units as templates and rewrites them on the way in.

Four Units And One Import

`install/` holds five `.service` files. Four are the framework's own:

```
install/compose.service      18 lines   docker compose stack
install/learn.service        18 lines   a Spring Boot jar
install/mem.service          14 lines   memory watchdog
install/poll.service         16 lines   GitHub repo poller
install/zabbix-agent.service  15 lines   vendored from the distro package
```

The fifth is not a template at all. Its first line is a comment giving its canonical location, `/lib/systemd/system/zabbix-agent.service`, and its `ExecStart` points at `/usr/local/sbin/zabbix_agentd` — a path no part of this repository creates. It is a copy of a packaged unit kept alongside the 399-line `install/zabbix_agentd.conf` it goes with, so that the pair can be diffed against whatever a given host actually has.

The four project units share a shape: `Type=simple`, `User=pi`, `Group=pi`, `After=network.target`, and `ExecStart=/usr/bin/env bash` against a script under `/home/pi/.zpwir/scripts`. The `/usr/bin/env` prefix is doing real work here — systemd does not run `ExecStart` through a shell, so it will not search `PATH`, and `env` is the standard way to get a `PATH` lookup back without hardcoding an interpreter location that differs between distributions.

`compose.service` is the odd one, and its oddity is worth stating precisely:

```
[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/bin/env bash /home/pi/.zpwir/scripts/startContainers.sh
ExecStart=/usr/bin/env bash /home/pi/.zpwir/scripts/stopContainers.sh
```

`systemd.service(5)` states that “Unless `Type=` is `oneshot`, exactly one command must be given”, and that “If more than one command is configured, the commands are invoked sequentially in the order they appear in the unit file.” This unit is `Type=oneshot`, so both lines are legal and both run — starting the compose stack and then immediately stopping it. The two scripts are 12 lines each and are exact mirrors, `docker-compose ... up -d` against `docker-compose ... down`. The spelling that produces the intended behaviour is `ExecStop=` for the second, which is what the other three units use for their teardown.

The Environment Blocks Are The Real Payload

A unit started by systemd inherits almost nothing. There is no login shell, no `.zshrc`, no `.zpwr_env.sh` — which means every variable the framework normally exports has to be restated in the unit or looked up by the script itself. What each unit chooses to restate is a compact statement of what its script actually needs:

```
compose.service  ZPWR_SCRIPTS  HOME
poll.service     ZPWR      ZSH_CUSTOM  ZPWR_SCRIPTS
learn.service    SCRIPTS  REPO_NAME  ZSH_COMP_REPO_NAME
mem.service      EMAIL
```

`compose.service` restating `HOME` is not redundant. `systemd` does set `HOME` for units with a `User=`, but the value follows the account database rather than the unit, and the two `compose` scripts reference `$HOME/docker` — pinning it makes the unit's behaviour independent of what the `passwd` entry says.

`poll.service` is the one worth checking against its script. `scripts/remoteRepoMonitorDaemon.sh` references six variables: `$ZPWR`, `$ZPWR_COMP_REPO_NAME`, `$ZPWR_LOGFILE`, `$ZPWR_TMUX`, `$ZPWR_ZTMUX` and `$ZSH_CUSTOM`. The unit supplies two of them, plus a third — `ZPWR_SCRIPTS` — that the script never reads. The gap is smaller than it looks, because the script self-defines what it can at its own lines 10 and 11:

```
REPO_NAME='zpwr'
ZPWR_COMP_REPO_NAME='zsh-more-completions'
```

That is the right instinct for a script that runs both from a shell and from a unit: values that never vary are hardcoded in the script, values that vary per machine come from the environment. What is left unset is `$ZPWR_LOGFILE`, `$ZPWR_TMUX` and `$ZPWR_ZTMUX`, which under a unit expand to nothing.

`learn.service` restates three names that appear nowhere in the shell environment — `SCRIPTS`, `REPO_NAME` and `ZSH_COMP_REPO_NAME`, as opposed to the framework's `ZPWR_SCRIPTS` and `ZPWR_REPO_NAME`. They are not the shell's variables at all; they are configuration for the Java process the unit starts, which reads its own names. The unit is the boundary between two naming conventions, and it is the only file in the tree where both appear.

The Verbs Are Gated Three Times

Nothing about the `systemd` surface exists on a machine without `systemd`, and the gate is applied at three separate layers rather than checked once.

`install/.zshrc:320` adds the autoload directory to `fpath` and pulls in the OMZ `systemd` plugin only when the command exists:

```
if zpwrCommandExists systemctl; then
    ZPWR_OMZ_PLUGINS+=( systemd )
    fpath+=( $ZPWR_AUTOLOAD_SYSTEMCTL )
fi
```

`env/.shell_aliases_functions.sh:184` marks the same directory's contents autoloadable, again behind the same predicate:

```
if zpwrCommandExists systemctl; then
    if zpwrIsZsh; then
        eval 'autoload -z $ZPWR_AUTOLOAD_SYSTEMCTL/*(:t)'
    fi
fi
```

And `env/zpwr.zsh:485` registers the three verbs:

```

if zpwrCommandExists systemctl; then
    ZPWR_VERBS[restart]='restart=restart zpwr systemd service'
    ZPWR_VERBS[serviceup]='ssu=start and enable systemd service'
    ZPWR_VERBS[servicedown]='ssd=stop and disable systemd service'
fi

```

Three gates is not redundancy; each one protects a different failure. Without the first, `fpath` grows by a directory of functions that can never run. Without the second, the names exist but are unresolved. Without the third, `zpwr -h` advertises verbs that fail on invocation. Only the third is visible to a user, and it is the reason the verb list has a different length on a Linux server than on a laptop.

Rewriting A Unit On The Way To `/etc`

`autoload/systemctl/restart` is the interesting one. Its argument is a bare service name, and it resolves that to a template in the repository:

```

service="$1"
src_dir="$ZPWR_INSTALL"
service_path="$src_dir/$service.service"

```

Three existence checks follow, and the middle one is a fallback rather than a failure: if `$ZPWR_INSTALL/$service.service` does not exist, `service_path` is reset to `$1` itself, so `restart /path/to/some.service` also works. Only if that is not a file either does the function give up. Then it checks for `/etc/systemd/system` and refuses if `systemd` is not installed — a fourth gate, this time on the target rather than the client.

Then it pulls, and copies the template out of the repository before touching it:

```

( builtin cd "$src_dir" && git pull; )
group=$(id -gn)
cp "$service_path" "$ZPWR_LOCAL"
service_path="$ZPWR_LOCAL/$service.service"

```

The subshell means the `cd` does not survive; `builtin cd` bypasses any `cd` function in the user's environment. The copy into `$ZPWR_LOCAL` is what keeps the repository clean: every rewrite below operates on the copy, so the working tree is never dirtied by an install and `git pull` on the next run never hits a conflict.

The rewrites are five in-place substitutions, all through `stryke`:

```

stryke -i -pe "s@\bpi\b@$USER@g" "$service_path"
...
stryke -i -pe "s@^Group=.*@Group=$group@g" "$service_path"
stryke -i -pe "s@/home/$USER/\\.zpwr/scripts@$ZPWR_SCRIPTS@g" "$service_path"
stryke -i -pe "s@/home/$USER/\\.zpwr@$ZPWR@g" "$service_path"

```

The order is load-bearing and easy to misread. The first substitution replaces the bare word `pi` everywhere, which converts `User=pi`, `Group=pi` and every `/home/pi/` prefix in one pass. Only after that do the path rewrites make sense, because they are written against `/home/$USER/`, the string the first rewrite just produced. The longer path is replaced before the shorter one, so `/home/x/.zpwr/scripts` becomes `$ZPWR_SCRIPTS` rather than `$ZPWR/scripts` — the two are usually the same directory, but only usually.

The `$UID` branch exists because root's home is not under `/home`. For a root install the first rewrite has turned `/home/pi` into `/home/root`, so the branch opens by undoing that: `s@/home/root@/root@`.

Two edges survive the rewriting. `\bpi\b` also matches inside `WorkingDirectory=/home/pi/docker`, giving `/home/$USER/docker`, which is correct as far as the home directory goes and unremapped as far as `docker` goes — the compose unit still expects a directory of that name. And `learn.service`'s `ExecStart` points at `/home/pi/learn.jar`, which becomes `/home/$USER/learn.jar`; the jar is not in the repository, so the unit is a template for a build artifact produced elsewhere.

One Substitution That Cannot Run

Between the `pi` rewrite and the `Group=` rewrite sits the mail-address rewrite:

```
if [[ -n "$EMAIL" ]]; then
    stryke -i -pe "s@\bmyemail\b'$EMAIL'@g" "$service_path"
fi
```

This is the line that gives `mem.service` its `Environment="EMAIL=myemail"` placeholder a real value, and as written it cannot. A substitution needs three delimiters. This has two: the pattern runs to the second `@`, and there is no closing one. Running the expression through a parser confirms it — `perl -ce` on the same program text reports `Substitution replacement not terminated`, while the intended spelling, with the quotes outside the delimiters, reports `syntax OK`. The single quotes were meant to protect the expansion and landed one character too far left.

The blast radius is bounded by the `if`: the line only runs when `EMAIL` is set, and only for whichever unit is being installed. When it does run, the interpreter exits non-zero on a parse error and `-i` writes nothing, so the file is left with the literal `myemail` — the same state as skipping the branch. Nothing is corrupted; the feature simply never worked, and the watchdog mails into a placeholder.

After The Copy

The tail is four privileged commands and a follow:

```
sudo cp "$service_path" /etc/systemd/system
sudo systemctl daemon-reload
sudo systemctl restart $service.service
sudo systemctl --no-pager -l status $service.service
sudo journalctl -f
```

`daemon-reload` before `restart` is required — `systemd` caches unit files, and without the reload the restart uses the previous contents. `--no-pager -l` on the status call is the same instinct that appears throughout the framework: a command whose output is being read by a human at the end of a script should not open a pager and should not truncate. The final `journalctl -f` never returns; `restart` is an interactive command that ends by handing the terminal to the log tail, and the way out is the interrupt key.

The two enable/disable verbs are much smaller — 15 lines each — and are pure loops over their arguments:

```
for service in "$@"; do
    zpwPrettyPrint "sudo systemctl start $service"
    zpwPrettyPrint "sudo systemctl enable $service"
    sudo systemctl start "$service"
    sudo systemctl enable "$service"
done
```

Printing the command before running it is the same echo-then-execute pattern the installer uses, and it matters more here than elsewhere: these commands change what the machine does at next boot, and the transcript is the only record of which units were touched.

`autoload/systemctl/restartZabbixAgent` is the fourth file in the directory and the only one specific to a single service. Its 19 lines are one `if` whose two branches are identical except that the first pipes the agent log through `zcolorizer` and the second does not — four commands duplicated to avoid building a pipeline conditionally. The uncoloured branch is the honest fallback that a great deal of shell code omits, at the cost of the two versions being able to drift.

Chapter 83 – The Git Configuration That Ships With The Shell

Three files in `install/` configure git: an 84-line `.gitconfig`, a 1,449-line `.gitignore_global`, and a 15-line `pre-push` hook. They are the framework's only opinions about a tool it does not own, and between them they are also the clearest example in the tree of a configuration that ships without an installer.

Eighty-Four Lines Of Opinion

`install/.gitconfig` opens with an empty identity:

```
[user]
  email =
  name =
```

That is the correct shape for a committed dotfile — a git config with a real address in it is a config that cannot be shared, and an empty `user.email` produces a loud error on the first commit rather than a wrong author on the hundredth.

The `[core]` block is where the behaviour is:

```
[core]
  editor = nvim
  excludesfile = ~/.gitignore_global
  autocrlf = input
  pager = delta | less --tabs=4 -RFX
  commentchar = |
  abbrev = 8
```

`pager` is a pipeline, which git permits because it runs the value through a shell. `delta` renders the diff, `less` displays it, and the `-RFX` triple is the combination that makes a pager tolerable for short output: pass through colour, quit immediately if the content fits on one screen, and do not send the terminal's alternate-screen sequences so that content stays visible after exit. `--tabs=4` matters because `delta` emits tabs and the default of 8 makes indented diffs wrap.

`commentchar = |` is the one that surprises. Git's default comment character in commit-message templates is `#`, which is unusable if you routinely write commit messages that reference issue numbers or shell comments; switching it to `|` frees `#` for prose at the cost of any commit message that legitimately starts a line with a pipe. `abbrev = 8` widens the short-hash from git's default of 7, which is a collision-avoidance choice that scales with repository size.

`help.autocorrect = 1` is the most consequential single line in the file, and its meaning is version-specific. The `git-config(1)` manual on a current git lists the accepted values, and for `1` it says: "run the suggested command immediately." Not "suggest it", not "wait a tenth of a second" — a mistyped subcommand with exactly one near match is executed. The value that only prints the suggestion is `0`.

`interactive.singlekey = yes` is the second behavioural change: `git add -p` and its relatives accept a bare keystroke rather than a key and Enter. It depends on a Perl module being available, and degrades to normal line-based input when it is not.

The remaining blocks are unremarkable individually and coherent together: `rerere.enabled` records conflict resolutions for replay, `rebase.autoStash` removes the dirty-tree refusal, `pull.rebase = false` keeps merge as the pull strategy, `status.submodulesummary = 1` makes submodule pointer moves visible in `git status`, and `init.defaultBranch = main`. Twelve aliases are defined, several of which are name repairs rather than shortcuts — `unstage`, `untrack`, `track` and `wipe` give English names to `reset HEAD --`, `rm --cached -r`, `add -N` and `checkout --`.

Two aliases are submodule-aware and belong together: `sdiff` runs a top-level diff and then `git submodule foreach 'git diff'`, and `spush` is `push --recurse-submodules=on-demand`. The three merge tools are all `nvim -d` with different window arrangements — `merge3` and `merge4` differ only in whether `$BASE` is included and which window is moved to the bottom.

The Ignore File Is A Concatenation

`install/.gitignore_global` is 1,449 lines: 418 comment lines, 263 blank, and 768 patterns. The section comments give away the provenance — `# Package Files`, `# Crashlytics plugin`, `# Local History for Visual Studio Code`, `# Output files from R CMD build`, `# Snowpack dependency directory` — these are the headings of the upstream per-language ignore templates, concatenated.

Concatenation is visible in the duplicate count. The 768 patterns reduce to 723 distinct ones, so 45 lines are repeats. The most repeated is `build/`, four times; `/build/`, `*.rej`, `*.orig`, `*.obj`, `*.o` and `*.exe` appear three times each. Duplicate ignore patterns are harmless to git, which is why nobody notices them, but they are a reliable fingerprint of a file assembled rather than written.

What is not harmless is which patterns ended up in a *global* ignore file. A global excludes file applies to every repository the user touches, and this one contains bare directory patterns:

```
bin/
build/
dist
dist/
docs/
logs/
node_modules/
out
out/
target/
```

`node_modules/` and `target/` are uncontroversial. `bin/` and `docs/` are not. Inside a language-specific `.gitignore` these are correct, because the file only applies where it sits. Promoted to the global level they mean that a new `bin/` or `docs/` directory in any repository is invisible to `git status` and silently skipped by `git add .` — including this repository's own `docs/`, which is tracked and so unaffected, but which would not have been addable had the pattern been in place first. The failure mode is a directory that does not appear in a commit and produces no warning, and the diagnosis requires `git check-ignore -v`.

`tags` appears twice in the file, which is the same `tags` that this book's chapter on the committed ctags index discusses. Globally ignoring the tags file is the right call; having it in there twice is the concatenation showing again.

The framework provides exactly one verb for the file, and it is three lines:

```
f="$(git config core.excludesfile)"
${=EDITOR} "${f/#\~/~/$HOME}"
```

Asking git for the path rather than hardcoding it is right — the answer follows whatever the user actually configured. The `${f/#\~/~/$HOME}` expansion is required because git stores the value with a literal tilde and returns it that way; the `#` anchors the replacement to the start so a tilde inside a directory name survives.

`${=EDITOR}` splits the variable on whitespace, which is how an `EDITOR` of `nvim -p` reaches the editor as two arguments instead of one.

The Hook That Only Runs On One Platform

`install/hooks/` contains a single file, and it is fifteen lines:

```
version="$(git describe --tags "$(git rev-list --tags --max-count=1)" 2>/dev/null)" ||
  exit 0
readme="$(git rev-parse --show-toplevel)/README.md"
[ -f "$readme" ] || exit 0
current="$(grep -o '| \_\_ v.*' "$readme" | head -1 | /usr/bin/sed 's#| \_\_ ##')"
[ "$current" = "$version" ] && exit 0
/usr/bin/sed -i ' ' 's#| \_\_ v.*#| \_\_ '$version'##' "$readme"
git add "$readme"
git commit --amend --no-edit --quiet
```

The intent is a stamp: the README's ASCII-art banner carries the version, and the banner should match the newest tag at push time. The early exits are careful — no tags, no README, or an already-correct value all exit 0, and a pre-push hook exiting 0 lets the push proceed.

The implementation has two problems, one portability and one design.

The portability problem is the `-i ' '` on line 13. That is the BSD spelling: BSD `sed` requires a separate backup-suffix argument and accepts an empty one. GNU `sed` takes the suffix attached to the flag, so it parses `' '` as the *script* and the real expression as a *filename*. Running the exact form against GNU `sed` demonstrates it:

```
$ gsed -i ' ' 's#v1#v2#' t.txt
gsed: can't read s#v1#v2#: No such file or directory
exit=2
```

The file is unmodified and the exit status is 2. And because that `sed` call is the last-but-two command in a hook with no `set -e` and no `|| true`, the non-zero status propagates: `githooks(5)` states of `pre-push` that “If this hook exits with a non-zero status, git push will abort without pushing anything.” Except that the two following commands, `git add` and `git commit --amend`, reset `$?` , so the observable behaviour on a GNU system is an unamended README, an error line, and an amend of the tip commit that changed nothing. Hardcoding `/usr/bin/sed` was an attempt to avoid a shadowed `sed` on `PATH`, and it succeeds on macOS, where `/usr/bin/sed` is BSD. On Linux `/usr/bin/sed` is GNU and the hardcoding guarantees the broken form.

The design problem is `git commit --amend` inside `pre-push`. Git has already computed the refs to push and passed them to the hook on standard input; the hook then rewrites the tip commit under it. Whether the push carries the amended or the pre-amend commit depends on when the ref is resolved, and the result is either a push of a commit that no longer exists locally or a local branch that has diverged from what was just pushed. The place for this work is a `pre-commit` hook or a release script.

Nothing Installs Any Of It

Grepping the tree for `.gitconfig` returns matches in nine files, and not one of them is an install step. Nine lines are in three test files, which assert that the shipped file exists and is non-empty. Two are in `docs/index.html` and two more in the wizard pages that site is generated from: `page_132.zsh` tells the reader to configure `delta` in `.gitconfig`, and `page_144.zsh` states that `.vimrc`, `.tmux.conf` and `.gitconfig` “all point back to `$ZPWR_INSTALL`”, which is true of the first two and not of the third. One is `autoload/common/zpwrGitConfig`, a three-line verb whose entire body is `${=EDITOR} "$HOME/.gitconfig"` — it opens the *live* file and never refers to the shipped one. One is a binary match

inside `install/g-tags/GPATH`, the committed ctags index. And one is `README.md:316`, which is the most direct evidence of all: it sits under the heading `ZPWR Black Market Extras (Not Installed)`.

Grepping for `install/hooks` finds two tests, which run `sh -n` and `zsh -n` over the hook, and one row of `docs/report.html` describing it as a “git hooks directory shipped into `$GIT_TEMPLATE_DIR` on install.” Nothing in the tree sets `init.templateDir`, `core.hooksPath` or `GIT_TEMPLATE_DIR`.

The symlink list that installs the other dotfiles is at `install/plugins_install.sh:58` and has ten entries — `.tmux.conf`, `.ideavimrc`, `.vimrc`, the four `grc` configuration files, `.iftopcolors` and `.inputrc`. Neither `.gitconfig` nor `.gitignore_global` is among them. The one piece of machinery that names `.gitignore_global` is `scripts/macOnly/tutorialConfigUpdater.sh:52`, and it runs the other direction:

```
cp "$HOME/.gitignore_global" "$ZPWR_DIR_INSTALL"
```

That is a capture step, not an install step — it copies the live file back into the repository so that changes made by hand are committed. Which is a coherent design for a personal dotfiles repo: the machine of record is the machine, and the repository is its backup. It only becomes a documentation problem when a generated report claims the reverse.

Chapter 84 – The Second Index

The framework carries two source indexes for two different tools. One is `tags`, the `ctags` index. The other is GNU GLOBAL, and it is the more elaborate of the two: a vendored 688-line configuration file, three committed binary database files, seven verbs, and a picker. It is also, as shipped, unreadable.

What GLOBAL Is For

`ctags` answers one question — where is this symbol defined. GNU GLOBAL answers that plus the reverse, where is this symbol *used*, which is why it keeps three files instead of one. `GPATH` holds definitions, `GRTAGS` holds references, and `GPATH` maps the internal file numbers back to paths. All three are Berkeley DB files, so all three are opaque to the tools that read the rest of the tree.

`install/g-tags/` contains all three, committed:

```
install/g-tags/GPATH    164,772 bytes
install/g-tags/GRTAGS   16,388 bytes
install/g-tags/GTAGS    91,081 bytes
```

The paths inside are stored relative to the home directory, in the form `./zpwir/autoload/systemctl/restart` — GLOBAL’s convention, and a fortunate one, because it means the committed database contains no absolute path from the machine that built it.

The Vendored Configuration

`install/.globalrc` is not the framework’s file. Its header says so: “This file is part of GNU GLOBAL”, with the upstream copyright block intact. It is the stock `gtags.conf`, dropped in whole.

Its format is termcap, which is worth a sentence because nothing else in the tree uses it. Entries are `label:` followed by colon-separated capabilities, and `tc=` includes another entry by name — the same inheritance mechanism termcap used for terminal definitions, applied to parser configuration. Eleven top-level labels are defined:

```
default native user ctags new-ctags pygments
native-pygments common gtags builtin-parser htags
```

`default` is one line, `:tc=native:`, and `native` is `:tc=gtags:tc=htags:`. The substance is in the 288 `langmap` and 287 `gtags_parser` lines below, which are the file-extension-to-parser tables — the reason a stock configuration is worth shipping at all, since assembling those by hand is a day’s work.

The selection mechanism is the `GTAGSLABEL` environment variable, which picks one of the eleven, and it is exactly what the framework’s verbs manipulate.

Seven Verbs, One Implementation

The registrations are scattered across `env/zpwir.zsh` but form one cluster:

```

line 132  emacshtags      zpwrEmacsZpwrGtags
line 133  emacshtagsedit  zpwrEmacsZpwrGtagsEdit
line 331  regengtags      zpwrRegenGtagsCtags
line 421  vimhtags        zpwrVimZpwrGtags
line 422  vimhtagsedit    zpwrVimZpwrGtagsEdit
line 472  regengtagspygments  zpwrRegenGtagsPygments
line 473  regengtagstype  zpwrRegenGtagsType

```

Six of the seven backing functions are wrappers of four to eight lines. Two of them are one line each:

```

function zpwrRegenGtagsCtags(){
    zpwrRegenGtagsType ctags
}

```

```

function zpwrRegenGtagsPygments(){
    zpwrRegenGtagsType pygments
}

```

Those two argument values, `ctags` and `pygments`, are two of the eleven labels in `.globalrc`. The verb surface is a set of named entry points into the configuration file's label table, which is why adding a parser means adding a four-line file rather than editing a dispatcher.

The vim and emacs pairs are the same shape one layer up — `zpwrVimZpwrGtags` calls `zpwrGetGtags "$ZPWR_VIM"` and `zpwrEmacsZpwrGtags` calls the same helper with `"$ZPWR_EMACS_CLIENT"` — so the editor is a parameter and the picker logic is shared. The emacs variant adds an existence check the vim variant does not have, refusing with `emacs must exist` rather than failing inside the helper.

The Regenerator

`autoload/common/zpwrRegenGtagsType` is 41 lines and does the real work. Its input is a list of files, built by a nine-glob loop rather than by letting `gtags` walk a directory:

```

for file in \
"$ZPWR/"* \
"$ZPWR_ENV/"*.sh \
"$ZPWR_INSTALL/"* \
"$ZPWR_AUTOLOAD_LINUX/"* "$ZPWR_AUTOLOAD_DARWIN/"* \
"$ZPWR_AUTOLOAD_SYSTEMCTL/"* "$ZPWR_AUTOLOAD_COMMON/"* \
"$ZPWR_COMPS/"* \
"$ZPWR_SCRIPTS/"* \
"$ZPWR_SCRIPTS_MAC/"*; do
if [[ -f "$file" ]]; then
    echo "$file"
fi
done | gtags --accept-dotfiles --gtagslabel=$type -f -

```

`-f -` reads the file list from standard input, and `--accept-dotfiles` is required because a recursive walk would otherwise skip everything under `$ZPWR_ENV`, where every file begins with a dot. The `[[ -f ]]` test filters the directories that the `*` globs pick up. The explicit list is a deliberate narrowing: it indexes the four autoload subdirectories, the completions, both script trees, the install directory and the dotfile-shaped environment files, and nothing else — no tests, no wizard pages, no vendored plugin sources.

The output goes to the home directory, not to the repository, because the whole block runs inside `( builtin cd "$HOME"; ... )`. That is the convention GLOBAL expects for a database that indexes paths under `$HOME`, and it is also why the function's second line inside the subshell is a bare removal in the user's home directory:

```

command rm GPATH GRTAGS GTAGS 2>/dev/null

```

`command rm` bypasses the framework's own `rm` function. Deleting three specifically-named files after a `cd`

into a known directory is about as narrow as such a step gets, but it remains a delete in `$HOME` guarded only by the subshell's `cd` having succeeded — and the `cd` is not checked.

The tail is the fallback that explains why the binary files are committed at all:

```
if [[ ! -s GTAGS ]]; then
  zpwLogConsoleErr "forced to copy back gtags because empty"
  echo command cp "$ZPWR_INSTALL/g-tags/"${GPATH,GRTAGS,GTAGS} "$HOME"
  command cp "$ZPWR_INSTALL/g-tags/"${GPATH,GRTAGS,GTAGS} "$HOME"
fi
```

If the freshly built `GTAGS` is empty — which happens when the requested parser label is not actually available, the commonest case being `pygments` without the Python plugin installed — the committed database is restored. The echo-then-run pair prints the command before executing it, and the brace expansion is expanded by both, so the transcript shows all three filenames.

The Committed Database Does Not Load

The fallback restores a database. Whether that database is usable is a separate question, and it can be answered directly. Copying the three committed files into a directory and pointing `GLOBAL` at them:

```
$ GTAGSROOT=$PWD GTAGSDBPATH=$PWD global -P '.'
global: dbop_get failed.
$ GTAGSROOT=$PWD GTAGSDBPATH=$PWD global -x zpwRegenGtagsType
global: dbop_get failed.
```

Every query fails the same way, against `global` (GNU `Global`) 6.7. A database built by that same binary in the same session, over a throwaway source directory, answers the identical `global -P '.'` invocation with no error. The committed files are not a drop-in for the installed tool.

The metadata does not explain it: both databases report `__VERSION 6` and carry the same `__COMPRESS` markers. Berkeley DB 1.85 files encode machine characteristics that a portable format would not, and these were built somewhere else — the units in the same directory are written for a single-board machine — which is the most likely reading, but the cause is not visible from the file and the observable fact is the one that matters: the fallback path restores an artifact that `global` refuses to read, and the picker that consumes it, `zpwGtagsIntoFzf`, begins with `global -x '.*'` and would therefore produce an empty list.

Drift, Measured Differently

Even setting the load failure aside, the committed index has aged. Extracting the path entries from `GPATH` with `strings` and filtering to the `./`-rooted ones yields 726 candidates. Some are truncation artifacts of that extraction method — `aut`, `autoload/common/zpw` — so the number is an upper bound rather than an exact count. Of those 726, 707 still name files that exist and 19 do not, and the missing ones are a legible history: `.travis.yml`, `install/alacritty.yml` (superseded by `alacritty.toml`), `scripts/storageShower.sh`, `scripts/runner.sh`, `scripts/banner.pl`.

The other direction is the larger gap. The index holds 646 entries under `autoload/` and `scripts/`; those two directories currently hold 712 files. Two thirds of the framework's growth since the database was built is invisible to it.

The picker is the one part of the stack that is worth reading regardless:

```
global -x '.*' |
eval "$ZPWR_FZF $FZF_GTAGS_OPTS" |
stryke -pe 's@^\(\\S*?)\\s+(\\d+)\\s+(\\S*)\\s+.*@+\\$2 "'"$HOME/"' '$3"@;s@\\n@ @g'
```

`global -x` prints symbol, line number, file, and source text in fixed columns. The substitution reorders three of those four into `+LINE "path"` — the argument form every vi-family editor accepts for “open this file at this line” — and the second substitution flattens newlines to spaces so that multiple selections become one argument list. The `$HOME/` interpolation re-absolutises the paths that GLOBAL stored relative to it. Three regular expressions convert a query result into a command line, which is the same trick the framework’s other `into-fzf` functions use, and it is independent of whether the database behind it loads.

Chapter 85 – One Corpus, Three Renderers

The 217 wizard pages under `docs/wizard_pages/` are the framework's reference documentation, and they are consumed by three completely different programs: a zsh function that prints them to a terminal, a static HTML site, and the generator that produces this book. The three agree on the content and disagree on the structure, and comparing them shows what a document format costs.

The Source Shape

Each page is a zsh file with three declarations — `PAGE_TITLE`, `PAGE_CHAPTER`, and a `PAGE_CONTENT()` function whose body is nothing but `print` statements with colour variables interpolated. That is a deliberate choice of source format: the primary consumer is a terminal, so the source is written in the terminal's own language and every other renderer pays the cost of parsing it.

There are 217 pages and 45 distinct `PAGE_CHAPTER` strings, so a chapter is a many-to-one grouping over pages, expressed as a repeated string rather than as a structure. Nothing in the source guarantees that pages sharing a chapter are adjacent in file order, and they are not.

Renderer One: The Terminal

The wizard is the reason the source looks the way it does. A page's body is executed, not parsed; the colour variables are locals declared at the top of each `PAGE_CONTENT`, so a page is self-contained and can be sourced and called in isolation. The `${#ZPWR_VERBS}` expansion that appears in the intro pages is live — the terminal renderer prints whatever the running shell's verb table currently holds, which is the only one of the three renderers that can be correct by construction.

Renderer Two: The Static Site

`docs/index.html` is 7,342 lines and contains the same corpus as marked-up HTML. It links two local stylesheets — `hud-static.css` at 656 lines and `tutorial.css` at 669 — plus a Google Fonts stylesheet, and loads exactly one script, `hud-theme.js` at 395 lines, from line 7340 of 7342. Everything else is inline: a `<style>` block starting at line 14, and then the content.

The structure it emits is a `<div class="enc-chapter">` per chapter and a `<section class="enc-section">` per page. The section count is exactly 217 — one per wizard page, no losses. The chapter count is 76.

Seventy-six chapter divisions over 45 distinct chapter names means the site starts a new chapter every time the `PAGE_CHAPTER` string changes as it walks pages in file order. The result is a book whose chapter numbering runs to 76 and which repeats itself:

```
CH 03  THE GIT DIMENSION
CH 50  THE GIT DIMENSION
CH 56  THE GIT DIMENSION
```

CREATIVE TOOLS appears four times. ZINIT & PLUGIN SYSTEM, UTILITY ARSENAL, NETWORK & CONTAINERS, INTROSPECTION, HEALTH & DIAGNOSTICS, BUILD SYSTEM and THE GIT DIMENSION appear three times each. The two appendices are numbered as chapters 74 and 75, and the closing page — chapter 43 of the encyclopedia, THE END — lands at CH 76.

Renderer Three: This Book

`docs/genEncyclopediaMd.py` reads the same 217 files and does not have that problem, because it groups before it emits. It builds a dictionary keyed by chapter string, appends each page to its chapter's list, and then sorts the chapters by a key that parses the string:

```
m = re.match(r'^APPENDIX\s+([A-Z])', raw)
if m:
    return (1, m.group(1))
m = re.match(r'^CHAPTER\s+(\d+)', raw)
return (0, int(m.group(1))) if m else (2, raw)
```

The tuple's first element is a bucket — numbered chapters, then appendices, then anything unrecognised — and the second is the sort within it, an integer for chapters so that 10 follows 9 rather than 1. The generator's own comment names the failure it exists to prevent: the pages “are not stored in chapter order — a batch of supplementary pages at the tail revisits earlier chapters — so we group and sort rather than emit a fresh (duplicate) heading every time the chapter string changes.”

That is precisely what the HTML renderer does. The two consumers were written against the same source and only one of them knows that the source has no ordering guarantee.

The generator also resolves what the terminal renderer leaves live. `#{#ZPWR_VERBS}` is substituted at build time by `count_verbs()`, which counts distinct assigned keys in `env/zpwr.zsh` by regular expression rather than by running a shell. Its docstring explains the choice: an earlier version shelled out to the build host's live plugin tree, which made the printed number depend on which plugins happened to be installed and cost about ninety seconds per build. Reading the pinned source instead makes the book reproducible — two runs over the same checkout emit the same number on any machine — at the cost of the number being a build-time snapshot rather than a runtime fact.

The Fourth File

`docs/report.html` is 505 lines and is not a renderer of the corpus at all. It is a metrics page, and every figure in it is typed into the HTML by hand. That makes it the one file in the tree where a number can be wrong without any code being wrong, and it is worth auditing figure by figure. Counted today, against the same checkout:

claim	stated	counted	command
tracked files	1,277	1,277	git ls-files wc -l
zsh+sh lines	107,387	107,389	git ls-files grep -E '\.(zsh sh)\$' xargs wc -l
autoload functions	533	533	find autoload -type f wc -l
verb keys	462	462	grep -cE '\s*ZPWR_VERBS\[
~ env/zpwr.zsh			
unique spellings	460	460	grep -oE 'ZPWR_VERBS\[([^\]]+)\]=' sort -u wc -l
zsh bindkeys	216	216	grep -rhcE '\s*bindkey' env install
~ autoload			
tmux binds	80	80	grep -rhcE '\s*bind-key \s*bind '
~ tmux/*.conf			
zinit plugins	48	48	grep -cvE '\s*(# \$)'
~ install/.zshplugins			
brew formulas/casks	654	654	formulae.sh 579 + casks.sh 75
test files	149	149	ls tests/*.zsh wc -l
test lines	79,657	79,657	cat tests/*.zsh wc -l
zunit cases	14,278	14,278	grep -rh '^@test' tests/*.zsh wc -l
wizard pages	217	217	ls docs/wizard_pages/page_*.zsh wc -l
~ -l			
tags	618	618	git tag wc -l
commits	6,348	6,353	git rev-list --count HEAD
all-branch commits	10,605	10,610	git rev-list --all --count

Thirteen of sixteen are exact. The three that miss are the three that move on every commit: both commit counts are short by exactly five — the page was accurate when written and the tree has had five commits since — and the line count is off by two for the same reason.

The second tier of figures — the stat cards below the summary, which nobody copies into a description and which therefore have no second copy to disagree with — holds up at least as well:

claim	stated	counted	command
zpwr.zsh lines	553	553	wc -l < env/zpwr.zsh
_zpwr completion lines	405	405	wc -l < autoload/comps/_zpwr
zinit snippet (OMZ) lines	4	4	grep -cE '\s*zinit snippet' install/.zshrc
shell installers	25	25	ls install/*.sh wc -l
install/ lines	82,150	82,150	find install -type f xargs wc -l
latest tag	v48.9.4	v48.9.4	git describe --tags \$(git rev-list --tags --max-count=1)

Six exact, including two five-figure numbers. Three cards do not reconcile, and each fails differently.

`zinit ice (turbo) invocations: 21` and `zinit load lines: 17` are both above what `install/.zshrc` contains: 19 `zinit ice` statements, 14 `zinit load`, one `zinit light`. Nineteen and fifteen. Neither stated figure is reachable from the file under any counting I could construct, and both are the kind of number that was correct before a consolidation.

`tmux Conf Files: 24` is the right number under the wrong label. `tmux/` holds 24 files but only 22 of them end in `.conf`; the other two are a shell script and a directory entry. The count is a file count that acquired a narrower caption.

`Zsh bindkeys` is the interesting one, because it appears as **228** on the stat card and **216** in the description, and both are correct. The difference is scope, and it resolves exactly:

```
$ grep -rHE '^[[:space:]]*bindkey' autoload | wc -l
216
$ grep -rHE '^[[:space:]]*bindkey' --exclude-dir=tests --exclude-dir=docs \
--exclude-dir=g-tags . | wc -l
228
```

Every one of the 216 lives in `autoload/`. The other twelve are in `scripts/`, which is where standalone scripts that build their own line editor bind their own keys. So 216 is “keybindings installed by the shell configuration” and 228 is “bindkey statements in the repository,” two different questions with two right answers and no note in the file saying which was asked. That is the characteristic failure of a hand-maintained metrics page — not a wrong number, but a number whose scope was implicit at the time it was written.

Two figures in the file do not agree with each other. The distinct-backing-function count appears as **373** four times — in the `<meta name="description">` tag on line 7, in the executive summary paragraph on line 102, in the repo-map table on line 172, and in the verb-dispatch paragraph on line 205 — and as **372** on line 210, in the stat card labelled **Distinct Functions**. One of the two is a typo and the file does not say which. Counting the first whitespace-delimited token of each verb value gives 361 by the most literal reading, which is a third number and demonstrates that the figure is sensitive to how the value is parsed — the values have the shape `function short-alias=description`, and whether the short alias, the conditional registrations, or the non-`zpw*` helpers are folded in changes the answer.

One claim in the file is checkable and wrong. The verb-dispatch section states that “`zpw verbs` (a.k.a. `zpwNumVerbs`, which echoes `$#ZPWR_VERBS`) reports the 462 key count.” `zpwNumVerbs` is three lines and does echo `$#ZPWR_VERBS`. But 462 is the count of *assignment lines*, and two keys — `search` and `zshsearch` — are assigned twice. An associative array holds one entry per key regardless of how many times it is assigned, which is demonstrable in two lines:

```
$ zsh -fc 'typeset -Ag A; A[x]=1; A[y]=2; A[x]=3; print ${#A}'
2
```

Three assignments, two entries. So `zpw verbs` reports 460, not 462, and the figure the README publishes at its lines 156 and 477 — 460 — is the number the command actually prints. The same duplicate pair is why `genEncyclopediaMd.py` counts verbs into a `set` and says so in its comment: “the count is the number of DISTINCT assigned keys — not the number of assignment lines (two keys are assigned twice, the later assignment winning).”

What The Comparison Shows

Three renderers, one source, and the divergences all point the same way. The terminal renderer is correct by construction because it executes the source. The book generator is correct because it re-derives structure the source does not carry. The HTML site assumes a structure the source never promised and produces 76 chapters where there are 45. The hand-typed report is the only one that can be wrong about the code without being wrong about the corpus, and it fails in exactly the two ways a hand-typed number fails — it goes stale, as the commit counts and the line count have, and it is copied, as the distinct-function figure was into five places where one copy came out different.

Chapter 86 – Eighty-Three Batches

The test suite has two halves with completely different construction. One half is the per-property families — one test per symbol, generated across the whole autoload tree, which is what makes 2,731-line test files exist. The other half is 83 files named `t-lib-batch*`, and they are hand-written, incremental, and deliberately shapeless.

The Census

```
files          83
lines         21,767
@test blocks   3,884
```

The names run `t-lib-batch2.zsh` through `t-lib-batch83.zsh`, then jump to `t-lib-batch86-r4.zsh`. There is no `batch1`, no `batch84`, no `batch85`. The gaps are the honest artifact of an append-only naming scheme: a batch that was merged into another or abandoned leaves a hole, and renumbering the survivors would break every reference to them. `docs/README.md` refers to the range as “`tests/t-lib-batch15.zsh` through `tests/t-lib-batch86-r4.zsh`”, which is how a sparse range gets described once nobody is counting.

Every file carries a `Purpose:` line in its header — 83 of 83 — and the purposes read like a work log rather than a taxonomy:

```
##### Purpose: env verb greps; wizard_pages zsh -n;
                zpwriHistoryVerbAccept-zpwriLint autoload zsh -n
##### Purpose: stryke on scripts/*.stk (beyond boxPrint), bash -n on
                install/*.sh, verb spots
##### Purpose: python3 compile on scripts/*.py, ci.yml extras, verbs,
                zpwriCommandExists
##### Purpose: autoload layout, install/hooks/pre-push, ci.yml pins, clean verbs
```

Several are explicit continuations. One header reads “Continues batch39 after `zpwriHistfile`” and names the convention it inherits: “`env greps use grep -Fq 'ZPWR_VERBS[key]='””. These are not thematic modules. They are checkpoints in a sweep — someone walked the tree, and each file is where they stopped.`

One Assertion, Repeated 3,884 Times

The uniformity is total. Extracting every `assert` line from the corpus and normalising the variable gives exactly one form, and comparing against the literal shows how narrow it is: 3,881 of the 3,884 assertions are

```
assert $state equals 0
```

and the remaining three are `assert $state equals 1`. There is no `assert $output contains`, no `assert ... is_empty`, no matcher of any other kind anywhere in 21,767 lines.

That is a real design decision with a real cost. Every test is a `run <command>` followed by an exit-status check, so every test asks “does this succeed” and none asks “is the answer right.” A test that a verb is registered cannot also check what it is registered to; a test that a file parses cannot also check what it parses to. The corpus verifies that nothing is broken, not that anything is correct.

What it buys is that every test is writable in four lines, which is why there are 3,884 of them. The trade is deliberate for the thing being tested: the framework is lazily-loaded shell, where the overwhelming failure mode is a syntax error in a file that nothing has sourced yet, and a syntax error is exactly what an exit status catches.

What Gets Run

The `run` targets divide into four groups, counted across the corpus:

```
run zsh -n          1,810
grep -Fq           370
run test -f        174
run bash -n         31
run python3 -m py_compile 3
run sh -n           1
```

`zsh -n` at 1,810 invocations is the corpus's centre of gravity. It parses a file and exits without executing it, which is the only compiler this codebase has. Against a tree whose functions are autoloading — where a broken file stays silent until the first call, possibly months later — a parse check per file is not a formality; it is the entire type system.

The three other interpreters appear in proportion to how much of each language the tree contains: 31 `bash -n` for `install/*.sh` and the bash script trees, one `sh -n` for the single POSIX-shell file (`install/hooks/pre-push`), and three `python3 -m py_compile` for the Python scripts. Every language in the repository has a parse gate and none is exempt.

The 370 `grep -Fq` invocations are the second family, and the `-F` is the point. Fixed-string matching against a file is how the corpus asserts registration without loading anything:

```
run test -n "${ZPWR_VERBS[emacshtags]+x}"
```

and its file-based sibling, `grep -Fq 'ZPWR_VERBS[key]='`. The `+x` parameter expansion is the correct spelling for “is this key set” — it yields `x` for a set key with an empty value and nothing for an unset one, which a plain `${ZPWR_VERBS[key]}` test would conflate. There are 1,169 lines in the corpus mentioning `ZPWR_VERBS[`, making the verb table the single most-asserted object in the suite.

`zpwrCommandExists` appears 431 times, which is the corpus checking that the framework's own predicate resolves the tools it depends on — a check that has to be a runtime assertion because the answer differs per machine.

Why This Shape Survives

A reviewer's instinct is to consolidate: 83 files with overlapping purposes and a single assertion form is, on its face, a corpus that wants to be a loop. The per-property families elsewhere in the suite are exactly that loop, and they exist, so the batches are not there because nobody thought of it.

The difference is what each half can express. A generated family applies one property to every symbol, which means it can only assert things true of every symbol. The batches assert things true of one file: that `ci.yml` still pins a particular version, that `install/hooks/pre-push` still parses as POSIX sh, that a specific verb cluster is registered together. Those are the checks that catch a regression in something a generator would have to be taught about first.

The corpus is also the reason the suite's headline number is what it is. Of the 14,278 `@test` cases in `tests/`, 3,884 — a bit over a quarter — are batches. The rest are the generated families. Both halves run in the same CI invocation — `zunit --verbose tests/*.zsh`, at `.github/workflows/ci.yml:145` — and neither can tell the difference.

Chapter 87 – Two Thousand Lines Of Vim Script

`install/.vimrc` is 1,998 lines. Its plugin loading is a small part of that and is covered elsewhere; what fills the rest is an editing layer built almost entirely out of `:normal!` sequences — a style of vim script that has mostly disappeared from modern configurations, and which does things a plugin-based configuration cannot easily do.

The Surface

```
set statements          67
function definitions    45
autocmd mentions       54
Plugin declarations    15
mapping statements     294
folds opened with "{{{ 15
```

294 mappings is the number that characterises the file, and the breakdown accounts for all of them:

```
nnoimap 118  map 34  nmap 7  xmap 5  omap 3
inoimap 57  noremap 21  imap 6  xnoremap 3  vmap 1
vnoremap 35  onoremap 3  smap 1
```

237 of the 294 are `noremap` forms and 57 are recursive. That is the right direction — the recursive `map`, `nmap` and `imap` forms are the ones that change meaning when a plugin redefines their right-hand side — and the 34 bare `map` statements are the file's largest single source of future surprise.

The leader is set once, at line 43, to space:

```
let mapleader = "\<space>"
```

with the comment “backslash replaced by space for leader keybindings.” Space as leader is the single highest-leverage line in the file, because it makes the prefix reachable by either thumb and frees the entire `<leader>` namespace from competing with vim's own backslash bindings.

The Fold Convention Is A Cross-Language Contract

The file is divided by 15 fold openers of this shape:

```
"{{{ MARK:Mappings
"*****
...
" }}}*****
```

`{{{` and `}}}` are vim's own fold markers, so the divisions are functional rather than decorative — the file opens as fifteen closed folds. The `MARK:` prefix is the framework's convention and appears across the tree in shell scripts and elsewhere.

What makes it a contract rather than a habit is that the file can generate it. `s:commonFold()` at line 1117 takes a comment character and builds the same three lines:

```
let l:start = a:char. ' {{{ MARK: '
let l:sec   = a:char. ' *****
let l:end   = a:char. ' }}} *****
```

and wraps the current visual selection in them, ending with `call feedkeys('a')` so the cursor lands in insert mode on the marker's label. The comment character comes from `s:fileTypeToComment`, a dictionary mapping extensions to leaders — `java`, `c`, `js`, `cpp` to `//`; `sh`, `zsh`, `py`, `pl`, `rb` to `#`; `lisp` to `;;`.

The same convention is implemented twice more, in two other languages. The emacs configuration's `zpwir/extract-fold` builds an identical banner, choosing its comment string from `derived-mode-p` rather than from a file extension, and its cases line up with the vim dictionary's groups. And `install/UltiSnips/all.snippets` carries it as a snippet as well:

```
snippet fm "complete fold marker"
#{{{ MARK:$1
#*****
$2
#}}}*****
endsnippet
```

with three more variants — `fmarker`, `endmarker`, `marker` — for the partial forms. Three implementations, one output format, which is what makes the marker a tree-wide convention instead of an editor preference.

The Quoting Family

Lines 456 to 713 hold the file's most distinctive code: a family of functions that wrap text in delimiters by scripting normal-mode motions.

```
function! InsertVar(toInsert, front, back)
  exe 'normal! lmb'
  exe 'normal! F'.a:front
  exe 'normal! i'.a:toInsert
  exe 'normal! f'.a:back
  exe 'normal! i'.a:toInsert
  exe 'normal! `b'
endfunction
```

Every function in the family opens with `lmb` and closes with ``b`: move right one character, set mark `b`, do the work, jump back to mark `b`. That is cursor restoration done manually, and the initial `l` compensates for the insertion shifting the cursor position. The body is a search backwards to the opening delimiter (`F`), an insert, a search forwards to the closing one (`f`), and a second insert.

Eight variants exist because the motions differ per delimiter class. `InsertEOLVar` uses `g_` — last non-blank character on the line — instead of a forward search, for the case where the closing position is the end of the line. `InsertEquals` uses `T` rather than `F`, stopping one character after the target instead of on it, which is what you want when the delimiter is `=`. `InsertMatchingPunct` uses `%` to jump to the matching bracket rather than searching for a literal, so it works on nested pairs. `InsertBackTick` searches backwards twice, because the opening backtick of a pair is the second one found going left.

`ReplaceBracket` is the family's most aggressive member: it deletes two characters at the opening delimiter (`hxx`), inserts the replacement twice, then does the same at the closing one — a bracket-type conversion implemented as four cursor-relative edits.

Writing this with `getline()/setline()` and a regular expression would be shorter and more testable. Writing it with `:normal!` makes it work on whatever vim's motions consider a word or a bracket, including in filetypes with custom `iskeyword` settings, without the function knowing anything about the language. That is the trade,

and for an editing layer that must work in every filetype the configuration targets, it is the defensible side of it.

One Broken Mapping

The quoter mappings sit at lines 732 to 745, four normal-mode and six visual-mode. One of the ten has a typo:

```
vnoremap <silent> <leader>' :call InsertQuoteVisualMode("single")<CR<CR>
```

`<CR<CR>` is not a key notation. The behaviour is worth establishing rather than guessing, and it can be reproduced in isolation — defining the same mapping against a recording function, invoking it, and catching the exception:

```
broken
ERR-a:Vim(call):E488: Trailing characters: <CR
good
```

So the function does run. Vim parses the right-hand side as the literal characters `<CR` followed by a real `<CR>` key, which means the command line becomes `:call InsertQuoteVisualMode("single")<CR` and Enter; `:call` evaluates its argument and then rejects the trailing text with E488. The user gets the quoting they asked for and an error message every single time, and because the error arrives after the effect, the mapping looks like it works.

The neighbouring nine mappings are correct, which is why the typo survived: it is one character in a block of visually identical lines, and its symptom is noise rather than failure.

Two Plugin Managers, One Live

The file's first fold, lines 19 to 38, is a neobundle block in which every functional line is commented out — `set runtimepath+=`, `call neobundle#begin`, `neobundlefetch`, `call neobundle#end`. Exactly one line in the fold is live:

```
set nocompatible " be improved
```

which is neobundle's prerequisite and also a setting the file wants anyway. The block is a fossil kept for its comments, and the cost of keeping it is that a reader has to get 130 lines in before finding the manager that is actually running.

The live one is Vundle, at lines 130 onwards, with 15 `Plugin` declarations. Two of the declarations are conditional:

```
if has('nvim')
    Plugin 'vim-airline/vim-airline'
    Plugin 'vim-airline/vim-airline-themes'
endif
```

which is the file recognising that it is loaded by two different editors and that one of them wants a statusline plugin the other does not.

The plugin configuration immediately after is where the file's autosave policy lives:

```
let g:auto_save = 1
let g:auto_save_silent = 1
let g:auto_save_events = ['InsertLeave', 'TextChanged', 'TextChangedI']
```

`TextChangedI` fires on every change in insert mode, so this configuration writes the buffer continuously while typing, silently. That is a coherent choice for someone whose editor and shell are the same workspace and whose build tools watch the filesystem — and it is the setting most likely to surprise anyone who inherits the file, because it makes `u` the only undo and removes the save-as-checkpoint habit entirely.

Autocommands As Mapping Guards

The 54 autocommand references are mostly filetype hooks, but one pattern recurs and is worth naming:

```
augroup cwindow_group
  autocmd!
  autocmd VimEnter * noremap <silent> <leader>cn :cnext<CR>
  autocmd VimEnter * noremap <silent> <leader>cp :cprev<CR>
  autocmd VimEnter * noremap <silent> <leader>cc :cclose<CR>
  autocmd VimEnter * noremap <silent> <leader>co :copen<CR>
augroup end
```

with the comment “conflicts with nerd commenter.” These are mappings deferred to `VimEnter` — after plugins have loaded and installed their own — specifically so that they win. It is the same technique the shell configuration uses for its late keybinding hooks, where `zpwriBindPenultimate` and `zpwriBindFinal` run at the end of the plugin ladder for exactly this reason. Wrapping them in a named `augroup` with a leading `autocmd!` is the correct hygiene: re-sourcing the vimrc clears the group’s previous definitions instead of stacking a second copy.

Chapter 88 — Forty Functions In A Package List

`install/.spacemacs` is 1,534 lines. Most of a Spacemacs dotfile is a settings form, and most of this one is too — but 40 of its functions sit in a project-local `zpwir/` namespace, and they turn a configuration file into the emacs half of the same editing layer the vimrc implements.

The Layer List Is A Language Census

`dotspacemacs-configuration-layers` runs from line 36 to line 114 and holds 58 bare layer names, of which 56 are distinct — `emacs-lisp` and `slack` are each listed twice, which Spacemacs tolerates and which is the same duplicate-in-a-hand-maintained-list pattern the shell's plugin file shows. Three further entries are written as `(layer :variables ...)` forms; two of those three, `colors` and `version-control`, also appear as bare names higher in the same list.

The list is worth reading as a statement of scope. It contains language layers for perl5, typescript, go, javascript, ruby, ruby-on-rails, django, python, scala, haskell, erlang, elixir, clojure, common-lisp, ocaml, swift, rust, c-c++, lua, asm, sql, latex, yaml, csv, markdown, html, react, terraform, ansible, nginx, docker and windows-scripts. That is 32 language and platform layers in one editor configuration.

Four entries are not languages and are the interesting ones:

```
gtags    GNU GLOBAL integration
fasd     the same frecency database the shell uses
tmux     pane navigation shared with the multiplexer
shell    an embedded terminal
```

Those four are where the editor joins the rest of the stack. `gtags` is the emacs consumer of the same GNU GLOBAL database the shell verbs build. `fasd` reads the same frecency store that the shell's jump commands write. `tmux` makes emacs windows and tmux panes navigate with one set of keys. The editor is not configured as a separate environment that happens to be installed; it is configured as a client of the shell's own indexes.

The one layer that takes a variable block and is not also listed bare is a policy:

```
(auto-completion :variables
  auto-completion-return-key-behavior 'complete
  auto-completion-idle-delay 3
  auto-completion-tab-key-behavior 'cycle
  auto-completion-enable-snippets-in-popup t
  auto-completion-enable-sort-by-usage t)
```

`auto-completion-idle-delay 3` is three seconds, which is an order of magnitude above the usual sub-second default. Combined with `return-key-behavior 'complete` — Enter accepts the completion rather than inserting a newline — the configuration is tuned so that the popup appears only when the user has actually stopped, and is then decisive when it does. That is the opposite of the aggressive-popup default and is consistent with the shell configuration's treatment of autosuggestions.

`dotspacemacs-additional-packages` adds 51 more package names outside the layer system, including five separate `helm-*` search front-ends, `evil-collection`, `evil-surround`, `evil-snipe`, and two GitHub-sourced recipes pinned by `:location (recipe :fetcher github :repo ...)`.

Vim Bindings Are Not Optional

```
dotspacemacs-editing-style '(vim :variables ...)
dotspacemacs-leader-key "SPC"
dotspacemacs-emacs-command-key "SPC"
dotspacemacs-ex-command-key ":"
dotspacemacs-major-mode-leader-key ","
```

The leader is space, which is the same leader the vimrc sets at its line 43. The two editors share a prefix key deliberately, so a <leader>-prefixed reflex works in both. `dotspacemacs-major-mode-leader-key ",` adds a second namespace for mode-specific commands, which vim has no equivalent of and which is the one place the two configurations diverge structurally rather than incidentally.

The `zpwr/` Namespace

40 functions are defined as `zpwr/...`, alongside 95 key-binding statements, 14 `add-hook` calls and one `advice-add`. Prefixing every one of them is the same discipline the shell uses with `zpwr*`: in a Lisp where all definitions share a global namespace and a package can define anything, a personal prefix is the only collision protection available.

A few show what the namespace is for.

`zpwr/extract-fold` is the emacs implementation of the vimrc's `s:commonFold`, building the same {{{
MARK: banner. Its comment-character selection uses `derived-mode-p` rather than a file extension:

```
((derived-mode-p 'zsh-mode 'sh-mode 'python-mode 'perl-mode 'cperl-mode
  'ruby-mode 'yaml-mode 'awk-mode) (setq comment "#"))
((derived-mode-p 'java-mode 'js-jsx-mode 'js2-mode 'c-mode 'c++-mode)
  (setq comment "//"))
((derived-mode-p 'inferior-emacs-lisp-mode 'emacs-lisp-mode 'lisp-mode)
  (setq comment ";;"))
((derived-mode-p 'vimrc-mode) (setq comment "\""))
```

`derived-mode-p` is the correct predicate here rather than an equality test, because it follows the mode hierarchy — a mode derived from `sh-mode` gets `#` without being listed.

`zpwr/complete-current-statement` is the same idea applied to line terminators, and it uses the identical mode grouping to decide whether the language wants a trailing semicolon. Two functions, one table, expressed twice — which is the cost of the mode dispatch being inline rather than in a shared alist.

`zpwr/add-timestamp-message` and `zpwr/current-time-microseconds` together retrofit timestamps onto the `*Messages*` buffer, which emacs does not do by default. `zpwr/inhibit-sentinel-messages-echo-area` and its buffer-targeted twin do the reverse — suppress process-sentinel chatter — and both are written as advice wrappers taking `(fun &rest args)`, the modern `advice-add` calling convention rather than the deprecated `defadvice` macro.

`zpwr/copy-to-clipboard` and `zpwr/paste-from-clipboard` exist because emacs running in a terminal has no access to the system clipboard, and the pair shells out. `zpwr/bypass-confirmation-all` wraps a function so that every `y-or-n-p` inside it answers yes — the emacs spelling of the same preference the shell expresses by never prompting on destructive operations.

Installation Is Two Lines

`install/spacemacs_install.sh` is 20 lines, of which 15 are the header and the directory guard. The work is:

```
ln -sf "$(pwd)/.spacemacs" "$HOME/.spacemacs"
git clone https://github.com/syl20bnr/spacemacs ~/.emacs.d
```

The symlink is the right mechanism — edits to the live dotfile land in the repository automatically, which is what keeps a 1,534-line configuration from drifting. Note that `.spacemacs` is not in the ten-entry symlink list at `install/plugins_install.sh:58`; this script is the only thing that links it, and it has to be run explicitly.

The clone is unpinned and unshallowed: no `--depth`, no `--branch`, no tag. Every install gets whatever the upstream default branch holds that day, against a 1,534-line configuration and 51 pinned-by-name packages. The framework pins elsewhere — the CI workflow has version pins that the batch tests assert — so the omission here is a gap rather than a policy.

Two Snippet Engines

The tree ships snippets for both editors, in both formats.

```
install/UltiSnips/          7 files   358 lines   42 snippets
install/emacs/snippets/sh-mode 23 snippet files + a README.md
```

UltiSnips is the vim side, keyed by filetype: `all.snippets` (12), `sh.snippets` (16), `perl.snippets` (7), `lisp.snippets` (2), `python.snippets` (2), `vim.snippets` (2), `javascript.snippets` (1). Yasnippet is the emacs side, and all of its shell snippets are in a single `sh-mode` directory as one file per snippet, which is the yasnippet convention, against UltiSnips' one file per filetype.

The two corpora are not translations of each other. The yasnippet names are spelled out — `if`, `ifn`, `ife`, `ifee`, `ifne`, `ifnee`, `ifeeee`, `case`, `select`, `until`, `while`, `function`, `args`, `bang`, `safe-bang`, `script-dir`, and three logging abbreviations (`logg`, `loggerr`, `loggdebug`) that expand to the framework's own console-logging functions. The UltiSnips shell names are contracted — `ie`, `ine`, `iee`, `inee`, `ieeee`, `ineee`, `fun`, `getopt`, `lo`, `loe`, `lod`, `pr`. Intersecting the two lists gives four shared names: `ff`, `fkv`, `ifn` and `p`.

So the abbreviations diverged even though the expansions cover the same constructs, which is what happens when two snippet corpora are maintained against two engines with no shared source. What did not diverge is the fold banner, which is byte-identical across the vimrc's `s:commonFold`, the `zpwir/extract-fold` in this file, and the `fm` snippet in `all.snippets`.

That is the theme the whole configuration demonstrates. The emacs setup is not an alternative to the vim setup; it is a translation of it, sharing the leader key, the fold convention, the mode-to-comment table, and — through the `gtags`, `fasd` and `tmux` layers — the shell's own indexes. Where the translation was done by hand and never reconciled, as with the snippet names, the two halves drift.

Appendix A – Keybinding Reference

Tmux Prefix Keys: Navigation & Panes

pane traversal, splitting, resizing. the grid of power.

Pane Selection (hjkl Repeatable):

prefix h	select pane left
prefix j	select pane down
prefix k	select pane up
prefix l	select pane right
prefix ;	last-pane (toggle to previous)
prefix o	select next pane in order
prefix q	display-panes (5s timeout)

Pane Resize (HJKL Repeatable, 5 Cells):

prefix H	resize pane left 5
prefix J	resize pane down 5
prefix K	resize pane up 5
prefix L	resize pane right 5
prefix Ctrl-Up/Down	resize pane by 1
prefix Alt-Up/Down	resize pane by 5

Pane Splitting:

prefix -	split vertical (same dir)
prefix \	split horizontal (same dir)
prefix "	split vertical (default dir)
prefix %	split horizontal (default dir)
prefix	split horizontal
prefix _	split vertical

Pane Operations:

prefix z	zoom/unzoom current pane
prefix !	break pane into own window
prefix x	kill current pane
prefix m	mark current pane
prefix +	split vertical then close prev pane
prefix Y	split vertical then close prev pane
prefix {	rotate window
prefix }	swap pane down

Arrow Key Selection:

```
prefix Up/Down      select pane up/down
prefix Left/Right   select pane left/right
Alt-Up/Down         select pane (root table, no prefix)
Alt-Left/Right      select pane (root table, no prefix)
```

```
All hjkl/HJKL bindings are repeatable (-r flag).
Arrow keys in root table work without prefix for speed.
```

```
// the wizard navigates panes by instinct
// hjkl is muscle memory, arrows are for civilians
```

Tmux Prefix Keys: Windows & Sessions

```
window management, session juggling, the multiplexer core.
```

Window Creation & Selection:

```
prefix c            new window
prefix a            last-window (toggle to previous)
prefix 0-9          select window by number
prefix '            select window by index prompt
prefix n            next window (repeatable)
prefix p            previous window (repeatable)
prefix Ctrl-n       next window
prefix Ctrl-p       previous window
```

Window Operations:

```
prefix ,           rename current window
prefix &           kill window (with confirm)
prefix .           move window to target
prefix f           find window by name
prefix w           choose-tree window view
prefix i           display-message (window info)
prefix t           clock-mode
```

Session Management:

```
prefix $           rename current session
prefix (           switch to previous client (repeatable)
prefix )           switch to next client (repeatable)
prefix d           detach client
prefix s           choose-tree session view
prefix Ctrl-z      suspend client
```

Reload & Persist:

```
prefix r          reload tmux config + display confirmation
prefix Ctrl-s     save session (tmux-resurrect)
prefix Ctrl-r     restore session (tmux-resurrect)
```

Root Table Shortcuts:

```
Ctrl-\\          switch to next client (no prefix)
Ctrl-]           switch to prev client (no prefix)
```

Meta:

```
prefix ?         list all keys
prefix :         command prompt
prefix /         describe a key binding
prefix ~         show messages
```

```
// sessions are parallel universes
// the wizard keeps many realities open at once
```

Tmux Prefix Keys: Layouts & Pane Configs

```
preconfigured pane arrangements. instant workspaces.
```

zpwr Layout Presets:

```
prefix D          control-window layout
prefix E          four vertical panes
prefix F          four panes layout
prefix G          eight panes layout
prefix O          sixteen panes layout
prefix R          thirty-two panes REPL layout
prefix T          config files layout
prefix M          learn layout
```

```
Each layout sources a conf file from $ZPWR/tmux/:
D -> control-window.conf
E -> fourVertical.conf
F -> four-panes.conf
G -> eight-panes.conf
O -> sixteen-panes.conf
R -> thirtytwo-panes-repl.conf
T -> config-files.conf
M -> learn.conf
```

Synchronize & Customize:

```
prefix S          toggle synchronize-panes (type in all)
prefix C          customize-mode
```

Built-in Layouts (Alt+number):

```
prefix Alt-1      even-horizontal
prefix Alt-2      even-vertical
prefix Alt-3      main-horizontal
prefix Alt-4      main-vertical
prefix Alt-5      tiled
prefix Alt-6      main-horizontal-mirrored
prefix Alt-7      main-vertical-mirrored
```

Window Rotation:

```
prefix Ctrl-o      rotate window panes
prefix Alt-o        rotate window panes (reverse)
```

Synchronize-panes (prefix S) sends keystrokes to ALL panes.
Perfect for running the same command across many servers.

```
// sixteen panes is not chaos
// sixteen panes is the wizard's dashboard
```

Tmux Prefix Keys: Special Operations

multi-pane execution, URL picking, buffers, plugins.

Multi-pane Execution:

```
prefix Space      run command in all panes (single)
prefix v          run command in all panes (multi)
prefix b          open URL/path in all panes
prefix g          google search in all panes
```

These source \$ZPWR/scripts/allPanes.zsh with different modes.
Space -> single mode (one command per pane)
v -> multi mode (batch operations)
b -> single open (open URL in each pane)
g -> single google (search in each pane)

Url Handling:

```
prefix e          arb --fzf URL picker -> open
prefix u          arb --fzf URL picker -> search
```

Both use tmux-fzf-url plugin with 30s timeout.

Paste & Buffers:

```
prefix P      paste buffer
prefix ]      paste buffer (bracketed)
prefix =      choose-buffer (arb --fzf style)
prefix #      list buffers
prefix Ctrl-v paste from system clipboard
```

Copy Mode:

```
prefix [      enter copy mode
prefix PgUp   enter copy mode and page up
```

Plugin Management (TPM):

```
prefix I      install plugins
prefix U      update plugins
prefix Alt-u   clean unused plugins
```

Send Prefix:

```
prefix Ctrl-a send prefix key to inner tmux
prefix Ctrl-] send literal Ctrl-]
```

Use prefix Ctrl-a when nesting tmux sessions.
The outer session captures prefix, Ctrl-a forwards it.

```
// buffers are the wizard's memory
// every copy is a spell stored for later
```

Tmux Copy Mode (vi)

tmux Copy Mode (VI Bindings)

```
scrollback navigation, selection, copy, search. pure vi.
```

Enter/exit:

```
prefix [      enter copy mode
q             cancel and exit copy mode
Escape        clear selection
```


Selection & Copy:

```
v          begin selection
V          select entire line
Ctrl-v     toggle rectangle selection
Space      begin selection (alternate)
y          copy selection to clipboard (no clear)
Enter      copy selection to clipboard and cancel
D          copy to end of line and cancel
A          append selection and cancel
```

Navigation:

```
h/j/k/l    cursor movement
Ctrl-h/j/k/l  move cursor 4 cells at a time
w/b/e      word forward/backward/end
W/B/E      WORD (space-delimited) movement
0 / $      start/end of line
^          first non-blank character
g / G      top / bottom of history
H / M / L  top / middle / bottom of screen
```

Scrolling:

```
Ctrl-u / Ctrl-d  half page up / down
Ctrl-b / Ctrl-f  full page up / down
Ctrl-y / Ctrl-e  scroll up / down one line
J / K           scroll down / up one line
```

Search:

```
/          search forward (incremental)
?          search backward (incremental)
n / N      next / previous match
*          search forward for word under cursor
#          search backward for word under cursor
```

zpwr Extensions:

```
s          copy + google search selection
x          copy + open URL selection
z          copy + google search selection
RightClick copy + google search selection
```

```
Copy goes to system clipboard via reattach-to-user-namespace pbcopy.
ZPWR extends vi copy mode with instant google/open actions.
```

```
// in copy mode the wizard moves through time
// scrollbar is archaeology, search is divination
```

Zsh Insert Mode: Basic & Movement

the primary editing mode. where commands are born.

Line Movement:

Ctrl-A	beginning-of-line
Ctrl-E	end-of-line
Ctrl-O	EOL or next tab stop

Editing:

Ctrl-U	clear entire line
Ctrl-W	delete word backward (autopair)
Ctrl-H	delete char backward (autopair)
Ctrl-Y	change surrounding quotes
Ctrl-Z	undo
Ctrl-R	redo
Ctrl-Q	duplicate last word

zpwz Special:

Space	zpwz supernatural expand (aliases+globs)
Ctrl-@	zpwz expand + terminate space
Ctrl-M (Enter)	zpwz magic enter (show status on empty)
.	zpwz rationalize dot (.. -> ../../)
Ctrl-B	zpwz clipboard paste
Ctrl-S	git add commit push (no check)
Ctrl-K	zsh-learn-Learn (flashcard)

Completion & Arb:

Tab (Ctrl-I)	fzf-completion
Ctrl-D	list-choices
Ctrl-L	clear-screen
Ctrl-T	arb --fzf file widget
Alt-c	arb --fzf cd widget

History:

Ctrl-N	down-history
Ctrl-P	up-history
Up/Down arrows	history substring search up/down

Mode Switching:

Escape	enter vi command mode
jf / fj	enter vi command mode (chord escape)

Surround & Pairs:

```
" ' ( [ { \ `      auto-pair/surround insert
) ] }              autopair close
Backspace          autopair-aware delete
```

Space is the most powerful key: it expands aliases, global aliases, and abbreviations before inserting.

```
// insert mode is where spells are typed
// every keystroke is an incantation
```

Zsh Insert Mode: Ctrl-F & Ctrl-V Chords

two-key combos that unlock the full zpwr arsenal.

Ctrl-F Chords (arb -fzf & Actions):

```
Ctrl-F Ctrl-D      pipe buffer into arb --fzf
Ctrl-F Ctrl-F      cd + vim arb --fzf file search (accept)
Ctrl-F Ctrl-G      google search
Ctrl-F Ctrl-H      lsof into arb --fzf
Ctrl-F Ctrl-I      ag (silver searcher) into arb --fzf
Ctrl-F Ctrl-J      zpwr verbs widget (accept)
Ctrl-F Ctrl-K      alternate quotes around buffer
Ctrl-F Ctrl-L      list choices
Ctrl-F Ctrl-M      zzcomplete
Ctrl-F Ctrl-N      zpwr verbs widget (browse)
Ctrl-F Ctrl-O      open URL from buffer
Ctrl-F Ctrl-P      basic sed substitution
Ctrl-F Ctrl-R      wrap buffer as variable
Ctrl-F Ctrl-S      git diff check (gacp)
Ctrl-F Ctrl-V      edit command line in $EDITOR
Ctrl-F j           zpwr verbs widget (accept)
Ctrl-F n           zpwr verbs widget (browse)
```

Ctrl-v Chords (vim/fzf/navigation):

```
Ctrl-V Ctrl-@      vim arb --fzf (open file in vim)
Ctrl-V Space       vim arb --fzf (open file in vim)
Ctrl-V Ctrl-F      fasd arb --fzf (frequency file open)
Ctrl-V Ctrl-G      arb --fzf cd widget
Ctrl-V Ctrl-K      emacs arb --fzf
Ctrl-V Ctrl-L      z arb --fzf (frequency dirs)
Ctrl-V Ctrl-N      vim arb --fzf sudo
Ctrl-V Ctrl-O      arb --fzf tab complete
Ctrl-V Ctrl-P      sudo command line toggle
Ctrl-V Ctrl-R      history search multi-word
Ctrl-V Ctrl-S      z arb --fzf (alternate)
Ctrl-V Ctrl-V      vim all files widget (accept)
Ctrl-V Ctrl-Z      arb --fzf history widget
Ctrl-V ,           arb --fzf environment variables
Ctrl-V .           arb --fzf all keybindings
```

Ctrl-V // locate arb -fzf

```
Ctrl-V /.      locate arb --fzf + edit
Ctrl-V ;       arb --fzf surround
Ctrl-V c       arb --fzf git commits
Ctrl-V k       arb --fzf vim keybindings
Ctrl-V m       vim all files widget (browse)
Ctrl-V v       vim all files widget (accept)
```

Ctrl-F is the action chord: search, edit, google, diff.
Ctrl-V is the vim/fzf chord: open files, browse, navigate.

```
// chords are the wizard's combos
// two keys unlock what single keys cannot reach
```

Zsh Visual & Menu Select Modes

text objects, selection, and completion menu navigation.

Visual Mode (enter With V/v In Vicmd):

```
Escape        deactivate region
j / k         move selection down / up
o             exchange point and mark
p             put replace selection
x             delete selection
u             downcase selection
U             upcase selection
~            swap case of selection
```

Text Objects (a = Around, I = Inner):

```
a" / i"       select around/inner double quotes
a' / i'       select around/inner single quotes
a\` / i\`      select around/inner backticks
a( / i(       select around/inner parentheses
a[ / i[       select around/inner brackets
a{ / i{       select around/inner braces
a< / i<       select around/inner angle brackets
aB / iB       select around/inner block
aw / iw       select a word / in word
aW / iW       select a blank word / in blank word
aa / ia       select a shell word / in shell word
```

Extended quote chars also work as text objects:
+ , - . / : ; = @ | \\\ are all valid quote delimiters.

Menu Select Mode (during Completion):

```
Ctrl-J / Ctrl-K      move down / up in menu
Ctrl-H / Ctrl-L      move left / right in menu
Tab / Ctrl-I         move forward (next item)
Ctrl-S / Shift-Tab   reverse menu complete
Ctrl-@ (Ctrl-Space)  accept line
Ctrl-D              accept and menu complete (stay open)
Ctrl-F              accept and infer next
Ctrl-M (Enter)       accept line (.accept-line)
Ctrl-V              insert mode within menu
Ctrl-N / Ctrl-P      forward / backward word in menu
Ctrl-X              incremental search forward in menu
?                   incremental search backward in menu
.                   self-insert (exit menu + type dot)
Backspace           undo last menu selection
```

Menu select uses hjkl-style navigation with Ctrl prefix.
Ctrl-D is key: accept current match but keep menu open.

```
// visual mode is precision surgery
// select only what you need, transform only what you touch
```

Vim Normal Mode Highlights

vim Normal Mode: zpwr Highlights

```
leader mappings, arb --fzf integration, tmux synergy.
leader key is Space. only zpwr-specific bindings listed.
```

File & Buffer (Space Prefix):

```
Space w              :w! (force write)
Space e              :q! (force quit)
Space q              :qa! (quit all)
Space wq             :wq! (write and quit)
Space n / Space p    next / prev buffer
Space an / Space ap  next / prev in arglist
Space Tab            alternate or next buffer
Space t              new tab
Space s / Space v    horizontal / vertical split
```

Arb Integration (Space Prefix):

```
Space f      :Files! (arb --fzf files)
Space b      :Buffers! (arb --fzf buffers)
Space j      :Lines! (arb --fzf lines)
Space ag     :Ag (silver searcher)
Space aa     :Agg (ag variant)
Space rg     :Rg (ripgrep)
Space z      :Commits! (arb --fzf git commits)
Space ;      :Commands! (arb --fzf commands)
Space ,      :FZFMaps (arb --fzf all mappings)
Space ke / , / m / i  arb --fzf keys / maps / nmap / imap
Space ta / ma / env  arb --fzf tags / marks / env vars
Space / / .         locate all / colorscheme picker
```

History (Space Prefix):

```
Space hh      :History! (arb --fzf file history)
Space h/      :History/! (arb --fzf search history)
Space h:      :History:! (arb --fzf command history)
```

Search & Replace (Space Prefix):

```
Space r      replace word under cursor (whole file)
Space g      global substitute word under cursor
Space x      repeat last substitution on line
n / N       next / prev search (centered)
z/          toggle auto-highlight word under cursor
```

Git & Tmux:

```
Space hs      GitGutter stage hunk
Space hu      GitGutter undo hunk
Space hp      GitGutter preview hunk
[c / ]c       prev / next git hunk
Space vj      save + TmuxRepeat file
Space os / ox / oc  google / open URL / copy word
```

Minimap & Comments:

```
Space mm      open minimap
Space mt      toggle minimap
Space cc      NERDCommenter comment
Space c Space NERDCommenter toggle
Space cu      NERDCommenter uncomment
```

```
// the leader key is Space because power needs room
// every binding is one thought away from execution
```

Vim Insert & Visual Mode + Keybinding Search

vim mode highlights and how to find any keybinding.

vim Insert Mode Highlights:

Ctrl-A	jump to first non-blank (like ^)
Ctrl-E	end of line or dismiss popup
Ctrl-D	delete char forward or default
Ctrl-F	move right or scroll
Ctrl-S	isurround (surround insert)
Ctrl-G s / S	isurround / ISurround (line)
Ctrl-Tab	list ultisnips snippets
Shift-Tab	SuperTab backward completion
Alt-d	delete word forward
" ' () [] \ ` { }	delimitMate auto-pairing

vim Visual Mode Highlights:

Space r	replace word in selection range
Space g	global substitute in selection range
Space x	repeat last sub on selected lines
Space " ' \ ` ([{	surround selection with quote/bracket
Space b	copy selection to tmux buffer
Space = / Space -	grow / shrink selection (4 lines)
J / K	move selected lines down / up
Y	yank and move to end
< / >	indent / outdent (stay in visual)
S	vim-surround visual surround
go	copy selection + open as URL
gs	copy selection + google search
Space vl / vk / vj	TmuxRepeat repl / visual / file
Space em / ev / ef	extract method / variable / fold

Sneak & Motion:

s / S	sneak forward / backward (2-char)
f / F / t / T	sneak-enhanced find char
; / ,	repeat / reverse last sneak/find
w / b / e	CamelCaseMotion word movement

=

How To Search Keybindings:

=

zpwir allsearch	search ALL keybindings in arb --fzf
zpwir vimsearch	search vim keybindings in arb --fzf
zpwir zshsearch	search zsh keybindings in arb --fzf

Keybinding Cache Files:

```
$ZPWR_LOCAL/zpwrAllKeybindings.txt  
$ZPWR_LOCAL/zpwr{Vim,Zsh,Tmux}Keybindings.txt
```

Regenerate:

```
zpwr regenkeybindings  regenerate all keybinding caches
```

Cache files are auto-generated. Run regenkeybindings after adding new bindings to refresh the searchable index.

```
// the wizard does not memorize 4000 keybindings  
// the wizard searches them at the speed of thought  
// arb --fzf is the oracle, the cache is the grimoire
```


Appendix B – Complete Keybinding Dump

ALL Tmux Prefix Keys (1/4)

```
bind-key -T prefix Space run-shell -b 'ZPWR/scripts/allPanes.zsh single
bind-key -T prefix . break-pane
bind-key -T prefix \ ' split-window
bind-key -T prefix \# list-buffers
bind-key -T prefix \ command-prompt -I '#S' { rename-session '%' }
bind-key -T prefix \% split-window -h
bind-key -T prefix & confirm-before -p 'kill-window #W? (y/n)' kill-win
bind-key -T prefix \ ' command-prompt -T window-target -p index { selec
bind-key -r -T prefix ( switch-client -p
bind-key -r -T prefix ) switch-client -n
bind-key -T prefix + split-window -v -c '#{pane_current_path}' \; sele
bind-key -T prefix , command-prompt -I '#W' { rename-window '%' }
bind-key -T prefix - split-window -v -c '#{pane_current_path}'
bind-key -T prefix . command-prompt -T target { move-window -t '%' }
bind-key -T prefix / command-prompt -k -p key { list-keys -lN '%' }
bind-key -T prefix 0 select-window -t :=0
bind-key -T prefix 1 select-window -t :=1
bind-key -T prefix 2 select-window -t :=2
bind-key -T prefix 3 select-window -t :=3
bind-key -T prefix 4 select-window -t :=4
bind-key -T prefix 5 select-window -t :=5
bind-key -T prefix 6 select-window -t :=6
bind-key -T prefix 7 select-window -t :=7
bind-key -T prefix 8 select-window -t :=8
bind-key -T prefix 9 select-window -t :=9
bind-key -T prefix : command-prompt
bind-key -T prefix \; last-pane
bind-key -T prefix < display-menu -T '#[align=centre]#{window_index}:#{
bind-key -T prefix = choose-buffer -Z
bind-key -T prefix > display-menu -T '#[align=centre]#{pane_index} ({#p
bind-key -T prefix ? list-keys -N
bind-key -T prefix C customize-mode -Z
bind-key -T prefix D source-file ZPWR/tmux/control-window.conf
bind-key -T prefix E source-file ZPWR/tmux/fourVertical.conf
bind-key -T prefix F source-file ZPWR/tmux/four-panes.conf
bind-key -T prefix G source-file ZPWR/tmux/eight-panes.conf
bind-key -r -T prefix H resize-pane -L 5
bind-key -T prefix I run-shell HOME/.tmux/plugins/tpm/bindings/install_
bind-key -r -T prefix J resize-pane -D 5
bind-key -r -T prefix K resize-pane -U 5
bind-key -r -T prefix L resize-pane -R 5
bind-key -T prefix M source-file ZPWR/tmux/learn.conf
bind-key -T prefix O source-file ZPWR/tmux/sixteen-panes.conf
bind-key -T prefix P paste-buffer
bind-key -T prefix R source-file ZPWR/tmux/thirtytwo-panes-repl.conf
bind-key -T prefix S set-window-option synchronize-panes
bind-key -T prefix T source-file ZPWR/tmux/config-files.conf
bind-key -T prefix U run-shell HOME/.tmux/plugins/tpm/bindings/update_p
```

ALL Tmux Prefix Keys (2/4)

```
bind-key -T prefix Y split-window -v -c '#{pane_current_path}' \; sele
bind-key -T prefix [ copy-mode
bind-key -T prefix \\\ split-window -h -c '#{pane_current_path}'
bind-key -T prefix ] paste-buffer -p
bind-key -T prefix _ split-window -v
bind-key -T prefix a last-window
bind-key -T prefix b run-shell -b 'ZPWR/scripts/allPanes.zsh single ope
bind-key -T prefix c new-window
bind-key -T prefix d detach-client
bind-key -T prefix e run-shell -b 'HOME/.tmux/plugins/tmux-fzf-url/fzf-
bind-key -T prefix f command-prompt { find-window -Z '%%' }
bind-key -T prefix g run-shell -b 'ZPWR/scripts/allPanes.zsh single goo
bind-key -r -T prefix h select-pane -L
bind-key -T prefix i display-message
bind-key -r -T prefix j select-pane -D
bind-key -r -T prefix k select-pane -U
bind-key -r -T prefix l select-pane -R
bind-key -T prefix m select-pane -m
bind-key -r -T prefix n next-window
bind-key -T prefix o select-pane -t :.+
bind-key -r -T prefix p previous-window
bind-key -T prefix q display-panes -d 5000
bind-key -T prefix r source-file ZPWR/tmux/init.conf \; display-messag
bind-key -T prefix s choose-tree -Zs
bind-key -T prefix t clock-mode
bind-key -T prefix u run-shell -b 'HOME/.tmux/plugins/tmux-fzf-url/fzf-
bind-key -T prefix v run-shell -b 'ZPWR/scripts/allPanes.zsh multi'
bind-key -T prefix w choose-tree -Zw
bind-key -T prefix x kill-pane
bind-key -T prefix z resize-pane -Z
bind-key -r -T prefix \{ rotate-window
bind-key -T prefix | split-window -h
bind-key -T prefix \} swap-pane -D
bind-key -T prefix \~ show-messages
bind-key -r -T prefix DC refresh-client -c
bind-key -T prefix PPage copy-mode -u
bind-key -r -T prefix Up select-pane -U
bind-key -r -T prefix Down select-pane -D
bind-key -r -T prefix Left select-pane -L
bind-key -r -T prefix Right select-pane -R
bind-key -T prefix M-1 select-layout even-horizontal
bind-key -T prefix M-2 select-layout even-vertical
bind-key -T prefix M-3 select-layout main-horizontal
bind-key -T prefix M-4 select-layout main-vertical
bind-key -T prefix M-5 select-layout tiled
bind-key -T prefix M-6 select-layout main-horizontal-mirrored
bind-key -T prefix M-7 select-layout main-vertical-mirrored
bind-key -T prefix M-n next-window -a
```

ALL Tmux Prefix Keys (3/4)

```
bind-key -T prefix M-o rotate-window -D
bind-key -T prefix M-p previous-window -a
bind-key -T prefix M-u run-shell HOME/.tmux/plugins/tpm/bindings/clean_
bind-key -r -T prefix M-Up resize-pane -U 5
bind-key -r -T prefix M-Down resize-pane -D 5
bind-key -r -T prefix M-Left resize-pane -L 5
bind-key -r -T prefix M-Right resize-pane -R 5
bind-key -T prefix C-] send-keys ^]
bind-key -T prefix C-a send-prefix
bind-key -T prefix C-n next-window
bind-key -T prefix C-o rotate-window
bind-key -T prefix C-p previous-window
bind-key -T prefix C-r run-shell HOME/.tmux/plugins/tmux-resurrect/scri
bind-key -T prefix C-s run-shell HOME/.tmux/plugins/tmux-resurrect/scri
bind-key -T prefix C-v run-shell 'tmux set buffer \'(reattach-to-user-
bind-key -T prefix C-z suspend-client
bind-key -r -T prefix C-Up resize-pane -U
bind-key -r -T prefix C-Down resize-pane -D
bind-key -r -T prefix C-Left resize-pane -L
bind-key -r -T prefix C-Right resize-pane -R
bind-key -r -T prefix S-Up refresh-client -U 10
bind-key -r -T prefix S-Down refresh-client -D 10
bind-key -r -T prefix S-Left refresh-client -L 10
bind-key -r -T prefix S-Right refresh-client -R 10
i <C-X><C-L> * fzf#vim#complete(fzf#wrap({ 'prefix': '^.*', 'source': 'r
<Space><Space> <Plug>(easymotion-prefix)
<Plug>(easymotion-prefix)N <Plug>(easymotion-N)
<Plug>(easymotion-prefix)n <Plug>(easymotion-n)
<Plug>(easymotion-prefix)k <Plug>(easymotion-k)
<Plug>(easymotion-prefix)j <Plug>(easymotion-j)
<Plug>(easymotion-prefix)gE <Plug>(easymotion-gE)
<Plug>(easymotion-prefix)ge <Plug>(easymotion-ge)
<Plug>(easymotion-prefix)E <Plug>(easymotion-E)
<Plug>(easymotion-prefix)e <Plug>(easymotion-e)
<Plug>(easymotion-prefix)B <Plug>(easymotion-B)
<Plug>(easymotion-prefix)b <Plug>(easymotion-b)
<Plug>(easymotion-prefix)W <Plug>(easymotion-W)
<Plug>(easymotion-prefix)w <Plug>(easymotion-w)
<Plug>(easymotion-prefix)T <Plug>(easymotion-T)
<Plug>(easymotion-prefix)t <Plug>(easymotion-t)
<Plug>(easymotion-prefix)s <Plug>(easymotion-s)
<Plug>(easymotion-prefix)F <Plug>(easymotion-F)
<Plug>(easymotion-prefix)f <Plug>(easymotion-f)
<Space><Space> <Plug>(easymotion-prefix)
<Plug>(easymotion-prefix)N <Plug>(easymotion-N)
<Plug>(easymotion-prefix)n <Plug>(easymotion-n)
<Plug>(easymotion-prefix)k <Plug>(easymotion-k)
<Plug>(easymotion-prefix)j <Plug>(easymotion-j)
```

ALL Tmux Prefix Keys (4/4)

```
<Plug>(easymotion-prefix)gE <Plug>(easymotion-gE)
<Plug>(easymotion-prefix)ge <Plug>(easymotion-ge)
<Plug>(easymotion-prefix)E <Plug>(easymotion-E)
<Plug>(easymotion-prefix)e <Plug>(easymotion-e)
<Plug>(easymotion-prefix)B <Plug>(easymotion-B)
<Plug>(easymotion-prefix)b <Plug>(easymotion-b)
<Plug>(easymotion-prefix)W <Plug>(easymotion-W)
<Plug>(easymotion-prefix)w <Plug>(easymotion-w)
<Plug>(easymotion-prefix)T <Plug>(easymotion-T)
<Plug>(easymotion-prefix)t <Plug>(easymotion-t)
<Plug>(easymotion-prefix)s <Plug>(easymotion-s)
<Plug>(easymotion-prefix)F <Plug>(easymotion-F)
<Plug>(easymotion-prefix)f <Plug>(easymotion-f)
```

ALL Tmux Copy-Mode-Vi (1/2)

```
bind-key -T copy-mode-vi Enter send-keys -X copy-pipe-and-cancel 'reatta
bind-key -T copy-mode-vi Escape send-keys -X clear-selection
bind-key -T copy-mode-vi Space send-keys -X begin-selection
bind-key -T copy-mode-vi \# send-keys -FX search-backward -- '#{copy_cu
bind-key -T copy-mode-vi \ send-keys -X end-of-line
bind-key -T copy-mode-vi \% send-keys -X next-matching-bracket
bind-key -T copy-mode-vi * send-keys -FX search-forward -- '#{copy_curso
bind-key -T copy-mode-vi , send-keys -X jump-reverse
bind-key -T copy-mode-vi / command-prompt -i -p '(search down)' 'send -X
bind-key -T copy-mode-vi 0 send-keys -X start-of-line
bind-key -T copy-mode-vi 1 command-prompt -N -I 1 -p (repeat) { send-key
bind-key -T copy-mode-vi 2 command-prompt -N -I 2 -p (repeat) { send-key
bind-key -T copy-mode-vi 3 command-prompt -N -I 3 -p (repeat) { send-key
bind-key -T copy-mode-vi 4 command-prompt -N -I 4 -p (repeat) { send-key
bind-key -T copy-mode-vi 5 command-prompt -N -I 5 -p (repeat) { send-key
bind-key -T copy-mode-vi 6 command-prompt -N -I 6 -p (repeat) { send-key
bind-key -T copy-mode-vi 7 command-prompt -N -I 7 -p (repeat) { send-key
bind-key -T copy-mode-vi 8 command-prompt -N -I 8 -p (repeat) { send-key
bind-key -T copy-mode-vi 9 command-prompt -N -I 9 -p (repeat) { send-key
bind-key -T copy-mode-vi : command-prompt -p '(goto line)' { send-keys -
bind-key -T copy-mode-vi \; send-keys -X jump-again
bind-key -T copy-mode-vi ? command-prompt -i -p '(search up)' 'send -X s
bind-key -T copy-mode-vi A send-keys -X append-selection-and-cancel
bind-key -T copy-mode-vi B send-keys -X previous-space
bind-key -T copy-mode-vi D send-keys -X copy-pipe-end-of-line-and-cancel
bind-key -T copy-mode-vi E send-keys -X next-space-end
bind-key -T copy-mode-vi F command-prompt -l -p '(jump backward)' { send
bind-key -T copy-mode-vi G send-keys -X history-bottom
bind-key -T copy-mode-vi H send-keys -X top-line
bind-key -T copy-mode-vi J send-keys -X scroll-down
bind-key -T copy-mode-vi K send-keys -X scroll-up
bind-key -T copy-mode-vi L send-keys -X bottom-line
bind-key -T copy-mode-vi M send-keys -X middle-line
bind-key -T copy-mode-vi N send-keys -X search-reverse
bind-key -T copy-mode-vi P send-keys -X toggle-position
bind-key -T copy-mode-vi T command-prompt -l -p '(jump to backward)' { s
bind-key -T copy-mode-vi V send-keys -X select-line
bind-key -T copy-mode-vi W send-keys -X next-space
bind-key -T copy-mode-vi X send-keys -X set-mark
bind-key -T copy-mode-vi ^ send-keys -X back-to-indentation
bind-key -T copy-mode-vi b send-keys -X previous-word
bind-key -T copy-mode-vi e send-keys -X next-word-end
bind-key -T copy-mode-vi f command-prompt -l -p '(jump forward)' { send-
bind-key -T copy-mode-vi g send-keys -X history-top
bind-key -T copy-mode-vi h send-keys -X cursor-left
bind-key -T copy-mode-vi j send-keys -X cursor-down
bind-key -T copy-mode-vi k send-keys -X cursor-up
bind-key -T copy-mode-vi l send-keys -X cursor-right
```

ALL Tmux Copy-Mode-Vi (2/2)

```
bind-key -T copy-mode-vi n send-keys -X search-again
bind-key -T copy-mode-vi o send-keys -X other-end
bind-key -T copy-mode-vi q send-keys -X cancel
bind-key -T copy-mode-vi r send-keys -X refresh-from-pane
bind-key -T copy-mode-vi s send-keys -X copy-pipe-no-clear 'reattach-to-
bind-key -T copy-mode-vi t command-prompt -l -p '(jump to forward)' { se
bind-key -T copy-mode-vi v send-keys -X begin-selection
bind-key -T copy-mode-vi w send-keys -X next-word
bind-key -T copy-mode-vi x send-keys -X copy-pipe-no-clear 'reattach-to-
bind-key -T copy-mode-vi y send-keys -X copy-pipe-no-clear 'reattach-to-
bind-key -T copy-mode-vi z send-keys -X copy-pipe-no-clear 'reattach-to-
bind-key -T copy-mode-vi \{ send-keys -X previous-paragraph
bind-key -T copy-mode-vi \} send-keys -X next-paragraph
bind-key -T copy-mode-vi WheelUpPane select-pane \; send-keys -X -N 5 s
bind-key -T copy-mode-vi WheelDownPane select-pane \; send-keys -X -N 5
bind-key -T copy-mode-vi DoubleClick1Pane select-pane \; send-keys -X s
bind-key -T copy-mode-vi TripleClick1Pane select-pane \; send-keys -X s
bind-key -T copy-mode-vi BSpace send-keys -X cursor-left
bind-key -T copy-mode-vi Home send-keys -X start-of-line
bind-key -T copy-mode-vi End send-keys -X end-of-line
bind-key -T copy-mode-vi NPage send-keys -X page-down
bind-key -T copy-mode-vi PPage send-keys -X page-up
bind-key -T copy-mode-vi Up send-keys -X cursor-up
bind-key -T copy-mode-vi Down send-keys -X cursor-down
bind-key -T copy-mode-vi Left send-keys -X cursor-left
bind-key -T copy-mode-vi Right send-keys -X cursor-right
bind-key -T copy-mode-vi M-x send-keys -X jump-to-mark
bind-key -T copy-mode-vi C-b send-keys -X page-up
bind-key -T copy-mode-vi C-c send-keys -X cancel
bind-key -T copy-mode-vi C-d send-keys -X halfpage-down
bind-key -T copy-mode-vi C-e send-keys -X scroll-down
bind-key -T copy-mode-vi C-f send-keys -X page-down
bind-key -T copy-mode-vi C-h send-keys -X -N 4 cursor-left
bind-key -T copy-mode-vi C-j send-keys -X -N 4 cursor-down
bind-key -T copy-mode-vi C-k send-keys -X -N 4 cursor-up
bind-key -T copy-mode-vi C-l send-keys -X -N 4 cursor-right
bind-key -T copy-mode-vi C-n send-keys -X page-down
bind-key -T copy-mode-vi C-p send-keys -X page-up
bind-key -T copy-mode-vi C-u send-keys -X halfpage-up
bind-key -T copy-mode-vi C-v send-keys -X rectangle-toggle
bind-key -T copy-mode-vi C-y send-keys -X scroll-up
bind-key -T copy-mode-vi C-Up send-keys -X scroll-up
bind-key -T copy-mode-vi C-Down send-keys -X scroll-down
```

ALL Zsh Insert Mode (1/3)

```
bindkey -M viins '^@' zpwrExpandTerminateSpace
bindkey -M viins '^A' beginning-of-line
bindkey -M viins '^B' zpwrClipboard
bindkey -M viins '^C' self-insert
bindkey -M viins '^D' list-choices
bindkey -M viins '^E' end-of-line
bindkey -M viins '^F^D' zpwrIntoFzf
bindkey -M viins '^F^F' zpwrCdVimFzfFilesearchWidgetAccept
bindkey -M viins '^F^G' zpwrGoogle
bindkey -M viins '^F^H' zpwrLsoffzf
bindkey -M viins '^F^I' zpwrIntoFzfAg
bindkey -M viins '^F^J' zpwrVerbsWidgetAccept
bindkey -M viins '^F^K' zpwrAlternateQuotes
bindkey -M viins '^F^L' list-choices
bindkey -M viins '^F^M' zzcomplete
bindkey -M viins '^F^N' zpwrVerbsWidget
bindkey -M viins '^F^O' zpwrOpen
bindkey -M viins '^F^P' basicSedSub
bindkey -M viins '^F^R' zpwrAsVar
bindkey -M viins '^F^S' zsh-gacp-CheckDiff
bindkey -M viins '^F^V' edit-command-line
bindkey -M viins '^Fj' zpwrVerbsWidgetAccept
bindkey -M viins '^Fn' zpwrVerbsWidget
bindkey -M viins '^G' what-cursor-position
bindkey -M viins '^H' autopair-delete
bindkey -M viins '^I' fzf-completion
bindkey -M viins '^J' accept-line
bindkey -M viins '^K' zsh-learn-learn
bindkey -M viins '^L' clear-screen
bindkey -M viins '^M' zpwrMagicEnter
bindkey -M viins '^N' down-history
bindkey -M viins '^O' zpwrEOLorNextTabStop
bindkey -M viins '^O^I' zc-logo
bindkey -M viins '^O^Z' zui-demo-various
bindkey -M viins '^P' up-history
bindkey -M viins '^Q' zpwrLastWordDouble
bindkey -M viins '^R' redo
bindkey -M viins '^S' zsh-gacp-NoCheck
bindkey -M viins '^T' fzf-file-widget
bindkey -M viins '^U' zpwrClearLine
bindkey -M viins '^V^@' zpwrVimFzf
bindkey -M viins '^V^F' zpwrFasdFZF
bindkey -M viins '^V^G' fzf-cd-widget
bindkey -M viins '^V^K' zpwrEmacsFzf
bindkey -M viins '^V^L' zpwrZFZF
bindkey -M viins '^V^N' zpwrVimFzfSudo
bindkey -M viins '^V^O' fzf-tab-complete
bindkey -M viins '^V^P' sudo-command-line
```


ALL Zsh Insert Mode (2/3)

```
bindkey -M viins '^V^R' history-search-multi-word
bindkey -M viins '^V^S' zpwrZFZF
bindkey -M viins '^V^V' zpwrVimAllWidgetAccept
bindkey -M viins '^V^Z' fzf-history-widget
bindkey -M viins '^V ' zpwrVimFzf
bindkey -M viins '^V,' zpwrFzfEnv
bindkey -M viins '^V.' zpwrFzfAllKeybind
bindkey -M viins '^V/.' zpwrLocateFzfEdit
```

bindkey -M viins '^V//' zpwrLocateFzf

```
bindkey -M viins '^V;' zpwrFzfSurround
bindkey -M viins '^Vc' zpwrFzfCommits
bindkey -M viins '^Vk' zpwrFzfVimKeybind
bindkey -M viins '^Vl' zpwrZFZF
bindkey -M viins '^Vm' zpwrVimAllWidget
bindkey -M viins '^Vv' zpwrVimAllWidgetAccept
bindkey -M viins '^W' autopair-delete-word
bindkey -M viins '^X^R' _read_comp
bindkey -M viins '^X^X' zpwrBufferXtrace
bindkey -M viins '^X?' _complete_debug
bindkey -M viins '^XC' _correct_filename
bindkey -M viins '^Xa' _expand_alias
bindkey -M viins '^Xc' _correct_word
bindkey -M viins '^Xd' _list_expansions
bindkey -M viins '^Xe' _expand_word
bindkey -M viins '^Xh' _complete_help
bindkey -M viins '^Xm' _most_recent_file
bindkey -M viins '^Xn' _next_tags
bindkey -M viins '^Xt' _complete_tag
bindkey -M viins '^X~' _bash_list-choices
bindkey -M viins '^Y' zpwrChangeQuotes
bindkey -M viins '^Z' undo
bindkey -M viins '^[' vi-cmd-mode
bindkey -M viins '^[_D' capitalize-word
bindkey -M viins '^[_E' zpwrExpandGlobalAliases
bindkey -M viins '^[_F' sub
bindkey -M viins '^[_L' down-case-word
bindkey -M viins '^[_M' self-insert-unmeta
bindkey -M viins '^[_O' zpwrRunner
bindkey -M viins '^[_P' zpwrUp8widget
bindkey -M viins '^[_T' transpose-words
bindkey -M viins '^[_U' up-case-word
bindkey -M viins '^[' zpwrSshRegain
bindkey -M viins '^[, ' _history-complete-newer
bindkey -M viins '^[/ ' _history-complete-older
bindkey -M viins '^[OA' history-substring-search-up
bindkey -M viins '^[OB' history-substring-search-down
bindkey -M viins '^[OC' vi-forward-char
bindkey -M viins '^[OD' vi-backward-char
```

ALL Zsh Insert Mode (3/3)

```
bindkey -M viins '^[OP' zpwrUp8widget
bindkey -M viins '^[OQ' sub
bindkey -M viins '^[OQ^[OQ' npm_toggle_install_uninstall
bindkey -M viins '^[OR' zpwrGetrcWidget
bindkey -M viins '^[Q' zi-browse-symbol
bindkey -M viins '^[[1;2D' sub
bindkey -M viins '^[[1;5A' zsh-gacp-CheckDiff
bindkey -M viins '^[[1;5B' zpwrUpdater
bindkey -M viins '^[[1;5C' zpwrTutsUpdate
bindkey -M viins '^[[1;5D' dbz
bindkey -M viins '^[[200~' bracketed-paste
bindkey -M viins '^[[5~' zpwrClipboard
bindkey -M viins '^[[A' history-substring-search-up
bindkey -M viins '^[[B' history-substring-search-down
bindkey -M viins '^[[C' vi-forward-char
bindkey -M viins '^[[D' vi-backward-char
bindkey -M viins '^[[Z' zpwrClipboard
bindkey -M viins '^[c' fzf-cd-widget
bindkey -M viins '^[man' man-command-line
bindkey -M viins '^[_' _bash_complete-word
bindkey -R -M viins '^\\\\"'^^' self-insert
bindkey -M viins '^_' zpwrAsVar
bindkey -M viins '^`' zpwrExpandSupernaturalSpace
bindkey -M viins '^.' self-insert
bindkey -M viins '^\' ' zpwrInterceptSurround
bindkey -R -M viins '^#-'&' self-insert
bindkey -R -M viins '^''-'(' zpwrInterceptSurround
bindkey -M viins '^)' autopair-close
bindkey -R -M viins '^*-'-' self-insert
bindkey -M viins '^.' zpwrRationalizeDot
bindkey -R -M viins '^/'-'Z' self-insert
bindkey -M viins '^[' zpwrInterceptSurround
bindkey -M viins '^\\\\"' self-insert
bindkey -M viins '^]' autopair-close
bindkey -R -M viins '^\\^-'_' self-insert
bindkey -M viins '^\' ' zpwrInterceptSurround
bindkey -R -M viins '^a-'-'f' self-insert
bindkey -M viins '^fj' vi-cmd-mode
bindkey -R -M viins '^g-'-'j' self-insert
bindkey -M viins '^jf' vi-cmd-mode
bindkey -R -M viins '^k-'-'z' self-insert
bindkey -M viins '^{' zpwrInterceptSurround
bindkey -M viins '^|' self-insert
bindkey -M viins '^}' autopair-close
bindkey -M viins '^~' self-insert
bindkey -M viins '^?' zpwrInterceptDelete
bindkey -R -M viins '^\\M-^@-'-'\\M-^?' self-insert
```

ALL Zsh Visual Mode (1/2)

```
bindkey -a 'V' visual-line-mode
bindkey -a 'v' visual-mode
bindkey -M visual '^[' deactivate-region
bindkey -M visual '^['OA' up-line
bindkey -M visual '^['OB' down-line
bindkey -M visual '^['[A' up-line
bindkey -M visual '^['[B' down-line
bindkey -M visual '-' vi-up-line-or-history
bindkey -M visual 'U' vi-up-case
bindkey -M visual 'a\'' select-quoted
bindkey -M visual 'a'' select-quoted
bindkey -M visual 'a(' select-bracketed
bindkey -M visual 'a)' select-bracketed
bindkey -M visual 'a+' select-quoted
bindkey -M visual 'a,' select-quoted
bindkey -M visual 'a-' select-quoted
bindkey -M visual 'a.' select-quoted
bindkey -M visual 'a/' select-quoted
bindkey -M visual 'a:' select-quoted
bindkey -M visual 'a;' select-quoted
bindkey -M visual 'a<' select-bracketed
bindkey -M visual 'a=' select-quoted
bindkey -M visual 'a>' select-bracketed
bindkey -M visual 'a@' select-quoted
bindkey -M visual 'aB' select-bracketed
bindkey -M visual 'aW' select-a-blank-word
bindkey -M visual 'a[' select-bracketed
bindkey -M visual 'a\\' select-quoted
bindkey -M visual 'a]' select-bracketed
bindkey -M visual 'a\'' select-quoted
bindkey -M visual 'aa' select-a-shell-word
bindkey -M visual 'ab' select-bracketed
bindkey -M visual 'aw' select-a-word
bindkey -M visual 'a{' select-bracketed
bindkey -M visual 'a|' select-quoted
bindkey -M visual 'a}' select-bracketed
bindkey -M visual 'i\'' select-quoted
bindkey -M visual 'i'' select-quoted
bindkey -M visual 'i(' select-bracketed
bindkey -M visual 'i)' select-bracketed
bindkey -M visual 'i+' select-quoted
bindkey -M visual 'i,' select-quoted
bindkey -M visual 'i-' select-quoted
bindkey -M visual 'i.' select-quoted
bindkey -M visual 'i/' select-quoted
bindkey -M visual 'i:' select-quoted
bindkey -M visual 'i;' select-quoted
bindkey -M visual 'i<' select-bracketed
```

ALL Zsh Visual Mode (2/2)

```
bindkey -M visual 'i=' select-quoted
bindkey -M visual 'i>' select-bracketed
bindkey -M visual 'i@' select-quoted
bindkey -M visual 'iB' select-bracketed
bindkey -M visual 'iW' select-in-blank-word
bindkey -M visual 'i[' select-bracketed
bindkey -M visual 'i\\' select-quoted
bindkey -M visual 'i]' select-bracketed
bindkey -M visual 'i\' ' select-quoted
bindkey -M visual 'ia' select-in-shell-word
bindkey -M visual 'ib' select-bracketed
bindkey -M visual 'iw' select-in-word
bindkey -M visual 'i{' select-bracketed
bindkey -M visual 'i|' select-quoted
bindkey -M visual 'i}' select-bracketed
bindkey -M visual 'j' down-line
bindkey -M visual 'k' up-line
bindkey -M visual 'o' exchange-point-and-mark
bindkey -M visual 'p' put-replace-selection
bindkey -M visual 'u' vi-down-case
bindkey -M visual 'x' vi-delete
bindkey -M visual '~' vi-oper-swap-case
```

ALL Vim Normal Mode (1/3)

```
n <Esc><C-C> * wvU
n <Esc><C-T> * :call TransposeWords()<CR>
n <Space>mt * :MinimapToggle<CR>
n <Space>mc * :MinimapClose<CR>
n <Space>mu * :MinimapUpdate<CR>
n <Space>mm * :Minimap<CR>
n <Space><Tab> :call AltOrNextBuffer()<CR>
n <Space>/ * :LocateAll<CR>
n <Space>ma * :Marks.<CR>
n <Space>] * <C-W>}<CR>
n <Space>ta * :Tags.<CR>
n <Space>h: * :History:.<CR>
n <Space>h/ * :History/.<CR>
n <Space>hh * :History.<CR>
n <Space>rg * :Rg<CR>
n <Space>oa * :ALEToggle<CR>
n <Space>m * :Map.<CR>
n <Space>j * :Lines.<CR>
n <Space>ke * :FZFKeys<CR>
n <Space>, * :FZFMaps<CR>
n <Space>aa * :Agg<CR>
n <Space>i * :Imap<CR>
n <Space>f * :Files.<CR>
n <Space>env * :ZFEnv<CR>
n <Space>; * :Commands.<CR>
n <Space>. * :Colors<CR>
n <Space>b * :Buffers.<CR>
n <Space>ag * :Ag<CR>
n <Space>vj * :w.<CR>:call TmuxRepeat('file')<CR>
n <Space>ev :call ExtractVariable()<CR>
n <Space>os * :call CopyCWordClip()<CR> :call system('bash ZPWR_TMUX/goo
n <Space>ox * :call CopyCWordClip()<CR> :call system('bash ZPWR_TMUX/goo
n <Space>oc * :call CopyCWordClip()<CR>
n <Space>x * :normal mzg&'zzz<CR>
n <Space>p * :bprev<CR>
n <Space>n * :bnext<CR>
n <Space>ap * :prev<CR>
n <Space>an * :next<CR>
n <Space>lo * :lopen<CR>
n <Space>lc * :lclose<CR>
n <Space>lp * :lprev<CR>
n <Space>ln * :lnext<CR>
n <Space>[ * :call Quoter('bracket')<CR>
n <Space>' * :call Quoter('back')<CR>
n <Space>' * :call Quoter('single')<CR>
n <Space>' * :call Quoter('double')<CR>
n <Space>z * :Commits.<CR>
n <Space>t * :tabnew<CR>
```

ALL Vim Normal Mode (2/3)

```
n <Space>s * :split<CR>
n <Space>v * :vsplit<CR>
n <Space>w * :w.<CR>
n <Space>e * :q.<CR>
n <Space>wq * :wq.<CR>
n <Space>q * :qa.<CR>
n <Space>r * :%s@\C\<<[R]<[W]>\>@[R]<[W]>@g<Left><Left>
n <Space>g * :%s@\C\<<[R]<[W]>\>@g<Left><Left>
n <Space>= * 4+
n <Space>- * 4-
n &g * :normal mzg&'zzz<CR>
n & * :&&<CR>
n <s<Esc> & <Nop>
n =s<Esc> & <Nop>
n >s<Esc> & <Nop>
n N * :call GoToLastSearch('?')<CR>
n Y * yy'>
n [o<Esc> & <Nop>
n [ ] k][%?]<CR>]]]]]]
n [[ ?]<CR>w99[{
n \K * :OnlineThesaurusCurrentWord<CR>
n ]o<Esc> & <Nop>
n ]] j0[[%/{<CR>
n ][ /]<CR>b99}}
n n * :call GoToLastSearch('/')<CR>
n yo<Esc> & <Nop>
n y<[<G> & :<[<U>call setreg(v:register, fugitive#Object(@%))<CR>
n z/ * :if AutoHighlightToggle()|set hls|endif<CR>
n <SNR>198_ * :<[<U><[R>=v:count ? v:count : ''<CR>
n <[<G> * :call multiple_cursors#new('n', 1)<CR>
n <F11> * :call conque_term#exec_file()<CR>
n <[<W>\ * :vsplit<CR>
n <[<W>- * :split<CR>
n <[<D>, * :FZFEnv<CR>
n <[<D>. * :FZFKeys<CR>
n <[<D>/ * :LocateAll<CR>
n <[<D>z * :TlistAddFiles *<CR>:TlistToggle<CR>
n <[<D>y * :update<CR>:SyntasticCheck<CR>
n <[<D>x * :Marks<CR>
n <[<D>w * :update<CR>
n <[<D>v * :w.<CR>:call TmuxRepeatGeneric()<CR>
n <[<D>u * :History:<CR>
n <[<D>] * <[<W>}<CR>
n <[<D>t * :Tags<CR>
n <[<D>s * :History/<CR>
n <[<D>rr * :Rg<CR>
n <[<D>rq * :silent .open -t %:p:h<CR>:redraw.<CR>
n <[<D>q * :SaveSession.<CR><Tab>
```

ALL Vim Normal Mode (3/3)

```
n <C-D>p * :call PasteClip()<CR>
n <C-D>r * :call GetRef()<CR>
n <C-D>o * :ALEToggle<CR>
n <C-D>n * :Snippets<CR>
n <C-D>m * :Map.<CR>
n <C-D>l * :Lines.<CR>
n <C-D>k * :ALEFix<CR>
n <C-D>j * :Agg<CR>
n <C-D>i * :Imap<CR>
n <C-D>h * :HistoryFiles<CR>
n <C-D>g * :Commits.<CR>
n <C-D>f * :Files.<CR>
n <C-D>e * :ALEInfo<CR>
n <C-D>d * :Commands<CR>
n <C-D>c * :Colors<CR>
n <C-D>b * :Buffers<CR>
n <C-D>a * :Ag<CR>
n <C-D><C-D> * :GitGutterUndoHunk<CR>
n <C-V> * :w.<CR>:call TmuxRepeat('file')<CR>
n <C-Bslash> * +
n <F7> * :TTags<CR>
n <F6> * :SyntasticToggleMode<CR>
n <F5> * :LOTRToggle<CR>
n <F4> * :MinimapToggle<CR>
n <F3> * :TlistAddFiles *<CR>:TlistToggle<CR>
n <F2> * :UndotreeToggle<CR>
n <F1> * :NERDTreeToggle<CR>
n <F8> * :%s@@@g<Left><Left><Left>
n <C-Up> * :<C-U>call GoToNextMarker('{{',1)<CR>
n <C-Down> * :<C-U>call GoToNextMarker('{{',0)<CR>
n <End> * G
n <Home> * gg
n <C-F> * :q.<CR>
n <C-C> * :wq.<CR>:qa.<CR>
n <C-T> * xp
n <C-L> * 4l
n <C-H> * 4h
n <C-K> * 4k
n <C-J> * 4j
n <D-v> * '*p
```

ALL Vim Insert Mode (1/2)

```
i <M-d> * <C-O>dw
i <F3l> * <C-O>dw
i <C-F> * col('.')>strlen(getline('.'))?'\\<C-F>':'\\<Right>'
i <C-E> * col('.')>strlen(getline('.'))||pumvisible()?'\\<C-E>':'\\<End>'
i <C-D> * col('.')>strlen(getline('.'))?'\\<C-D>':'\\<Del>'
i <C-X><C-A> * <C-A>
i <C-A> * <C-O>^
i <C-X> * <C-R>=<SNR>69_ManualCompletionEnter()<CR>
i <C-Tab> * <C-R>=UltiSnips#ListSnippets()<CR>
i <C-X><C-L> * fzf#vim#complete(fzf#wrap({'prefix': '^.*', 'source': 'r
i <C-X><C-K> * fzf#vim#complete#word({'left': '15%'})
i <F1l> * <C-X><C-T>
i <F10> * <C-X><C-K>
i <C-D>n * <C-X><C-O>
i <C-D>\ * <C-X><C-L>
i <C-D>, * <C-O>:FZFEnv<CR>
i <C-D>. * <C-O>:FZFKeys<CR>
i <C-D>/ * <C-O>:LocateAll<CR>
i <C-D>z * <Esc>:TlistAddFiles * <CR> :TlistToggle<CR>i
i <C-D>y * <Esc>:update<CR>:SyntasticCheck<CR>a
i <C-D>x * <C-O>:Marks<CR>
i <C-D>w * <C-O>:update<CR>
i <C-D>v * <Esc>:w.<CR>:call TmuxRepeatGeneric()<CR>a
i <C-D>u * <C-O>:History:<CR>
i <C-D>] * <C-W>}<CR>
i <C-D>t * <C-O>:Tags<CR>
i <C-D>s * <C-O>:History/<CR>
i <C-D>rr * <Esc>:Rg<CR>
i <C-D>rq * <Esc>:silent .open -t %:p:h<CR>:redraw.<CR>a
i <C-D>q * <C-O>:SaveSession.<CR><Tab>
i <C-D>p * <C-O>:call PasteClip()<CR>
i <C-D>r * <C-O>:call GetRef()<CR>
i <C-D>o * <C-O>:ALEToggle<CR>
i <C-D>m * <C-O>:Map<CR>
i <C-D>l * <C-O>:Lines<CR>
i <C-D>k * <C-O>:ALEFix<CR>
i <C-D>j * <C-O>:Agg<CR>
i <C-D>i * <C-O>:Imap<CR>
i <C-D>h * <C-O>:HistoryFiles<CR>
i <C-D>g * <C-O>:Commits.<CR>
i <C-D>f * <C-O>:Files<CR>
i <C-D>e * <C-O>:ALEInfo<CR>
i <C-D>d * <C-O>:Commands<CR>
i <C-D>c * <C-O>:Colors<CR>
i <C-D>b * <C-O>:Buffers<CR>
i <C-D>a * <C-O>:Ag<CR>
i <C-D><C-D> * <C-O>:GitGutterUndoHunk<CR>
i <C-Bslash> * <Esc>+
```


ALL Vim Insert Mode (2/2)

```
i <F7> * <Esc>:TTags<CR>
i <F6> * <Esc>:SyntasticToggleMode<CR>
i <F5> * <Esc>:LOTRToggle<CR>
i <F4> * <Esc>:MinimapToggle<CR>
i <F3> * <Esc>:TlistAddFiles *<CR>:TlistToggle<CR>
i <F2> * <Esc>:UndotreeToggle<CR>
i <F1> * <Esc>:NERDTreeToggle<CR>
i <F8> * <Esc>:%s@@@g<Left><Left><Left>
i <C-Up> * <Esc>:<C-U>call GoToNextMarker('{{',1)<CR>i
i <C-Down> * <Esc>:<C-U>call GoToNextMarker('{{',0)<CR>i
i <C-B><C-N> * <Esc>^2xji
i <C-B> * getline('.')=~'^\s*'&&col('.')>strlen(getline('.'))?'0\<C-D>
i <End> * <Esc>Gi
i <Home> * <Esc>ggi
i <C-C> * <Esc>:wq.<CR>:qa.<CR>
i <C-T> * i<BS><C-O>:silent. undojoin | normal. xp<CR>
i <C-D><C-T> * <C-O>:call TransposeWords()<CR>
i <C-L> * <Esc>mbgg=G'bzza
i <Tab> * <C-R>=UltiSnips#ExpandSnippetOrJump()<CR>
i fj <Esc>
i jf <Esc>
```

ALL Vim Visual Mode (1/1)

```
v <Space>vl * <Esc>:call TmuxRepeat('repl')<CR>gv
v <Space>vk * <Esc>:call TmuxRepeat('visual')<CR>gv
v <Space>vj * <Esc>:call TmuxRepeat('file')<CR>gv
v <Space>( * :call InsertQuoteVisualMode('paren')<CR>
v <Space>{ * :call InsertQuoteVisualMode('curlybracket')<CR>
v <Space>[ * :call InsertQuoteVisualMode('bracket')<CR>
v <Space>' * :call InsertQuoteVisualMode('back')<CR>
v <Space>' * :call InsertQuoteVisualMode('single')<CR><CR>
v <Space>' * :call InsertQuoteVisualMode('double')<CR>
v <Space>r * :<','>s@\C\<<C-R><C-W>\>@<C-R><C-W>@g<Left><Left>
v <Space>g * :<','>%s@\C\<<C-R><C-W>\>@@g<Left><Left>
v <Space>b * :w .tmux set-buffer '(cat)'<CR><CR>
v <Space>= * 4+
v <Space>- * 4-
v < * <gv
v > * >gv
v J * :m '> + <CR> gv
v K * :m '< -- <CR> gv
v Y * y'>j
v \K * y:Thesaurus <C-R>'<CR>
v go * :call CopyClip()<CR> :call system('bash ZPWR_TMUX/google.sh open'
v gs * :call CopyClip()<CR> :call system('bash ZPWR_TMUX/google.sh googl
v <C-D>d * :<C-C>:update<CR>
v <C-D>y * :<C-C>:update<CR>:SyntasticCheck<CR>
v <C-D>, * :call NERDComment('x','Toggle')<CR>'>
v <RightMouse> * :call CopyClip()<CR>'>
v <C-B> * :call CopyClip()<CR>'>
v <C-Up> * :m '< -- <CR> gv
v <C-Down> * :m '> + <CR> gv
v <C-Left> * <gv
v <C-Right> * >gv
v <C-F> * :<C-C>:q.<CR>
v <C-L> * 4l
v <C-K> * 4k
v <C-J> * 4j
v <D-x> '*d
v <D-c> '*y
v <D-v> '-d'*p
```

Colophon

This book is generated, not typed twice. The manuscript is `docs/book.md`, and `docs/book.md` is itself rendered from the environment it documents: `docs/genEncyclopediaMd.py` reads the wizard pages that ship inside the public `zpwr` source (`docs/wizard_pages/page_*.zsh`), strips the shell's colour machinery, classifies each printed line as chapter, section, command, comment or prose, and emits pandoc markdown. So the encyclopedia and the wizard a reader can page through in their own terminal are the same text, produced from the same file, and cannot drift apart.

From there `scripts/book_pdf.sh` renders the manuscript to PDF through pandoc and LuaLaTeX with the project's own theme, and `scripts/book_epub.sh` renders the same manuscript — plus this apparatus — to a reflowable EPUB 3. The cover is not hand-placed either: `scripts/book_cover.sh` draws it at 1600×2560 from the same palette the PDF theme uses, so the cover on a storefront and the chapter headings inside the book cannot drift apart.

The display faces are Orbitron and Share Tech Mono; body text is set in Arimo. All three are licensed under the SIL Open Font License 1.1.

The verb count printed on the cover and in the prose is not typed in. Both are derived at build time from the distinct keys of the `ZPWR_VERBS` associative array in the source's `env/zpwr.zsh` — the same table the dispatcher itself reads — so the number in this edition is the number the shell would answer with.

Reproducing the Book's Claims

The book describes a shell environment that is installed and running, so its central claims are checkable from a prompt inside it rather than from a build directory.

The size of the arsenal — the verb table's own count, which is what the cover and the opening chapter report:

```
zpwr verbscount
```

The same figure taken straight from the source the dispatcher loads, which is how the build derives it:

```
perl -ne 'while (/ZPWR_VERBS\[([^\]]+)\]=/g) { $k{$1}=1 }  
END { print scalar keys %k, "\n" }' "$ZPWR/env/zpwr.zsh"
```

Every verb, dumped rather than summarised:

```
zpwr subcommands
```

The loaded environment the performance and introspection chapters describe — aliases, functions, builtins, parameters, completions, counted as they stand:

```
zpwr environmentcounts
```

The health check the diagnostics chapter walks through, run against the reader's own machine rather than the author's:

```
zpwr doctor
```

And the manuscript itself, regenerated from the wizard pages so that the text of this book can be diffed against the environment it documents:

```
python3 docs/genEncyclopediaMd.py "$ZPWR/docs/wizard_pages" docs/book.md
```

Also from MenkeTechnologies

zpwr is one program in a stack that is documented the same way — from source, with the numbers cited to the tools that produce them. Companion volumes cover the shell, the terminal emulator, the multiplexer, the bytecode VM the language frontends share, and the language ports built on it.

Titles and availability change; the current catalog is published at github.com/MenkeTechnologies.